

ПОДГОТОВКА К СОБЕСЕДОВАНИЮ

Go для Backend-разработчика

Глубокий справочник: от битов в памяти до системного дизайна

10 глав · от типов и рантайма до баз данных и архитектуры
С разбором задач и диаграммами

Содержание

- 1 Глава 1. Типы данных и память
- 2 Глава 2. Runtime и память
- 3 Глава 3. Конкурентность
- 4 Глава 4. Интерфейсы и дизайн
- 5 Глава 5. Ошибки
- 6 Глава 6. HTTP и сети
- 7 Глава 7. gRPC и Protobuf
- 8 Глава 8. Архитектура и системный дизайн
- 9 Глава 9. SQL и базы данных
- 10 Глава 10. Алгоритмические и скрининг-задачи

Глава 1. Типы данных и память

Введение к главе

Прежде чем говорить о горутинах, каналах и планировщике, нужно понять фундамент — как Go хранит данные в памяти. Почти все «странные» вопросы на собеседовании («почему изменение одного слайса повлияло на другой?», «почему строку нельзя изменить по индексу?», «почему этот код не компилируется?») сводятся к одному принципу, который стоит запомнить с самого начала:

В Go всё передаётся по значению. Всегда. Без исключений.

Эта фраза кажется простой, но именно её непонимание порождает большинство ошибок. Мы будем возвращаться к ней снова и снова — на слайсах, на map, на структурах, на интерфейсах. Держи её в голове.

1.1. Базовые типы и zero values

Числовые типы

В Go числовые типы имеют **явно заданный размер в битах** — это отличает его от многих языков, где размер `int` зависит от платформы неявно.

Тип	Размер	Диапазон
<code>int8</code> / <code>uint8</code>	8 бит	-128..127 / 0..255
<code>int16</code> / <code>uint16</code>	16 бит	±32K / 0..65535
<code>int32</code> / <code>uint32</code>	32 бита	±2.1 млрд / 0..4.2 млрд
<code>int64</code> / <code>uint64</code>	64 бита	±9.2·10 ¹⁸ / 0..1.8·10 ¹⁹
<code>int</code> / <code>uint</code>	32 или 64 бита	зависит от платформы
<code>float32</code>	32 бита	~7 значащих цифр
<code>float64</code>	64 бита	~15 значащих цифр

Ключевой момент про `int`: это **не отдельный тип фиксированного размера**, а платформозависимый. На современных 64-битных системах `int` — это 64 бита. Но `int` и `int64` — это **разные типы** с точки зрения компилятора, и Go не будет автоматически их конвертировать:

```
var a int = 5
var b int64 = 10
c := a + b // ошибка компиляции: mismatched types int and int64
```

Это следствие философии Go: **никаких неявных числовых преобразований**. Если хочешь сложить — конвертируй явно: `int64(a) + b`. Разберём конвертацию подробнее в разделе 1.10.

byte и rune — это псевдонимы

Два типа, которые постоянно встречаются при работе со строками:

- `byte` — это **псевдоним** (alias) для `uint8`. Один байт, 0..255.
- `rune` — это **псевдоним** для `int32`. Представляет один Unicode code point (символ).

Слово «псевдоним» здесь точное: `byte` и `uint8` — это буквально один и тот же тип, просто `byte` читается понятнее, когда речь о данных, а `uint8` — когда о числах. То же с `rune` и `int32`.

Почему `rune` — это 32 бита? Потому что Unicode-символов больше, чем помещается в один байт. Символ `'a'` влезает в байт, а `'ë'`, `'中'` или эмодзи — нет, им нужно до 4 байт. Тип `rune` (`int32`) гарантированно вмещает любой Unicode code point. Подробно про руны и строки — в разделе 1.6.

bool и string

- `bool` — `true` или `false`. Не является числом (в отличие от C, где 0/1). Нельзя написать `if 1 {}`.
- `string` — неизменяемая последовательность байтов. Это отдельная большая тема, ей посвящён раздел 1.6.

Zero values — почему в Go нет null

Вот принципиальное дизайнерское решение Go, которое стоит понять глубоко.

В Go нет понятия «неинициализированная переменная». Как только ты объявил переменную, она **сразу** имеет значение — так называемое **нулевое значение** (zero value). У каждого типа оно своё:

Тип	Zero value
Числовые (<code>int</code> , <code>float64</code> , ...)	<code>0</code>
<code>bool</code>	<code>false</code>
<code>string</code>	<code>""</code> (пустая строка, не nil!)
Указатели, <code>slice</code> , <code>map</code> , <code>chan</code> , <code>func</code> , <code>interface</code>	<code>nil</code>
<code>struct</code>	структура, где каждое поле = своё zero value

```
var i int      // 0
var f float64  // 0.0
var b bool     // false
var s string   // "" — пустая строка, а НЕ nil
var p *int     // nil
```

Почему это важно и как влияет на дизайн:

1. Нет NullPointerException «на пустом месте». В Java `String s;` даёт `null`, и обращение к `s.length()` падает. В Go `var s string` даёт `""`, и `len(s)` спокойно возвращает `0`. Многие типы в Go спроектированы так, чтобы их zero value был **сразу пригоден к использованию** — это называют «zero value useful».

Классический пример — `sync.Mutex`:

```
var mu sync.Mutex
mu.Lock() // работает сразу, без инициализации
```

Тебе не нужно писать `mu = NewMutex()`. Нулевой мьютекс — это готовый к работе разблокированный мьютекс. Тот же принцип у `bytes.Buffer`, `sync.WaitGroup` и многих других типов стандартной библиотеки. Когда будешь проектировать свои структуры — стремись к тому же.

2. Но `nil` для ссылочных типов всё же существует — и вот тут начинаются нюансы, особенно с `nil slice` vs пустым слайсом. Это настолько частый вопрос на собеседовании, что вынесем его отдельно в раздел 1.5.

1.2. Константы и `iota`

Константы в Go — тема, которую недооценивают, а спрашивают часто. Здесь есть неочевидная глубина.

Типизированные и нетипизированные константы

Константа объявляется через `const`:

```
const Pi = 3.14159           // нетипизированная
const MaxUsers int = 1000    // типизированная
```

Разница огромная. **Нетипизированная константа** не имеет конкретного типа до момента использования — у неё есть только «вид» (kind): целочисленная, вещественная, строковая и т.д. Она приобретает тип только тогда, когда её куда-то присваивают или используют в выражении.

Почему это удобно — посмотри:

```
const c = 100 // нетипизированная целочисленная константа

var i int = c // c становится int
var f float64 = c // тот же c становится float64
var b byte = c // и byte — тоже ок (100 влезает)
```

Одна и та же константа `c` спокойно «подстраивается» под нужный тип. Если бы она была типизированной (`const c int = 100`), то `var f float64 = c` дало бы ошибку — пришлось бы писать `float64(c)`.

Это делает нетипизированные константы гибкими: они ведут себя как «идеальные числа» до последнего момента. Именно поэтому большинство констант в Go оставляют нетипизированными — не сковывая их типом без необходимости.

`iota` — генератор последовательностей

`iota` — это встроенный счётчик, который работает **внутри блока `const`**. Он начинается с `0` и увеличивается на `1` с каждой строкой блока.

```
const (
    A = iota // 0
    B = iota // 1
    C = iota // 2
)
```

Поскольку повторять `= iota` скучно, Go позволяет опустить выражение — оно **повторяется автоматически**:

```
const (
    A = iota // 0
    B        // 1 (повторяется = iota)
    C        // 2
)
```

Это самый частый паттерн: перечисления (enum). Например, дни недели или статусы:

```
const (
    StatusPending = iota // 0
    StatusActive         // 1
    StatusBlocked        // 2
)
```

`iota` и битовые маски

А вот где `iota` раскрывается по-настоящему — битовые флаги. Классический приём: сдвигаем `1` влево на `iota` бит.

```
const (
    FlagRead    = 1 << iota // 1 << 0 = 1 (0b001)
    FlagWrite    // 1 << 1 = 2 (0b010)
    FlagExecute  // 1 << 2 = 4 (0b100)
)
```

Теперь можно комбинировать флаги через побитовое ИЛИ и проверять через И:

```
perms := FlagRead | FlagWrite // 0b011 = права на чтение и запись

if perms&FlagWrite != 0 {
    // есть право на запись
}
```

Каждый флаг занимает свой бит, и они не пересекаются — поэтому их можно свободно комбинировать в одном числе. Такой приём ты увидишь в стандартной библиотеке (например, флаги открытия файла `os.O_RDONLY`, `os.O_CREATE` и т.д.).

Небольшой трюк: если нужно пропустить значение `0`, используют `_`:

```
const (
    _ = iota // пропускаем 0
    KB = 1 << (10 * iota) // 1 << 10 = 1024
    MB // 1 << 20
    GB // 1 << 30
)
```

Здесь одной конструкцией сгенерированы размеры KB/MB/GB — красиво и читаемо.

1.3. Указатели и семантика передачи по значению

Возвращаемся к главному принципу главы. Разберём его до конца, потому что на нём стоит половина книги.

Что такое указатель

Указатель — это переменная, которая хранит **адрес** другой переменной в памяти.

- `&x` — взять адрес переменной `x` (получить указатель на неё).
- `*p` — разыменовать указатель `p` (получить значение по адресу).

```
x := 42
p := &x // p — указатель на x, тип *int
fmt.Println(*p) // 42 — разыменовали, получили значение
*p = 100 // меняем значение по адресу
fmt.Println(x) // 100 — x изменился!
```

Тип указателя записывается со звёздочкой: `*int` — «указатель на int».

Главный принцип: всё копируется

Когда ты передаёшь переменную в функцию, Go **всегда создаёт копию**. Функция работает с копией, а не с оригиналом:

```
func increment(n int) {  
    n = n + 1 // меняем КОПИЮ  
}  
  
func main() {  
    x := 5  
    increment(x)  
    fmt.Println(x) // 5 — оригинал не изменился  
}
```

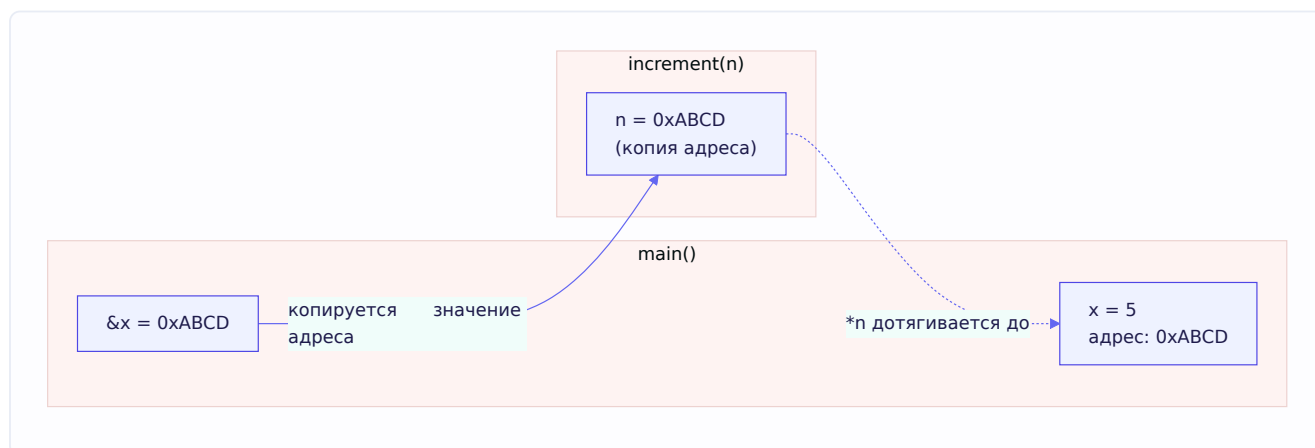
Чтобы функция могла изменить оригинал, нужно передать **адрес** — указатель:

```
func increment(n *int) {  
    *n = *n + 1 // меняем значение по адресу  
}  
  
func main() {  
    x := 5  
    increment(&x) // передаём адрес x  
    fmt.Println(x) // 6 — оригинал изменился  
}
```

Тонкость, которая ломает мозг новичкам

«Но подожди — указатель ведь тоже копируется?» Да! И это правильное понимание. Когда ты передаёшь `&x` в функцию, **копируется сам указатель** (то есть копируется адрес). Но копия адреса указывает на **то же самое место в памяти**, что и оригинал. Поэтому через `*n` ты достигаешь до исходной переменной.

Разберём на схеме:



Указатель `n` внутри функции — это **копия** адреса. Но раз это тот же адрес, `*n` меняет ту же ячейку памяти, что видна в `main` как `x`.

Вот почему фраза «всё передаётся по значению» не противоречит указателям: значение указателя — это адрес, и копируется именно адрес. Никакой магии «передачи по ссылке» в Go нет — есть только копирование, просто иногда копируется адрес.

new

Функция `new(T)` выделяет память под значение типа `T`, заполняет её нулевым значением и возвращает **указатель** на неё:

```
p := new(int) // p — это *int, указывает на 0
fmt.Println(*p) // 0
*p = 42
```

На практике `new` используют нечасто — обычно берут адрес через `&`:

```
p := &SomeStruct{} // идиоматичнее, чем new(SomeStruct)
```

Оба варианта дают `*SomeStruct`, но через `&` можно сразу задать поля.

Почему одни типы «эталонно» копируются, а другие — «как будто нет»

А вот теперь — ключевой мост к следующим разделам. Ты наверняка слышал, что «слайсы и тар передаются по ссылке». **Это неточная формулировка, и на собеседовании за неё могут зацепиться.**

Правда в том, что слайсы и тар тоже копируются по значению — как и всё в Go. Но их **значение само по себе содержит указатель внутри**. Слайс — это маленькая структура из трёх полей (указатель на массив, длина, вместимость). Когда ты копируешь слайс, копируются эти три поля, но указатель внутри продолжает показывать на **тот же самый массив данных**. Поэтому изменения данных видны «снаружи» — но не потому что «передача по ссылке», а потому что скопированный указатель ведёт туда же.

Это различие — не придирка к словам, а корень целой пачки задач с собеседований. Мы разберём его в мельчайших деталях в разделе 1.5, когда дойдём до `SliceHeader`. Пока просто зафиксируй мысль:

«Передаётся по значению» и «внутри значения лежит указатель» — совместимые вещи. Слайс, тар, канал — это про второе, а не про «передачу по ссылке».

1.4. Массивы и слайсы

Это фундамент, на котором стоит львиная доля Go-кода. И именно здесь кроется большинство «что выведет программа?»-задач с собеседований. Разберёмся медленно и по-настоящему.

Массив — фиксированный размер

Массив в Go — это последовательность элементов **фиксированной длины**, заданной при объявлении. Длина — часть типа:

```
var a [3]int      // массив из 3 int, все = 0
b := [3]int{1, 2, 3}
c := [...]int{1, 2, 3} // компилятор сам посчитает длину = 3
```

Важнейший факт: `[3]int` и `[4]int` — это **разные типы**. Нельзя присвоить один другому, нельзя передать `[4]int` в функцию, ждущую `[3]int`.

И вот ключевое свойство, вытекающее из принципа «всё копируется»: **массив при присваивании и передаче в функцию копируется целиком**.

```
func modify(arr [3]int) {
    arr[0] = 100 // меняем КОПИЮ
}

func main() {
    a := [3]int{1, 2, 3}
    modify(a)
    fmt.Println(a) // [1 2 3] — оригинал не тронут!
}
```

Скопировался весь массив — все три элемента. Если бы массив был на миллион элементов, скопировался бы миллион. Именно поэтому массивы в чистом виде используют редко — дорого копировать. Вместо них используют слайсы.

Слайс — окно в массив

Слайс (slice) — это **гибкая, динамическая обёртка над массивом**. Он не хранит данные сам — он *ссылается* на участок какого-то массива (который называют «нижележащим», underlying array).

Создать слайс можно несколькими способами:

```
s := []int{1, 2, 3}           // литерал (массив создаётся неявно)
s := make([]int, 3)           // длина 3, все нули: [0 0 0]
s := make([]int, 3, 10)       // длина 3, вместимость 10
```

Разница с массивом видна сразу: у слайса в квадратных скобках **нет числа** — `[]int`, а не `[3]int`. Длина не является частью типа, поэтому слайс может расти.

И — внимание — слайс, в отличие от массива, при передаче в функцию **ведёт себя иначе**. Но чтобы понять, почему, нужно заглянуть под капот. Этим и займёмся в 1.5 — это настолько важно, что заслуживает отдельного раздела.

1.5. SliceHeader под капотом — сердце всех slice-задач

Вот раздел, ради которого стоило начинать. Если ты поймёшь его по-настоящему, ты решишь **любую** задачу на слайсы, которую тебе дадут. А их дают почти всегда.

Из чего состоит слайс

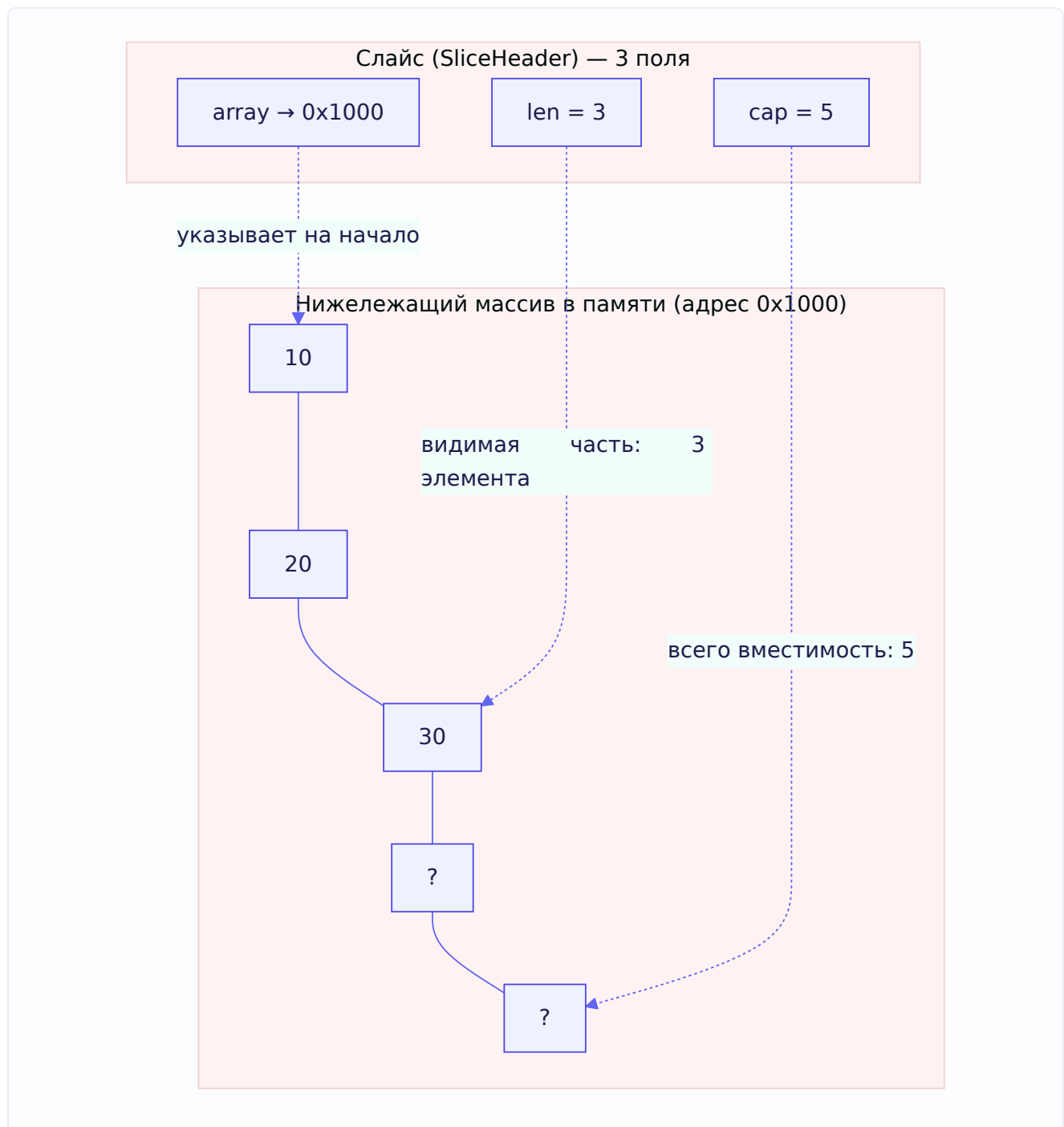
Слайс — это структура из **трёх полей**. В рантайме она описана примерно так:

```
type slice struct {  
    array unsafe.Pointer // указатель на начало данных в нижележащем массиве  
    len    int           // длина: сколько элементов сейчас в слайсе  
    cap    int           // вместимость: сколько влезет до реаллокации  
}
```

Три поля, ничего больше:

- **array** — адрес, где начинаются данные. Сами данные лежат в отдельном массиве в памяти.
- **len** — сколько элементов ты «видишь» через этот слайс. Именно это возвращает `len(s)`.
- **cap** — сколько элементов помещается в нижележащем массиве, начиная от `array`. Это возвращает `cap(s)`.

Визуально:



Слайс видит первые 3 элемента (`len=3`), но массив под ним рассчитан на 5 (`cap=5`). Ячейки за пределами `len` существуют физически, но «не видны» через этот слайс — пока.

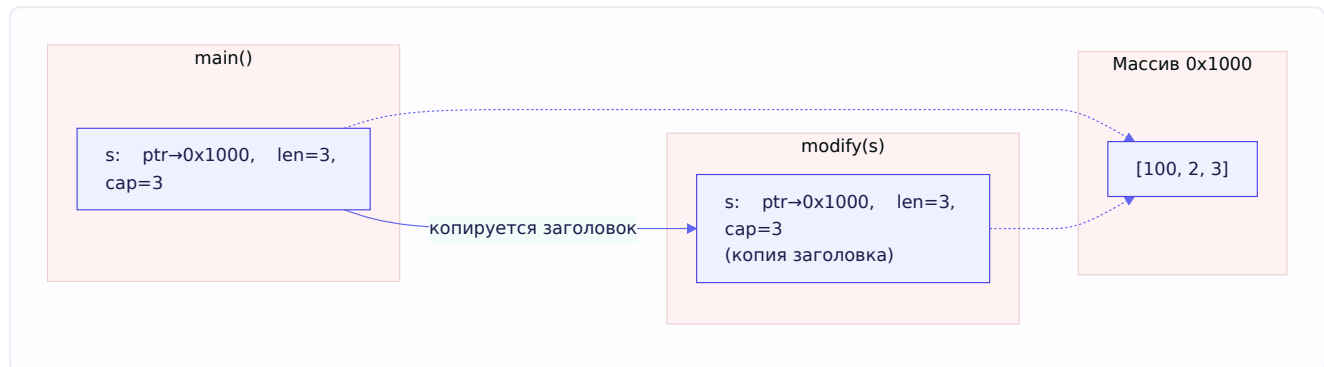
Почему слайс «передаётся по ссылке» (на самом деле нет)

Теперь вернёмся к вопросу из 1.3. Когда ты передаёшь слайс в функцию, копируется **SliceHeader** — эти три поля. Но поле `array` — указатель — в копии показывает **на тот же самый массив**. Поэтому:

```
func modify(s []int) {
    s[0] = 100 // меняем данные в нижележащем массиве
}

func main() {
    s := []int{1, 2, 3}
    modify(s)
    fmt.Println(s) // [100 2 3] — изменилось!
}
```

Копия заголовка, но тот же массив под ним → изменение элемента видно снаружи.



Оба заголовка — оригинал и копия — показывают на один массив. Вот и вся «магия».

А теперь — append и главная ловушка

Здесь начинается самое интересное. Что делает `append`?

`append` добавляет элемент в слайс. Но слайс — это окно фиксированной вместимости (`cap`). Что если места больше нет? Тогда `append` **создаёт новый, больший массив**, копирует туда старые данные, добавляет новый элемент и возвращает слайс, указывающий уже на **новый** массив.

Отсюда правило:

Если `len < cap` — `append` пишет в **существующий** массив (места хватает). Если `len == cap` — `append` **выделяет новый** массив (реаллокация).

И вот отсюда растёт классическая задача. Разберём её — она из задачника Озона дословно:

```
func a() {
    x := []int{}
    x = append(x, 0)
    x = append(x, 1)
    x = append(x, 2)
    y := append(x, 3)
    z := append(x, 4)
    fmt.Println(y, z)
}
```

Что выведет? Интуиция говорит `[0 1 2 3] [0 1 2 4]`. **Но выведет `[0 1 2 4] [0 1 2 4]`**. Почему?

Проследим по шагам, следя за `len` и `cap`:

```
x := []int{}      → len=0, cap=0
append(x, 0)      → реаллокация, len=1, cap=1
append(x, 1)      → len=1==cap=1, реаллокация, len=2, cap=2
append(x, 2)      → len=2==cap=2, реаллокация, len=3, cap=4 ← cap вырос с запасом!
```

После трёх `append` слайс `x` имеет `len=3`, `cap=4`. **Есть одна свободная ячейка!** Массив под ним выглядит так: `[0, 1, 2, _]`, где `_` — свободное место.

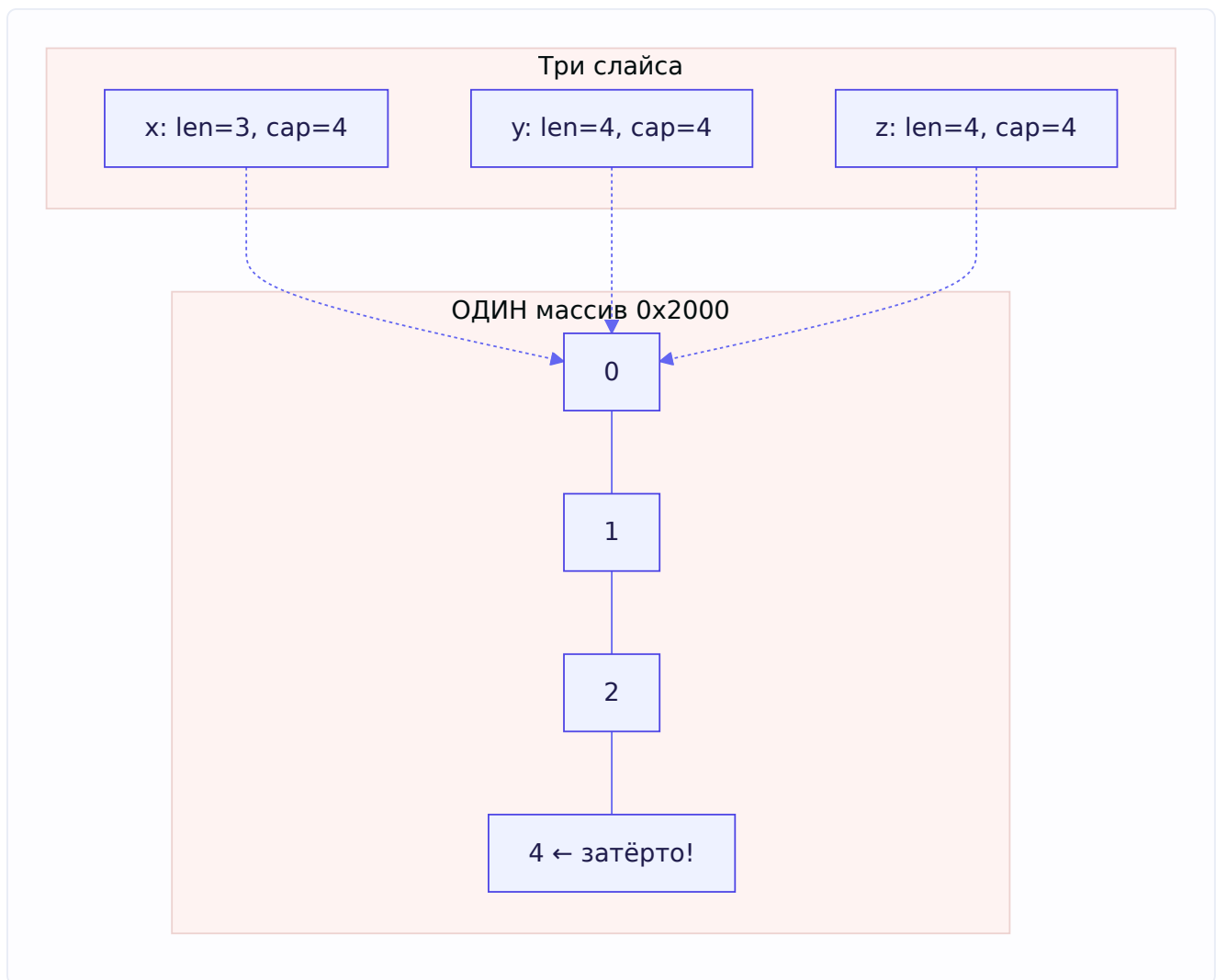
Теперь:

```
y := append(x, 3) → len(3) < cap(4), места хватает!
                  Пишем 3 в свободную ячейку того же массива: [0,1,2,3]
                  y = {ptr→тот же массив, len=4, cap=4}
```

Массив стал `[0, 1, 2, 3]`. А `y` показывает на него.

```
z := append(x, 4) → x всё ещё len=3, cap=4 (x не менялся!)
                  len(3) < cap(4), места хватает
                  Пишем 4 в ТУ ЖЕ свободную ячейку (индекс 3): [0,1,2,4]
                  z = {ptr→тот же массив, len=4, cap=4}
```

Вот в чём подвох: и `y`, и `z` пишут в **одну и ту же ячейку** одного и того же массива, потому что оба стартовали от `x` с `len=3`. Второй `append` (для `z`) **затёр** значение, записанное первым (для `y`). Поэтому в ячейке с индексом 3 теперь `4`, и `y`, и `z` показывают на этот массив:



Отсюда `[0 1 2 4] [0 1 2 4]`.

Мораль, которую хотят услышать на собеседовании: `append` безопасен только если ты присваиваешь результат обратно той же переменной (`x = append(x, ...)`). Как только ты «ветвишь» `append` от одного слайса в разные переменные, а места в `cap` хватает — они начинают делить массив и затирать друг друга.

Вторая ловушка: `append` в функции не виден снаружи

Ещё одна задача из задачника — на первый взгляд противоречит тому, что слайс «по ссылке»:

```
func main() {
    nums := []int{1, 2, 3}
    addNum(nums[0:2])
    fmt.Println(nums) // ?
}

func addNum(nums []int) {
    nums = append(nums, 4)
}
```

Что выведет? `[1 2 4]`. Разберём, почему именно так — тут двойной подвох.

`nums[0:2]` — это слайс от исходного массива: `len=2, cap=3` (вместимость считается до конца нижележащего массива, а он на 3 элемента). Внутри `addNum` делаем `append`. Поскольку `len(2) < cap(3)`, места хватает — `append` пишет `4` в **третью ячейку исходного массива** (туда, где была `3`). Массив становится `[1, 2, 4]`.

Но! Присваивание `nums = append(...)` меняет только **локальную копию** заголовка внутри функции — новый `len=3` наружу не возвращается. Поэтому снаружи `nums` по-прежнему `len=3` и показывает на массив, который теперь `[1, 2, 4]`.

Итог: значение `3` затёрлось на `4` (потому что `append` писал в общий массив), но «удлинения» слайса снаружи не видно (потому что новый заголовок остался в функции). Отсюда `[1 2 4]`.

Это идеальная иллюстрация принципа: **данные массива — общие** (через указатель), **но заголовок слайса — копия**. Изменения элементов видны, изменения `len / cap` — нет.

Full slice expression — как защититься

Go даёт способ явно ограничить `cap` при слайсинге — трёхиндексная форма `s[low:high:max]`:

```
x := []int{0, 1, 2, 3, 4}
y := x[0:2:2] // len=2, cap=2 (ограничили max третьим индексом)
```

Теперь `cap(y) == len(y) == 2`. Любой `append` к `y` **гарантированно** вызовет реаллокацию (новый массив) и не тронет массив `x`. Это защита от нечаянного разделения массива — если хочешь, чтобы `append` всегда создавал новую копию, ограничивай `cap`.

Как растёт cap при append

Полезно знать алгоритм роста (спрашивают):

- Если нужного размера ещё нет, и текущая **вместимость меньше 1024** элементов — новая вместимость примерно **удваивается** ($\times 2$).
- Если вместимость **больше 1024** — растёт примерно на **четверть** ($\times 1.25$) за раз.

(Точные коэффициенты менялись между версиями Go и зависят от размера элемента, но порядок именно такой: сначала агрессивный рост $\times 2$, потом более экономный $\times 1.25$.)

Практический вывод: если ты знаешь примерный итоговый размер — **задай cap сразу** через `make([]T, 0, n)`. Иначе слайс будет многократно реаллоцироваться и копироваться по мере роста, что бьёт по производительности.

Безопасное копирование слайса

Раз слайсы делят массивы, как сделать **настоящую независимую копию**? Два способа:


```
// Способ 1: copy()
dst := make([]int, len(src))
copy(dst, src) // копирует min(len(dst), len(src)) элементов

// Способ 2: append к пустому (Go 1.21+ можно slices.Clone)
dst := append([]int(nil), src...)
```

Оба создают новый массив, никак не связанный с оригиналом.

Ловушка make с ненулевым len

Ещё частый вопрос. Что не так с этим кодом?

```
s := make([]int, 3) // len=3, cap=3, содержимое: [0, 0, 0]
s = append(s, 1)    // добавит В КОНЕЦ: [0, 0, 0, 1]
```

Люди часто думают, что `make([]int, 3)` — это «пустой слайс на 3 элемента», и `append` запишет в начало. Нет! `make([]int, 3)` создаёт слайс с **тремя реальными нулями**, а `append` добавляет **после** них. Если хочешь пустой слайс с запасом вместимости — используй `make([]int, 0, 3)` (длина 0, вместимость 3).

Когда GC удалит нижележащий массив

Массив живёт, пока на него ссылается **хоть один** слайс. Это создаёт неочевидную утечку: если ты взял маленький слайс от огромного массива, весь огромный массив останется в памяти.

```
huge := make([]byte, 10_000_000) // 10 МБ
small := huge[:10]               // взяли 10 байт
// huge вышел из области видимости, но массив на 10 МБ НЕ освободится,
// потому что small ссылается на тот же массив!
```

Решение — сделать честную копию через `copy`, тогда `small` будет ссылаться на новый маленький массив, а десятимегабайтный сможет собраться сборщиком мусора.

1.6. Строки: immutable, []byte, []rune

Строки в Go устроены хитрее, чем кажется, и на этом строят несколько задач.

Строка — это неизменяемый срез байтов

Внутри строка — это структура из **двух** полей: указатель на байты и длина.

```
type stringStruct struct {
    str unsafe.Pointer // указатель на байты
    len int           // длина в БАЙТАХ (не в символах!)
}
```

Ключевое свойство: строка **неизменяема** (immutable). Байты, на которые она ссылается, менять нельзя. Отсюда — задача из задачника:

```
s := "test"
println(s[0]) // ?
s[0] = 'R'    // ?
println(s)    // ?
```

Ответ: **этот код не скомпилируется**. Строка `s[0]` для чтения работает (вернёт байт `116` — ASCII-код буквы `t`), но `s[0] = 'R'` — ошибка компиляции: `cannot assign to s[0]`. Строки нельзя менять по индексу.

Почему так? Неизменяемость даёт важные гарантии: строки можно безопасно шарить между горутинами без блокировок, использовать как ключи map, а компилятор может их оптимизировать (например, две одинаковые строковые константы могут указывать на одну память).

Чтобы «изменить» строку, нужно превратить её в `[]byte`, изменить, и собрать обратно:

```
b := []byte(s) // копируем байты в изменяемый слайс
b[0] = 'R'
s = string(b)  // собираем новую строку
```

Обрати внимание: конвертации `[]byte(s)` и `string(b)` **копируют данные** — именно потому, что строка неизменяема, а слайс изменяем, их нельзя «шарить».

Индексация даёт байты, не символы

`s[i]` возвращает **байт** (тип `byte`), а не символ. Для строк из ASCII это одно и то же, но как только появляются многобайтные символы — разница критична:

```
s := "привет"
fmt.Println(len(s))    // 12, а не 6! (каждая кириллическая буква = 2 байта в UTF-8)
fmt.Println(s[0])      // 208 – первый БАЙТ, а не буква "п"
```

Go хранит строки в **UTF-8**. В этой кодировке ASCII-символы занимают 1 байт, кириллица — 2 байта, некоторые символы — 3-4 байта. Поэтому `len` строки — это число байтов, а не символов.

byte vs rune

- **byte** (`uint8`) — один байт. Итерация по байтам — когда важны именно байты (например, парсинг протокола).
- **rune** (`int32`) — один Unicode code point (символ). Итерация по рунам — когда важны символы.

Магия происходит в цикле `for range` по строке — он итерирует по **рунам**, а не байтам, автоматически декодируя UTF-8:

```
s := "привет"
for i, r := range s {
    fmt.Printf("%d: %c\n", i, r) // r — руна (символ), i — байтовый индекс
}
// индексы будут 0, 2, 4, 6, 8, 10 — прыгают на 2, т.к. каждый символ = 2 байта
```

Чтобы посчитать **СИМВОЛЫ** (а не байты), есть два пути:

```
utf8.RuneCountInString(s) // 6 — считает руны
len([]rune(s))            // 6 — но создаёт слайс рун (дороже по памяти)
```

Итого запомни правило: `[]byte(s)` — когда работаешь с байтами (I/O, протоколы), `[]rune(s)` — когда работаешь с символами (подсчёт, разворот строки, доступ к i-му символу).

1.7. Мар — разбираем обе реализации

Это самая объёмная тема главы. Мар спрашивают почти на каждом собеседовании, и глубина ответа сильно влияет на грейд. Мы разберём: общую теорию хеш-таблиц, классическую реализацию Go (с overflow-бакетами), новую реализацию на Swiss Tables, и разные подводные камни.

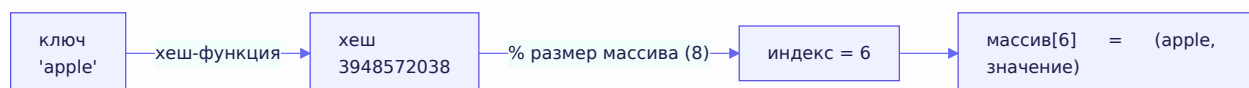
1.7.1. Общая теория: как устроена хеш-таблица

Начнём с идеи, не привязываясь к Go (именно так просят в задачнике: «интересует не реализация, а общая идея»).

Задача: хотим хранить пары ключ→значение и получать значение по ключу **быстро** — в идеале за O(1). Как?

Под капотом лежит обычный **массив** фиксированного размера. Весь фокус в том, как по ключу вычислить индекс в этом массиве:

1. **Хеш-функция** превращает ключ (например, строку) в большое число — хеш. Хорошая хеш-функция распределяет ключи равномерно.
2. Это большое число приводим к размеру массива через **остаток от деления**: `index = hash % len(array)`.
3. По этому индексу кладем пару ключ-значение.



Получение значения работает так же: хешируем ключ, берём остаток, идём по индексу.

Проблема — коллизии. Разные ключи могут дать один индекс (хешей бесконечно много, ячеек массива — мало). Два основных способа их решить:

- **Chaining (цепочки):** в каждой ячейке массива хранится не одна пара, а **список** пар. При коллизии добавляем в список. При поиске идём по списку и сравниваем ключи.
- **Open addressing (открытая адресация):** если ячейка занята, ищем ближайшую свободную по какому-то правилу (например, следующую по порядку).

Рехеширование (resize). Когда пар становится слишком много относительно размера массива (высокий load factor — коэффициент заполнения), коллизии учащаются, и $O(1)$ деградирует. Тогда создаётся массив побольше, и все пары **перераспределяются** (перехешируются) в него. Это дорогая операция, поэтому её стараются делать редко и/или размазывать во времени.

Это общая теория. Теперь — как это сделано в Go.

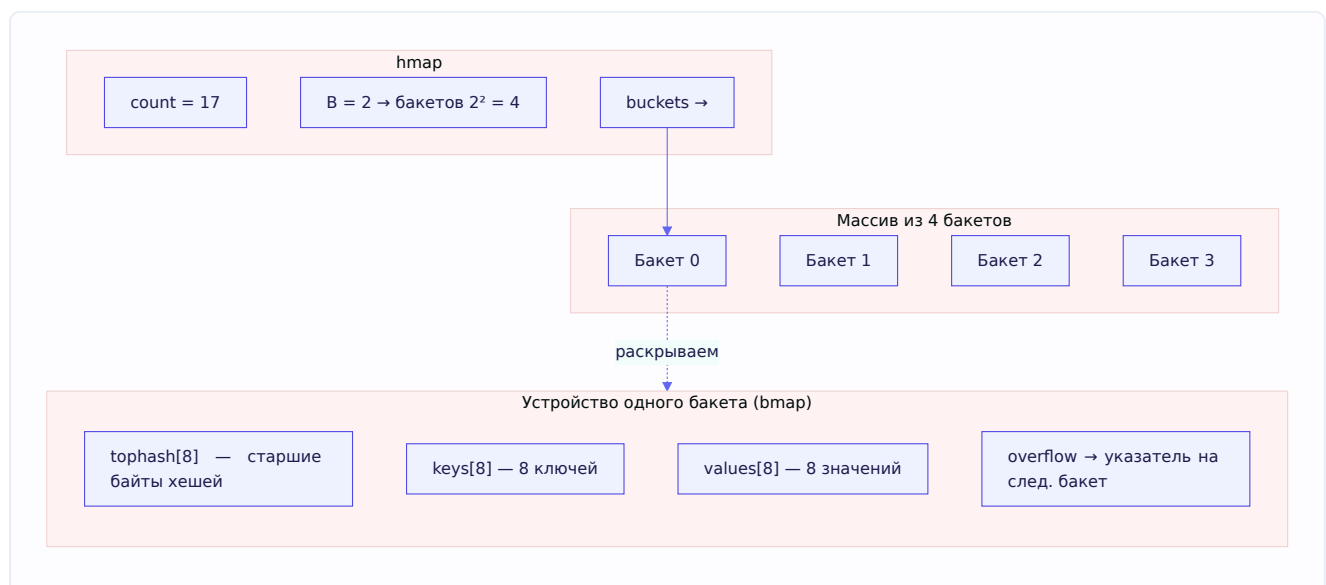
1.7.2. Классическая реализация Go (до 1.24): hmap и бакеты

Историческая реализация map в Go (она работала больше десяти лет и всё ещё важна для понимания) использует chaining, но с интересной оптимизацией — **бакеты на 8 элементов**.

Главная структура называется **hmap**:

```
type hmap struct {
    count    int    // количество элементов (это возвращает len(m))
    B        uint8   // log2 числа бакетов (бакетов = 2^B)
    buckets  unsafe.Pointer // указатель на массив бакетов
    oldbuckets unsafe.Pointer // старые бакеты во время resize
    // ... другие поля
}
```

Данные лежат не в отдельных ячейках, а в **бакетах**. Каждый бакет (**bmap**) хранит до **8 пар** ключ-значение:

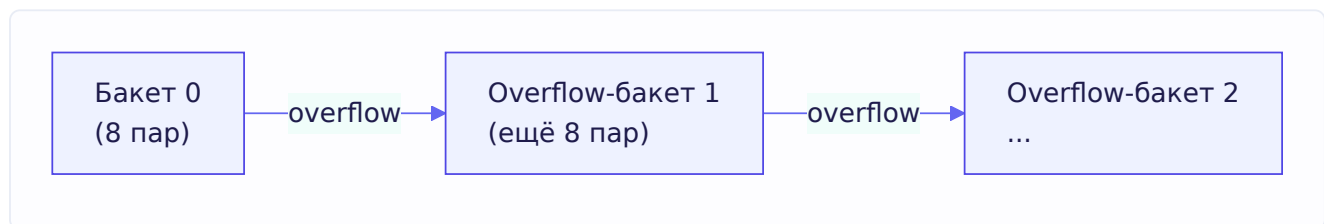


Как происходит **поиск ключа**:

1. Считаем хеш ключа.
2. **Младшие биты** хеша выбирают номер бакета ($hash \& (2^B - 1)$).

3. Внутри бакета сравниваем **tophash** — старший байт хеша — с сохранёнными. Это быстрая предфильтрация: прежде чем сравнивать ключи целиком (дорого), сравниваем один байт (дёшево). Если tophash не совпал — точно не тот ключ.
4. Если tophash совпал — сравниваем ключ целиком, чтобы исключить случайное совпадение байта.

Overflow-бакеты. А что если в один бакет попало больше 8 ключей? Тогда к бакету цепляется **overflow-бакет** — дополнительный бакет, на который указывает поле **overflow**. Получается связный список бакетов. Это и есть chaining, но на уровне бакетов, а не отдельных пар:



Почему по 8? Это оптимизация под кеш процессора: 8 пар лежат рядом в памяти, их эффективно перебирать. И tophash всех восьми лежат подряд — можно быстро отсканировать.

Resize (пост). Когда среднее число элементов на бакет превышает порог (load factor ≈ 6.5), или развелось слишком много overflow-бакетов, map удваивает число бакетов (**B** увеличивается на 1). Но перенос делается **инкрементально** («evacuation»): чтобы не было долгой паузы, старые бакеты (**oldbuckets**) переносятся в новые постепенно, по мере операций записи. Пока идёт перенос, map держит и старые, и новые бакеты, и при чтении проверяет оба места.

1.7.3. Новая реализация: Swiss Tables (Go 1.24+)

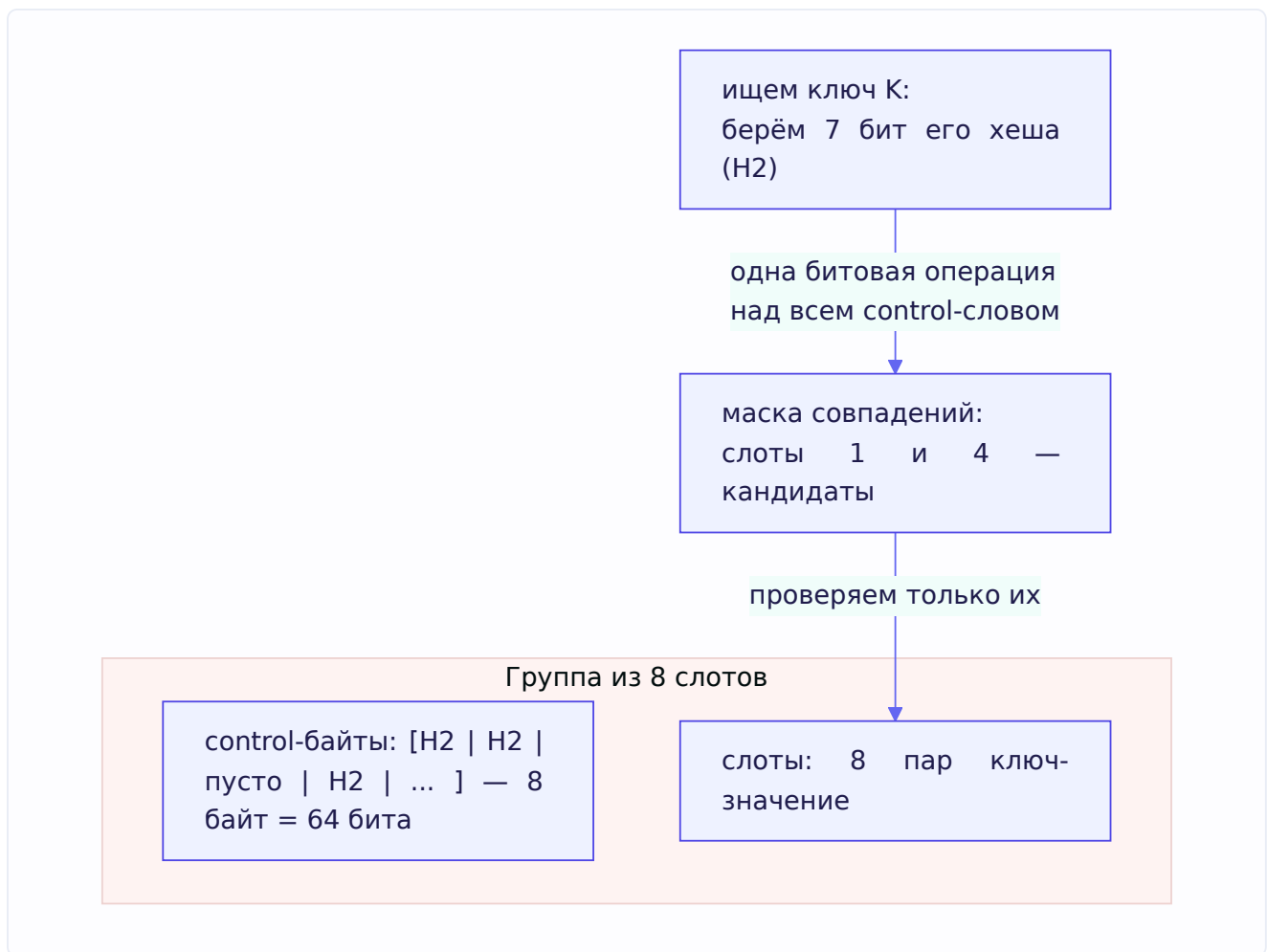
Начиная с Go 1.24, реализация map была заменена на структуру, вдохновлённую **Swiss Tables** (изначально из C++ мира, от Google). Это заметный прирост производительности, и знание об этом — свежий козырь на собеседовании.

Ключевые отличия от старой схемы:

Открытая адресация вместо цепочек. Вместо overflow-бакетов Swiss Tables использует open addressing внутри групп. Нет длинных цепочек — есть группы слотов.

Группы (groups) и control-байты. Данные организованы в **группы по 8 слотов**. Для каждой группы есть массив из 8 **control-байтов** (control word) — по одному байту метаданных на слот. Каждый control-байт хранит: пустой ли слот, удалён ли, или (если занят) — 7 бит хеша ключа.

Параллельный поиск (SIMD-подобный). Вот главная идея. 8 control-байтов группы — это ровно 64 бита, то есть **одно машинное слово**. Благодаря этому можно проверить **все 8 слотов группы одновременно** битовыми операциями над одним 64-битным словом — вместо последовательного перебора. Одной операцией мы получаем маску: в каких слотах группы потенциально лежит наш ключ. Это резко сокращает число сравнений.



Почему быстрее старой реализации:

- Меньше промахов кеша: control-байты компактны, лежат подряд, сканируются одним словом.
- Нет разыменовывания указателей overflow-бакетов (которые раскиданы по памяти).
- Проверка 8 слотов «пачкой» вместо поштучной.

На практике Swiss Tables дают заметное ускорение доступа, особенно для крупных map. Если на собеседовании упомянешь, что с Go 1.24 map перешёл на Swiss Tables с control-байтами и групповым поиском — это сильный сигнал, что ты следишь за языком.

Что говорить на собеседовании: если спрашивают «как устроена map в Go» — начни с общей теории (хеш, коллизии), потом опиши классическую схему (hmap, бакеты по 8, top hash, overflow, инкрементальный resize), и добавь, что с 1.24 реализация перешла на Swiss Tables (группы, control-байты, поиск по всей группе одним словом). Такой ответ покрывает все грейды.

1.7.4. Почему итерация в случайном порядке

Задача из задачника:

```
m := map[string]int{"a": 1, "b": 2, "c": 3}
for k, v := range m {
    fmt.Println(k, v)
}
```

Порядок вывода — **случайный** и меняется от запуска к запуску. Это **сделано намеренно** разработчиками Go.

Зачем? Чтобы разработчики **не полагались** на порядок итерации. Порядок в хеш-таблице определяется расположением по бакетам, которое зависит от хешей и может меняться при resize. Если бы Go давал стабильный порядок в одной версии, люди бы начали на него закладываться, а в следующей версии (или после resize) он бы сломался. Чтобы пресечь это в корне, Go **специально рандомизирует** стартовую точку итерации.

Практический вывод: если нужен предсказуемый порядок — собери ключи в слайс и отсортируй:

```
keys := make([]string, 0, len(m))
for k := range m {
    keys = append(keys, k)
}
sort.Strings(keys)
for _, k := range keys {
    fmt.Println(k, m[k])
}
```

1.7.5. Incomparable types как ключи

Не любой тип может быть ключом map. Ключом может быть только **сравнимый** (comparable) тип — тот, для которого работают операторы `==` и `!=`. Причина понятна из устройства: чтобы разрешить коллизию, map должен сравнивать ключи.

Что **нельзя** использовать как ключ (это incomparable-типы):

- **слайсы** (`[]int`) — не сравнимы;
- **map** — не сравнимы;
- **функции** — не сравнимы;
- **структуры, содержащие несравнимые поля** (например, struct со слайсом внутри);
- **массивы из несравнимых типов** (например, `[3][]int`).

```
m := map[][]int]string{} // ошибка компиляции: invalid map key type []int
```

Что **можно**: числа, строки, булевы, указатели, каналы, интерфейсы, структуры и массивы, состоящие только из сравнимых полей.

Отдельный коварный случай — интерфейс как ключ. Он comparable на этапе компиляции, но если положить в него несравнимое значение (например, слайс), будет **паника в рантайме** при использовании как ключа. Так что «comparable» проверяется компилятором для статических типов, но для интерфейсов финальная проверка — в рантайме.

1.8. Почему map не concurrent-safe

Критически важная тема — и одна из самых частых на собеседовании.

Проблема

Map в Go **не защищён** от одновременного доступа из нескольких горутин. Если одна горутина пишет в map, а другая в это же время читает или пишет — это **race condition**, и Go его активно детектирует.

```
m := make(map[int]int)
go func() { m[1] = 2 }()
go func() { m[1] = 7 }()
go func() { m[1] = 10 }()
time.Sleep(100 * time.Millisecond)
fmt.Println(m[1])
```

Этот код (при `GOMAXPROCS != 1`) упадёт с явной паникой:

```
fatal error: concurrent map writes
```

Обрати внимание: это **не обычная паника**, которую можно поймать через `recover`, а **fatal error** — рантайм намеренно роняет программу целиком. Разработчики Go решили, что конкурентная запись в map — это настолько серьёзный баг, что программу лучше уронить сразу, чем позволить ей работать с повреждённой хеш-таблицей.

Почему так сделано

Почему бы не встроить блокировку прямо в map? Потому что это замедлило бы **все** map, включая те, что используются в одной горутине (а таких большинство). Философия Go: не платить за то, что не используешь. Нужна потокобезопасность — добавь её явно. Не нужна — работай с быстрым незащищённым map.

Решения

Есть несколько подходов, от простого к продвинутому — их полезно перечислить на собеседовании (это прямо просят в задачнике «Ревью кода кеша» и «In-memory cache»):

1. sync.Mutex — обычная блокировка. Оборачиваем map и мьютекс в структуру, лочим на каждой операции:


```

type SafeMap struct {
    mu sync.Mutex
    m map[int]int
}

func (s *SafeMap) Set(k, v int) {
    s.mu.Lock()
    defer s.mu.Unlock()
    s.m[k] = v
}

func (s *SafeMap) Get(k int) (int, bool) {
    s.mu.Lock()
    defer s.mu.Unlock()
    v, ok := s.m[k]
    return v, ok
}

```

Просто и надёжно. Минус — любая операция (даже чтение) блокирует все остальные.

2. sync.RWMutex — отдельные блокировки чтения и записи. Если чтений намного больше, чем записей, `RWMutex` позволяет **множеству читателей** работать одновременно, блокируя всех только на время записи:

```

func (s *SafeMap) Get(k int) (int, bool) {
    s.mu.RLock()           // блокировка на чтение — читателей может быть много
    defer s.mu.RUnlock()
    v, ok := s.m[k]
    return v, ok
}

```

Но есть нюанс, который любят спрашивать: **RWMutex не всегда быстрее**. При очень высокой частоте операций сам механизм RWMutex дороже обычного Mutex (внутри он сложнее), и на коротких критических секциях выигрыша от параллельного чтения может не быть — накладные расходы съедят его. RWMutex оправдан, когда критическая секция достаточно «тяжёлая» и читателей действительно много.

3. Шардирование (sharding). Самое масштабируемое решение. Разбиваем данные на N независимых частей (шардов), у каждого — свой мьютекс. Ключ по хешу попадает в конкретный шард:

```

type ShardedMap struct {
    shards []*SafeMap
}

func (s *ShardedMap) getShard(key string) *SafeMap {
    h := fnv32(key)           // хешируем ключ
    return s.shards[h%uint32(len(s.shards))] // выбираем шард
}

```

Идея: если у нас 16 шардов, то 16 горутин, работающих с разными ключами, скорее всего попадут в разные шарды и **не будут мешать друг другу** — каждая лочит только свой мьютекс.

Это снимает главную проблему единого мьютекса — lock contention (борьбу за один замок).

Сколько шардов делать? Из задачника: **не меньше числа потоков (GOMAXPROCS), а лучше больше и кратно** — чтобы вероятность попадания двух горутин в один шард была мала.

4. sync.Map. Готовая потокобезопасная map из стандартной библиотеки. Оптимизирована под специфический сценарий: **много чтений, мало записей**, и когда ключи пишутся один раз и потом только читаются. Внутри использует два хранилища (read-only часть без блокировок + dirty-часть под мьютексом). Для сценария «читаем часто, пишем редко» — очень быстро. Но для равномерной смеси чтений/записей обычный `Mutex + map` обычно не хуже. `sync.Map` — не универсальная замена, а инструмент под конкретный паттерн.

Хороший ответ на собеседовании про кеш «80% чтений / 20% записей» (это прямо из задачника): начать с `RWMutex` (раз чтений много), упомянуть, что при росте нагрузки `RWMutex` упрётся в contention, и перейти к шардированию как основному решению. Про `sync.Map` сказать, что он хорош при преобладании чтений, но здесь 20% записей — многовато для его идеального сценария.

1.9. Struct, padding и alignment

Тема, которая кажется мелкой, но показывает глубину понимания памяти.

Структура — просто набор полей подряд

Struct в Go — это составной тип, поля которого лежат в памяти **последовательно**, одно за другим. Казалось бы — сложи размеры полей и получишь размер структуры. Но нет: из-за **выравнивания** (alignment) реальный размер может быть больше суммы полей.

Что такое выравнивание

Процессор читает память не побайтно, а «словами» — блоками по 4 или 8 байт. Он работает эффективнее, когда данные лежат по адресам, кратным их размеру: `int64` (8 байт) — по адресам, кратным 8, `int32` — кратным 4, и т.д. Это называется выравниванием.

Чтобы обеспечить выравнивание, компилятор вставляет между полями **пустые байты — padding** (набивку). Из-за этого порядок полей влияет на размер структуры.

Пример: тот же набор полей, разный размер

```
type BadStruct struct {
    a bool // 1 байт
    b int64 // 8 байт
    c bool // 1 байт
}

type GoodStruct struct {
    b int64 // 8 байт
    a bool // 1 байт
    c bool // 1 байт
}
```

Посчитаем размеры (на 64-битной платформе, выравнивание $\text{int64} = 8$):

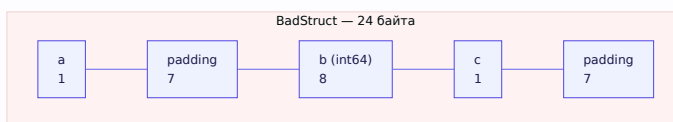
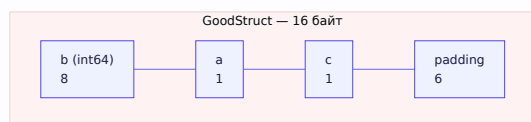
BadStruct:

```
a (bool) — 1 байт
[padding] — 7 байт (чтобы b начался с адреса, кратного 8)
b (int64) — 8 байт
c (bool) — 1 байт
[padding] — 7 байт (чтобы весь размер был кратен 8)
Итого: 24 байта
```

GoodStruct:

```
b (int64) — 8 байт
a (bool) — 1 байт
c (bool) — 1 байт
[padding] — 6 байт (до кратности 8)
Итого: 16 байт
```

Один и тот же набор полей — **24 против 16 байт**, разница в полтора раза, только из-за порядка!



Практический вывод

Правило оптимизации структур: **располагай поля от больших к меньшим** ($\text{int64} \rightarrow \text{int32} \rightarrow \text{int16} \rightarrow \text{bool}$). Тогда padding минимизируется. Для большинства кода это неважно, но когда структур миллионы (например, элементы большого слайса или кеша) — экономия памяти становится ощутимой.

Проверить реальный размер можно через `unsafe.Sizeof(x)`.

Пустая структура struct{}

Особый случай — `struct{}{}` (пустая структура) занимает **0 байт**. Её используют, когда нужен факт наличия, а не значение. Классика — множество (set) на основе map:

```
set := map[string]struct{}{}
set["key"] = struct{}{}           // добавили — значение ничего не весит
_, exists := set["key"]           // проверили наличие
```

`map[string]struct{}` эффективнее `map[string]bool`, потому что значения не занимают памяти вообще. Ты уже видел этот приём в задаче на генерацию уникальных чисел из задачника — там `resMap := make(map[int]struct{})`.

1.10. Type conversion vs type assertion

Завершим главу важным различием, которое часто путают.

Conversion — преобразование типов

Конвертация превращает значение одного типа в **другой конкретный тип**. Синтаксис — `T(value)`:

```
var i int = 65
var f float64 = float64(i) // 65.0
var b byte = byte(i)       // 65
var s string = string(rune(i)) // "A" (символ с кодом 65)
```

Конвертация работает между **совместимыми** типами и происходит на этапе компиляции — компилятор знает оба типа. Она может **потерять данные**, и об этом важно помнить:

```
var big int32 = 300
var small int8 = int8(big) // 300 не влезает в int8 (max 127)!
fmt.Println(small)         // 44 — переполнение, «обрезка» старших битов
```

Здесь `300` в `int8` переполняется и превращается в `44` — компилятор не предупредит, ответственность на тебе. То же с `uint`:

```
var x int = -1
var u uint = uint(x) // огромное число (все биты в единицах), НЕ ошибка
```

Отдельно про строки и числа — популярная ловушка:

```
string(65) // "A" — это НЕ "65"! Конвертирует число в символ по коду Unicode
```

Чтобы получить `"65"` из числа `65`, нужен `strconv.Itoa(65)`, а не `string(65)`. (Современные версии Go предупреждают о `string(int)` через vet, но знать это надо.)

Assertion — извлечение из интерфейса

Type assertion — принципиально другая операция. Она **не преобразует** тип, а **извлекает** конкретное значение из **интерфейса**, проверяя, что там лежит именно ожидаемый тип. Синтаксис — `value.(T)`:

```
var i interface{} = "hello"

s := i.(string) // извлекаем строку из интерфейса
fmt.Println(s)  // "hello"
```

Разница фундаментальна:

- **Conversion** `T(v)` — «преврати это значение в тип T» (работает с конкретными типами, на этапе компиляции).
- **Assertion** `v.(T)` — «внутри этого интерфейса лежит T? дай мне его» (работает только с интерфейсами, проверка в рантайме).

Если в интерфейсе лежит не тот тип, assertion в однозначной форме **паникует**:

```
var i interface{} = "hello"
n := i.(int) // паника: interface conversion: interface {} is string, not int
```

Чтобы не паниковать, есть безопасная форма «comma, ok»:

```
n, ok := i.(int)
if !ok {
    fmt.Println("это не int") // не паникуем, просто ok=false
}
```

Assertion, type switch и вся механика интерфейсов подробно разбираются в главе 4 — там мы вернёмся к этому уже со стороны устройства интерфейсов (iface/eface). Пока главное — **не путать conversion и assertion**: это разные операции с разным синтаксисом (`T(v)` против `v.(T)`) и разным смыслом.

Итоги главы

Мы прошли фундамент, на котором стоит всё остальное. Главное, что стоит унести:

Сквозной принцип: в Go всё передаётся по значению. Слайсы, тар и каналы кажутся «ссылочными» только потому, что внутри их значения лежит указатель — но копируется всё равно значение (сам указатель).

Слайсы — три поля (ptr, len, cap). Почти все slice-задачи решаются, если проследить, когда `append` пишет в общий массив (`len < cap`), а когда создаёт новый (`len == cap`). Данные массива общие, заголовок слайса — копия.

Строки неизменяемы и хранятся в UTF-8; индексация даёт байты, `for range` — руны.

Map — хеш-таблица: общая теория (хеш, коллизии, resize), классическая реализация Go (hmap, бакеты по 8, top hash, overflow-бакеты, инкрементальный resize) и новая на Swiss Tables (группы, control-байты, поиск по всей группе одним словом с 1.24). Map не потокобезопасен — защищаемся Mutex/RWMutex, шардированием или sync.Map под нужный сценарий.

Struct — поля лежат подряд, padding из-за выравнивания влияет на размер; порядок полей от больших к меньшим экономит память.

Conversion vs assertion — `T(v)` превращает конкретный тип (компиляция, может терять данные), `v.(T)` извлекает из интерфейса (рантайм, может паниковать).

В следующей главе спускаемся ещё глубже — в рантайм: stack vs heap, escape analysis, и подробнейший разбор GMP-планировщика.

Глава 2. Runtime и память

Введение к главе

В первой главе мы разобрали, как устроены данные. Теперь спускаемся на уровень ниже — где эти данные живут и кто исполняет наш код. Это территория **рантайма** Go — той части, которая работает вместе с твоей программой и незаметно делает огромную работу: планирует горутины, выделяет и освобождает память, общается с операционной системой.

Тут кроется то, что отличает джуна от джуна+ на собеседовании. Многие знают, что горутины «лёгкие», но не могут объяснить почему. Многие слышали про сборщик мусора, но не понимают, что такое STW. К концу этой главы ты будешь понимать всё это на уровне механики.

Глава делится на три больших пласта: 1. **Память программы** — stack vs heap, escape analysis, аллокатор, сборщик мусора. 2. **Планировщик** — модель GMP, как горутины исполняются на потоках ОС. 3. **OS-уровень** — что под всем этим: страницы памяти, сисколлы, процессы, дескрипторы. Это тот фундамент, на котором стоит рантайм, и его активно спрашивают.

Начнём с памяти.

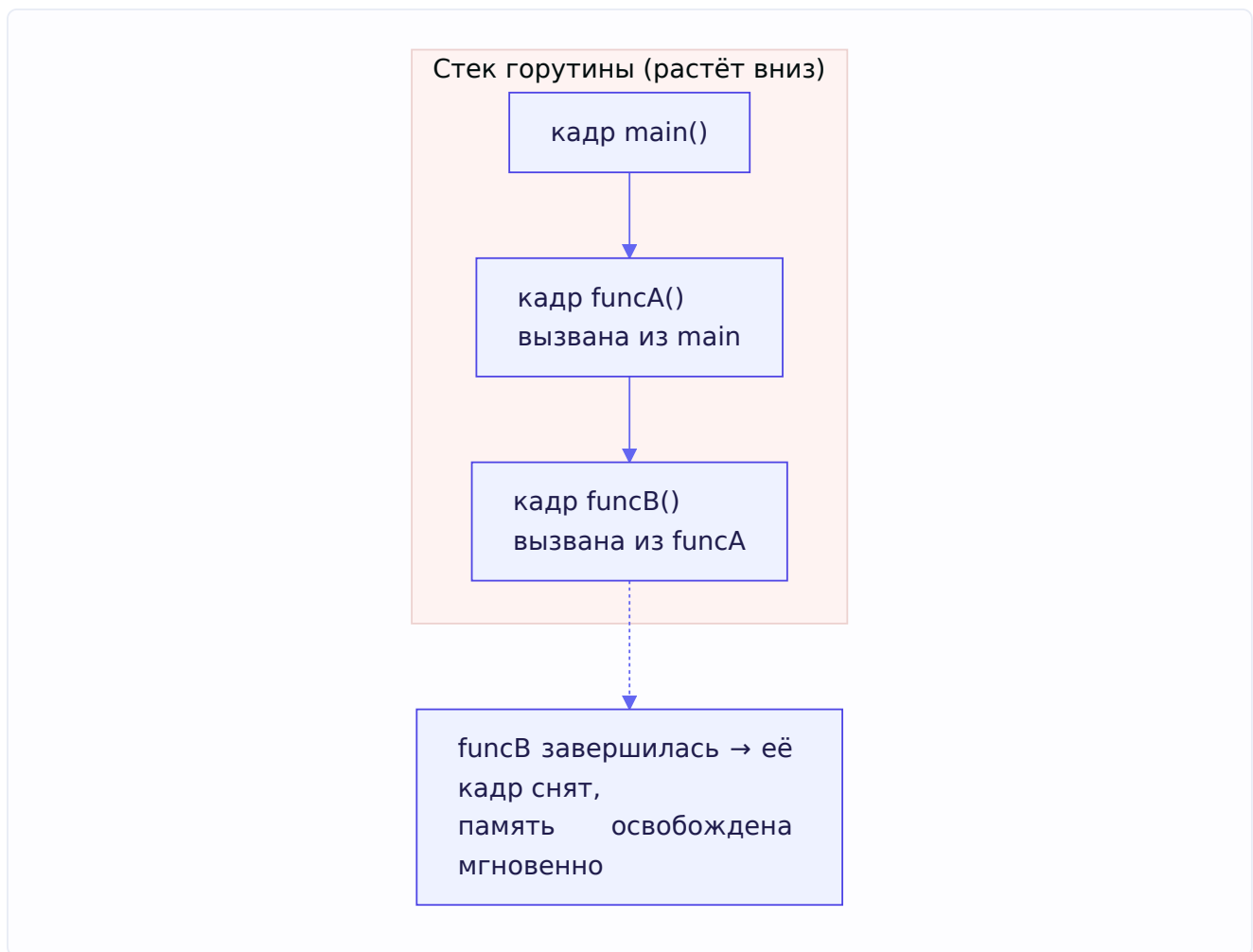
2.1. Stack vs Heap

Чтобы понять escape analysis, сборщик мусора и вообще производительность Go, нужно чётко различать два вида памяти: стек и кучу.

Стек (stack)

Стек — это память, привязанная к вызову функции. Каждый раз, когда вызывается функция, ей выделяется участок стека (**stack frame**, кадр стека), где живут её локальные переменные и аргументы. Когда функция завершается — её кадр просто «снимается» со стека, и память мгновенно освобождается.

Стек работает по принципу LIFO (последним пришёл — первым вышел), как стопка тарелок:



Почему стек **очень быстрый**: - Выделение памяти — это буквально сдвиг одного указателя (указателя вершины стека). Одна инструкция процессора. - Освобождение — сдвиг указателя обратно. Тоже мгновенно. - Не нужен сборщик мусора: память освобождается автоматически при выходе из функции.

Минус: данные на стеке живут только пока живёт функция. Как только она вернулась — данные исчезли.

Куча (heap)

Куча — это область памяти для данных, которые должны жить **дольше**, чем вызов одной функции. Если ты создал объект в функции и вернул на него указатель — объект не может лежать на стеке (кадр-то снимется), он должен переехать в кучу.

Куча гибче, но дороже: - Выделение памяти в куче — сложная операция (нужно найти свободный участок подходящего размера). - Память в куче **не** освобождается автоматически при выходе из функции. За ней следит **сборщик мусора** (GC), который периодически ищет объекты, на которые больше никто не ссылается, и освобождает их. А работа GC — это накладные расходы (об этом в 2.6–2.7).

Ключевой вопрос: где разместить переменную?

Итак, стек — быстро, но недолговечно. Куча — гибко, но дорого (аллокация + нагрузка на GC). Значит, хочется класть на стек всё, что можно, и в кучу — только то, что необходимо.

Кто принимает это решение? В C это делает программист вручную (`malloc` — куча, локальная переменная — стек). В Go это решает **компилятор автоматически** с помощью анализа, который называется **escape analysis**. Ему и посвящён следующий раздел.

Важный нюанс, который любят на собеседовании: в Go ты **не управляешь** напрямую тем, где окажется переменная. Ключевое слово `new` **не** гарантирует кучу, а обычная локальная переменная **не** гарантирует стек. Всё решает escape analysis на этапе компиляции.

2.2. Escape Analysis

Escape analysis (анализ убегания) — это процесс на этапе компиляции, в ходе которого компилятор решает для каждой переменной: может ли она безопасно жить на стеке, или она «убегает» (escapes) в кучу.

Основная идея

Правило простое по формулировке: если компилятор **может доказать**, что переменная не переживёт функцию, в которой создана, — она остаётся на стеке. Если переменная как-то «утекает» за пределы функции (на неё где-то останется ссылка после возврата) — она отправляется в кучу.

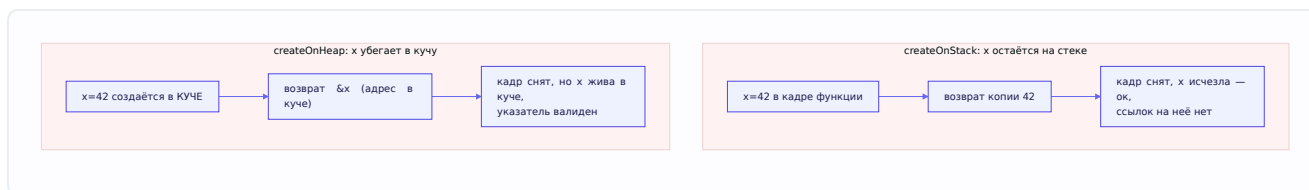
Классический пример убегания

```
func createOnStack() int {
    x := 42 // x не убегает — живёт и умирает внутри функции
    return x // возвращаем КОПИЮ значения, не ссылку
}

func createOnHeap() *int {
    x := 42 // x убегает!
    return &x // возвращаем АДРЕС локальной переменной
}
```

Разберём `createOnHeap` внимательно. Мы создаём `x` внутри функции и возвращаем `&x` — указатель на неё. Но функция сейчас завершится, её кадр стека будет снят. Если бы `x` осталась на стеке — возвращённый указатель показывал бы на освобождённую память (это называется «висячий указатель», dangling pointer, — источник страшных багов в C).

Go этого не допускает. Компилятор видит: «на `x` останется ссылка после возврата функции» → значит, `x` **должна пережить** функцию → размещаем её в куче. Тогда указатель остаётся валидным, а GC освободит `x`, когда на неё перестанут ссылаться.



Как посмотреть решения компилятора

Go позволяет увидеть, что именно убежало, через флаг компиляции:

```
go build -gcflags="-m" main.go
```

Компилятор выведет строки вроде:

```
./main.go:8:2: moved to heap: x  
./main.go:3:2: x does not escape
```

Это полезный навык — уметь проверить, где реально оказываются твои объекты.

Что заставляет переменную убегать

Типичные причины escape (полезно знать для собеседования):

- **Возврат указателя** на локальную переменную (как выше).
- **Сохранение указателя в структуру/слайс/map**, которые живут дольше функции.
- **Передача в интерфейс**. Когда конкретное значение кладётся в интерфейс, оно часто убегает, потому что компилятор не всегда знает, что с ним сделают через интерфейс. Например, `fmt.Println(x)` принимает `interface{}` — и `x` нередко убегает.
- **Замыкания**, захватывающие переменную по ссылке (переменная должна пережить функцию, если замыкание её переживёт).
- **Слишком большой размер** — очень крупные объекты могут размещаться в куче, даже если формально не убегают, потому что не влезают в стек разумного размера.
- **Неизвестный на этапе компиляции размер** — например, слайс, размер которого определяется в рантайме.

Почему это важно на практике

Escape analysis напрямую влияет на производительность. Каждый убегающий объект — это: 1. Более дорогая аллокация (куча вместо сдвига указателя стека). 2. Дополнительная работа для GC потом.

Поэтому в горячем коде (который вызывается миллионы раз) стремятся минимизировать escape: не возвращать без нужды указатели на мелкие структуры, переиспользовать буферы (например, через `sync.Pool`), избегать лишних интерфейсных обёрток. Но — важно — это оптимизация «по месту»: не стоит жертвовать читаемостью кода везде подряд ради экономии, которая заметна лишь в горячих точках. Сначала профилируй (pprof, раздел 2.9), потом оптимизируй.

Тонкость для собеседования: escape analysis объясняет, почему в Go можно спокойно возвращать указатель на локальную переменную (`return &x`) — в отличие от C, где это баг. Компилятор сам позаботится, чтобы переменная переехала в кучу и указатель остался валидным.

2.3. GMP Scheduler — планировщик горутин

Вот она — центральная тема главы и одна из важнейших во всей книге. Если ты поймёшь GMP по-настоящему, ты сможешь объяснить почти любой вопрос про конкурентность в Go. Разберём медленно, по кирпичикам.

Зачем вообще нужен планировщик

Начнём с проблемы. Ты хочешь запустить миллион горутин. Операционная система умеет исполнять код только на **потоках ОС** (OS threads). Но потоки ОС — тяжёлые: каждый занимает мегабайты памяти под стек, а переключение между ними (context switch) дорогое, потому что требует перехода в ядро.

Если бы каждая горутина была отдельным потоком ОС — миллион горутин означал бы миллион потоков, что убило бы систему. Значит, нужен способ исполнять **много горутин на небольшом числе потоков ОС**. Этим и занимается планировщик Go — он **мультиплексирует** (распределяет) горутин по поверх потоков.

Три сущности: G, M, P

Модель называется **GMP** по трём главным сущностям. Разберём каждую.

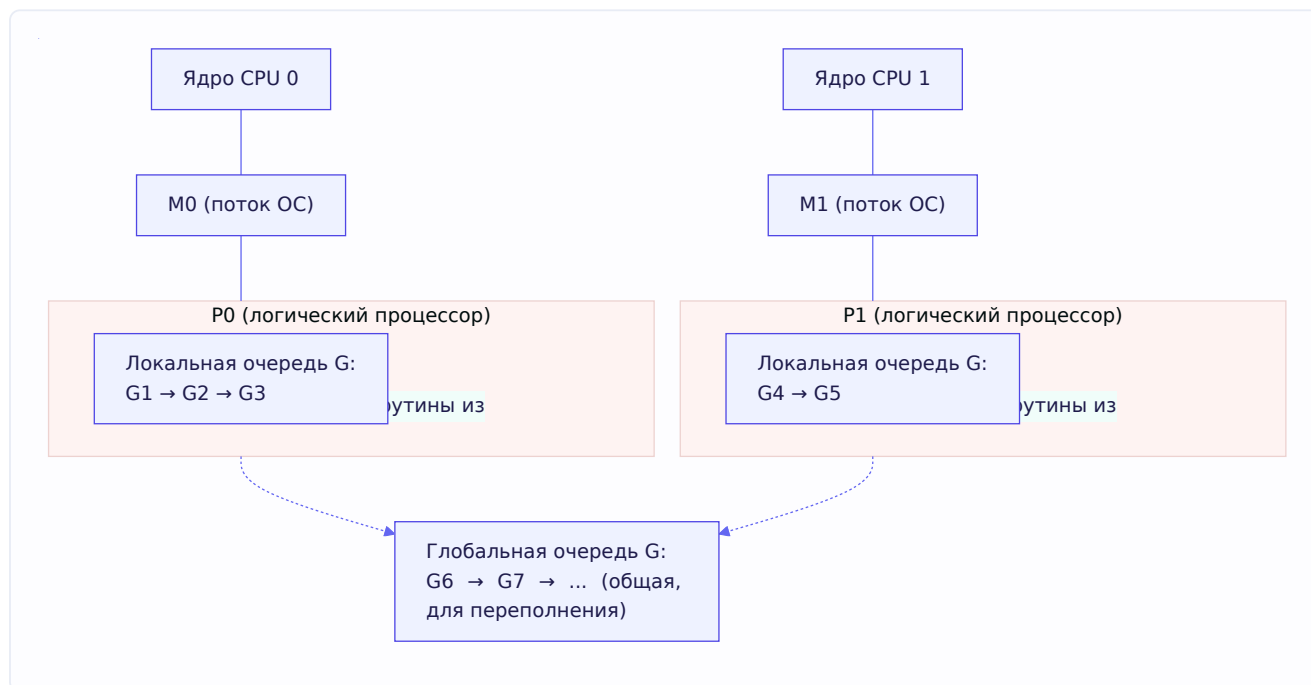
G — Goroutine (горутина). Это твоя горутина — единица работы. Внутри рантайма это структура `g`, которая хранит: стек горутин, счётчик инструкций (где именно она остановилась), состояние. Горутин очень лёгкие — стек стартует с малого размера (несколько килобайт) и растёт по мере надобности. Создать горутину — дёшево, поэтому их можно миллионы.

M — Machine (поток ОС). Это реальный поток операционной системы (OS thread). Только на M может по-настоящему исполняться код — процессор работает именно с потоками ОС. M — это «рабочая лошадь», которая крутит машинные инструкции. M-ов относительно мало (обычно порядка числа ядер, плюс запасные для блокирующих операций).

P — Processor (контекст планирования). Это самая неочевидная сущность. P — это **логический процессор**, посредник между G и M. P хранит **локальную очередь горутин**, готовых к исполнению, и ресурсы для их запуска. Чтобы M мог исполнять горутин, ему нужен P. Число P фиксировано и равно `GOMAXPROCS` (по умолчанию — числу ядер CPU).

Зачем нужен именно P, а не просто «M берёт горутин из общей очереди»? Потому что общая очередь на всех потоков стала бы узким местом — все M дрались бы за один замок при каждом взятии горутин. P даёт каждому «рабочему месту» **свою локальную очередь**, за которую не надо конкурировать. Это ключ к масштабируемости.

Как это выглядит вместе



Читается так: связка **М+Р** **исполняет горутин**. М — это исполнитель (поток), Р — это контекст с очередью работы. М берёт горутину G из локальной очереди своего Р и исполняет её на ядре процессора. Когда одна G отработала своё или уступила — берётся следующая из очереди.

Формула, которую полезно проговорить: «G хочет исполниться, для этого её должен подхватить М, а чтобы М мог работать — ему нужен Р с очередью. Р столько, сколько GOMAXPROCS; М создаётся по необходимости; G — сколько угодно».

Локальные и глобальная очереди

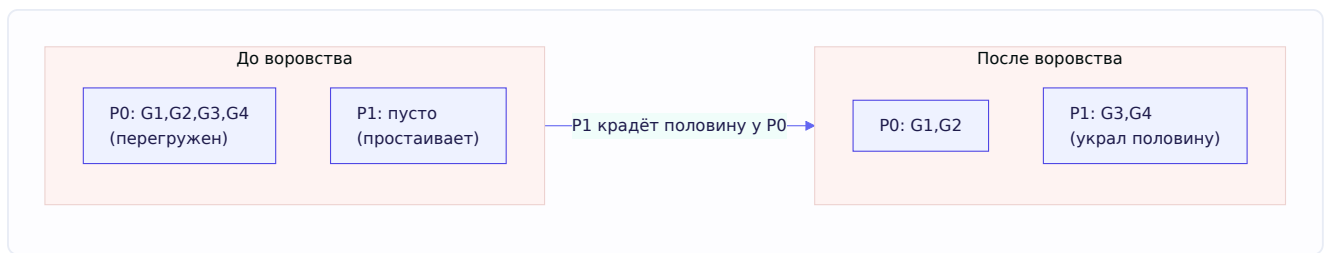
Каждый Р имеет **локальную очередь** горутин (вместимостью 256). Новые горутин обычно попадают в локальную очередь того Р, где их создали. Это быстро — не нужны блокировки, ведь очередь «своя».

Есть также **глобальная очередь** — общая для всех Р. Она используется, когда локальная очередь переполнилась, а также для горутин, которые «вернулись» после долгих операций. Доступ к глобальной очереди требует блокировки, поэтому её используют реже.

Work stealing — воровство работы

Что если у Р0 очередь забита горутинами, а Р1 всё разгрёб и простаивает? Было бы глупо, чтобы Р1 бездельничал, пока Р0 завален. Поэтому в Go есть **work stealing** (воровство работы):

Когда у Р заканчиваются свои горутин, он не засыпает сразу. Сначала он пытается **украсть** половину горутин из локальной очереди другого, случайно выбранного Р. Только если и красть неоткуда (и в глобальной очереди пусто) — Р засыпает.



Это автоматически балансирует нагрузку между процессорами — без участия программиста. Красивое инженерное решение: работа сама «перетекает» к свободным исполнителям.

Самое интересное: что происходит при блокирующем syscall

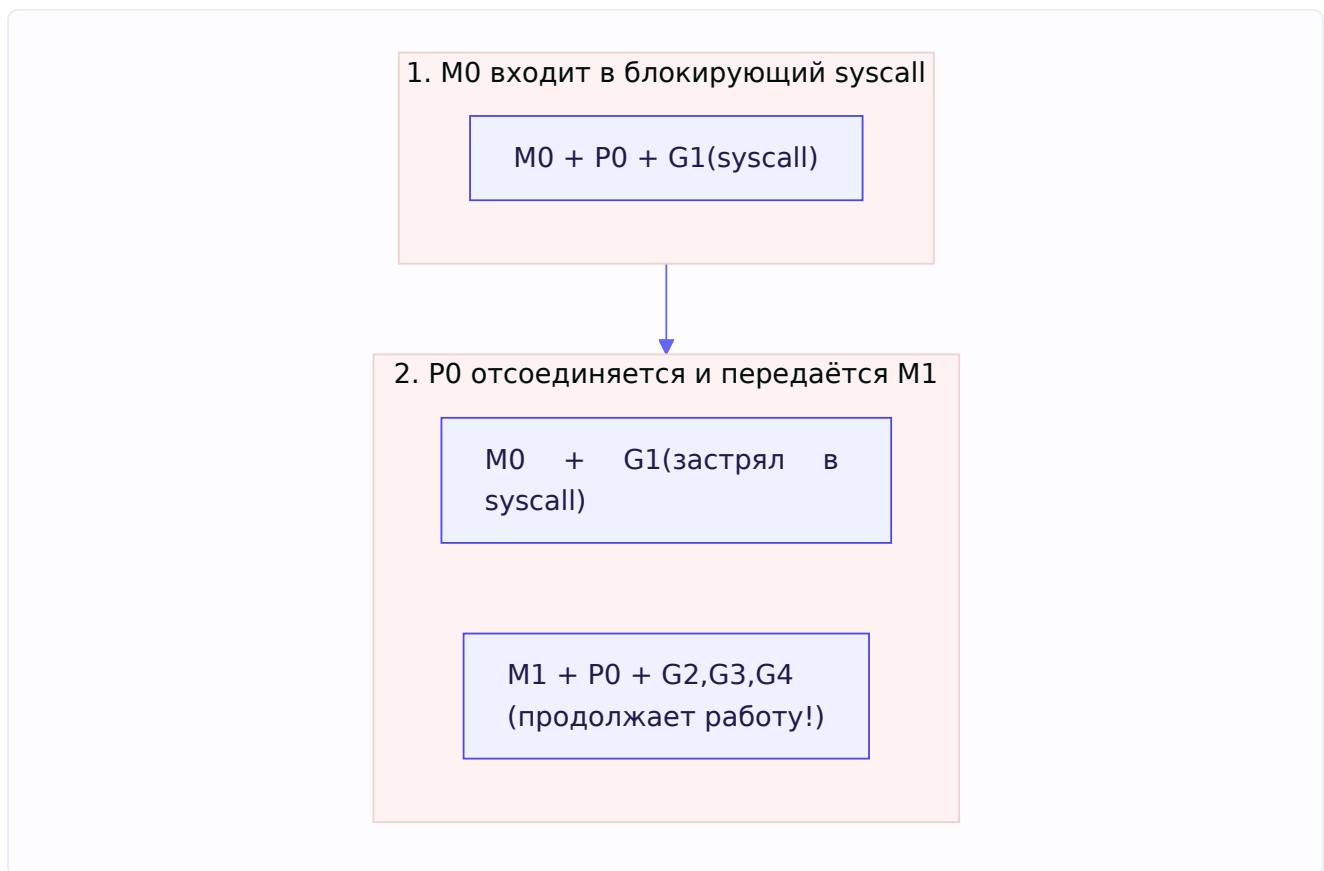
А вот теперь — механизм, который прямо связан с задачей из задачника Озона («может ли программа с GOMAXPROCS=4 потреблять больше 4 ядер?»). Ответ — да, и вот почему.

Представь: горутина G выполняет **блокирующий системный вызов** — например, читает файл с диска или ждёт данные из сети синхронно. Пока идёт syscall, поток M, на котором она работает, **застревает в ядре** — он не может делать ничего другого, он «заблокирован» этим вызовом.

Если бы Go ничего не делал, то P, привязанный к этому M, простаивал бы всё время syscall'a — а на нём висит целая очередь других готовых горутин! Они бы не исполнялись зря.

Поэтому рантайм делает хитрый ход. Когда M входит в блокирующий syscall, происходит **hand-off** (передача) P:

1. M с застрявшей горутинной **отсоединяется** от своего P.
2. P **отдаётся** другому потоку M (свободному или вновь созданному), чтобы тот продолжил исполнять остальные горутинны из очереди.
3. Когда syscall завершается, застрявший M пытается снова заполучить какой-нибудь P, чтобы продолжить свою горутину. Если свободного P нет — горутина отправляется в очередь, а M засыпает про запас.



Вот отсюда и ответ на вопрос задачника: в момент блокирующего syscall Go создаёт **дополнительный поток** М, чтобы Р не простаивал. В эти моменты число активных потоков ОС может **временно превысить GOMAXPROCS**, и программа может кратковременно занимать больше ядер, чем GOMAXPROCS. GOMAXPROCS ограничивает число Р (сколько горутин выполняется *одновременно* в обычном коде), но не жёсткое число потоков ОС.

Netpoller — почему сетевые операции не блокируют поток

Особый случай — **сетевые** операции. Казалось бы, чтение из сокета — блокирующая операция, и по логике выше на каждое такое чтение нужен был бы отдельный застрявший М. При тысячах сетевых соединений это было бы расточительно.

Go решает это через **netpoller** — интегрированный в рантайм механизм на основе эффективных системных примитивов ОС (**epoll** в Linux, **kqueue** в BSD/macOS, IOCP в Windows). Работает так:

Когда горутина хочет читать из сокета, а данных ещё нет, она **не** блокирует поток М. Вместо этого горутина «паркуется» (переводится в ожидание), а М освобождается и берёт другую работу. Netpoller запоминает: «эту горутину надо разбудить, когда в сокете появятся данные». Когда данные приходят (ОС уведомляет через epoll), netpoller **будит** горутину и возвращает её в очередь на исполнение.

Результат: тысячи сетевых горутин могут «ждать» данные, занимая при этом **ноль** потоков ОС. Именно поэтому Go так хорош для сетевых серверов — можно держать десятки тысяч соединений на горстке потоков.

Связь с задачиком: в разделе про сисколлы упоминается **epoll** — «позволяет асинхронно получать события о том, что из дескриптора можно вычитать кусок информации». Netpoller Go

построен именно на этом.

Preemption — как горутину прерывают

Ещё один важный вопрос: что если горутина заиклилась и не отдаёт управление? Не заблокирует ли она навечно свой P?

Тут история эволюционировала:

До Go 1.14 — кооперативная вытеснение (cooperative preemption). Планировщик мог «отобрать» управление у горутины только в определённых точках — при вызове функций (в пролог функции была встроена проверка «а не пора ли уступить?»). Проблема: если горутина крутит тесный цикл без вызовов функций (например, тяжёлое вычисление в цикле `for`), точек для вытеснения нет — и такая горутина могла заблокировать P надолго, не давая другим исполняться.

С Go 1.14 — асинхронная вытеснение (asynchronous preemption). Планировщик умеет прерывать горутину **в любой момент**, посылая потоку сигнал ОС (`SIGURG`). Обработчик сигнала останавливает горутину и передаёт управление планировщику. Теперь даже тесный цикл без вызовов функций можно вытеснить. Это устранило целый класс проблем с «застрявшими» горутинками.

GOMAXPROCS — итог

Соберём про `GOMAXPROCS` всё вместе (это прямой вопрос из задачника):

- `GOMAXPROCS` = число **P**, то есть максимальное число горутин, исполняющихся **одновременно** (параллельно) в обычном Go-коде.
- По умолчанию равно числу ядер CPU (с учётом лимитов контейнера в свежих версиях Go).
- Можно задать через переменную окружения `GOMAXPROCS` или вызовом `runtime.GOMAXPROCS(n)`.
- **Не** ограничивает жёстко число потоков ОС: при блокирующих syscall'ах рантайм создаёт дополнительные M, и потребление CPU может кратковременно превысить GOMAXPROCS.

Как отвечать на собеседовании «что такое GOMAXPROCS»: «Это число логических процессоров P — сколько горутин могут исполняться параллельно. По умолчанию — число ядер. При этом реальное число потоков ОС может быть больше, потому что на блокирующие syscall'ах рантайм заводит дополнительные потоки — поэтому программа с GOMAXPROCS=4 в моменты блокирующих вызовов может занимать больше 4 ядер».

2.4. Горутины под капотом

Мы уже знаем, что горутина — это сущность G в модели GMP. Теперь разберём детальнее, из чего она состоит и почему такая лёгкая.

Структура горутины

Внутри рантайма горутина — это структура **g**. Она хранит всё, что нужно, чтобы приостановить горутину и потом продолжить с того же места:

- **Стек** горутины (её собственная память под локальные переменные).
- **Указатель инструкции** — на каком месте кода горутина сейчас находится (чтобы возобновить после паузы).
- **Состояние** — выполняется, ждёт, готова к запуску и т.д.
- Служебные поля — ссылка на связанный М, информация для планировщика, для сборщика мусора.

Когда планировщик приостанавливает горутину (например, она уступила управление или заблокировалась на канале), он сохраняет её регистры и указатель инструкции в структуру **g**. Когда позже горутину возобновляют — восстанавливают это состояние, и она продолжает как ни в чём не бывало.

Растущий стек — вот в чём лёгкость

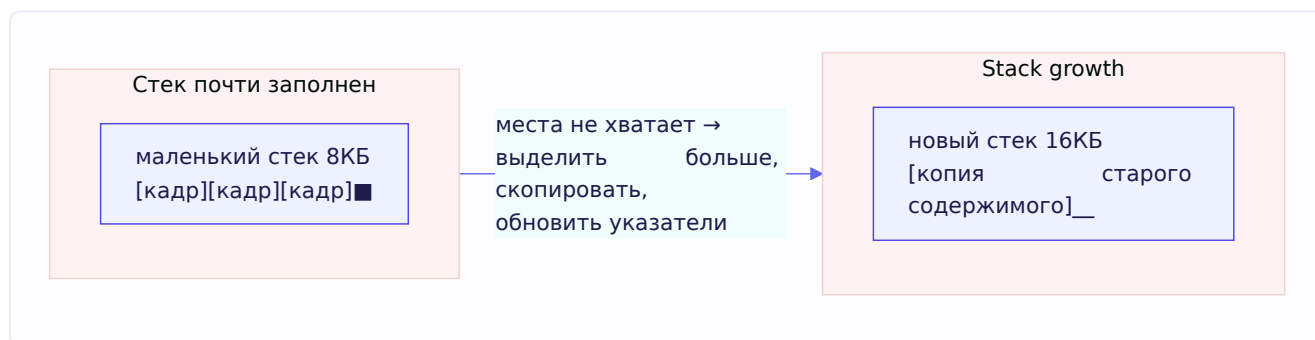
Главный секрет лёгкости горутин — в устройстве их стека.

Вспомним поток ОС: его стек **фиксирован и велик** — обычно 1–8 МБ, выделяется сразу при создании потока. Миллион потоков = терабайты только под стеки. Невозможно.

Стек горутины устроен иначе. Он **начинается с очень маленького размера** (исторически 8 КБ, сейчас порядка 2–8 КБ) и **растёт по мере необходимости**. Как это работает:

При каждом вызове функции проверяется, хватит ли места на стеке горутины для её кадра. Если стека вот-вот не хватит — рантайм делает **stack growth**: 1. Выделяет **новый, больший** участок памяти под стек (примерно вдвое больше). 2. **Копирует** содержимое старого стека в новый. 3. Аккуратно **обновляет все указатели**, которые ссылались на старый стек, чтобы они показывали на новое место. 4. Освобождает старый стек.

Стек может расти вплоть до предела (по умолчанию до 1 ГБ на 64-битных системах, до 250 МБ на 32-битных). А когда стек становится сильно недозагруженным, рантайм может его и **уменьшить** (stack shrink) во время сборки мусора.



Вот почему горутин можно создавать сотнями тысяч: каждая стартует с пары килобайт, а не с мегабайтов. Память под стек берётся ровно по потребности.

Сколько можно создать горутин

Из задачника: **количество горутин не ограничено** искусственно — можно создать столько, на сколько хватит памяти. Если каждая горутина в среднем занимает несколько килобайт, то на машине с несколькими гигабайтами оперативной памяти реально держать сотни тысяч и даже миллионы горутин.

2.5. Горутины vs OS threads

Теперь можно чётко сформулировать разницу — это частый вопрос собеседования (в задачнике он идёт отдельным пунктом). Соберём всё в сравнение.

Характеристика	Горутина	Поток ОС
Кто управляет	Планировщик Go (в user space)	Планировщик ОС (в kernel space)
Начальный стек	Малый (~2-8 КБ), растёт динамически	Большой (1-8 МБ), фиксирован
Стоимость создания	Очень дешево	Дорого (системный вызов в ядро)
Стоимость переключения	Дешево (без перехода в ядро)	Дорого (context switch через ядро)
Сколько можно создать	Сотни тысяч / миллионы	Тысячи (упрёшься в память)
Идентификатор	Нет публичного ID (намеренно)	Есть thread ID

Почему переключение горутин дешевле

Ключевая причина — **где** происходит планирование.

Переключение потоков ОС требует перехода из user space в kernel space (это дорогая операция — про user/kernel space подробно в 2.12) и обратно, плюс ОС сохраняет/восстанавливает полный контекст потока. Это сотни-тысячи тактов процессора.

Переключение горутин планировщик Go делает **сам, в user space**, без обращения к ядру. Ему нужно сохранить меньше состояния, и не надо платить за переход в ядро. Поэтому «переключить» горутину в разы дешевле, чем поток.

Отсюда вывод из задачника: горутины дают **меньше переключений контекста на уровне ОС**. Планировщик Go старается держать горутины на одних и тех же потоках М и переключать их внутри user space, минимизируя дорогие переходы в ядро.

Модель M:N

Формально это называют планированием **M:N**: М горутин мультиплексируются поверх N потоков ОС (где $N \approx$ числу ядер). В отличие от модели 1:1 (одна горутина = один поток, как в некоторых языках) это позволяет иметь огромное число «потоков исполнения» при малом числе реальных потоков ОС.

2.6. Garbage Collector — сборщик мусора

Переходим к сборщику мусора (GC). Это то, что освобождает тебя от ручного управления памятью — но важно понимать, как он работает, потому что он влияет на производительность, и его любят спрашивать.

Зачем нужен GC

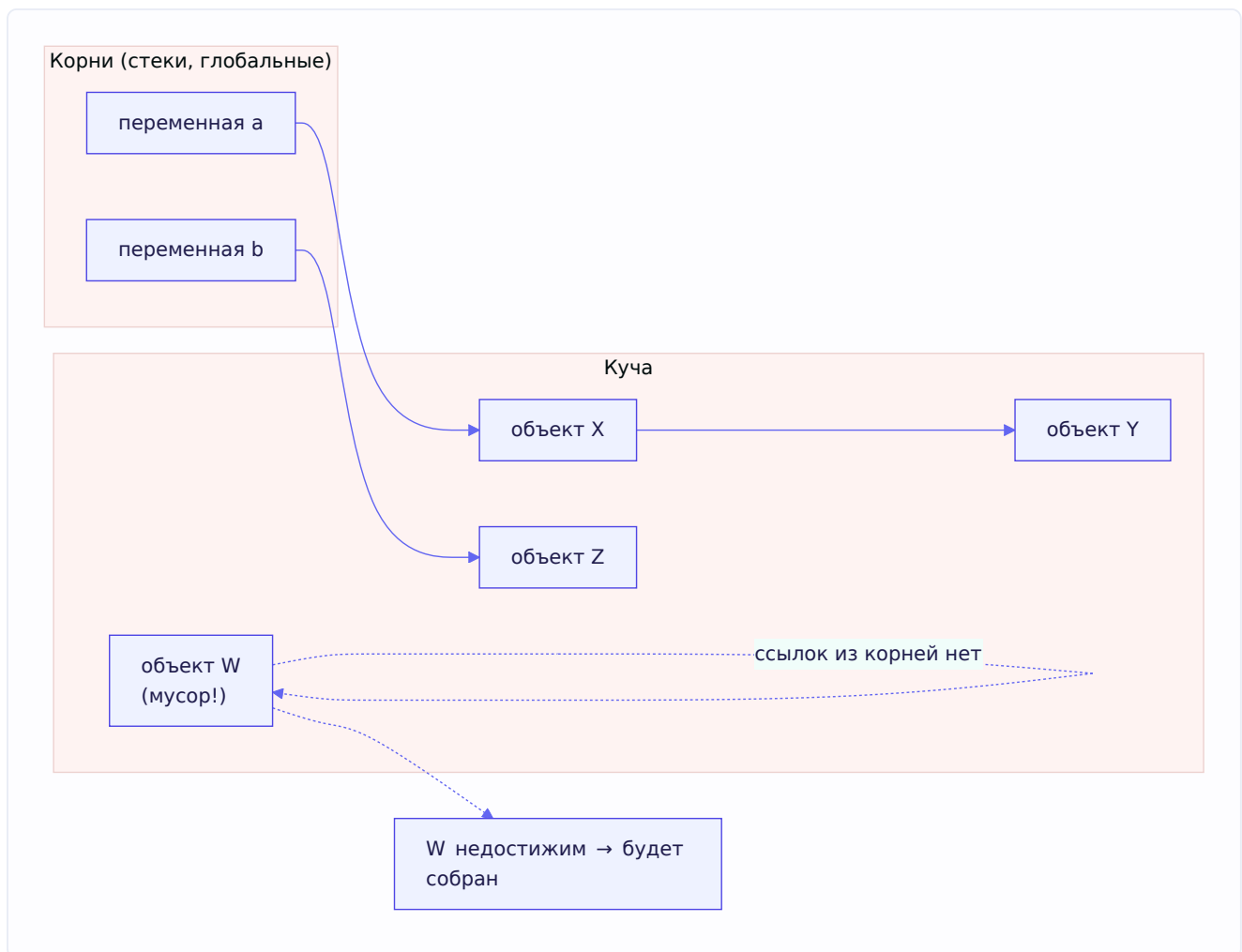
Вспомним: объекты, которые убежали в кучу (раздел 2.2), не освобождаются автоматически при выходе из функции. Кто-то должен определить, что объект больше не нужен, и вернуть его память. В C это делает программист вручную (`free`) — и ошибается: забыл освободить → утечка памяти; освободил дважды или рано → краш.

Сборщик мусора делает это автоматически. Он периодически определяет, какие объекты в куче **достижимы** (на них есть ссылки из работающей программы), а какие — **мусор** (недостижимы, на них никто не ссылается), и освобождает мусор.

Что значит «достижимый»

GC начинает с **корней** (roots) — заведомо живых ссылок. Корни — это: - Локальные переменные на стеках всех горутин. - Глобальные переменные. - Регистры процессора.

От корней GC идёт по ссылкам: если объект достижим из корня (напрямую или через цепочку указателей) — он живой. Всё, до чего дотянуться нельзя, — мусор.



Какой GC в Go

GC в Go — **конкурентный, трёхцветный, с mark-and-sweep**. Разберём эти слова:

- **Mark-and-sweep** (пометить и подмести) — базовый алгоритм: сначала фаза **mark** (помечаем все достижимые объекты), затем фаза **sweep** (освобождаем всё непомеченное).
- **Конкурентный** (concurrent) — GC работает **одновременно** с твоей программой, не останавливая её надолго. Это критично для отзывчивости.
- **Трёхцветный** (tri-color) — конкретный способ пометки, который позволяет работать конкурентно. Разберём отдельно в 2.7, это важно.

Ключевая цель GC в Go — **низкие паузы**. Разработчики сознательно жертвуют некоторой пропускной способностью ради того, чтобы программа не «замирала» надолго. Целевая пауза — доли миллисекунды.

2.7. STW, tri-color marking, write barrier

Это самая техническая часть про GC — и самая ценная на собеседовании. Разберём медленно.

STW — Stop The World

STW (Stop The World) — это момент, когда рантайм **останавливает все горютины**, чтобы GC мог что-то сделать без помех. Во время STW твоя программа буквально стоит — ничего не выполняется.

Очевидно, что долгие STW-паузы — это плохо (программа «фризится»). Вся эволюция GC в Go — это борьба за сокращение STW. В современном Go STW-паузы очень короткие (обычно доли миллисекунды), потому что основную работу GC делает **конкурентно**, а STW использует лишь для коротких критичных операций (например, чтобы согласованно включить/выключить фазы).

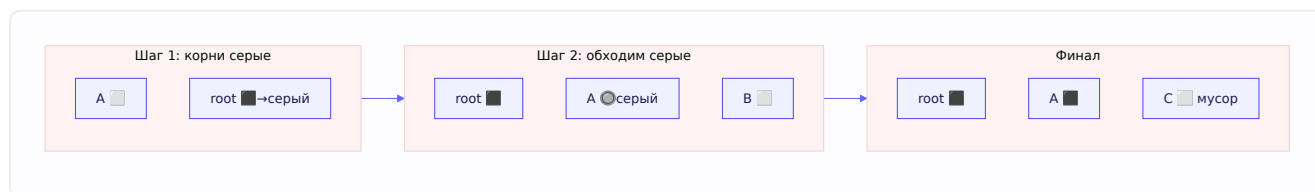
Трёхцветный алгоритм (tri-color marking)

Как пометить живые объекты, **не останавливая программу**? Ведь программа во время пометки меняет ссылки — это как считать людей в здании, из которого они продолжают входить и выходить. Решение — трёхцветная схема. Каждый объект окрашивается в один из трёх цветов:

- **Белый** — ещё не обработан. В начале все объекты белые. В конце все оставшиеся белые = мусор.
- **Серый** — объект достижим (помечен как живой), но его ссылки **ещё не просмотрены**. Это «фронт работы» — очередь на обработку.
- **Чёрный** — объект достижим И все его ссылки уже просмотрены. Полностью обработан.

Алгоритм:

1. **Старт:** все объекты белые. Корни красим в серый.
2. **Шаг:** берём любой серый объект. Смотрим все объекты, на которые он ссылается, — красим их из белого в серый. Сам объект, закончив с его ссылками, красим в чёрный.
3. **Повторяем**, пока есть серые объекты.
4. **Финал:** серых не осталось. Все чёрные — живые. Все оставшиеся белые — мусор, их память освобождается.



(Легенда:  белый — не обработан,  серый — в очереди,  чёрный — готов.)

Прелесть в том, что серые объекты образуют «границу» между обработанным (чёрным) и необработанным (белым). Пока есть серые — работа не закончена. Это позволяет **прерываться и возобновляться**, работая параллельно с программой.

Проблема: программа меняет ссылки во время пометки

Но есть коварная проблема. GC работает конкурентно — программа в это время **продолжает менять указатели**. Представь опасный сценарий:

1. GC пометил объект A чёрным (полностью обработал, больше не вернётся к нему).
2. Программа создаёт новую ссылку: A теперь указывает на белый объект C.
3. Программа удаляет все другие ссылки на C — теперь C достижим **только** через чёрный A.
4. Но GC к чёрному A больше не вернётся! Он не увидит новую ссылку A→C.
5. C останется белым → GC ошибочно сочтёт его мусором и **освободит живой объект**.
Катастрофа.

Это нарушение так называемого **трёхцветного инварианта**: «чёрный объект не должен ссылаться напрямую на белый». Как его защитить?

Write barrier — барьер записи

Write barrier (барьер записи) — это решение. Это небольшой код, который рантайм **вставляет вокруг операций записи указателей** во время работы GC. Когда программа записывает указатель (меняет ссылку), барьер это перехватывает и информирует GC о новой связи.

Упрощённо: когда чёрный объект начинает ссылаться на белый, барьер **перекрашивает** нужный объект (например, красит белый в серый), чтобы GC его точно обработал и не потерял. Инвариант восстанавливается, живой объект не теряется.



Барьер записи включается только на время цикла GC (в остальное время его нет, чтобы не платить за него постоянно). Это компромисс: небольшая цена на каждую запись указателя во время GC — ради возможности работать конкурентно без потери объектов.

Как выглядит полный цикл GC

Соберём вместе:

1. **STW (короткая)**: включить барьеры записи, подготовиться.
2. **Mark (конкурентно)**: пометить живые объекты трёхцветным алгоритмом, параллельно с работой программы; барьеры записи ловят изменения ссылок.
3. **STW (короткая)**: завершить пометку, согласовать состояние.
4. **Sweep (конкурентно)**: освободить память белых объектов, тоже параллельно с программой.

Две STW-паузы — очень короткие. Основная работа (mark и sweep) идёт конкурентно. Именно так Go достигает низких пауз.

Как запускается GC

GC не работает непрерывно — он запускается периодически. Главный триггер — **рост кучи**. Есть параметр `GOGC` (по умолчанию 100), который означает: запускать GC, когда куча выросла на 100% (вдвое) относительно размера после прошлой сборки. Увеличив `GOGC`, ты делаешь сборки более редкими (меньше нагрузка на CPU, но больше памяти); уменьшив — наоборот. Есть и предохранитель по времени (GC запустится хотя бы раз в 2 минуты, даже если куча не выросла).

Как рассказывать про GC на собеседовании: «В Go конкурентный трёхцветный mark-and-sweep сборщик, оптимизированный под низкие паузы. Он помечает живые объекты тремя цветами — белый/серый/чёрный, работая параллельно с программой. Чтобы не потерять объекты, чьи ссылки меняются во время пометки, используется write barrier — барьер записи, поддерживающий трёхцветный инвариант. STW-паузы очень короткие, основная работа идёт конкурентно».

2.8. Memory allocator — mcache, mcentral, mheap

Последний кусок про память рантайма — как вообще выделяется память в куче. Ты просил глубину, так что разберём и это, хоть спрашивают реже.

Проблема, которую решает аллокатор

Когда горутине нужна память (объект убежал в кучу), кто-то должен её выдать. Простейший вариант — при каждой аллокации звать системный вызов к ОС и просить памяти. Но это **очень медленно** (сисколл дорогой, см. 2.11) и вызывало бы дикую конкуренцию между горутинами за общий ресурс.

Аллокатор Go (он вдохновлён аллокатором TCMalloc от Google) решает это **иерархией кешей**, чтобы большинство аллокаций происходило **без блокировок и без сисколлов**.

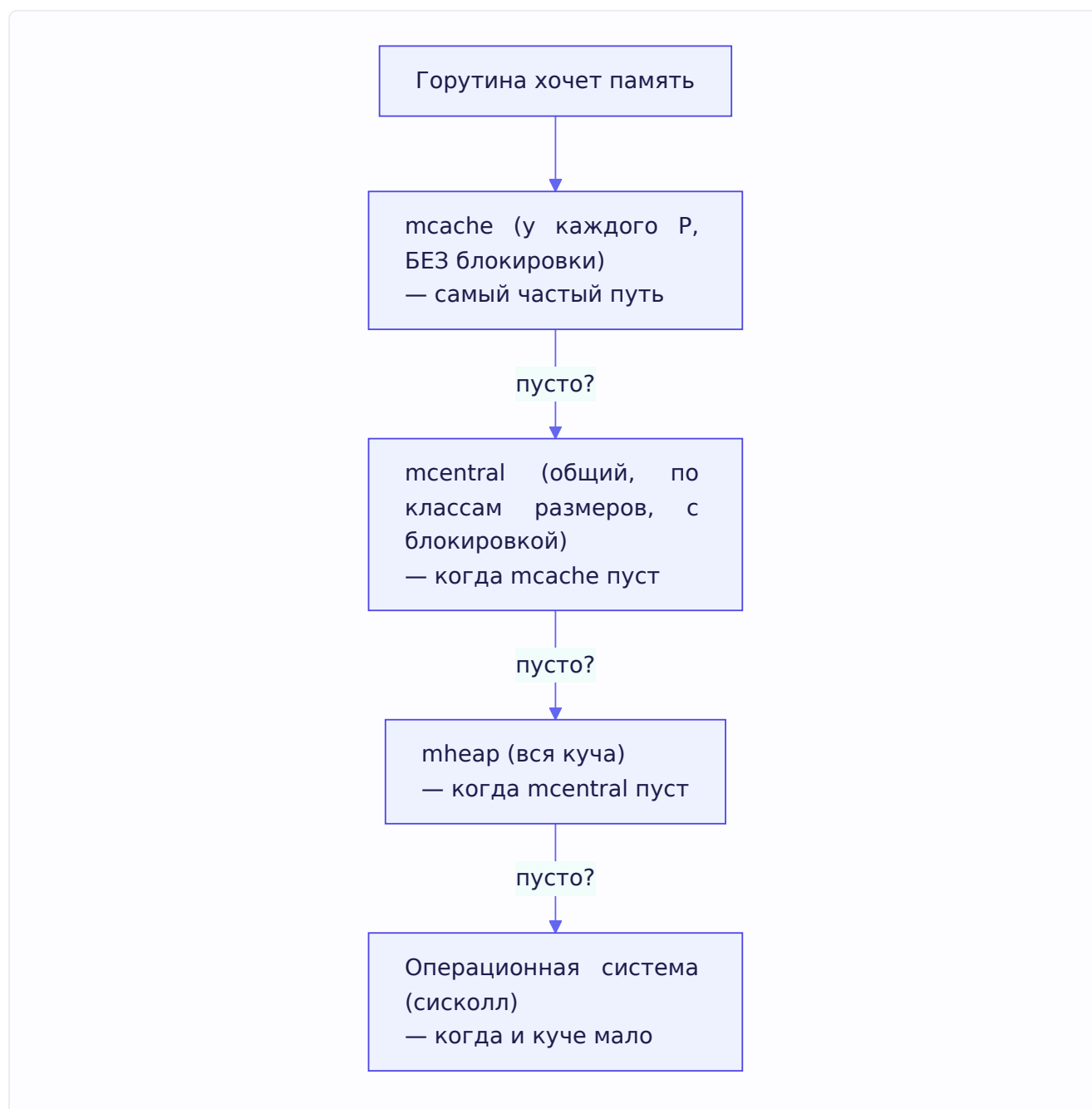
Три уровня

mcache — кеш каждого P. У **каждого P** есть свой **mcache** — локальный кеш свободных блоков памяти. Когда горутине нужна память под небольшой объект, она берёт её из mcache **своего P**.

А раз `msache` привязан к `P` и на нём в каждый момент работает одна горутина — **блокировка не нужна**. Это самый быстрый путь, по которому идёт большинство аллокаций.

`msentral` — центральный склад по размерам. Когда в `msache` закончились блоки нужного размера, он идёт за новой порцией в `msentral`. `msentral` — общий на все `P` склад блоков **конкретного класса размера**. Доступ к нему уже требует блокировки, но обращаются к нему относительно редко (только когда локальный кеш опустел).

`mheap` — вся куча. Если и в `msentral` нет памяти, запрос идёт в `mheap` — это представление всей кучи целиком. `mheap` управляет памятью крупными участками (страницами) и, если и ему не хватает, **запрашивает память у операционной системы** через `сисколл`. Это самый редкий и самый дорогой путь.



Классы размеров (size classes)

Аллокатор не выделяет память байт-в-байт под запрос. Вместо этого есть заранее заданные **классы размеров** (например, 8, 16, 32, 48... байт). Запрос округляется вверх до ближайшего класса. Это упрощает переиспользование освобождённых блоков (блок из-под объекта в 30 байт подойдёт под любой новый объект класса 32) и снижает фрагментацию.

Объекты делят на категории: **tiny** (совсем мелкие, до 16 байт — их упаковывают вместе), **small** (до 32 КБ — идут по классам размеров через mcache), **large** (больше 32 КБ — выделяются напрямую из mheap).

Зачем это знать

Практический смысл: аллокации в Go **дешёвы в типичном случае** (mcache без блокировок), но не бесплатны. Массовые аллокации мелких объектов в горячем коде всё равно нагружают систему и, что важнее, создают работу для GC. Поэтому приёмы вроде переиспользования буферов (`sync.Pool`), предварительного выделения слайсов с нужным `cap` (помнишь из главы 1?) реально снижают нагрузку — они уменьшают число аллокаций и, как следствие, давление на GC.

2.9. Профилирование — pprof

Мы много говорили о производительности: escape analysis, аллокации, GC. Но как понять, что тормозит в **конкретной** программе? Гадать бесполезно — нужно измерять. Инструмент для этого в Go — **pprof**, и вопрос про него есть в задачнике (грейды с 17 по 21, то есть спрашивают на всех уровнях).

Что такое pprof

pprof — это встроенный в Go инструмент профилирования. Он собирает данные о том, на что программа тратит ресурсы, и позволяет их визуализировать. Основные виды профилей:

- **CPU-профиль** — где программа проводит процессорное время (какие функции «горячие»).
- **Heap/alloc-профиль** — что и где выделяет память в куче.
- **Mutex-профиль** — где горутины конкурируют за мьютексы (contention).
- **Block-профиль** — где горутины блокируются (на каналах, мьютексах).
- **Goroutine-профиль** — сколько горутин и где они «застряли».

Семплирующий профайлер — ключевая идея

Вот что важно понять (это прямо спрашивают на 20 грейде): pprof — **семплирующий** (sampling) профайлер. Что это значит?

Профайлер сохраняет **не каждое** событие, а **каждое N-ное** — берёт «пробы» (семплы) через определённые интервалы. Для CPU-профайлера это работает так: примерно 100 раз в секунду срабатывает таймер, и профайлер записывает **стекарейс** — какие функции сейчас выполняются на каждом потоке. Собрав тысячи таких снимков, можно статистически

восстановить, где программа проводит время: если функция попадалась в 30% снимков — значит, она съедает примерно 30% CPU.

Почему семплирование, а не запись каждого события? Потому что записывать **каждый** вызов функции — колоссальный overhead, он исказил бы саму программу (это как измерять скорость машины, посадив в неё десять наблюдателей). Семплирование даёт достаточно точную картину при минимальном влиянии.

Почему CPU-профиль требует времени, а heap — мгновенный

Тонкий вопрос из задачника. Разница вытекает из природы данных:

- **Heap/alloc-профиль получается мгновенно.** Данные об аллокациях собираются **постоянно** в фоне (это часть работы аллокатора). Когда ты запрашиваешь heap-профиль, рантайм просто отдаёт уже накопленный снимок текущего состояния кучи. Ждать нечего.
- **CPU-профиль требует подождать.** Чтобы понять, где тратится процессорное время, нужно **набрать статистику семплов** за какой-то период. Если снять CPU-профиль мгновенно — данных не будет, ведь семплы набираются во времени. Поэтому CPU-профилирование запускают на несколько секунд, в течение которых собираются стектрейсы.

Overhead профилирования

Ещё из задачника: - Для профилей **heap, alloc, mutex, block** overhead **минимальный** — эти данные и так собираются рантаймом почти всегда. - **CPU** собирается только по требованию и **добавляет overhead** (нужно регулярно прерываться и писать стектрейсы). Насколько — зависит от программы, надо мерить.

Как снимать профили

Два способа: 1. **Программный API** — пакет `runtime/pprof`, ручной запуск/остановка профилирования из кода. 2. **HTTP-эндпоинт** — подключить пакет `net/http/pprof`, и профили станут доступны по HTTP (`/debug/pprof/...`). Очень удобно для живых сервисов: зашёл по URL — снял профиль с работающего сервера.

Граф и флеймграф

Собранный профиль визуализируют двумя основными способами:

- **Граф вызовов** — узлы это функции, стрелки — кто кого вызывает, размер/цвет узла — сколько ресурса он потребляет. Помогает увидеть общую структуру.
- **Флеймграф (flame graph)** — горизонтальные полосы, где ширина полосы = доля ресурса. Стек вызовов идёт снизу вверх. Широкая полоса = «горячее» место. Флеймграф читается быстро: ищешь самые широкие полосы — это и есть узкие места.

trace — отдельный, более подробный инструмент

Ещё из задачника: `trace` отличается от профилей. Если профили дают **статистическую** картину («где примерно тратится время»), то `trace` даёт **подробную хронологию событий**

выполнения: - создание, блокировка, разблокировка горутин; - системные вызовы; - работа сборщика мусора; - размер кучи, число потоков и горутин во времени.

За эту детальность платят: `trace` добавляет **больше overhead**, чем обычные профили, и генерирует много данных. Его используют, когда нужно разобраться в тонком поведении планировщика и конкурентности, а не просто найти горячую функцию.

2.10. Мониторинг микросервисов

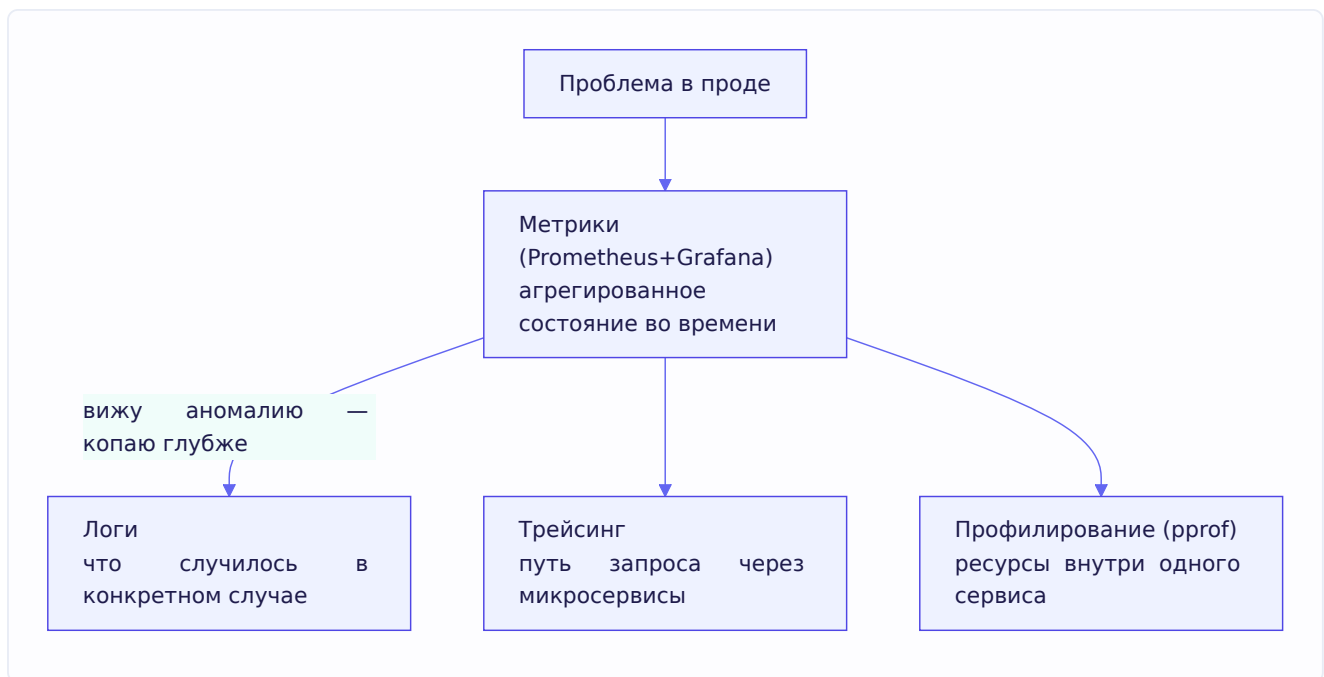
Профилирование отвечает на вопрос «что тормозит в этом процессе прямо сейчас». Но в проде нужен ещё и постоянный **мониторинг** — наблюдение за здоровьем сервисов во времени. Это тоже вопрос из задачника. Разложим по полочкам: есть **четыре разных инструмента наблюдения**, и важно не путать их назначение.

1. Логи (logs). Текстовые записи о событиях: что произошло, ошибки, важные шаги. Логи хороши, когда нужно разобраться в **конкретном** случае — «почему упал вот этот запрос». В Kubernetes логи пода смотрят через `kubectl logs`.

2. Метрики (metrics). Числовые показатели во времени: нагрузка, RPS, время ответа, использование памяти/CPU, число ошибок. Метрики дают **агрегированную** картину состояния сервиса. Стандартный стек: **Prometheus** собирает метрики, **Grafana** рисует по ним графики и дашборды. Именно метрики — основа алертинга (автоматических уведомлений «что-то пошло не так»).

3. Трейсинг (tracing). Прослеживание пути **одного запроса** через **много** микросервисов. В распределённой системе запрос проходит через десяток сервисов — трейсинг показывает эту цепочку и где именно потерялось время. Незаменим для поиска узких мест в микросервисной архитектуре.

4. Профилирование (profiling). То, что мы разобрали в 2.9 — детальный разбор потребления ресурсов **внутри** сервиса (CPU, память). Через `pprof`.



Правильная стратегия: метрики показывают, **что** сервису плохо (растёт latency, память), а логи/трейсинг/профилирование помогают понять, **почему**. Отдельно из задачника: эндпоинт `/debug/vars` (пакет `expvar`) отдаёт отладочную информацию о процессе (например, статистику памяти и GC) в формате, удобном для машинного считывания.

2.11. OS-уровень: страницы памяти и виртуальная память

Дальше — пласт про операционную систему. Может показаться, что это не про Go, но: рантайм Go стоит **на** ОС, и половина «глубоких» вопросов из задачника Озона — именно про OS-уровень (страницы, сисколы, дескрипторы, OOM). Бэкендер обязан это понимать. Разберём по-человечески.

Виртуальная память — базовая идея

Каждый процесс в современной ОС видит своё собственное, **непрерывное** адресное пространство памяти — как будто вся память принадлежит только ему. Это иллюзия, которую создаёт **виртуальная память**. На самом деле физической памяти (RAM) меньше, и её делят все процессы.

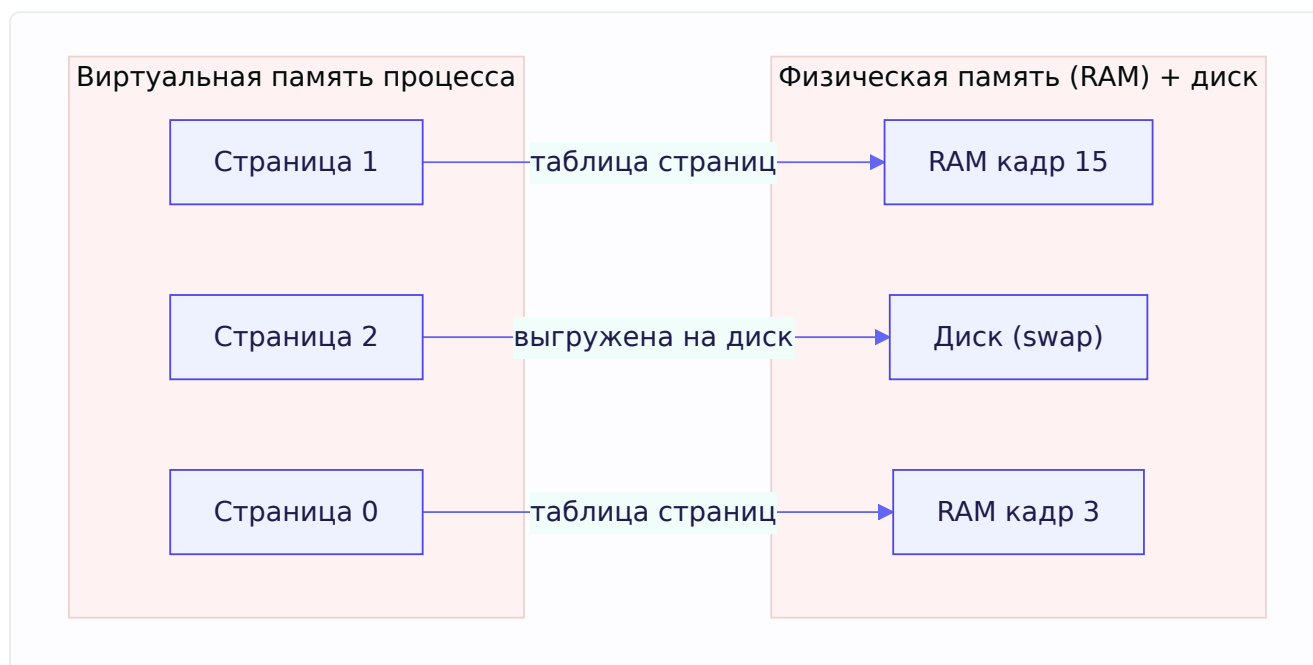
Как это работает: адреса, которыми оперирует программа (**виртуальные адреса**), не соответствуют напрямую физическим адресам в RAM. Между ними стоит слой трансляции, который переводит виртуальные адреса в физические.

Зачем это нужно (из задачника): - **Изоляция процессов**. Каждый процесс в своём адресном пространстве и не может влезть в память другого — основа безопасности. - **Защита памяти**. Можно пометить участки памяти как «только для чтения» или «нельзя исполнять код» — защита от порчи данных и от эксплойтов. - **Гибкость**. Виртуальная память может отображаться (мапиться) не только на RAM, но и на диск, на память устройств, на файлы.

Страница памяти

Транслировать каждый байт по отдельности было бы нереально. Поэтому память делят на **страницы** (pages) — блоки фиксированного размера (стандартно **4 КБ**). Трансляция работает на уровне страниц: вся страница виртуальной памяти отображается на страницу физической (или на диск, или на файл).

Определение из задачника: **страница памяти — это единица структурирования виртуальной памяти**. Она позволяет отображать виртуальную память на физическую либо на файл, а также **защищать** отдельные участки памяти (например, запрещать запись).



Как трансляция работает быстро — TLB

Резонный вопрос из задачника: «Если каждое обращение к памяти — это поиск в таблице отображения страниц, как это может быть быстро?»

Ответ: у процессора есть специальный кеш для этих отображений — **TLB (Translation Lookaside Buffer)**. Он хранит недавно использованные переводы «виртуальный адрес → физический». Поскольку программы обычно работают с **близкими** адресами (обращаются к одним и тем же страницам многократно — это свойство называется локальностью), нужный перевод почти всегда уже лежит в TLB, и трансляция происходит очень быстро, без обращения к полной таблице страниц в памяти.

Hugepages

Ещё из задачника — **hugepages** (большие страницы). Это механизм, позволяющий менять размер страницы со стандартных 4 КБ на **гораздо большие** значения (например, 2 МБ или даже 1 ГБ). Зачем?

Чем больше страница, тем **меньше** записей нужно в таблице страниц и TLB, чтобы покрыть тот же объём памяти. Для программ, работающих с огромными объёмами памяти (базы данных, кеши), это резко снижает число промахов TLB и обращений к таблице страниц — а значит,

ускоряет работу. Плата — менее гранулярное управление памятью (страница большая, её нельзя частично выгрузить).

2.12. User Space vs Kernel Space и системные вызовы

Мы уже упоминали переходы «в ядро». Теперь разберём это как следует — тоже прямой вопрос из задачника.

Два режима работы

Процессор работает в одном из двух режимов, и это разделение — фундамент безопасности ОС:

User Space (пользовательское пространство) — режим, в котором работают **прикладные программы** (в том числе твой Go-сервис). В этом режиме процессу доступна только его собственная виртуальная память, и **часть низкоуровневых команд процессора недоступна**. Программа не может напрямую лезть к устройствам, управлять памятью на низком уровне или трогать другие процессы.

Kernel Space (пространство ядра) — режим, в котором работает **операционная система** (ядро). Здесь доступен весь арсенал процессора: прямой доступ к устройствам, управление памятью, планирование процессов и т.д.

Зачем разделение

Из задачника: разделение нужно для **безопасности** — чтобы никакое прикладное ПО не могло сломать систему. Если бы программы имели полный доступ к железу, любая ошибка (или вредонос) могла бы обрушить всю машину или прочитать чужие данные. Ядро выступает надёжным посредником: оно одно имеет полный доступ, а программы обращаются к нему через строго определённый интерфейс.

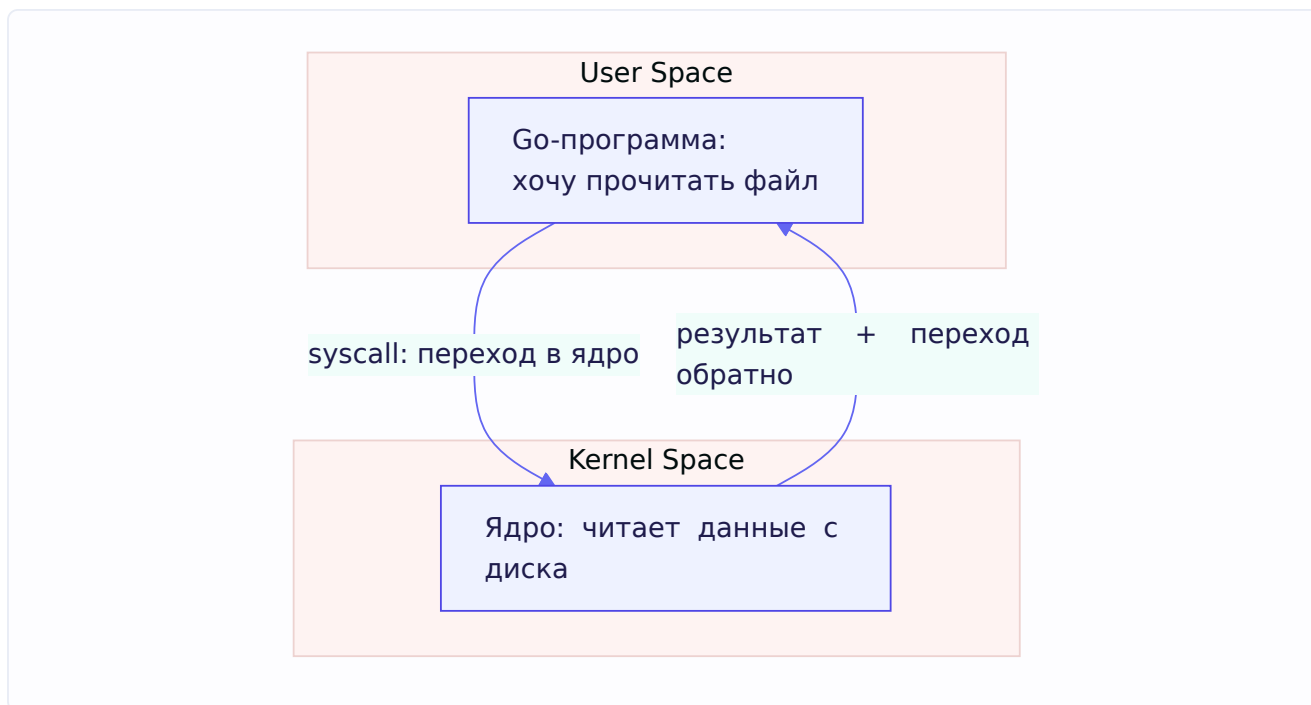
Системный вызов (syscall)

Этот «строго определённый интерфейс» и есть **системный вызов** (syscall). Можно считать его **API операционной системы**. Когда программе нужно что-то, требующее привилегий, — прочитать файл, отправить данные по сети, создать процесс, — она делает syscall.

Определение из задачника: syscall — это **вызов функции в операционной системе** (например, чтение файла).

Что происходит во время syscall

Из задачника, по шагам: происходит **переключение из User Space в Kernel Space**. Программа передаёт ядру номер нужной операции и аргументы, процессор переходит в привилегированный режим, ядро выполняет операцию (например, реально читает данные с диска), затем управление и результат возвращаются в программу, и процессор переключается обратно в User Space.



Важное следствие: syscall **не бесплатен**. Переключение режимов стоит процессорных тактов. Вот почему рантайм Go старается минимизировать сисколлы (кеширует память в `mscache` вместо запросов к ОС; использует `netpoller`, чтобы не делать блокирующих сетевых сисколлов). И вот почему блокирующий syscall в модели GMP приводит к `hand-off P` (раздел 2.3) — поток застревает в ядре.

Примеры важных сисколлов из задачника

- **fork** — запускает **копию** текущего процесса (создаёт дочерний процесс). Основа порождения процессов в Unix.
- **epoll** — позволяет **асинхронно** получать события о готовности дескрипторов (например, «в этот сокет пришли данные, можно читать»). Это фундамент высокопроизводительных сетевых серверов — и, как мы знаем, основа `netpoller`'а в Go.

2.13. Файловые дескрипторы

Раз зашла речь о чтении файлов и сокетов — разберём файловые дескрипторы (вопрос из задачника).

Что это

Файловый дескриптор — это идентификатор (по сути, небольшое целое число), который операционная система выдаёт процессу при открытии файла. Через этот идентификатор процесс потом совершает операции: чтение, запись и т.д.

Но — важное расширение из задачника — дескриптор это **универсальный** идентификатор не только для файлов на диске. Через дескрипторы в Unix-философии («всё есть файл») работают: -

реальные файлы на диске; - виртуальные файлы, в том числе замалленные на память; - устройства ввода-вывода; - **пайпы** (каналы между процессами); - **сокеты** (сетевые соединения).

Это красивая унификация: read/write работают одинаково хоть для файла, хоть для сетевого сокета — разница только в дескрипторе.

Стандартные дескрипторы

Из задачника: у каждого процесса при запуске есть три стандартных дескриптора с фиксированными номерами: - **stdin (0)** — стандартный ввод. - **stdout (1)** — стандартный вывод. - **stderr (2)** — стандартный вывод ошибок.

При запуске из консоли stdout/stderr обычно перенаправлены в терминал.

Практическая задача: «нет места на диске, хотя оно есть»

Классическая задача из задачника, которая проверяет понимание дескрипторов. Ситуация: **df** показывает, что место на диске есть, но записать ничего нельзя — система говорит «нет места».

Разгадка связана с тем, **когда реально освобождается место при удалении файла**. Когда ты удаляешь файл, но какой-то процесс всё ещё держит **открытый дескриптор** на него, — файл физически **не удаляется**. Место освобождается только когда закрыты **все** дескрипторы на файл всеми процессами. Поэтому если программа пишет в большой лог, лог «удалили», но программа держит дескриптор открытым — место не вернётся. Решение: найти процесс, держащий дескриптор (например, через **lsdf**), и заставить его закрыть дескриптор (перезапустить или сигналом).

2.14. Процессы, потоки и сигналы

Завершим главу процессами, их отличием от потоков и управлением ими. Всё это — вопросы из задачника.

Процесс vs системный поток

Из задачника, суть отличия:

Процесс — это исполняющаяся программа со своим изолированным адресным пространством. Процессы по умолчанию **не разделяют память** между собой.

Системный поток запускается **внутри** процесса. Потоки одного процесса **разделяют** память этого процесса (общую кучу, глобальные данные), но каждый имеет свой стек.

Тонкость, которую ценят на собеседовании: в терминах ядра Linux и процесс, и поток — это **task** (задача). Разница в том, что процессы по умолчанию не шарят память, а потоки внутри процесса — шарят. При этом процессы **тоже могут** разделять память — через специальные механизмы (**shm_open** , **mmap**).

Как процессы обмениваются информацией (IPC)

Раз процессы изолированы, как им общаться? Через механизмы **межпроцессного взаимодействия** (IPC), из задачника: - **сокеты** — в том числе по сети; - **пайпы** — каналы между процессами; - **сигналы** — простые уведомления; - **разделяемая память** (shm/mmap) — общий участок памяти.

Сигналы

Сигнал — это способ ОС (или другого процесса) уведомить процесс о событии. Это простое асинхронное сообщение. Примеры: - **SIGTERM** — вежливая просьба завершиться. Процесс может её перехватить и корректно завершить работу (закрыть файлы, сохранить данные) — это называется graceful shutdown. - **SIGKILL** — приказ немедленно умереть. **Нельзя** перехватить или проигнорировать — ядро убивает процесс безусловно. - **SIGURG** — тот самый сигнал, который Go использует для асинхронного вытеснения горутин (раздел 2.3).

Убийство процесса

Из задачника, как убить процесс в Linux: - `kill <pid>` — по умолчанию шлёт SIGTERM (вежливо). - `kill -9 <pid>` — шлёт SIGKILL (жёстко). - `killall <name>` — по имени процесса. - В `top / htop` — найти процесс, нажать `k`.

Важный нюанс из задачника: `kill -9 (SIGKILL)` — **крайняя мера**, и это плохо как основной способ. Почему? Потому что SIGKILL не даёт процессу завершиться корректно: он не закроет файлы, не сбросит буферы, не освободит ресурсы штатно — данные могут повредиться. Сначала стоит послать SIGTERM (`kill` без `-9`), дать процессу шанс завершиться по-человечески, и только если он не реагирует — применять `-9`.

Найти PID процесса можно через `ps aux | grep <name>` или в `top / htop`.

Как читать высокий процент CPU

Классическая задача из задачника: `top` показывает, что процесс потребляет **146% CPU**. Реально ли это и надо ли паниковать?

Да, абсолютно реально. Проценты в `top` считаются **на ядро**: 100% = одно полностью загруженное ядро. Значит, 146% = процесс использует почти **полтора ядра** — он многопоточный (или в случае Go — многогорутинный на нескольких P) и работает на нескольких ядрах одновременно. Для многопоточной программы это **нормально**.

Красный флаг на собеседовании (прямо отмечено в задачнике) — если кандидат говорит «ну это наверное перегрузка системы». Само по себе значение выше 100% ни о какой перегрузке не говорит — это просто параллельная работа на нескольких ядрах. Нужно ли что-то делать — зависит от контекста (сколько всего ядер, какая ожидаемая нагрузка), но сам факт >100% тревоги не вызывает.

OOM Killer и swap

Финальная OS-задача из задачника: на «голом» сервере (без контейнеров) в сервисе утечка памяти. Что произойдёт, когда память кончится?

Ответ зависит от того, включён ли **swap** (подкачка на диск):

- **Swap включён:** когда физическая память (RAM) заканчивается, система начинает вытеснять редко используемые страницы памяти на диск (в swap). Система может работать в таком состоянии довольно долго — но медленно, потому что диск гораздо медленнее RAM.
- **Swap отключён:** когда RAM кончается, вмешивается **OOM Killer** (Out Of Memory Killer) — механизм ядра, который начинает **убивать процессы**, чтобы освободить память. Не факт, что он убьёт именно наш сервис (он выбирает жертву по своей эвристике), но рано или поздно доберётся и до него.

Тонкий момент из задачника: при решении, кого убить, система оценивает **не виртуальную** память процесса, а только **реально используемую** — ту, что фактически отображена в физическую память. Процесс может зарезервировать гигантское виртуальное адресное пространство, но пока он им реально не пользуется (страницы не отображены в RAM), это не делает его первым кандидатом на отстрел.

Итоги главы

Это была большая и плотная глава. Главное, что стоит унести:

Память программы: стек быстрый, но недолговечный; куча гибкая, но дорогая (аллокация + GC). Где окажется переменная, решает **escape analysis** на этапе компиляции — если переменная переживает функцию (например, вернули `&x`), она убегает в кучу.

GMP-планировщик: G (горутины) мультиплексируются поверх M (поток ОС) через P (логические процессоры с локальными очередями). Work stealing балансирует нагрузку. При блокирующем syscall происходит hand-off P на другой поток — поэтому потребление CPU может превышать GOMAXPROCS. Netpoller (epoll) позволяет тысячам сетевых горутин ждать, не занимая потоков. Preemption с 1.14 — асинхронная, через сигналы.

Горутины лёгкие из-за малого растущего стека (старт с килобайтов против мегабайтов у потоков) и планирования в user space (дешёвое переключение без перехода в ядро).

Сборщик мусора: конкурентный трёхцветный mark-and-sweep, оптимизированный под низкие паузы. Трёхцветная схема (белый/серый/чёрный) позволяет работать параллельно с программой; write barrier защищает от потери объектов при изменении ссылок во время пометки; STW-паузы очень короткие.

Аллокатор: иерархия mcache (у P, без блокировок) → mcentral (общий, по классам размеров) → mheap (вся куча) → ОС. Большинство аллокаций — быстрый путь через mcache.

OS-уровень: виртуальная память через страницы (4 КБ), быстрая трансляция через TLB, hugepages для больших объёмов. Разделение user/kernel space ради безопасности; syscall — API ОС с переключением режимов (недешево). Файловые дескрипторы — универсальные идентификаторы (файлы, сокеты, пайпы). Процессы изолированы, потоки шарят память; сигналы (SIGTERM вежливо, SIGKILL жёстко); >100% CPU — норма для многопоточности; OOM Killer при нехватке RAM без swap.

В следующей главе поднимаемся обратно к прикладному уровню и разбираем конкурентность на практике: каналы, `select`, мьютексы и паттерны — с опорой на модель GMP, которую мы только что построили.

Глава 3. Конкурентность

Введение к главе

Конкурентность — это то, за что Go любят. Именно ради удобной работы с параллельными задачами язык и создавался. В предыдущей главе мы разобрали *механику* — как планировщик GMP исполняет горутины. Теперь поднимаемся на уровень выше: как этим *пользоваться* правильно.

Сразу проясним важное различие, которое путают, — **конкурентность (concurrency)** и **параллелизм (parallelism)**:

- **Конкурентность** — это про *структуру*: способ организовать программу как набор независимых задач, которые могут выполняться в перекрывающиеся периоды времени. Это способ *думать* о задаче.
- **Параллелизм** — это про *исполнение*: когда задачи реально выполняются одновременно на разных ядрах.

Известная формулировка Роба Пайка (одного из создателей Go): «Конкурентность — это не параллелизм». Конкурентность — это дизайн, разбиение на независимые части. Параллелизм — лишь один из возможных способов их исполнить. Можно писать конкурентный код на одном ядре (горутины будут чередоваться), а можно — на многих (будут идти параллельно). Go даёт удобные инструменты именно для конкурентного *дизайна*, а параллелизм получается «бесплатно», если ядер несколько.

Главная философия конкурентности в Go выражена в девизе:

«**Don't communicate by sharing memory; share memory by communicating.**» Не общайтесь через разделяемую память — разделяйте память через общение.

Что это значит на практике: вместо того чтобы несколько горутин лезли в одну переменную под мьютексом (общение через разделяемую память), предпочтительно **передавать данные между горутинами по каналам** (общение через сообщения). Тот, кто владеет данными, — единственный, кто с ними работает; остальные получают их через канал. Это снижает целый класс ошибок. Мьютексы при этом никуда не деваются — просто каналы часто дают более ясный дизайн. Разберём оба подхода.

3.1. Горутины vs OS threads (кратко)

Детали мы разобрали в главе 2 (разделы 2.4–2.5), здесь — краткое напоминание как отправная точка, потому что вся глава строится вокруг горутин.

Горутина — это лёгкая единица конкурентного исполнения, управляемая планировщиком Go. Ключевое: - Запускается словом `go` перед вызовом функции. - Стартует с крошечного стека

(килобайты), который растёт по мере надобности. - Планируется в user space, переключение дёшево. - Их можно создавать сотнями тысяч.

```
go doSomething()           // запустить функцию в новой горутине
go func() {                // или анонимную функцию
    fmt.Println("привет из горутины")
}()
```

Есть важнейший нюанс, с которого начинаются все проблемы новичков: **main не ждёт горутины**. Когда функция **main** завершается, программа заканчивается — **вместе со всеми ещё работающими горутинами**, независимо от того, доделали они свою работу или нет. Разберём это подробно на первой же задаче.

Задача: почему горутины «не успевают»

Классика из задачника:

```
func main() {
    for i := 0; i < 5; i++ {
        go func() {
            fmt.Println(i)
        }()
    }
}
```

Что выведет? **Скорее всего — ничего**. Почему? Цикл мгновенно запускает 5 горутин и заканчивается. Сразу за циклом заканчивается и **main**. Программа завершается — а горутины даже не успели дойти до **fmt.Println**. Они создались, но планировщик не успел дать им поработать до того, как программа умерла.

Здесь на самом деле **две** проблемы, и это важно проговорить (задачник отмечает обе):

1. **main не ждёт горутин** — программа завершается раньше, чем они отработают.
2. **Захват переменной цикла i** — даже если бы горутины успели, до Go 1.22 они бы вывели одно и то же число (об этом — раздел 3.10, это отдельная большая ловушка).

Как дождаться горутин? Нужен явный механизм синхронизации. Самый простой (но плохой) — **time.Sleep**, самый правильный — **sync.WaitGroup**. К ним придём в разделе 3.5. А пока запомни: **создать горутину и дождаться её результата — это разные вещи**, и о втором нужно позаботиться явно.

3.2. Каналы и hchan

Каналы — главный инструмент общения между горутинами в Go. Разберём их и снаружи (как использовать), и изнутри (как устроены).

Что такое канал

Канал (channel) — это типизированная труба, по которой горутин передают друг другу значения. Одна горутин пишет в канал, другая читает. Канал сам по себе **потокбезопасен** — к нему могут одновременно обращаться несколько горутин, и рантайм гарантирует корректность.

```
ch := make(chan int) // канал для передачи int
ch <- 42              // запись: отправить 42 в канал
x := <-ch             // чтение: получить значение из канала
```

Стрелка `<-` показывает направление потока данных: `ch <- x` — данные текут в канал, `x := <-ch` — данные текут из канала.

Буферизированные и небуферизированные каналы

Есть два вида каналов, и разница между ними принципиальна.

Небуферизированный канал (`make(chan int)`) — не хранит значений вообще. Отправка блокируется, пока кто-то не начнёт читать, и наоборот. Это точка **синхронной встречи** (rendezvous): отправитель и получатель должны «встретиться». Отправитель ждёт получателя, получатель ждёт отправителя.

```
ch := make(chan int) // небуферизированный
// ch <- 1 // заблокируется здесь навсегда, если никто не читает
```

Буферизированный канал (`make(chan int, 3)`) — имеет буфер на N значений. Отправка блокируется только когда буфер **полон**; чтение — только когда буфер **пуст**. Пока в буфере есть место, отправитель не ждёт получателя.

```
ch := make(chan int, 2) // буфер на 2
ch <- 1 // не блокируется, в буфере есть место
ch <- 2 // не блокируется, буфер заполнился
// ch <- 3 // ЗАБЛОКИРУЕТСЯ — буфер полон
```

Аналогия: небуферизированный канал — это передача из рук в руки (нужно, чтобы оба присутствовали). Буферизированный — это почтовый ящик на N писем (можно положить и уйти, пока есть место).



Устройство изнутри: hchan

Теперь заглянем под капот (это спрашивают на 19 грейде в задачнике). Канал в рантайме — это структура `hchan`. Упрощённо она содержит:

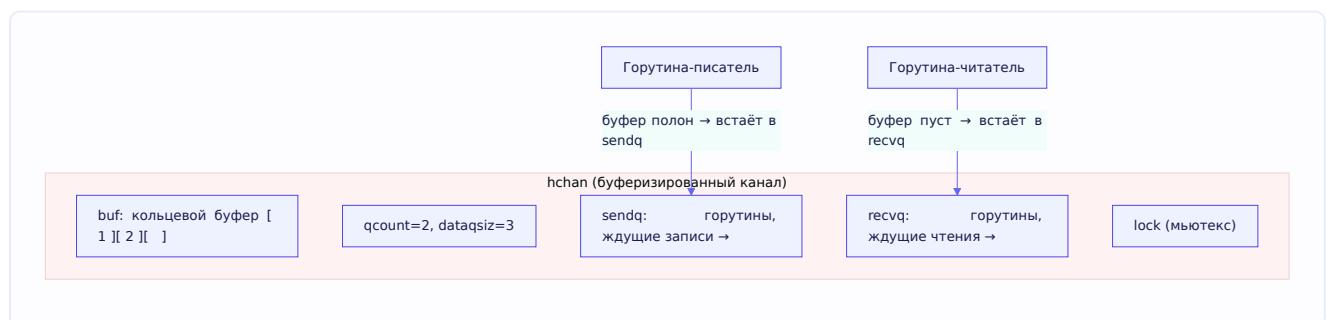
```

type hchan struct {
    qcount    uint    // сколько элементов сейчас в буфере
    dataqsiz  uint    // размер буфера (cap канала)
    buf       unsafe.Pointer // сам кольцевой буфер (для буферизированных)
    closed    uint32    // флаг: закрыт ли канал
    sendx     uint    // индекс для записи в кольцевой буфер
    recvx     uint    // индекс для чтения из кольцевого буфера
    recvq     waitq    // очередь горутин, ждущих чтения
    sendq     waitq    // очередь горутин, ждущих записи
    lock      mutex    // мьютекс для защиты всей структуры
}

```

Разберём ключевые поля:

- **buf** — кольцевой буфер (для буферизированных каналов), куда складываются значения.
`sendx` / `recvx` — позиции записи и чтения в этом кольце.
- **recvq** и **sendq** — очереди **заблокированных горутин**. Если горутина хочет читать, а данных нет — она встаёт в `recvq` и «засыпает». Если хочет писать, а места нет — встаёт в `sendq`. Когда появляется возможность, планировщик будит горутины из очереди.
- **lock** — да, внутри канала есть мьютекс. Именно он делает канал потокобезопасным: все операции с каналом происходят под этой блокировкой.



Вот почему из задачника ответ на «как устроен канал внутри» звучит так: это структура с метаданными (размер буфера, открыт/закрыт, индексы), кольцевым буфером, **очередями ожидающих горутин** и **мьютексом** для безопасного доступа. Механизм очередей `recvq/sendq` — это и есть та «магия», благодаря которой блокировка на канале не сжигает процессор: горутина не крутится в цикле ожидания, а засыпает и будится планировщиком (вспомни `park/unpark` из главы 2).

Поведение в особых случаях — обязательно знать

Это самая частая группа вопросов про каналы. Собери в голове чёткую таблицу — её любят гонять на собеседовании.

Операции с закрытым каналом:

- **Чтение из закрытого канала** — возвращает **нулевое значение** типа немедленно, без блокировки. Причём можно отличить закрытие через вторую переменную: `v, ok := <-ch` — если `ok == false`, канал закрыт и значения больше не будет.
- **Запись в закрытый канал** — **паника** (`send on closed channel`).

- **Повторное закрытие** закрытого канала — тоже **паника**.

Операции с nil-каналом (переменная-канал, которой не сделали `make`):

- **Чтение из nil-канала** — блокировка **навсегда**.
- **Запись в nil-канал** — блокировка **навсегда**.
- **Закрытие nil-канала** — **паника**.

Сведём в таблицу:

Операция	Открытый канал	Закрытый канал	nil-канал
Чтение <code><-ch</code>	значение / блокировка	zero value, <code>ok=false</code>	блокировка навсегда
Запись <code>ch<-</code>	передача / блокировка	паника	блокировка навсегда
Закрытие <code>close(ch)</code>	закрывает	паника	паника

Эти правила кажутся произвольными, но за ними есть логика. Чтение из закрытого канала возвращает zero value, чтобы получатели могли спокойно «дочитать» и узнать о закрытии — это основа паттерна завершения (см. ниже). Запись в закрытый паникует, потому что это явная логическая ошибка: писать туда, куда объявили, что больше не пишут. Блокировка на nil навсегда — на первый взгляд бесполезна, но пригождается в `select` для «отключения» ветки (об этом в 3.3).

Кто должен закрывать канал

Важное правило дизайна: **закрывать канал должен отправитель, а не получатель**. Почему? Потому что закрытие — это сигнал «данных больше не будет». Знать это может только тот, кто пишет. Если получатель закроет канал, а отправитель попытается записать — паника. Отсюда принцип: закрывает тот, кто владеет отправкой, и обычно — только один владелец.

Закрывать канал вообще **не обязательно**, если в этом нет нужды: каналы собираются сборщиком мусора, как и всё остальное, когда на них не остаётся ссылок. Закрытие нужно именно как *сигнал*, а не для «освобождения ресурса».

Задача: range по каналу без close

Классическая задача из задачника, показывающая, зачем нужен close:

```
func main() {
    ch := make(chan int)
    go func() {
        for i := 0; i < 5; i++ {
            ch <- i
        }
    }()
    for n := range ch {
        fmt.Println(n)
    }
}
```

Что произойдёт? Программа выведет числа от 0 до 4, а потом **упадёт с паникой** `all goroutines are asleep - deadlock!`.

Почему? Конструкция `for n := range ch` читает из канала, пока он **не закрыт**. Горутина отправила 5 чисел и завершилась — но канал не закрыла. Цикл `range` прочитал все 5 значений и ждёт следующего. А отправлять больше некому — горутина уже мертва. Все горутин (включая `main`) спят в ожидании — рантайм это обнаруживает и роняет программу с сообщением о взаимоблокировке (deadlock).

Исправление — закрыть канал, когда отправка закончена:

```
go func() {
    for i := 0; i < 5; i++ {
        ch <- i
    }
    close(ch) // ← сигнал: данных больше не будет
}()
```

Теперь `range` прочитает 5 значений, увидит закрытие канала и **корректно завершит цикл**. Никакой паники.

Запомни связку: `for range` по каналу завершается **только** при закрытии канала. Если продюсер не закрывает канал — консьюмер зависнет навсегда (или упадёт с deadlock, если больше некому работать).

3.3. select

`select` — это управляющая конструкция специально для каналов. Если коротко из задачника: **это switch для каналов**. Но за этой краткостью много важных деталей.

Как работает

`select` позволяет горутине ждать **сразу нескольких** канальных операций и выполнить ту, которая станет готова первой:

```
select {
case v := <-ch1:
    fmt.Println("получили из ch1:", v)
case ch2 <- x:
    fmt.Println("отправили в ch2")
case <-time.After(time.Second):
    fmt.Println("таймаут")
}
```

Правила выбора: 1. `select` смотрит на все `case` и находит те, что **готовы** (операция может выполняться без блокировки). 2. Если готов **один** `case` — выполняется он. 3. Если готовы **несколько** — выбирается **случайный** из готовых (чтобы не было систематического перекоса в пользу какого-то канала). 4. Если **ни один** не готов и есть `default` — выполняется `default`.

(неблокирующий режим). 5. Если ни один не готов и `default` нет — `select` блокируется, пока какой-нибудь case не станет готов.

Задача: select с отправкой и чтением

Тонкая задача из задачника — разберём по шагам, она отлично проверяет понимание:

```
func main() {
    ch := make(chan int, 1) // буфер на 1
    for i := 0; i < 10; i++ {
        select {
            case x := <- ch:
                print(x)
            case ch <- i:
        }
    }
}
```

Что выведет? `0 2 4 6 8`. Разберём, почему.

Канал буферизированный, вместимость 1. На каждой итерации `select` выбирает между «прочитать из канала» и «записать в канал». Проследим:

```
i=0: канал пуст. Чтение невозможно (нечего читать), запись возможна (место есть).
    → выполняется запись: ch <- 0. Канал теперь [0].
i=1: канал [0]. Чтение возможно (есть 0), запись невозможна (буфер полон).
    → выполняется чтение: x=0, print(0). Канал теперь []. Вывод: 0
i=2: канал пуст. → запись ch <- 2. Канал [2].
i=3: канал [2]. → чтение x=2, print(2). Канал []. Вывод: 2
i=4: → запись ch <- 4.
i=5: → чтение print(4). Вывод: 4
...
```

Видишь закономерность? На чётных `i` происходит запись (в пустой буфер), на нечётных — чтение (из полного буфера). Печатаются только значения, записанные на предыдущем (чётном) шаге: 0, 2, 4, 6, 8. Итог — `0 2 4 6 8`.

Дополнение из задачника: если сделать канал **небуферизированным** (`make(chan int)`), программа **упадёт с deadlock**. Почему? У небуферизированного канала запись требует одновременного читателя, а чтение — писателя. Внутри одной горутины в `select` ни одна из двух операций не может завершиться сама по себе (нет второй стороны). Ни один case не готов, `default` нет → `select` блокируется навсегда → deadlock.

select с пустым телом и nil-каналы

Пара трюков из задачника:

Блокировка навсегда — пустой `select`:

```
select {} // блокируется навсегда, ни одного case
```

Это иногда используют, чтобы «подвесить» `main`, пока работают другие горутины (хотя обычно есть способы яснее).

Отключение ветки через `nil`-канал. Помнишь, что операции с `nil`-каналом блокируются навсегда? В `select` это полезно: если присвоить каналу в `case` значение `nil`, эта ветка **никогда не станет готовой** — фактически отключается. Приём применяют для динамического включения/выключения направлений в цикле `select`:

```
var sendCh chan int = actualCh
// ...
if больше не нужно отправлять {
    sendCh = nil // ветка "case sendCh <- x" теперь никогда не работает
}
```

Здесь `nil`-канал из «бесполезного» превращается в изящный инструмент управления.

3.4. Mutex и RWMutex

Каналы — не единственный инструмент. Иногда проще и правильнее защитить общие данные классической блокировкой. Мьютексы мы уже затрагивали в главе 1 (защита `map`), здесь — систематически.

Mutex — взаимное исключение

Mutex (mutual exclusion, взаимное исключение) — примитив синхронизации, который гарантирует, что в защищённый участок кода (**критическую секцию**) в каждый момент времени входит **только одна** горутина.

```
var mu sync.Mutex

mu.Lock() // захватить блокировку (если занята — ждать)
// критическая секция: только одна горутина здесь одновременно
counter++
mu.Unlock() // освободить блокировку
```

Механика: когда горутина вызывает `Lock()` на уже захваченном мьютексе, она **блокируется** (засыпает) до тех пор, пока владелец не вызовет `Unlock()`. Определение из задачника: мьютекс нужен для эксклюзивного доступа к ресурсу; если ресурс нужен другой горутине, она блокируется, пока первая не освободит.

Важная деталь: `sync.Mutex` **готов к работе в нулевом значении** (помнишь «zero value useful» из главы 1?). Не нужно его инициализировать — `var mu sync.Mutex` уже рабочий разблокированный мьютекс.

Устройство под капотом: state и sema

А вот теперь — то, ради чего стоит копать глубже (в задачнике прямо отмечено: «может рассказать как устроен mutex под капотом» — это уже уровень повыше). Разберём

внутренности.

Сам `sync.Mutex` — на удивление маленькая структура, всего **два поля** типа `int32`:

```
type Mutex struct {  
    state int32 // состояние мьютекса, упаковано побитно  
    sema  uint32 // семафор для «парковки»/«побудки» горутин  
}
```

Всего 8 байт. Вся сложность спрятана в том, **как** используются эти два числа.

Поле `state` — битовая упаковка. Это самое интересное. `state` — одно 32-битное число, но разные его биты означают разные вещи. Младшие три бита — это флаги, а старшие биты хранят счётчик:

state (int32) — упакован побитно

биты 3..31
СЧЁТЧИК ожидающих
горутин

бит 2
Starving
(режим голодания)

бит 1
Woken
(кого-то уже будят)

бит 0
Locked
(захвачен?)

Разберём каждую часть:

- **Бит 0 — `Locked`.** Захвачен ли мьютекс. `1` — занят, `0` — свободен. Это главный флаг.

- **Бит 1 — Woken**. «Уже разбудили одну горутину». Оптимизация: чтобы при разблокировке не будить лишних, если одна уже проснулась и вот-вот захватит мьютекс.
- **Бит 2 — Starving**. Флаг режима голодания (см. ниже) — переключает мьютекс в «честный» режим.
- **Биты 3..31 — счётчик ожидающих** (waiters count). Сколько горутин сейчас спят в ожидании этого мьютекса.

Такая упаковка в одно число нужна, чтобы всю операцию захвата/освобождения можно было делать **атомарно** через одну CAS-инструкцию процессора (Compare-And-Swap, про атомарность — раздел 3.5). Меняем всё состояние одним атомарным действием — не нужен отдельный внутренний замок для самого мьютекса. Красивый приём: мьютекс синхронизирует других, а сам синхронизируется через атомик над упакованным `state`.

Поле `sema` — семафор для сна и пробуждения. Флаги в `state` говорят, что происходит, но горутины ведь нужно как-то реально усыпить и разбудить. Для этого — `sema`. Это низкоуровневый семафор рантайма (runtime semaphore). Когда горутина не смогла захватить мьютекс и должна ждать, она «паркуется» на этом семафоре (`runtime_SemacquireMutex`) — то есть засыпает, снимаясь с процессора (вспомни `park` из главы 2, это тот же механизм). Когда владелец освобождает мьютекс, он «будит» одну из спящих через семафор (`runtime_Semrelease`). Так реализуются очередь и пробуждение.

Два режима: `normal` и `starvation`

Теперь самое тонкое, что действительно ценят на собеседовании. У мьютекса Go **два режима работы**, и переключение между ними решает важную проблему.

Normal (обычный) режим. По умолчанию. Когда мьютекс освобождается, разбуженная горутина **не получает его автоматически** — она **конкурирует** за него с новыми горутинками, которые как раз сейчас вызвали `Lock()` и ещё крутятся на процессоре. Это несправедливо к разбуженной (она только что проснулась, а свежая горутинка уже «на ходу» и часто перехватывает мьютекс первой), зато **быстро**: активная горутинка сразу работает, не тратя время на переключения. Проблема: при высокой конкуренции «невозвучая» горутинка может ждать очень долго — её постоянно опережают.

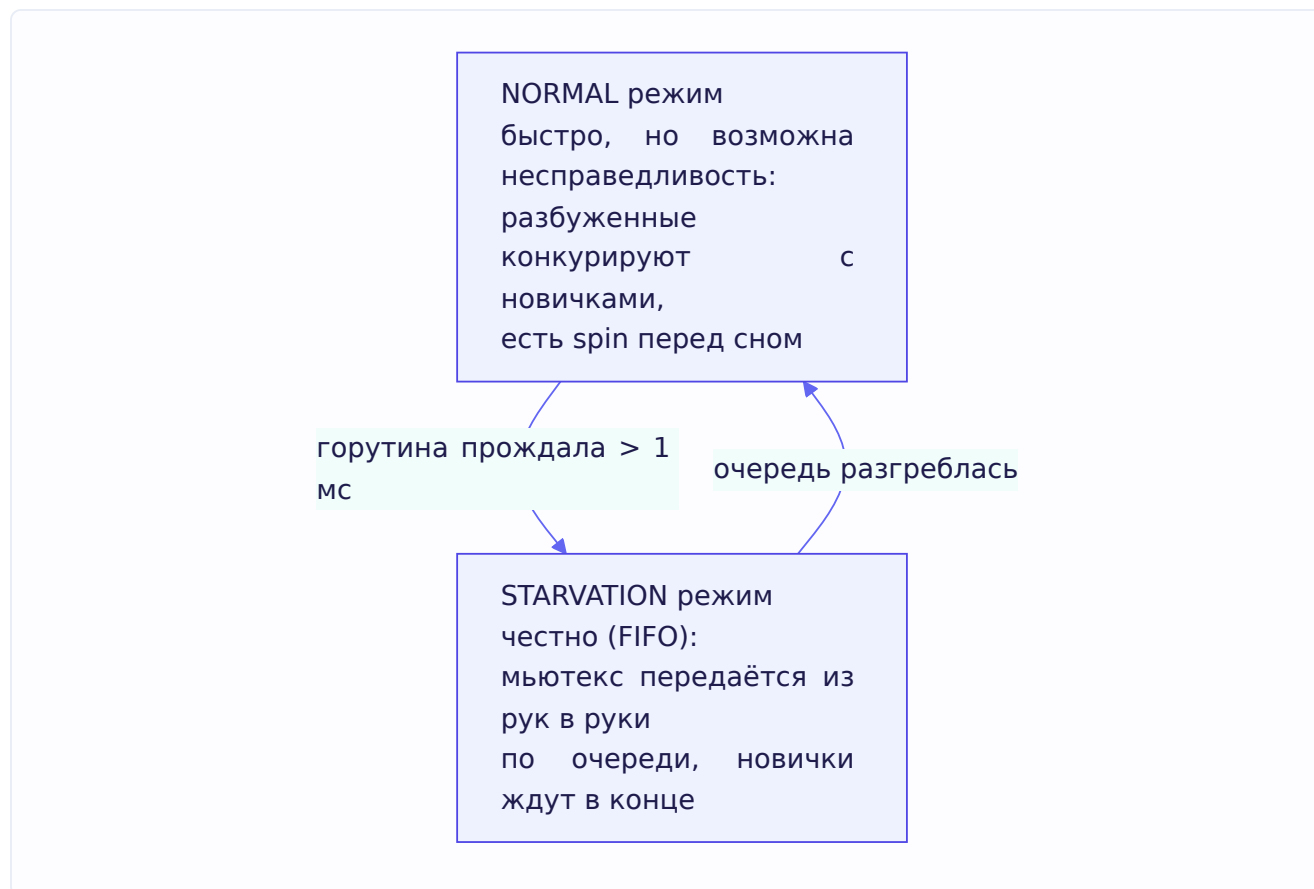
Spinning (кручение) перед сном. Ещё одна оптимизация обычного режима. Прежде чем засыпать (что дорого — переключение контекста), горутинка может немного «покрутиться» в активном ожидании (`spin`) — несколько раз проверить, не освободился ли мьютекс. Если критическая секция короткая, мьютекс освободится за эти несколько тактов, и горутинка захватит его **без** дорогого усыпления/пробуждения. `Spin` оправдан только при определённых условиях (мало итераций, есть свободные ядра).

Starvation (режим голодания). Чтобы «невозвучие» горутинки не ждали вечно, есть предохранитель. Если горутинка ждёт мьютекс дольше **1 миллисекунды**, мьютекс переключается в режим голодания (взводится бит `Starving`). В этом режиме правила меняются на «честные»:

- Освобождённый мьютекс **передаётся напрямую** первой горутинке в очереди (FIFO) — никакой конкуренции с новичками.

- Новые горутины, которые только что вызвали `Lock()`, **не пытаются** захватить мьютекс и не крутятся — они сразу встают **в конец очереди**.

Это гарантирует, что никто не голодает бесконечно. Когда очередь разгребается (последняя горутина забрала мьютекс, или её ожидание стало меньше 1 мс), мьютекс возвращается в обычный быстрый режим.



Итог: мьютекс Go — это компромисс между **скоростью** (обычный режим со spin, оптимизирован под типичный случай короткой секции) и **справедливостью** (режим голодания как предохранитель от бесконечного ожидания). Именно поэтому на вопрос «как устроен мьютекс под капотом» хороший ответ звучит так:

«`sync.Mutex` — это два поля: `state` и `sema`. В `state` побитно упакованы флаг захвата, служебные биты (`woken`, `starving`) и счётчик ожидающих — всё меняется атомарно через CAS. `sema` — семафор рантайма, на котором горутины паркуются и пробуждаются. У мьютекса два режима: обычный (быстрый, с активным кручением-spin перед сном, но возможной несправедливостью) и режим голодания (включается после 1 мс ожидания, честная FIFO-передача мьютекса), чтобы никто не ждал вечно».

defer для разблокировки

Идиоматичный приём — освободить мьютекс через `defer`, сразу после захвата:

```
mu.Lock()
defer mu.Unlock() // выполнится при выходе из функции, что бы ни случилось
// ... критическая секция ...
```

Почему это важно: `defer` гарантирует, что `Unlock` вызовется даже если внутри критической секции произойдёт паника или ранний `return`. Без `defer` паника оставила бы мьютекс навечно заблокированным — и все горютины, ждущие его, зависли бы. `defer` защищает от этого класса ошибок. (Подробно про `defer` — в главе 5.)

RWMutex — разделяемое чтение

RWMutex (Read-Write Mutex) — более специализированный мьютекс, различающий два типа доступа:

- **RLock() / RUnlock()** — блокировка на **чтение**. Читателей может быть **много одновременно** — они не мешают друг другу.
- **Lock() / Unlock()** — блокировка на **запись**. Писатель эксклюзивен: пока он пишет, никто не читает и не пишет.

Идея из задачника: RWMutex нужен, когда есть горютины, которым нужен доступ только на чтение. Много читателей могут работать параллельно, и только запись всех блокирует.

```
var mu sync.RWMutex

// Читатель:
mu.RLock()
_ = data[key] // читателей может быть много одновременно
mu.RUnlock()

// Писатель:
mu.Lock()
data[key] = value // эксклюзивно
mu.Unlock()
```

Устройство RWMutex под капотом

Раз уж мы вскрыли обычный мьютекс, посмотрим и на этот. `sync.RWMutex` устроен поверх обычного `Mutex` плюс несколько счётчиков и семафоров:

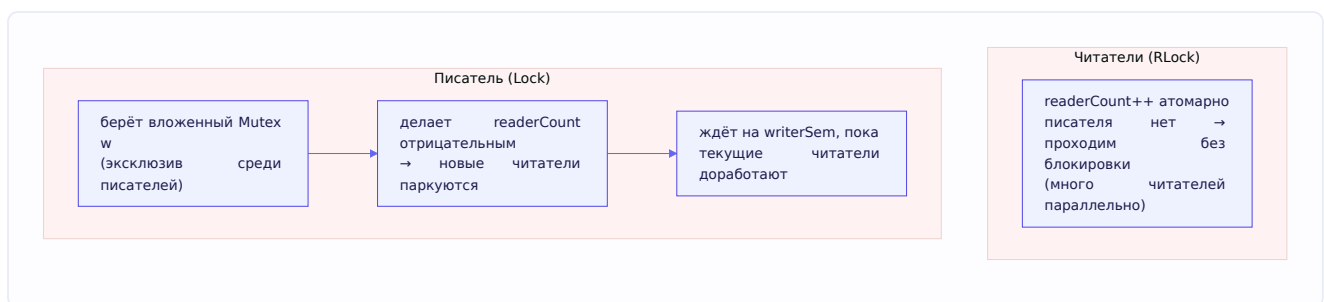
```
type RWMutex struct {
    w      Mutex // обычный мьютекс — его берут писатели между собой
    writerSem uint32 // семафор, на котором писатель ждёт завершения читателей
    readerSem uint32 // семафор, на котором читатели ждут завершения писателя
    readerCount int32 // число активных читателей (со спец. трюком, см. ниже)
    readerWait int32 // сколько читателей писатель ещё ждёт
}
```

Как это работает по частям:

- **W (вложенный Mutex)** — писатели конкурируют между собой через обычный мьютекс: одновременно писать может только один. То есть вся логика взаимного исключения

писателей — это уже разобранный `Mutex` со своими `state/sem`.

- `readerCount` — счётчик активных читателей. Когда читатель делает `RLock()`, он атомарно увеличивает этот счётчик; при `RUnlock()` — уменьшает. Пока писателя нет, читатели просто инкрементят счётчик и проходят — **без всякой блокировки друг друга**. Вот почему читателей может быть много одновременно.
- **Трюк с отрицательным `readerCount`**. Когда приходит писатель, он атомарно **вычитает большое число** (константу `rwmutexMaxReaders`) из `readerCount`, делая его отрицательным. Отрицательное значение — сигнал новым читателям: «пришёл писатель, не входите». Новые `RLock()` видят отрицательный счётчик и паркуются на `readerSem`. При этом по абсолютной величине писатель всё ещё знает, сколько читателей было активно до его прихода.
- `readerWait` и `writerSem`. Писатель должен дождаться, пока **уже начавшие** читатели закончат. Он записывает их число в `readerWait` и паркуется на `writerSem`. Каждый уходящий читатель (`RUnlock`) уменьшает `readerWait`, и последний из них будит писателя.



Теперь понятнее, откуда берутся накладные расходы, о которых пойдёт речь ниже: `RWMutex` — это не «мьютекс попроще», а надстройка с несколькими атомарными счётчиками, двумя семафорами и вложенным обычным мьютексом. За возможность параллельного чтения платят этой сложностью.

Когда `RWMutex` медленнее — важный нюанс

Вопрос с подвохом (задачник спрашивает на 18 грейде в задаче про кеш). Кажется, что `RWMutex` всегда лучше обычного `Mutex`, раз позволяет параллельное чтение. Но это не так.

Две причины, по которым `RWMutex` может проиграть:

1. **`RWMutex` сложнее внутри** — ему нужно вести учёт числа читателей, координировать читателей и писателей. Эти накладные расходы на *каждую* операцию выше, чем у простого `Mutex`. Если критическая секция очень короткая (например, чтение одного значения из `map`), эти накладные расходы могут **перевесить** выигрыш от параллельного чтения.
2. **Lock contention при высокой частоте**. При очень большом потоке операций сама координация становится узким местом. К тому же при потоке записей читатели всё равно блокируются, и параллельность чтения не реализуется.

Практический вывод: `RWMutex` оправдан, когда **читателей действительно много, записей мало**, и **критическая секция достаточно тяжёлая**, чтобы выигрыш от параллельности перекрыл накладные расходы. Для коротких секций под высокой нагрузкой обычный `Mutex` (или вовсе шардирование, глава 1) часто быстрее. Как всегда — измеряй профилировщиком, не угадывай.

Мьютекс vs канал — что выбрать

Раз есть два инструмента синхронизации, когда какой? Грубое правило:

- **Мьютекс** — когда несколько горутин работают с **общим состоянием** (счётчик, кеш, map), и нужно просто защитить доступ. Проще и быстрее для «охраны данных».
- **Канал** — когда нужно **передавать владение данными** или **координировать этапы** работы между горутинami (конвейер, раздача задач). Реализует философию «share memory by communicating».

Не догма, но ориентир: охраняешь данные — мьютекс; передаёшь данные или сигналы — канал.

3.5. WaitGroup, Once и atomic

Три инструмента из пакета `sync` (и `sync/atomic`), которые решают частые задачи синхронизации. Разберём каждый — что делает, как устроен, где ловушки.

WaitGroup — дождаться группы горутин

Помнишь проблему из раздела 3.1 — «main не ждёт горутин»? `sync.WaitGroup` — штатное решение. Это счётчик, который позволяет одной горутине **дождаться завершения** группы других.

Три метода: - `Add(n)` — увеличить счётчик на `n` (обычно — сколько горутин запускаем). - `Done()` — уменьшить счётчик на 1 (горутина сообщает «я закончила»). По сути это `Add(-1)`. - `Wait()` — заблокироваться, пока счётчик не станет равен 0.

Идея простая: заводим счётчик по числу горутин, каждая по завершении делает `Done`, а главная горутина ждёт на `Wait`, пока все не отчитаются. Вот правильная версия той задачи из задачника:

```
func main() {
    var wg sync.WaitGroup
    for i := 0; i < 5; i++ {
        wg.Add(1) // +1 к счётчику перед запуском горутин
        go func() {
            defer wg.Done() // -1 при завершении, через defer — надёжно
            fmt.Println(i)
        }()
    }
    wg.Wait() // ждём, пока счётчик не обнулится
}
```

Теперь `main` не завершится, пока все 5 горутин не сделают `Done`. Обрати внимание на `defer wg.Done()` — по той же причине, что и с мьютексом: `defer` гарантирует, что счётчик уменьшится даже при панике или раннем `return`, иначе `Wait()` зависнет навсегда.

Критически важные правила WaitGroup (за нарушение любого — либо паника, либо зависание, и это любят спрашивать):

1. **Add** вызывать ДО запуска горутины, а не внутри неё. Если сделать **Add** внутри горутины, **Wait()** в главной может успеть выполниться раньше, чем горутина стартует и вызовет **Add** — и **Wait** пройдёт мимо, ничего не дождавшись. Поэтому **Add(1)** стоит в цикле, до **go**.
2. Число **Done** должно точно совпадать с суммой **Add**. Больше **Done**, чем **Add** → счётчик уходит в минус → паника (**negative WaitGroup counter**). Меньше → **Wait()** зависнет навсегда.
3. Нельзя копировать **WaitGroup** после начала использования — только передавать по указателю (***sync.WaitGroup**). Внутри у неё состояние, копия его «расщепит» и всё сломается. (**go vet** это ловит.)

Кратко об устройстве: внутри **WaitGroup** — атомарный счётчик (упакованный вместе со счётчиком ожидающих в одно 64-битное значение) и семафор рантайма, на котором паркуется вызвавший **Wait()**. Механизм тот же, что мы видели у мьютекса: атомарные операции над состоянием + семафор для сна/побудки. Когда счётчик достигает нуля, семафор будит всех ждущих на **Wait()**.

Once — выполнить ровно один раз

sync.Once гарантирует, что некоторый код выполнится **ровно один раз**, даже если к нему одновременно рвётся много горутин. Классический сценарий — ленивая инициализация (создать ресурс при первом обращении).

```
var once sync.Once
var config *Config

func GetConfig() *Config {
    once.Do(func() {
        config = loadConfig() // выполнится РОВНО один раз, даже при гонке
    })
    return config
}
```

Даже если **GetConfig** вызовут 1000 горутин одновременно, **loadConfig()** отработает **единожды**, а остальные 999 **дождутся** её завершения и увидят готовый результат. Это важная деталь: **Once.Do** не просто «пропускает» повторные вызовы — он **блокирует** их, пока первый не закончит. Иначе кто-то мог бы получить недоинициализированный **config**.

Как устроен: внутри **Once** — флаг **done** (проверяется атомарно, быстрый путь) и мьютекс (медленный путь для тех, кто пришёл во время первого выполнения). Быстрая проверка **done** без блокировки делает повторные вызовы почти бесплатными.

atomic — операции без блокировок

Пакет **sync/atomic** даёт **атомарные** операции над числами и указателями — то есть операции, которые выполняются как единое неделимое целое, без риска, что другая горутина вмешается в середине.

Зачем это нужно? Простейший пример — счётчик. Казалось бы, **counter++** — одна операция. Но на уровне процессора это **три** действия: прочитать значение, прибавить 1, записать обратно.

Если две горутины делают это одновременно, они могут обе прочитать одно и то же старое значение и обе записать `+1` — одно увеличение потеряется. Это гонка (см. 3.8).

`atomic` решает это без мьютекса:

```
var counter int64

atomic.AddInt64(&counter, 1) // атомарно +1
v := atomic.LoadInt64(&counter) // атомарно прочитать
atomic.StoreInt64(&counter, 100) // атомарно записать
```

Также есть ключевая операция **CAS (Compare-And-Swap)** — «сравни и обменяй»:

```
atomic.CompareAndSwapInt64(&x, old, new) // если x == old, записать new; вернуть успех
```

CAS — фундамент многих lock-free алгоритмов (и, как мы видели, самого мьютекса). Идея: «поменяй значение, но только если его никто не трогал с тех пор, как я его прочитал». Если тронул — операция не удалась, пробуем снова.

atomic vs mutex — что выбрать. `atomic` быстрее мьютекса (это буквально одна процессорная инструкция, без парковки горутин), но применим только к **простым** операциям над одним числом/указателем. Как только нужно защитить **несколько** связанных действий или сложную структуру — нужен мьютекс. Правило: одиночный счётчик или флаг — `atomic`; составная критическая секция — мьютекс.

Современный Go (1.19+) добавил удобные типы-обёртки: `atomic.Int64`, `atomic.Bool`, `atomic.Pointer[T]` и т.д. — с методами `.Load()`, `.Store()`, `.Add()`, которые читаются приятнее, чем функции со `&`:

```
var counter atomic.Int64
counter.Add(1)
v := counter.Load()
```

Связь с задачиком: в задаче про счётчик, агрегирующий данные из разных горутин, ожидается один из двух ответов — либо `atomic`, либо отдельная горутина-агрегатор, собирающая значения через канал. Это ровно выбор «share memory (atomic) vs communicate (канал)».

3.6. Context и propagation

`context.Context` — один из важнейших механизмов в бэкенд-Go. Он решает задачу **управления временем жизни** операций: отмену, таймауты и передачу связанных с запросом данных через всю цепочку вызовов.

Зачем нужен Context

Представь типичный сценарий: приходит HTTP-запрос, обработчик идёт в базу, оттуда в кеш, оттуда во внешний сервис. И тут клиент **отменяет** запрос (закрыл вкладку) или истёк **таймаут**. Что должно произойти? Вся цепочка операций должна **остановиться** — незачем ходить в базу ради ответа, который никто не ждёт. Но как «спустить» сигнал отмены через все слои?

Именно это делает **Context**: он **распространяет сигнал отмены** (и дедлайн) от точки входа вниз по всей цепочке горутин и вызовов. Это и называется **propagation** — распространение.

Дерево контекстов

Контексты образуют **дерево**. Есть корневой контекст, от него порождаются дочерние, от них — свои дочерние. Отмена родителя **автоматически отменяет всех потомков** (но не наоборот).

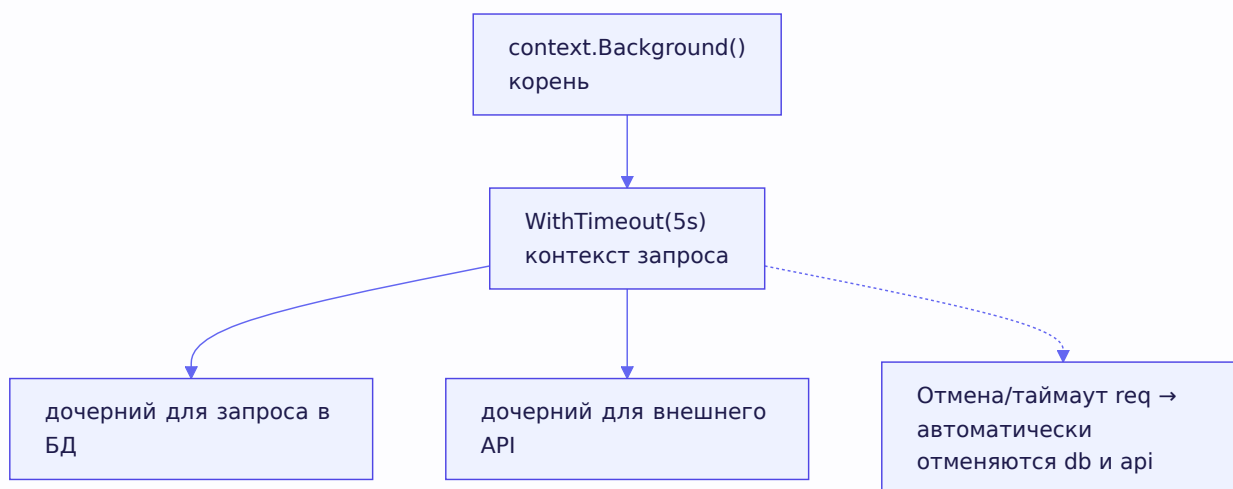
Корневые контексты: - `context.Background()` — пустой корневой контекст, точка старта (обычно в `main` или на входе запроса). - `context.TODO()` — заглушка, когда пока не решил, какой контекст использовать.

От них порождают дочерние:

```
// С отменой вручную:
ctx, cancel := context.WithCancel(parent)
defer cancel() // ВАЖНО: всегда вызывать cancel, чтобы освободить ресурсы

// С таймаутом (отменится сам через 5 секунд):
ctx, cancel := context.WithTimeout(parent, 5*time.Second)
defer cancel()

// С дедлайном (отменится в конкретный момент):
ctx, cancel := context.WithDeadline(parent, time.Now().Add(time.Minute))
defer cancel()
```



Как слушать отмену

Ключевой метод — `ctx.Done()`. Он возвращает **канал**, который **закрывается**, когда контекст отменён (вспомни из 3.2: чтение из закрытого канала мгновенно возвращает значение — вот он, сигнал). Горутина слушает этот канал, обычно через `select`:

```
func worker(ctx context.Context) {
    for {
        select {
            case <-ctx.Done(): // контекст отменён или истёк таймаут
                fmt.Println("останавливаюсь:", ctx.Err())
                return
            case work := <-tasks:
                process(work)
        }
    }
}
```

`ctx.Err()` расскажет, **почему** отменили: `context.Canceled` (вызвали `cancel`) или `context.DeadlineExceeded` (истёк таймаут).

Правила работы с Context (важно для собеседования)

1. **Context передаётся первым аргументом** функции, по соглашению именуется `ctx`: `func Do(ctx context.Context, ...)`.
2. **Не храни Context в структуре** — передавай явно через аргументы. (Есть исключения, но по умолчанию — так.)
3. **Всегда вызывай `cancel`** (обычно через `defer cancel()`), даже если контекст завершился сам по таймауту. Иначе — утечка: контекст держит ресурсы и горутины-таймер, пока его не отменят.
4. **Context неизменяем**. Каждый `WithCancel` / `WithValue` создаёт **новый** контекст-обёртку, не меняя родителя. Отсюда и дерево.
5. `context.WithValue` позволяет прокинуть данные по цепочке (например, request ID для логирования), но им **не злоупотребляют** — только для сквозных «запросных» данных, не для передачи обычных параметров.

Связь с задачником

В интерактивной задаче про URL из задачника (4-й пункт) как раз просят: при получении двух первых ответов «200 OK» **штатно прервать** остальные запросы. Каноничное решение — общий `context.WithCancel`: как только насчитали два успеха, вызываем `cancel()`, и все остальные горутины, слушающие `ctx.Done()`, аккуратно завершаются. Это и есть propagation отмены на практике — один сигнал останавливает пачку операций.

3.7. Паттерны конкурентности

Теперь самое практическое — типовые схемы организации горутин и каналов. Их спрашивают почти всегда (в задачнике: «паттерны конкурентного программирования, использующие каналы»), и они реально применяются в проде. Разберём каждый: что решает, как выглядит, где ловушки.

Все паттерны строятся из двух ролей: - **Producer (продюсер)** — горутина, которая производит данные и шлёт их в канал. - **Consumer (консьюмер)** — горутина, которая читает данные из канала и обрабатывает.

Из комбинаций этих ролей и складываются все паттерны.

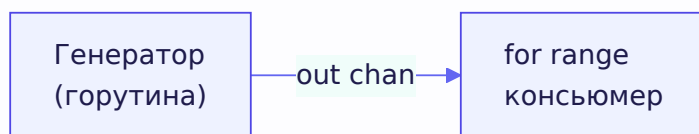
Генератор (Generator)

Самый базовый паттерн. **Генератор** — горутина, которая производит поток значений и отдаёт их через канал. Позволяет «лениво» порождать данные, не вычисляя всё сразу.

```
func generator(nums ...int) <-chan int {
    out := make(chan int)
    go func() {
        defer close(out) // закрываем, когда всё отправили
        for _, n := range nums {
            out <- n
        }
    }()
    return out // возвращаем канал ТОЛЬКО ДЛЯ ЧТЕНИЯ
}

// использование:
for n := range generator(1, 2, 3) {
    fmt.Println(n)
}
```

Обрати внимание на тип возврата — `<-chan int` (receive-only, только для чтения). Это хороший приём: генератор возвращает канал, из которого можно только читать, но нельзя писать или закрыть. Компилятор не даст вызывающему коду случайно записать в него или закрыть — инкапсуляция направления. Закрытие остаётся ответственностью генератора (помнишь правило «закрывает отправитель»).



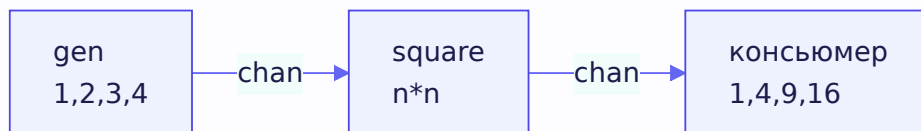
Pipeline (конвейер)

Pipeline — цепочка стадий, где каждая стадия читает из входного канала, что-то делает с данными и пишет в выходной канал, который становится входом для следующей стадии. Как конвейер на заводе.

```
// Стадия 1: генерирует числа
func gen(nums ...int) <-chan int {
    out := make(chan int)
    go func() {
        defer close(out)
        for _, n := range nums {
            out <- n
        }
    }()
    return out
}

// Стадия 2: возводит в квадрат
func square(in <-chan int) <-chan int {
    out := make(chan int)
    go func() {
        defer close(out)
        for n := range in {
            out <- n * n
        }
    }()
    return out
}

// использование: соединяем стадии
for result := range square(gen(2, 3, 4)) {
    fmt.Println(result) // 4, 9, 16
}
```



Красота pipeline: каждая стадия — независимая горутина, они работают **параллельно**. Пока `square` возводит в квадрат первое число, `gen` уже готовит следующее. Данные «текут» через конвейер. Каждая стадия закрывает свой выходной канал, когда её вход закрылся, — так закрытие «распространяется» по конвейеру до конца.

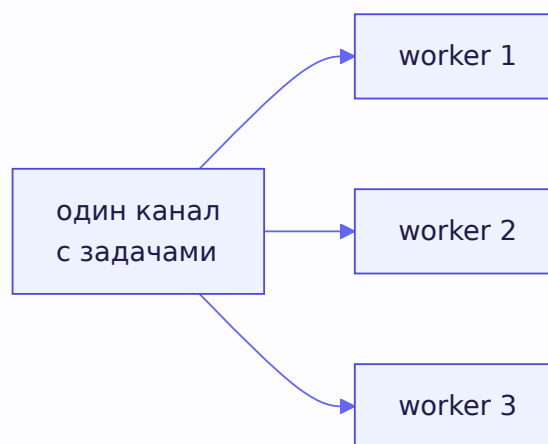
Fan-out (веерное распределение)

Fan-out — когда **несколько** горутин читают из **одного** канала, распределяя работу между собой. Применяется, когда одна задача производит работу быстрее, чем один консьюмер успевает обрабатывать. Запускаем несколько консьюмеров на один канал — они разбирают задачи параллельно.

```
tasks := generator(1, 2, 3, 4, 5, 6)

// запускаем 3 обработчика на ОДИН канал tasks
for i := 0; i < 3; i++ {
    go func(id int) {
        for task := range tasks {
            fmt.Printf("worker %d обработал %d\n", id, task)
        }
    }(i)
}
```

Каждую задачу из канала заберёт **какой-то один** из трёх воркеров (канал гарантирует, что значение получит только один читатель). Так нагрузка распределяется.



Fan-in (всестороннее сведение / merge)

Fan-in — обратная операция: свести **несколько** каналов в **один**. Несколько продюсеров пишут каждый в свой канал, а мы объединяем их выходы в единый поток. Это ровно задача «merge каналов» из задачника — разберём её каноничное решение подробно.

```

func merge(cs ...chan int) <-chan int {
    var wg sync.WaitGroup
    out := make(chan int)

    // для каждого входного канала — своя горутина, копирующая в out
    output := func(c <-chan int) {
        defer wg.Done()
        for n := range c { // читаем c до закрытия
            out <- n       // и пересылаем в общий out
        }
    }

    wg.Add(len(cs))
    for _, c := range cs {
        go output(c) // по горутине на каждый входной канал
    }

    // отдельная горутина: дождаться всех и закрыть out
    go func() {
        wg.Wait()
        close(out)
    }()

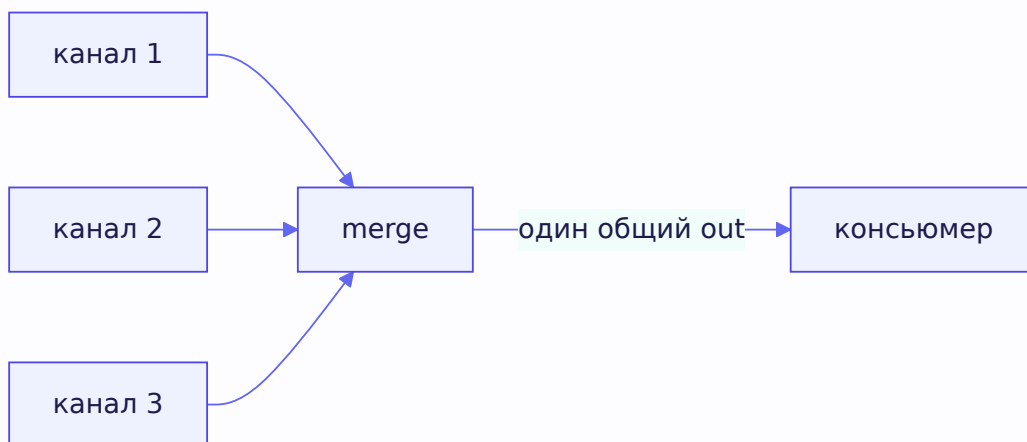
    return out
}

```

Разберём логику по частям — здесь красиво сходятся несколько изученных механизмов:

1. Для каждого входного канала запускаем горутину `output`, которая читает из него всё до закрытия и пересылает в общий `out`.
2. `WaitGroup` считает эти горутин — их ровно `len(cs)`.
3. **Отдельная** горутина ждёт `wg.Wait()` (все копирующие горутин завершились = все входные каналы закрыты и вычитаны) и только тогда закрывает `out`.

Почему закрытие `out` вынесено в отдельную горутину, а не сделано после цикла `for`? Потому что `wg.Wait()` **блокирующий** — если вызвать его в основной горутине до `return out`, функция зависнет и никогда не вернёт канал. А консьюмер снаружи не начнёт читать, пока не получит канал. Тупик. Поэтому ожидание и закрытие живут в фоновой горутине, а `merge` сразу возвращает `out`, готовый к чтению. Это тонкий, но важный момент — его стоит уметь объяснить.



Worker Pool (пул воркеров)

Worker Pool — пожалуй, самый практичный паттерн. Это комбинация fan-out и fan-in: фиксированное число воркеров разбирает задачи из общего канала и складывает результаты в другой общий канал. Ключевое преимущество — **ограничение параллелизма**: не «миллион горутин на миллион задач», а ровно N воркеров, сколько мы решили.

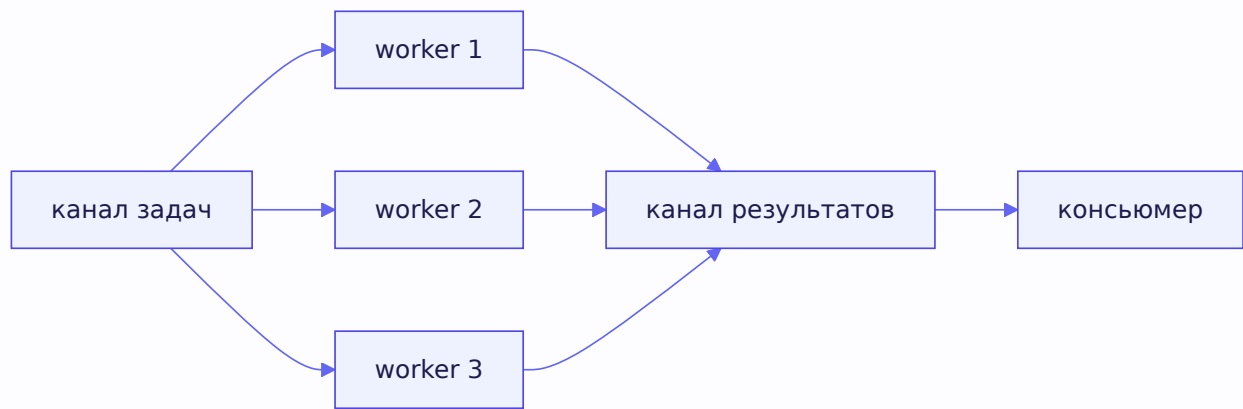
```
func workerPool(numWorkers int, tasks <-chan int) <-chan int {
    results := make(chan int)
    var wg sync.WaitGroup

    // запускаем фиксированное число воркеров
    for i := 0; i < numWorkers; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            for task := range tasks { // fan-out: разбираем общий канал задач
                results <- process(task) // fan-in: пишем в общий канал результатов
            }
        }()
    }

    // закрываем results, когда все воркеры закончили
    go func() {
        wg.Wait()
        close(results)
    }()

    return results
}
```

Зачем ограничивать число воркеров, если горутины дешёвые? Несколько причин: - **Внешние ограничения**. Если каждая задача — запрос к БД или внешнему API, нельзя запустить 10000 одновременных запросов — упрёшься в лимиты соединений или в rate limit сервиса (вспомни задачу про аналитику из задачника — там ровно про это). - **Ресурсы**. Каждая задача может потреблять память/CPU; неограниченный параллелизм исчерпает ресурсы. - **Предсказуемость**. Фиксированное число воркеров даёт стабильную, управляемую нагрузку.



Именно worker pool ожидается в интерактивной задаче из задачника (обработка списка URL): вместо горютины на каждый URL — фиксированный пул воркеров, читающих URL из канала. Плюс `context` для отмены и `WaitGroup` для ожидания завершения. Все механизмы главы сходятся в одном паттерне.

Семафора (ограничение через буферизированный канал)

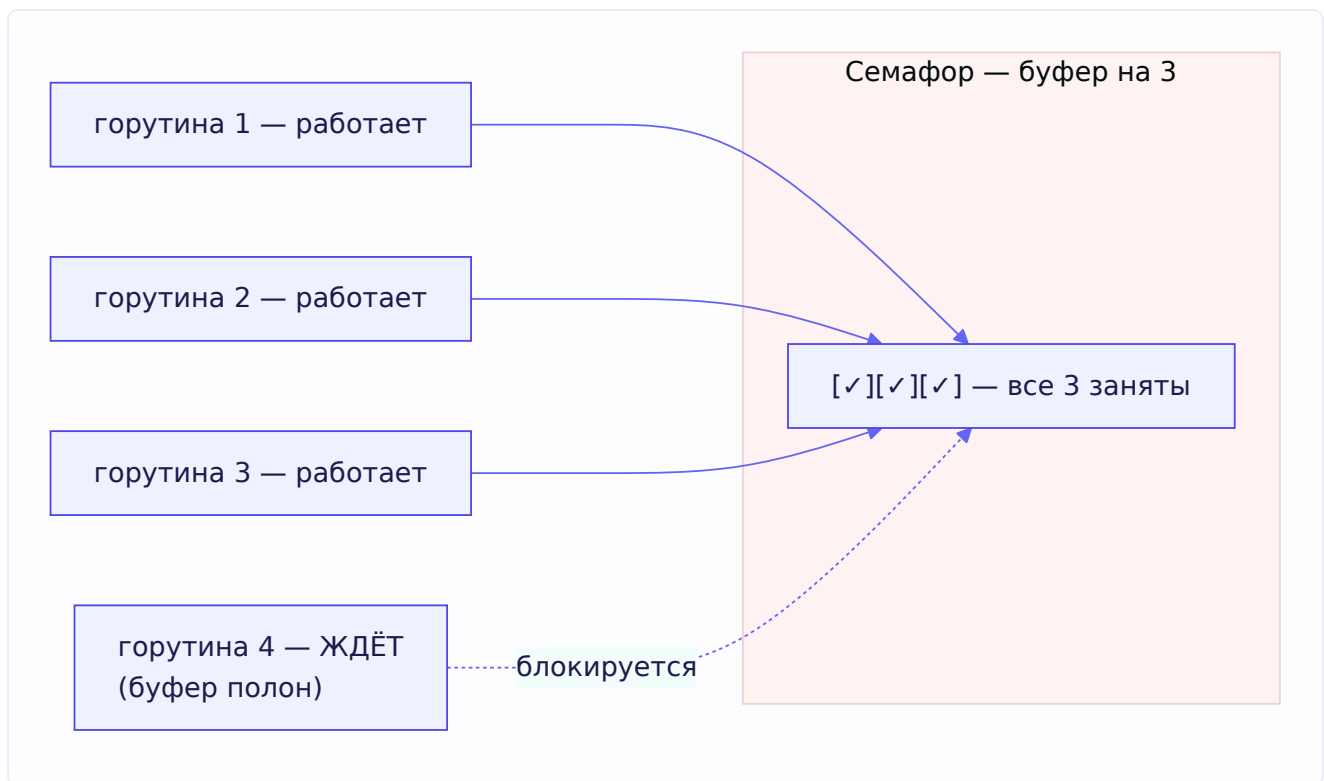
Семафора решает ту же задачу — ограничить число одновременных операций, — но более лёгким способом, без явного пула воркеров. Трюк: буферизированный канал вместимости N работает как счётчик разрешений.

Идея: перед началом операции горютина «берёт разрешение» (пишет в канал), после — «возвращает» (читает из канала). Когда все N разрешений заняты (буфер полон), следующая горютина блокируется на записи, пока кто-то не освободит место. Так одновременно работает не больше N .

```
func main() {
    sem := make(chan struct{}, 3) // семафор на 3 одновременные операции
    var wg sync.WaitGroup

    for _, task := range tasks {
        wg.Add(1)
        go func(t Task) {
            defer wg.Done()
            sem <- struct{}{} // взять разрешение (блокируется, если 3 заняты)
            defer func() { <-sem }() // вернуть разрешение по завершении
            process(t)
        }(task)
    }
    wg.Wait()
}
```

Почему `chan struct{}`? Потому что нам важен сам факт занятости слота, а не значение — а пустая структура `struct{}` занимает 0 байт (помнишь из главы 1?). Идеально для «разрешений», которые ничего не несут.



Семафора vs Worker Pool — оба ограничивают параллелизм, в чём разница? Worker pool — это N **долгоживущих** горутин, разбирающих задачи из канала (хорошо, когда задач много и они однотипные). Семафора — это разрешение на запуск, при котором на каждую задачу заводится **своя** горутина, но одновременно работает не больше N (проще, когда задачи разнородны или их немного). Выбор по вкусу и ситуации.

3.8. Race condition (состояние гонки)

Мы много раз упоминали гонки — теперь разберём систематически. Это фундаментальная тема безопасности конкурентного кода.

Что такое гонка

Race condition (состояние гонки) возникает, когда две или более горутины **одновременно** обращаются к одним данным, и **хотя бы одна** из них **пишет**, без синхронизации. Результат становится непредсказуемым — зависит от того, кто кого «обгонит».

Мы уже видели пример в главе 1 — конкурентная запись в map. Вот пример с обычной переменной:

```
var counter int
for i := 0; i < 1000; i++ {
    go func() {
        counter++ // ГОНКА: читают-прибавляют-пишут одновременно
    }()
}
```

Как обсуждали в разделе про `atomic`, `counter++` — это три операции (прочитать, +1, записать). Две горутины могут прочитать одно значение, обе прибавить, обе записать — одно увеличение теряется. Итоговый `counter` окажется меньше 1000, и каждый запуск даст разное число.

Гонка с `map` — паника

Особый случай из задачника: гонка при записи в `map` — это не просто «неверный результат», а **фатальная паника** рантайма:

```
x := make(map[int]int, 1)
go func() { x[1] = 2 }()
go func() { x[1] = 7 }()
go func() { x[1] = 10 }()
time.Sleep(100 * time.Millisecond)
fmt.Println(x[1])
```

При `GOMAXPROCS != 1` этот код упадёт с `fatal error: concurrent map writes`. Рантайм **специально** детектирует конкурентную запись в `map` и роняет программу (мы разбирали почему в главе 1: повреждённая хеш-таблица опаснее краха). Решение — синхронизация (мьютекс), как показано в главе 1.

Детектор гонок

Go даёт мощнейший инструмент — **race detector**. Запусти программу или тесты с флагом `-race`:

```
go run -race main.go
go test -race ./...
```

Детектор инструментирует обращения к памяти и **во время выполнения** ловит гонки, печатая подробный отчёт: какая горутина, к какой переменной, из какого места кода обращалась конкурентно. Это обязательный инструмент — гоняй тесты с `-race` в CI. Важно понимать: детектор находит только те гонки, которые **реально произошли** во время конкретного запуска, — поэтому нужны тесты, которые провоцируют конкурентный доступ.

Как предотвращать гонки

Собрав всю главу, перечислим арсенал (это прямой вопрос задачника «какие знаешь средства для предотвращения race condition»):

1. **Мьютекс / `RWMutex`** — защитить критическую секцию.
2. **`atomic`** — для простых операций над числом/указателем.
3. **Каналы** — передавать данные вместо разделения (философия «share by communicating»: если данными владеет одна горутина, гонки в принципе нет).
4. **Шардирование** — разнести данные так, чтобы горутины работали с разными участками (глава 1).
5. **Immutability** — если данные не меняются после создания, их можно свободно читать из многих горутин без синхронизации.

3.9. Deadlock и Livelock

Два способа, которыми конкурентная программа может «застрять». Разберём оба.

Deadlock (взаимоблокировка)

Deadlock — ситуация, когда горютины навсегда заблокированы, ожидая друг друга (или ожидая события, которое никогда не наступит). Мы уже видели два случая: `range` по незакрытому каналу и небуферизированный канал в `select` — оба падают с `all goroutines are asleep - deadlock!`.

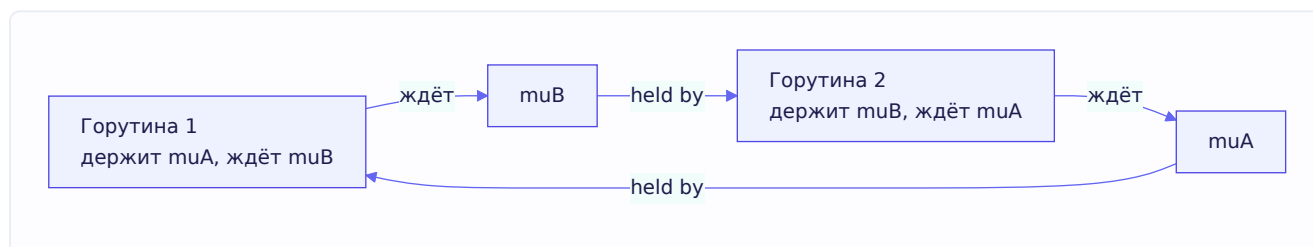
Классический «учебный» deadlock — двойной захват мьютексов в разном порядке:

```
var muA, muB sync.Mutex

// Горутина 1:
go func() {
    muA.Lock()
    muB.Lock() // ждёт muB...
    // ...
}()

// Горутина 2:
go func() {
    muB.Lock()
    muA.Lock() // ждёт muA...
    // ...
}()
```

Гонка сценариев: горутина 1 захватила `muA` и ждёт `muB`; горутина 2 захватила `muB` и ждёт `muA`. Каждая держит то, что нужно другой, и ждёт то, что держит другая. Ни одна не сдвинется — вечная блокировка.



Условия возникновения deadlock (классические четыре условия Коффмана — полезно знать): взаимное исключение, удержание с ожиданием, отсутствие вытеснения, циклическое ожидание. Убери любое — deadlock невозможен.

Главный практический способ избежать deadlock на мьютексах — **всегда захватывать блокировки в одном и том же порядке** во всех горютинах. Если бы обе горютины выше брали сначала `muA`, потом `muB` — цикла ожидания не возникло бы.

Livlock (активная блокировка)

Livlock — более коварный родственник. Горутины **не** заблокированы (они активно работают, тратят CPU), но **не могут продвинуться вперёд**, потому что постоянно реагируют друг на друга, меняя состояние, но не завершая работу.

Житейская аналогия: два человека в узком коридоре пытаются разойтись. Оба шагают влево — снова напротив. Оба вправо — снова напротив. Они двигаются (не «заблокированы»), но пройти не могут. Это livelock.

В коде livelock часто возникает при наивных попытках избежать deadlock: горутина берёт замок, видит, что второй занят, **отпускает** первый, ждёт, пробует снова — и если все делают так синхронно, они бесконечно «уступают» друг другу, не продвигаясь. Отличие от deadlock: при deadlock всё **стоит** (горутины спят), при livelock всё **крутится** впустую (горутины активны, но без прогресса). Livelock труднее заметить — программа выглядит «работающей», грузит CPU, но полезной работы нет.

Лечится обычно внесением **асимметрии** — например, случайных задержек перед повторной попыткой (как в сетевых протоколах: случайный backoff), чтобы горутины перестали синхронно копировать поведение друг друга.

3.10. Итерационные переменные (loop variable capture)

Возвращаемся к ловушке, которую отложили в 3.1. Это, пожалуй, **самая знаменитая** ошибка в Go — её обязательно проверяют на собеседовании, и она есть в задачнике в нескольких вариациях.

Суть проблемы (до Go 1.22)

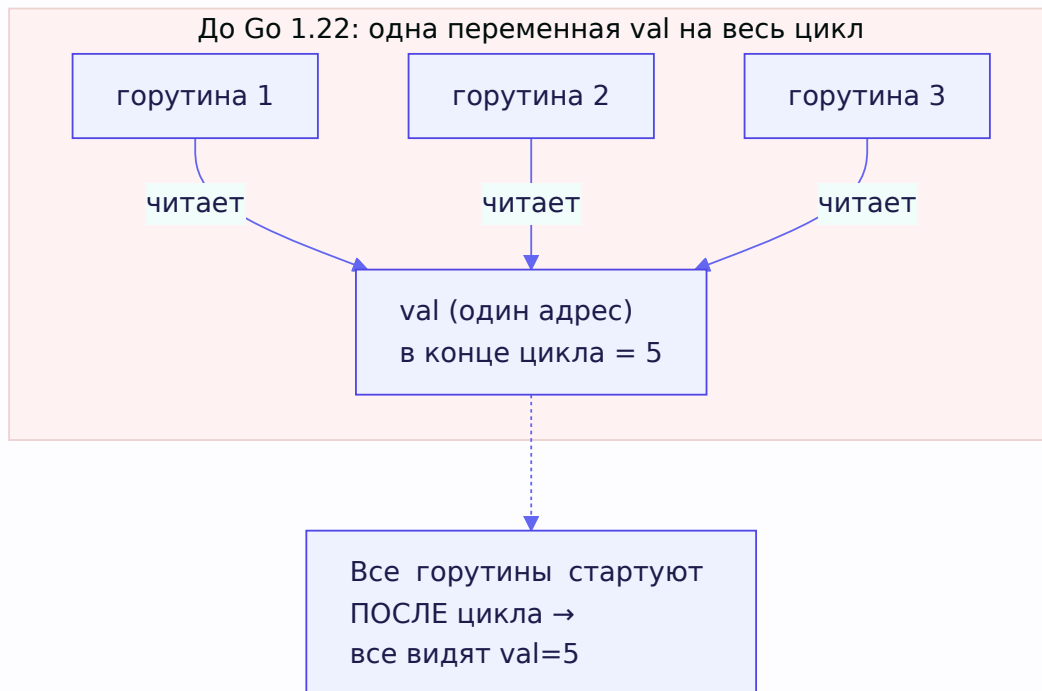
```
func main() {
    values := []int{1, 2, 3, 4, 5}
    for _, val := range values {
        go func() {
            fmt.Println(val)
        }()
    }
    time.Sleep(100 * time.Millisecond)
}
```

Что выведет этот код **в Go до версии 1.22**? Не **1 2 3 4 5** (в каком-то порядке), как можно подумать, а **5 5 5 5 5** — пять раз последнее значение!

Почему? В старых версиях Go переменная цикла **val** — **одна на весь цикл**. На каждой итерации в неё лишь **записывается** новое значение, но это та же самая переменная, тот же адрес в памяти. Горутины создаются через замыкание (см. 3.11), и все они **захватывают одну и ту же переменную val** — по ссылке, не копию.

К моменту, когда горутины реально начинают исполняться (а это происходит после того, как цикл уже пронёсся — вспомни, **main** не ждёт горутин, они стартуют с задержкой), цикл давно

закончился, и в общей переменной `val` лежит **последнее** значение — 5. Все горутины читают эту переменную и видят 5.



Два классических исправления

Способ 1 — копия переменной внутри цикла:

```
for _, val := range values {
    val := val // создаём НОВУЮ переменную на каждой итерации
    go func() {
        fmt.Println(val)
    }()
}
```

Строка `val := val` создаёт **новую** локальную переменную на каждой итерации, со своим адресом. Каждая горутина захватывает свою копию — и видит правильное значение. Выглядит странно (`val := val`), но это идиома.

Способ 2 — передать как аргумент:

```
for _, val := range values {
    go func(v int) {
        fmt.Println(v)
    }(val) // передаём val в горутины как аргумент
}
```

Здесь `val` передаётся **аргументом** — а аргументы, как мы знаем из главы 1, **копируются по значению**. Каждая горутина получает свою копию `v`. Этот способ часто считают чище: явно видно, что передаётся в горутины.

Что изменилось в Go 1.22

В Go 1.22 (2024) семантику **изменили**: теперь переменная цикла — **новая на каждой итерации** по умолчанию. Тот же код без исправлений в Go 1.22+ выведет `1 2 3 4 5` — проблема устранена на уровне языка.

Но знать про эту ловушку **обязательно**, потому что: - Много кода написано на старых версиях. - На собеседовании прямо спрашивают, понимаешь ли ты механику (и историю). - Хороший ответ: «В Go 1.22 это починили — переменная цикла теперь своя на каждой итерации. В более старых версиях был баг с захватом общей переменной, и его чинили копией `val := val` или передачей аргументом».

Отдельно отметить связку: в самой первой задаче главы (`go func(){ fmt.Println(i) }()` в цикле) были **две** проблемы — и «main не ждёт горутин», и этот захват переменной. Полностью правильное решение объединяет `WaitGroup` (дождаться) и корректную передачу переменной (правильные значения).

3.11. Замыкания (closures)

Раз захват переменных всплыл дважды, разберём механизм под ним — замыкания. Это тоже вопрос из задачника.

Что такое замыкание

Замыкание (closure) — это функция, которая **ссылается на переменные из внешней (охватывающей) области видимости**. Функция «замыкается» вокруг этих переменных — она их «помнит» и может читать и менять, даже когда внешняя функция уже завершилась.

Определение из задачника: замыкание — функция, которая ссылается на переменные вне её тела; она имеет доступ к этим переменным и может присваивать им значения.

Классический пример — аккумулятор

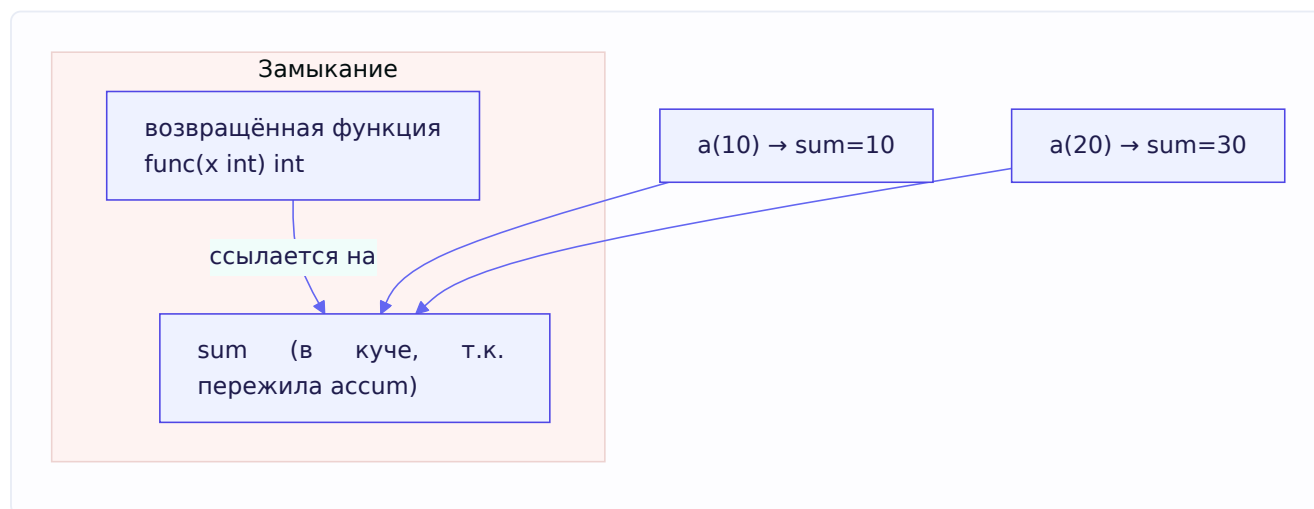
```
func accum() func(int) int {
    sum := 0 // переменная внешней функции
    return func(x int) int { // возвращаем функцию, замыкающую sum
        sum += x // замыкание меняет sum
        return sum
    }
}

// использование:
a := accum()
fmt.Println(a(10)) // 10
fmt.Println(a(20)) // 30 — sum "запомнилась" между вызовами!
fmt.Println(a(5))  // 35
```


Разберём, что происходит. Функция `accum` создаёт переменную `sum` и возвращает **другую функцию**, которая эту `sum` использует. Хотя `accum` уже завершилась после `a := accum()`, переменная `sum` **не исчезла** — возвращённое замыкание продолжает на неё ссылаться. Каждый вызов `a(...)` меняет ту же `sum`, поэтому она накапливается: $10 \rightarrow 30 \rightarrow 35$.

Связь с escape analysis

Помнишь escape analysis из главы 2? Вот прямая связь. Переменная `sum` объявлена локально в `accum`, но она **переживает** свою функцию (на неё ссылается возвращённое замыкание). Значит, `sum` не может жить на стеке (кадр `accum` снимется) — компилятор **отправляет её в кучу**. Замыкание, захватывающее переменную, — одна из типичных причин escape, о которых мы говорили. Красиво, как темы связываются.



Замыкания и ловушка захвата

Теперь понятно, откуда росла проблема из 3.10. Горутина `go func(){ fmt.Println(val) }()` — это замыкание, захватывающее `val`. Оно захватывает **переменную** (ссылку), а не её текущее значение. Поэтому когда несколько замыканий захватывают одну переменную, они все видят её **финальное** значение. Способы исправления из 3.10 — это способы заставить каждое замыкание захватить **свою** переменную (копию).

Полезное правило: замыкание захватывает переменные **по ссылке**, а не по значению. Если нужно «заморозить» текущее значение — передавай его копией (аргументом или через `x := x`).

Итоги главы

Большая глава про то, как заставить горуты работать вместе правильно. Ключевое:

Философия: конкурентность — про дизайн (разбиение на независимые задачи), параллелизм — про исполнение. «Share memory by communicating» — предпочитай передачу данных по каналам разделению памяти под мьютексом, где это уместно.

Каналы: внутри — `hchan` с буфером, очередями ожидающих горутин (`recvq/sendq`) и мьютексом. Небуферизированные — синхронная встреча, буферизированные — почтовый ящик.

Помни таблицу поведения: чтение из закрытого → zero value, запись в закрытый → паника, любая операция с nil → блокировка навсегда. Закрывает отправитель. `range` по каналу требует close, иначе deadlock.

select — switch для каналов, случайный выбор среди готовых; nil-канал отключает ветку.

Синхронизация: Mutex (внутри — state с битовыми флагами + sema, два режима normal/starvation), RWMutex (много читателей, но не всегда быстрее), WaitGroup (дождаться группы, Add до go), Once (ровно один раз), atomic (быстрые операции без блокировок, CAS).

Context — распространение отмены и таймаутов по дереву вызовов через закрытие канала `Done()`.

Паттерны: генератор (поток значений), pipeline (цепочка стадий), fan-out (много консьюмеров на канал), fan-in/merge (свести каналы в один), worker pool (N воркеров, ограничение параллелизма), семафора (буфер-канал как счётчик разрешений).

Опасности: race condition (лови `-race`, предотвращай мьютексом/atomic/каналами), deadlock (единый порядок захвата замков), livelock (крутится без прогресса, лечится асимметрией), захват переменной цикла (до 1.22 — `5 5 5 5 5`, чинится копией/аргументом; в 1.22 исправлено), замыкания (захватывают по ссылке, переживают функцию → escape в кучу).

В следующей главе переходим от конкурентности к дизайну кода: интерфейсы под капотом (iface/eface), nil interface trap, и принципы проектирования — SOLID, DI, паттерны.

Глава 4. Интерфейсы и дизайн

Введение к главе

Интерфейсы — это то, что делает Go-код гибким и тестируемым. Но за простым синтаксисом скрывается механика, которую многие не понимают, — и именно она порождает коварнейшую ловушку языка (`nil interface trap`), на которой спотыкаются даже опытные разработчики.

Эта глава состоит из двух пластов: 1. **Как интерфейсы устроены** — `iface`, `eface`, таблицы методов, и почему из этого вытекает `nil interface trap`. Это глубокая механика, которую спрашивают на грейд повыше. 2. **Как проектировать** — композиция, SOLID, DI, паттерны проектирования, дженерики. Это про инженерное мышление, которое отличает джуна от джуна+.

Начнём с фундаментального вопроса: что вообще такое интерфейс в Go?

4.1. Интерфейсы под капотом: `iface` и `eface`

Что такое интерфейс с точки зрения программиста

Интерфейс — это набор сигнатур методов. Тип «удовлетворяет» интерфейсу, если у него есть все методы из этого набора. Определение из задачника: интерфейс — это абстракция поведения, описание методов, которые должны быть у типа.

```
type Writer interface {  
    Write([]byte) (int, error)  
}
```

Любой тип, у которого есть метод `Write([]byte) (int, error)`, автоматически удовлетворяет `Writer`. Ключевое слово — **автоматически**.

Duck typing — неявное удовлетворение

Вот принципиальное отличие Go от классических ООП-языков (Java, C#), и его любят спрашивать. В Java ты должен **явно** написать `class MyType implements Writer`. В Go — **ничего писать не нужно**. Если у типа есть нужные методы — он удовлетворяет интерфейсу, и точка.

Это называется **duck typing** (утиная типизация): «Если что-то ходит как утка и крикает как утка — значит, это утка». Go не требует объявлять родство — он смотрит на поведение (методы).

```

type MyBuffer struct{}
func (b *MyBuffer) Write(p []byte) (int, error) { return len(p), nil }

// Нигде не написано "implements Writer"!
// Но MyBuffer удовлетворяет Writer, потому что имеет метод Write:
var w Writer = &MyBuffer{} // работает

```

Почему это важно: **слабая связанность**. Твой тип не «знает» про интерфейс. Можно определить интерфейс *после* типа, в другом пакете, под свои нужды. В Go принято определять интерфейсы **на стороне потребителя** («принимай интерфейсы, возвращай структуры»), а не навязывать их производителю. К этому вернёмся в разделе про дизайн.

Внутреннее устройство: два вида интерфейсов

А теперь — под капот. Внутри рантайма интерфейсная переменная — это **структура из двух указателей**. Но есть два варианта этой структуры, в зависимости от того, есть ли у интерфейса методы.

eface — пустой интерфейс (`interface{}` / `any`). Для интерфейса без методов используется `eface` (empty interface):

```

type eface struct {
    _type *_type // указатель на информацию о типе
    data unsafe.Pointer // указатель на сами данные
}

```

Два поля: - `_type` — указатель на метаданные типа: какой конкретный тип лежит внутри (int? string? User?), его размер, как его сравнивать и т.д. - `data`* — указатель на само значение (данные лежат где-то в памяти, здесь адрес).

iface — интерфейс с методами. Для интерфейса, у которого есть методы (`Writer`, `error`, etc.), используется `iface`:

```

type iface struct {
    tab *itab // указатель на таблицу методов (itable)
    data unsafe.Pointer // указатель на сами данные
}

```

Тоже два поля, но первое — сложнее: - `tab` — указатель на `itab` (interface table, таблица интерфейса). Это ключевая структура, разберём ниже. - `data` — как и в `eface`, указатель на само значение.

itab — таблица методов

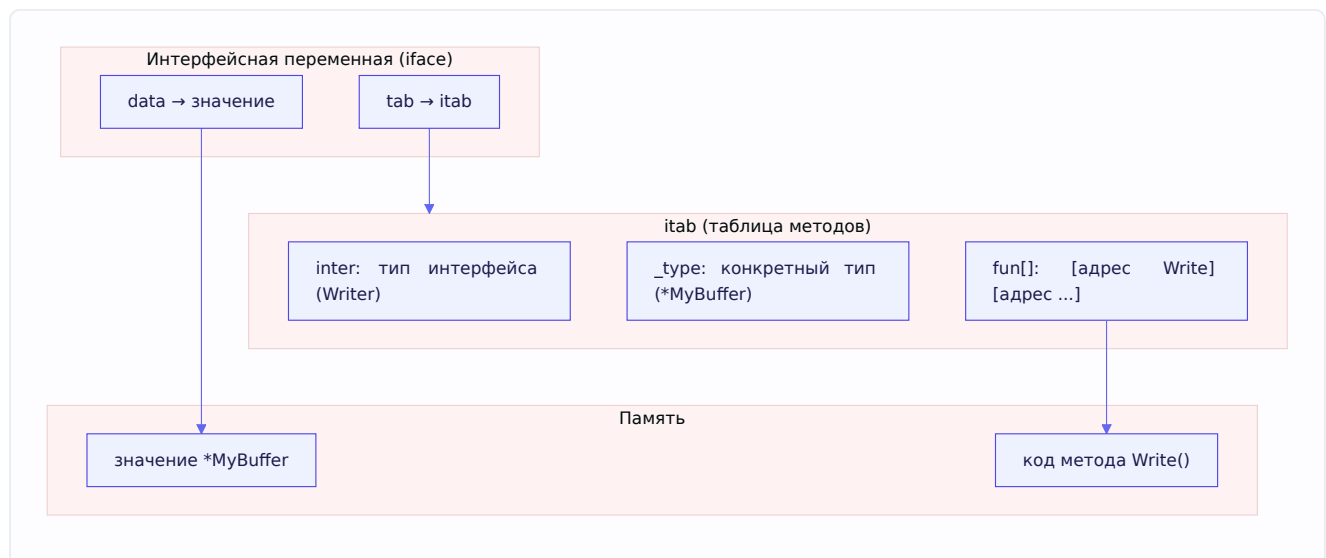
`itab` — сердце интерфейса с методами. Она связывает **конкретный тип** с **интерфейсом** и хранит адреса методов:

```

type itab struct {
    inter *interfacetype // какой это интерфейс (напр. Writer)
    _type *_type          // какой конкретный тип внутри (напр. *MyBuffer)
    fun   [1]uintptr      // МАССИВ УКАЗАТЕЛЕЙ НА МЕТОДЫ (реально переменной длины)
    // ... хеш и прочее
}

```

Самое важное поле — **fun** — массив указателей на реализации методов конкретного типа. Когда ты вызываешь метод через интерфейс, рантайм смотрит в **fun**, находит адрес нужного метода и прыгает туда. Это называется **динамическая диспетчеризация** — решение о том, какой код вызвать, принимается в рантайме, а не на этапе компиляции.



Читается так: интерфейсная переменная держит два указателя — на таблицу типа (что за тип и где его методы) и на сами данные. Вызов метода = найти адрес в таблице → прыгнуть по нему.

Ключевой вывод: интерфейс — это пара (тип, значение)

Запомни эту формулу, она объясняет всё дальнейшее, включая главную ловушку:

Интерфейсное значение — это пара из двух частей: информация о типе (**tab / **_type**) и указатель на данные (**data**).**

Интерфейс «пуст» (равен **nil**) **только когда обе части — nil**: и тип, и данные. Это и есть корень nil interface trap, к которому переходим.

Задача из задачника: type assertion между интерфейсами

Разберём задачу из задачника, которая проверяет понимание устройства:

```

type Foo struct{}
func (f *Foo) A() {}
func (f *Foo) B() {}
func (f *Foo) C() {}

type AB interface { A(); B() }
type BC interface { B(); C() }

func main() {
    var f AB = &Foo{}
    y := f.(BC) // работает ли этот type-assertion?
    y.A()       // а этот вызов?
    _ = y
}

```

Разберём по пунктам: - `y := f.(BC)` **сработает**. Хотя `f` имеет статический тип `AB`, внутри него лежит `*Foo`, а `y` `*Foo` есть методы `B()` и `C()` — то есть он удовлетворяет `BC`. Type assertion проверяет **конкретный тип внутри** интерфейса (через `itab`), а `*Foo` реализует и `AB`, и `BC`. Assertion успешен, `y` получает тип `BC`. - `y.A()` **НЕ скомпилируется**. У интерфейса `BC` есть только методы `B()` и `C()`. Метода `A()` в его наборе нет — компилятор не даст вызвать `A()` через переменную типа `BC`, даже если реальный `*Foo` под ним умеет `A()`. Тип переменной ограничивает доступные методы тем, что объявлено в интерфейсе.

Это показывает разницу: то, что **лежит** в интерфейсе (конкретный тип со всеми методами), и то, что **видно** через интерфейс (только методы объявленного интерфейса), — разные вещи.

4.2. Nil interface trap — главная ловушка

Вот она — одна из самых коварных ошибок в Go, на которой попадают даже опытные. Понять её можно **только** зная устройство интерфейса из 4.1. Разберём медленно.

Демонстрация проблемы

```

type MyError struct{}
func (e *MyError) Error() string { return "что-то сломалось" }

func doSomething() error {
    var err *MyError = nil // конкретный указатель, равен nil
    return err             // возвращаем его как error
}

func main() {
    err := doSomething()
    if err != nil {
        fmt.Println("ошибка!", err) // ЭТО ВЫПОЛНИТСЯ! Хотя err "как бы nil"
    } else {
        fmt.Println("всё ок")
    }
}

```

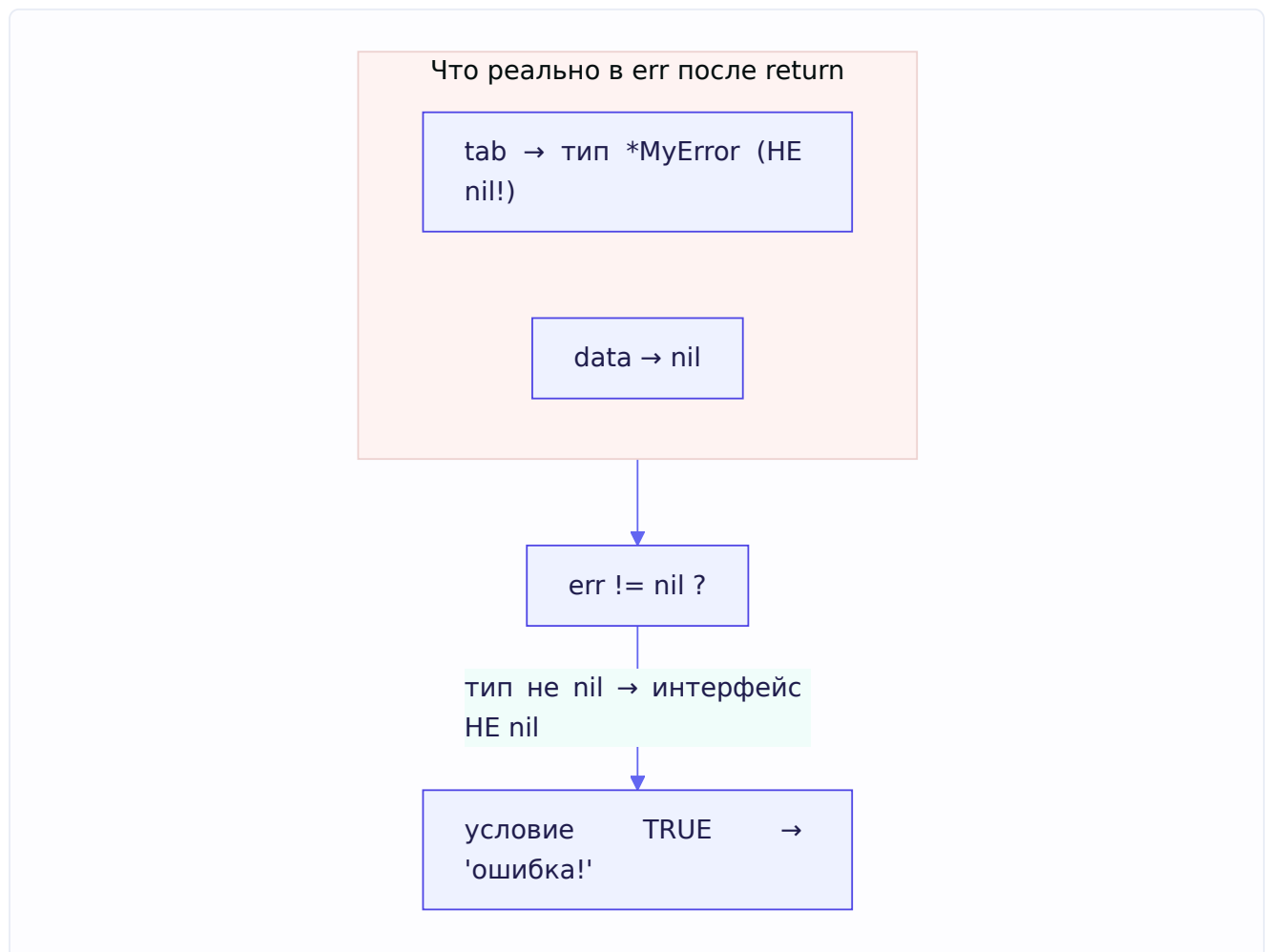
Программа выведет «**ошибка!**» — хотя интуитивно мы вернули `nil`, и ждём «всё ок». Это шокирует новичков. Почему так?

Объяснение через устройство интерфейса

Вспомни формулу: **интерфейс = пара (тип, данные)**, и он равен `nil` только если **обе** части `nil`.

Разберём по шагам, что происходит:

1. Внутри `doSomething` переменная `err` имеет тип `*MyError` и значение `nil`. Это **нетипизированный** `nil`-указатель конкретного типа.
2. Когда мы возвращаем `err` как `error` (интерфейс), Go **упаковывает** его в интерфейсное значение. И вот ключевое: в пару кладётся:
3. **тип** = `*MyError` (тип-то известен! он не `nil`),
4. **данные** = `nil` (значение указателя действительно `nil`).
5. Получается интерфейс, у которого **тип НЕ `nil`** (`*MyError`), а данные — `nil`.
6. Проверка `err != nil` сравнивает интерфейс с `nil`. Но интерфейс равен `nil` только когда **обе** части `nil`. Здесь тип = `*MyError` $\neq$ `nil`. Значит, **интерфейс НЕ равен `nil`!**



Вот в чём подвох: `nil`-указатель `*MyError`, упакованный в интерфейс, даёт **не-`nil` интерфейс**, потому что информация о типе присутствует. «Пустой указатель конкретного типа» и «пустой

интерфейс» — разные вещи.

Как избежать

Правильный способ — возвращать **буквальный** `nil`, а не `nil`-переменную конкретного типа:

```
func doSomething() error {
    // если ошибки нет — возвращаем именно nil (не типизированный указатель):
    return nil // тип интерфейса тоже nil → интерфейс истинно nil
}
```

А если логика сложнее — не объявляй промежуточную переменную конкретного типа, работай сразу с `error`:

```
func doSomething() error {
    var err error // тип error, а не *MyError!
    if somethingWrong {
        err = &MyError{}
    }
    return err // если ошибки не было, err истинно nil
}
```

Разница критична: `var err *MyError` создаёт типизированную переменную, которая при упаковке в интерфейс «протащит» свой тип. `var err error` — это уже интерфейс, и если в него ничего не положили, он истинно `nil`.

Как объяснять на собеседовании: «Интерфейс в Go — это пара (тип, данные). Он равен `nil`, только если обе части `nil`. Если положить в интерфейс `nil`-указатель конкретного типа, интерфейс получит непустую информацию о типе и перестанет быть равным `nil` — хотя данные внутри `nil`. Поэтому нельзя возвращать типизированный `nil`-указатель как `error`; надо возвращать буквальный `nil` или работать с переменной интерфейсного типа».

Эта ловушка — прямое следствие того, что мы разобрали в 4.1. Без понимания «пары (тип, данные)» она выглядит магией; с пониманием — очевидна.

4.3. Type assertion и type switch

Мы уже касались `type assertion` в главе 1 (в сравнении с `conversion`) и в задаче из 4.1. Теперь — систематически.

Type assertion — извлечение конкретного типа

Type assertion извлекает конкретное значение из интерфейса. Синтаксис — `value.(T)`:

```
var i interface{} = "привет"
s := i.(string) // извлекаем string
```


Две формы, и разница между ними важна:

Однозначная форма — паникует при несовпадении:

```
s := i.(string) // если внутри не string → ПАНИКА
```

Форма «comma, ok» — безопасная, не паникует:

```
s, ok := i.(string)
if ok {
    // внутри действительно string, s — валидна
} else {
    // не string, s — нулевое значение, но паники нет
}
```

Правило: если ты **не уверен** в типе — всегда используй форму с `ok`. Однозначную — только когда абсолютно уверен (иначе паника уронит программу).

Механика через itab

Как assertion работает под капотом? Вспомни itab из 4.1 — там хранится информация о конкретном типе. Assertion `i.(string)` сравнивает тип, записанный в интерфейсе (в `tab / _type`), с запрошенным типом `string`. Если совпадает — возвращает данные; если нет — паника или `ok=false`. Это быстрая проверка указателей на типы, не «глубокое» сравнение.

Type switch — множественная проверка

Когда нужно проверить интерфейс на **несколько** возможных типов, используют **type switch**:

```
func describe(i interface{}) string {
    switch v := i.(type) {
    case int:
        return fmt.Sprintf("целое число: %d", v)
    case string:
        return fmt.Sprintf("строка длины %d", len(v))
    case bool:
        return fmt.Sprintf("булево: %t", v)
    case nil:
        return "ничего (nil)"
    default:
        return fmt.Sprintf("неизвестный тип: %T", v)
    }
}
```

Синтаксис `i.(type)` работает **только** внутри `switch`. В каждом `case` переменная `v` автоматически получает соответствующий конкретный тип — в ветке `int` переменная `v` имеет тип `int`, в ветке `string` — `string`. Это удобно и безопасно: не нужно писать отдельные assertion'ы с проверкой `ok`.

Где применяется

Type assertion и type switch нужны, когда работаешь с `interface{} / any` и нужно «распаковать» конкретный тип: обработка JSON (`map[string]interface{}`, глава про JSON), обработка разнородных данных, реализация функций вроде `fmt.Println` (которая через type switch решает, как печатать разные типы). Но важно: **частая нужда в type switch — сигнал, что дизайн можно улучшить**. Если ты постоянно «распаковываешь» типы, возможно, стоило использовать полиморфизм через методы интерфейса. Type switch — инструмент для конкретных задач (сериализация, роутинг по типу), а не замена нормальному ООП-дизайну.

Задача из задачника: assertion с указателем

Вспомним задачу «про звёздочку» из задачника (мы её упоминали в главе 1):

```
func (s *Storage) Set(wh *warehouse.Warehouse) {
    s.cache.Put(wh.Id, *wh) // кладём ЗНАЧЕНИЕ (разыменовали!)
}

func (s *Storage) Get(id types.WarehouseId) *warehouse.Warehouse {
    item, ok := s.cache.Get(id)
    if ok {
        if wh, ok := item.(*warehouse.Warehouse); ok { // достаём УКАЗАТЕЛЬ
            return wh
        }
    }
    return nil
}
```

Баг: в `Set` в кеш кладётся **значение** `warehouse.Warehouse` (через `*wh` разыменованное), а в `Get` делается assertion на **указатель** `*warehouse.Warehouse`. Тип в кеше (`warehouse.Warehouse`) не совпадает с запрошенным (`*warehouse.Warehouse`) — assertion всегда возвращает `ok=false`, `Get` всегда возвращает `nil`, и кеш фактически **не работает** (всегда «промах»). Именно поэтому в задаче кеш не разгрузил хранилище. Урок: тип при assertion должен **точно** совпадать с тем, что положили, — значение и указатель на него это **разные** типы в itab.

4.4. Композиция vs наследование

Переходим от механики к дизайну. И начинаем с фундаментального выбора, который Go сделал за нас: **в Go нет наследования**. Совсем. Вместо него — композиция. Разберём, что это значит и почему так.

Наследования нет — и это осознанное решение

В классических ООП-языках (Java, C++, Python) есть наследование: класс `Dog` наследует от класса `Animal`, получая его поля и методы, и может их переопределять. Строятся иерархии: `Animal → Mammal → Dog → Puppy`.

В Go этого нет. Нельзя написать «тип B наследует от типа A». Создатели Go считали, что наследование порождает больше проблем, чем решает: - **Хрупкие иерархии**. Изменение

базового класса ломает всех потомков непредсказуемым образом (проблема «хрупкого базового класса»). - **Жёсткая связанность**. Потомок намертво привязан к родителю, тащит всю его реализацию, даже ненужную. - **Проблема «банан-горилла-джунгли»**. Хотел банан — но через наследование получил гориллу, которая его держит, и все джунгли впридачу. То есть наследуя один нужный метод, тащишь весь класс.

Вместо наследования Go предлагает два механизма: **встраивание (embedding)** для переиспользования кода и **интерфейсы** для полиморфизма.

Встраивание (embedding) — переиспользование через композицию

Встраивание — это включение одного типа в другой без имени поля. Встроенный тип «поднимает» свои поля и методы во внешний тип.

```
type Animal struct {
    Name string
}

func (a Animal) Eat() string { return a.Name + " ест" }

type Dog struct {
    Animal // ВСТРАИВАНИЕ (нет имени поля — только тип)
    Breed string
}

func main() {
    d := Dog{
        Animal: Animal{Name: "Рекс"},
        Breed:  "овчарка",
    }
    fmt.Println(d.Name)    // "Рекс" — поле Animal доступно напрямую!
    fmt.Println(d.Eat())   // "Рекс ест" — метод Animal тоже поднялся!
}
```

Обрати внимание: `d.Name` и `d.Eat()` работают, хотя `Name` и `Eat` определены в `Animal`. Встраивание «продвинуло» их в `Dog`. Выглядит похоже на наследование — но это **не оно**. Разница фундаментальна.

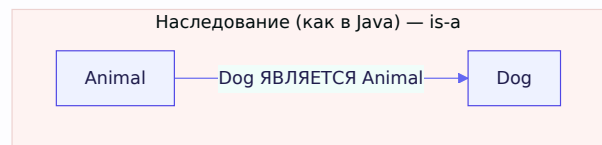
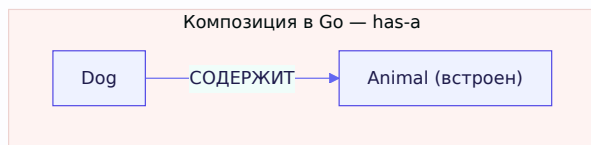
Чем встраивание отличается от наследования

Это важно понять, чтобы не путать понятия:

1. Нет полиморфизма подтипов через встраивание. `Dog` — это **не** `Animal`. Нельзя передать `Dog` туда, где ждут `Animal`. При наследовании `Dog` был бы подтипом `Animal`. В Go для полиморфизма используются **интерфейсы**, а не встраивание.

2. Это композиция «has-a», а не «is-a». `Dog` **содержит** `Animal` (has-a), а не **является** `Animal` (is-a). Встроенный тип — это как поле, просто без имени. Можно обращаться и явно: `d.Animal.Eat()`.

3. Нет переопределения в классическом смысле. Если у `Dog` определить свой метод `Eat()`, он «затенит» встроенный — но это не виртуальный вызов. Метод `Animal`, вызывающий `Eat()` внутри себя, вызовет **свой** `Eat`, а не `Dog`-овский (нет динамического диспетчинга между встроенным и внешним типом, как при наследовании).



Как получить полиморфизм — через интерфейсы

Раз встраивание не даёт полиморфизма, как в Go написать «функцию, работающую с разными типами единообразно»? Через **интерфейсы**:

```
type Eater interface {
    Eat() string
}

func Feed(e Eater) { // принимает ЛЮБОЙ тип с методом Eat()
    fmt.Println(e.Eat())
}

// И Dog, и Cat, и кто угодно с Eat() подойдёт:
Feed(Dog{Animal: Animal{Name: "Рекс"}})
Feed(Cat{Name: "Мурка"})
```

Вот полное разделение обязанностей в Go-дизайне: - **Встраивание** — переиспользовать реализацию (не писать код дважды). - **Интерфейсы** — полиморфизм (работать с разными типами единообразно).

В ООП-языках наследование пыталось делать **оба** дела сразу — и переиспользование, и полиморфизм, — отсюда путаница и жёсткость. Go разделил их на два чистых механизма. Это и есть знаменитый принцип «composition over inheritance» (композиция вместо наследования), доведённый до логического конца.

Встраивание интерфейсов

Встраивать можно не только структуры, но и интерфейсы — так собирают большие интерфейсы из маленьких. Классика из стандартной библиотеки:

```
type Reader interface { Read(p []byte) (n int, err error) }
type Writer interface { Write(p []byte) (n int, err error) }

type ReadWriter interface { // составлен встраиванием
    Reader
    Writer
}
```

ReadWriter автоматически требует и **Read**, и **Write**. Это чистый и переиспользуемый способ строить интерфейсы.

4.5. SOLID в Go

SOLID — пять принципов проектирования. Их придумали для классического ООП, но они прекрасно ложатся на Go — просто выражаются иначе (через интерфейсы и композицию, а не наследование). Разберём каждый с Go-примером.

S — Single Responsibility (единственная ответственность)

У каждого типа/функции должна быть одна причина для изменения — то есть одна ответственность.

В Go это проявляется в маленьких, сфокусированных типах и функциях. Плохо, когда структура `User` сама себя валидирует, сохраняет в БД, отправляет email и рендерит в JSON — это четыре разные ответственности. Лучше разнести: `UserValidator`, `UserRepository`, `EmailSender`, отдельная сериализация.

Признак нарушения: тип, который меняется по многим несвязанным причинам. Если «добавили новое поле в БД» и «сменили шаблон письма» оба трогают один тип — ответственности смешаны.

O — Open/Closed (открыт для расширения, закрыт для изменения)

Поведение можно расширять, не меняя существующий код. В Go это достигается через интерфейсы: код зависит от интерфейса, а новые реализации добавляются без правки этого кода.

```
type PaymentMethod interface {
    Pay(amount int) error
}

func Checkout(pm PaymentMethod, amount int) error { // закрыт для изменения
    return pm.Pay(amount)
}

// Добавить новый способ оплаты = новая реализация, БЕЗ правки Checkout:
type CardPayment struct{}
func (c CardPayment) Pay(amount int) error { /* ... */ return nil }

type CryptoPayment struct{} // новый способ – Checkout не трогаем!
func (c CryptoPayment) Pay(amount int) error { /* ... */ return nil }
```

`Checkout` закрыт для изменения (его код не меняется), но открыт для расширения (новые `PaymentMethod` добавляются свободно). Это прямое следствие полиморфизма через интерфейсы.

L — Liskov Substitution (подстановка Лисков)

Любую реализацию интерфейса можно подставить вместо другой, не сломав программу. То есть все реализации должны честно соблюдать «контракт» интерфейса — не только сигнатуры методов, но и ожидаемое поведение.

В Go компилятор гарантирует совпадение сигнатур, но **не** семантику. Пример нарушения: интерфейс `io.Reader` предполагает, что `Read` возвращает прочитанные байты. Если твоя

реализация вместо чтения удаляет файл — сигнатура совпадает, но контракт нарушен, и код, ожидающий нормальный Reader, сломается. Соблюдай не только форму, но и смысл контракта.

I — Interface Segregation (разделение интерфейсов)

Лучше много маленьких специализированных интерфейсов, чем один большой. Клиент не должен зависеть от методов, которые не использует.

Это, пожалуй, **самый Go-шный** принцип. В Go идиома — крошечные интерфейсы, часто на один метод. Стандартная библиотека — эталон: `io.Reader` (один метод `Read`), `io.Writer` (один метод `Write`), `io.Closer` (один `Close`). Их комбинируют по мере надобности (`io.ReadWriter`).

```
// Плохо: жирный интерфейс, реализовать который дорого,  
// и клиенты зависят от всего сразу:  
type BigFileInterface interface {  
    Read(p []byte) (int, error)  
    Write(p []byte) (int, error)  
    Close() error  
    Seek(offset int64, whence int) (int64, error)  
    Truncate(size int64) error  
    Sync() error  
}  
  
// Хорошо: мелкие интерфейсы, клиент берёт только нужное:  
type Reader interface { Read(p []byte) (int, error) }  
type Writer interface { Write(p []byte) (int, error) }
```

Функция, которой нужно только читать, принимает `io.Reader` — и её можно вызвать с чем угодно читаемым (файл, сеть, буфер в памяти). Это невероятно гибко и тестируемо.

Мнемоника Роба Пайка: «The bigger the interface, the weaker the abstraction» — чем больше интерфейс, тем слабее абстракция. Маленькие интерфейсы сильнее.

D — Dependency Inversion (инверсия зависимостей)

Зависеть надо от абстракций, а не от конкретных реализаций. Высокоуровневый код не должен напрямую зависеть от низкоуровневых деталей — оба должны зависеть от интерфейса.

```
// Плохо: сервис намертво привязан к конкретной Postgres-реализации:
type UserService struct {
    db *PostgresDB // конкретный тип — жёсткая зависимость
}

// Хорошо: сервис зависит от интерфейса (абстракции):
type UserRepository interface {
    GetUser(id int) (*User, error)
    SaveUser(u *User) error
}

type UserService struct {
    repo UserRepository // интерфейс — можно подставить любую реализацию
}
```

Теперь `UserService` не знает, где хранятся данные — Postgres, MongoDB, in-memory. Можно подменить хранилище, не трогая сервис. А для тестов — подставить фейковую реализацию (об этом дальше). Инверсия зависимостей — фундамент тестируемого кода, и она напрямую ведёт нас к следующей теме.

4.6. Dependency Injection (внедрение зависимостей)

Dependency Injection (DI, внедрение зависимостей) — это приём, при котором зависимости объекта **передаются ему извне**, а не создаются им самим внутри. Звучит просто, но эффект огромный.

Проблема без DI

```
// БЕЗ DI: сервис сам создаёт свои зависимости:
type UserService struct {
    repo *PostgresRepository
}

func NewUserService() *UserService {
    return &UserService{
        repo: NewPostgresRepository("postgres://..."), // создаёт ВНУТРИ
    }
}
```

Что здесь плохо: - **Жёсткая связанность**. `UserService` намертво привязан к `PostgresRepository`. Хочешь другую БД — правь сам сервис. - **Невозможно тестировать**. Как протестировать `UserService` без реальной Postgres? Никак — он всегда лезет в настоящую БД. Тесты становятся медленными, хрупкими, требуют развёрнутой БД.

Решение через DI

```
// С DI: зависимость передаётся снаружи:
type UserService struct {
    repo UserRepository // ИНТЕРФЕЙС, а не конкретный тип
}

func NewUserService(repo UserRepository) *UserService {
    return &UserService{repo: repo} // получаем готовую зависимость
}
```

Теперь **тот, кто создаёт** `UserService`, решает, какой репозиторий ему дать:

```
// В проде — настоящая Postgres:
service := NewUserService(NewPostgresRepository("postgres://..."))

// В тестах — фейковая реализация:
service := NewUserService(NewMockRepository())
```

Почему это меняет всё — тестируемость

Вот главная выгода. Раз `UserService` зависит от **интерфейса** `UserRepository`, в тесте можно подсунуть **мок** (mock) — поддельную реализацию, которая возвращает заранее заданные данные без всякой БД:

```
type MockRepository struct {
    users map[int]*User
}

func (m *MockRepository) GetUser(id int) (*User, error) {
    return m.users[id], nil // возвращаем из памяти, без БД
}

func (m *MockRepository) SaveUser(u *User) error {
    m.users[u.ID] = u
    return nil
}

// Тест:
func TestUserService(t *testing.T) {
    mock := &MockRepository{users: map[int]*User{
        1: {ID: 1, Name: "Тест"},
    }}
    service := NewUserService(mock) // внедряем мок вместо БД
    // теперь тестируем логику сервиса без реальной базы — быстро и надёжно
}
```

Именно поэтому в Go так любят маленькие интерфейсы: они делают код **тестируемым через подстановку моков**. Интерфейс `UserRepository` — это «шов» (seam), в который можно вставить настоящую реализацию в проде и фейковую в тесте. Связь тем: DI (4.6) опирается на Dependency Inversion (4.5), которая опирается на интерфейсы (4.1). Всё выстраивается в цепочку.

DI в Go — обычно вручную

Важное отличие от Java/C#: там для DI часто используют тяжёлые фреймворки (Spring, контейнеры внедрения). В Go DI обычно делают **вручную** — просто передавая зависимости в конструкторы, как показано выше. Это называют «конструкторным DI». Язык и так удобен для этого, фреймворк не нужен. Есть инструменты вроде `google/wire` для генерации кода связывания в больших проектах, но базовый подход — ручная передача через конструкторы, и для большинства проектов этого достаточно.

Идиома, которую стоит запомнить и озвучить на собеседовании: **«accept interfaces, return structs»** — принимай интерфейсы (гибкость, тестируемость), возвращай конкретные структуры (ясность для вызывающего). Конструктор возвращает `*UserService` (конкретный тип), но принимает `UserRepository` (интерфейс).

4.7. Паттерны: Factory, Repository, Strategy

Паттерны проектирования — типовые решения повторяющихся задач. В Go их реализуют через интерфейсы и функции, часто проще, чем в классическом ООП. Разберём три самых востребованных на бэкенде.

Factory (фабрика)

Factory — паттерн, где создание объекта вынесено в отдельную функцию, скрывающую детали инициализации. Вызывающий говорит «дай мне объект», не зная, как именно он собирается.

В Go роль фабрики обычно играет **функция-конструктор** (по соглашению `NewXxx`):

```
type Storage interface {
    Save(key, value string) error
    Load(key string) (string, error)
}

// Фабрика: по типу возвращает нужную реализацию,
// вызывающий получает интерфейс и не знает деталей:
func NewStorage(kind string) (Storage, error) {
    switch kind {
    case "memory":
        return &MemoryStorage{data: make(map[string]string)}, nil
    case "redis":
        return &RedisStorage{ /* ... */ }, nil
    case "postgres":
        return &PostgresStorage{ /* ... */ }, nil
    default:
        return nil, fmt.Errorf("неизвестный тип хранилища: %s", kind)
    }
}
```

Зачем: вызывающий код работает с интерфейсом `Storage` и не привязан к конкретной реализации. Сменить хранилище — поменять аргумент фабрики. Плюс фабрика инкапсулирует

сложную инициализацию (подключения, настройки), чтобы вызывающему не пришлось знать эти детали.

Repository (репозиторий)

Repository — паттерн, абстрагирующий доступ к хранилищу данных. Он прячет за интерфейсом то, как и где хранятся данные, предоставляя логике простые методы вроде «получить пользователя», «сохранить заказ».

```
type UserRepository interface {
    GetByID(id int) (*User, error)
    Create(u *User) error
    Update(u *User) error
    Delete(id int) error
}

// Реализация для Postgres:
type PostgresUserRepo struct {
    db *sql.DB
}

func (r *PostgresUserRepo) GetByID(id int) (*User, error) {
    // SQL-запрос к базе...
    return user, nil
}
```

Мы уже видели репозиторий в разделах про SOLID и DI — это не случайно. Repository — практическое воплощение Dependency Inversion: бизнес-логика зависит от **интерфейса** репозитория, а не от конкретной БД. Выгоды: - **Тестируемость** — в тестах подставляем мок-репозиторий (как в 4.6). - **Заменяемость** — сменить Postgres на Mongo = новая реализация интерфейса, логика не трогается. - **Разделение ответственности** — бизнес-логика не засорена SQL-запросами.

Repository — один из столпов чистой архитектуры на бэкенде, к которой придём в 4.10.

Strategy (стратегия)

Strategy — паттерн, позволяющий выбрать алгоритм (поведение) в рантайме, подставляя разные реализации одного интерфейса.

```

type SortStrategy interface {
    Sort([]int) []int
}

type QuickSort struct{}
func (QuickSort) Sort(data []int) []int { /* быстрая сортировка */ return data }

type BubbleSort struct{}
func (BubbleSort) Sort(data []int) []int { /* пузырьковая */ return data }

type Sorter struct {
    strategy SortStrategy // текущая стратегия
}

func (s *Sorter) SetStrategy(strategy SortStrategy) {
    s.strategy = strategy // меняем алгоритм на лету
}

func (s *Sorter) Execute(data []int) []int {
    return s.strategy.Sort(data)
}

```

Один и тот же `Sorter` может использовать разные алгоритмы в зависимости от подставленной стратегии — например, быстрый для больших массивов, простой для маленьких. Стратегию можно менять в рантайме.

В Go Strategy часто упрощают до **функционального типа** — раз функции first-class, зачем интерфейс на один метод:

```

type SortFunc func([]int) []int

func Process(data []int, sort SortFunc) []int {
    return sort(data) // передали алгоритм как функцию
}

// использование:
Process(data, quickSort) // одна стратегия
Process(data, bubbleSort) // другая

```

Это идиоматичнее для простых случаев: вместо типа-обёртки с одним методом — просто передаём функцию. Отличный пример того, как Go часто делает паттерны легче классических.

4.8. Functional Options pattern

Это специфичный для Go паттерн, который решает проблему **гибкой конфигурации** объектов. Его любят спрашивать, потому что он показывает владение идиомами языка.

Проблема, которую он решает

Представь конструктор сервера с кучей необязательных настроек: порт, таймауты, максимум соединений, TLS, логгер... Как их задавать?

Плохой вариант 1 — куча параметров:

```
func NewServer(port int, timeout time.Duration, maxConn int, tls bool, ...) *Server
// вызов нечитаем: NewServer(8080, 30*time.Second, 100, true, ...)
// что означает true? какой параметр за что? Легко перепутать порядок.
```

Плохой вариант 2 — структура конфига:

```
func NewServer(cfg Config) *Server
// лучше, но: как отличить "не задано" от "задано нулевое значение"?
// нужно ли заполнять ВСЕ поля? что с дефолтами?
```

Решение — функциональные опции

Идея: конструктор принимает **переменное число функций-опций**, каждая из которых настраивает один аспект. Незаданные опции остаются со значениями по умолчанию.

```

type Server struct {
    port      int
    timeout time.Duration
    maxConn int
}

// Тип опции — функция, модифицирующая Server:
type Option func(*Server)

// Каждая опция — функция, возвращающая Option:
func WithPort(port int) Option {
    return func(s *Server) { s.port = port }
}
func WithTimeout(t time.Duration) Option {
    return func(s *Server) { s.timeout = t }
}
func WithMaxConn(n int) Option {
    return func(s *Server) { s.maxConn = n }
}

// Конструктор принимает переменное число опций:
func NewServer(opts ...Option) *Server {
    // 1. Значения по умолчанию:
    s := &Server{
        port:      8080,
        timeout: 30 * time.Second,
        maxConn: 100,
    }
    // 2. Применяем переданные опции поверх дефолтов:
    for _, opt := range opts {
        opt(s)
    }
    return s
}

```

Теперь использование — чистое и читаемое:

```

// Только дефолты:
s1 := NewServer()

// Настроить порт и таймаут, остальное — по умолчанию:
s2 := NewServer(
    WithPort(9090),
    WithTimeout(60*time.Second),
)

```

Почему это красиво

- **Читаемость.** `WithPort(9090)` самодокументирован — сразу видно, что настраивается. Не перепутаешь порядок, как в варианте с позиционными аргументами.
- **Необязательность.** Задаёшь только то, что нужно; остальное — разумные дефолты.
- **Расширяемость.** Добавить новую опцию = новая функция `WithXxx`, не ломая существующие вызовы (сигнатура `NewServer(opts ...Option)` не меняется). Это принцип Open/Closed в

действии.

- **Валидация.** Опция может возвращать ошибку или паниковать при невалидном значении.

Ты встретишь этот паттерн повсюду в серьёзных библиотеках Go (gRPC, различные клиенты БД). Умение объяснить и написать functional options — хороший сигнал на собеседовании.

4.9. Generics (дженерики)

Дженерики (обобщённое программирование) появились в Go 1.18 (2022) — это было крупнейшее изменение языка за годы. Они позволяют писать код, работающий с **разными типами**, без дублирования и без потери типобезопасности.

Проблема, которую решают дженерики

До дженериков, если ты хотел функцию «максимум из двух», приходилось либо писать её для каждого типа (`MaxInt`, `MaxFloat`, ...), либо использовать `interface{}` с потерей типобезопасности и накладными расходами на упаковку. Оба варианта плохи.

```
// До дженериков – дублирование:
func MaxInt(a, b int) int { if a > b { return a }; return b }
func MaxFloat(a, b float64) float64 { if a > b { return a }; return b }
// ... и так для каждого типа
```

Синтаксис дженериков

Дженерики вводят **параметры типа** в квадратных скобках:

```
func Max[T constraints.Ordered](a, b T) T {
    if a > b {
        return a
    }
    return b
}

// Использование – тип выводится автоматически:
Max(3, 5)           // T = int → 5
Max(2.7, 1.1)       // T = float64 → 2.7
Max("a", "b")       // T = string → "b"
```

Здесь `[T constraints.Ordered]` — параметр типа `T` с **ограничением** `Ordered`. Одна функция работает для всех упорядочиваемых типов, сохраняя полную типобезопасность (компилятор проверяет типы).

Ограничения (constraints)

Constraint — это интерфейс, описывающий, какие типы допустимы как параметр. Он задаёт, что должен «уметь» тип.

```
// Ограничение: любой из перечисленных типов (union через |):
type Number interface {
    ~int | ~int64 | ~float64
}

func Sum[T Number](nums []T) T {
    var total T
    for _, n := range nums {
        total += n // можно +, т.к. constraint гарантирует числовые типы
    }
    return total
}
```

Тонкость — **тильда** `~`: `~int` означает «int и любой тип, чей базовый тип int» (например, `type MyInt int` тоже подойдёт). Без тильды — только точно `int`.

Есть готовые ограничения в пакете `constraints` (`Ordered`, `Integer`, `Float`), а `any` — это ограничение «любой тип» (то же, что `interface{}`).

Дженерик-типы

Параметризовать можно не только функции, но и типы — например, обобщённый стек:

```
type Stack[T any] struct {
    items []T
}

func (s *Stack[T]) Push(item T) {
    s.items = append(s.items, item)
}

func (s *Stack[T]) Pop() (T, bool) {
    var zero T
    if len(s.items) == 0 {
        return zero, false
    }
    item := s.items[len(s.items)-1]
    s.items = s.items[:len(s.items)-1]
    return item, true
}

// Использование:
intStack := &Stack[int]{} // стек int
strStack := &Stack[string]{} // стек string
```

Один код `Stack` работает для любого типа, типобезопасно: `intStack` принимает только `int`, `strStack` — только `string`.

Когда использовать, а когда нет

Важный нюанс для собеседования — дженерики **не заменяют интерфейсы**, у них разные задачи:

- **Дженерики** — когда нужна одна логика для многих типов, и важна связь между типами (напр. «функция принимает `[]T` и возвращает `T`»). Структуры данных (стеки, деревья, множества), утилитарные функции (Map, Filter, Reduce).
- **Интерфейсы** — когда нужно **разное поведение** за общим контрактом (полиморфизм). Разные реализации `Storage`, `Reader`.

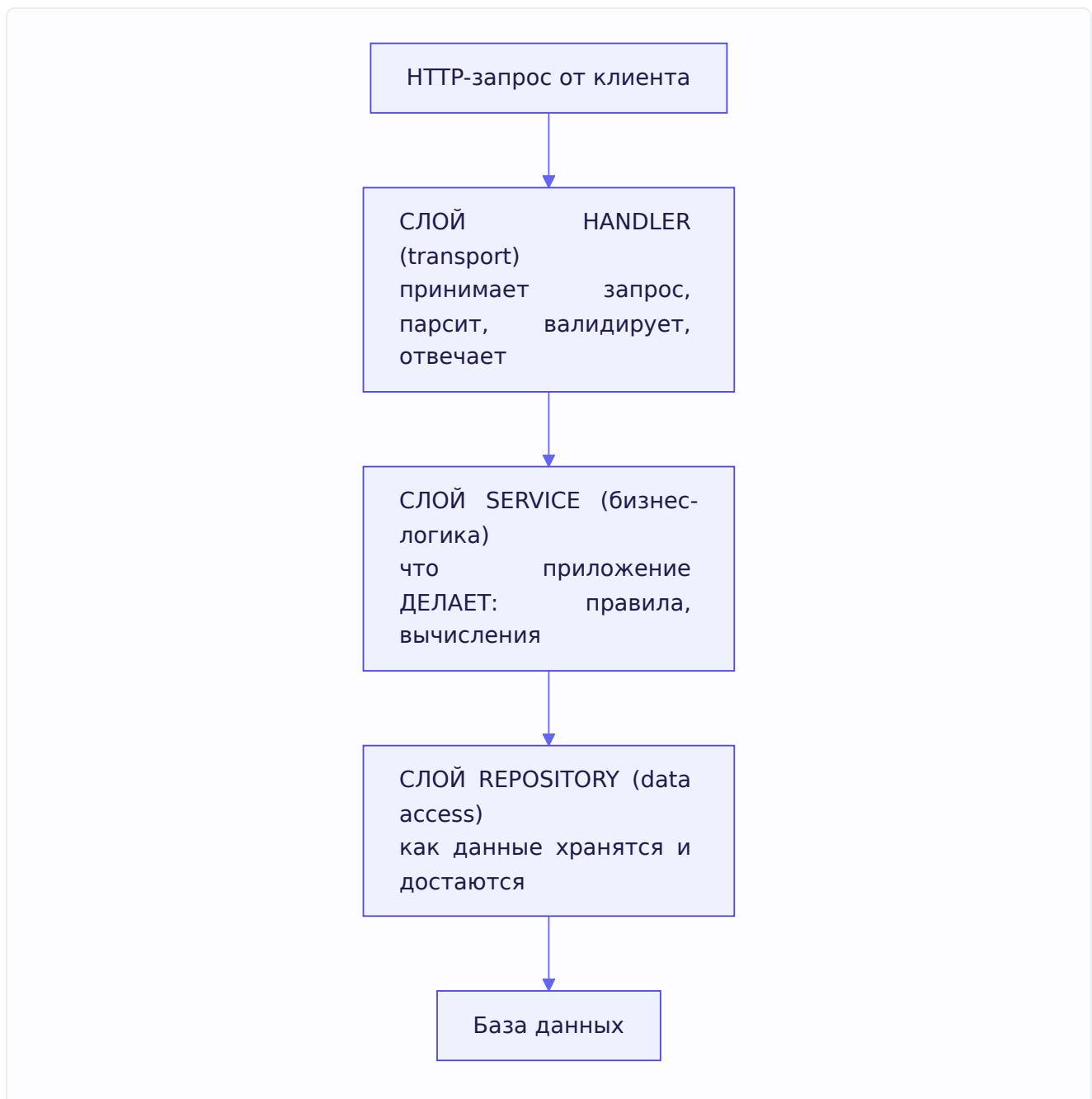
Правило: не тащи дженерики везде. Если задача решается интерфейсом — используй интерфейс (он привычнее и часто яснее). Дженерики — для случаев, где интерфейсы приводили к дублированию или потере типобезопасности. Официальный совет команды Go: «пиши обычный код, пока не увидишь, что дженерики реально устраняют дублирование».

4.10. Слоистая архитектура

Завершаем главу тем, как всё изученное складывается в **структуру целого приложения**. Это то, что реально спрашивают на бэкэнд-собеседованиях: «как ты организуешь код сервиса?»

Идея слоёв

Бэкэнд-приложение принято делить на **слои** по ответственности. Классическое трёхслойное деление:



Разберём каждый слой:

1. Handler (транспортный слой). Отвечает за общение с внешним миром по HTTP (или gRPC). Его задачи: принять запрос, распарсить входные данные (JSON, параметры), проверить их формат, вызвать нужный метод сервиса, превратить результат в ответ (JSON + статус-код). Handler **не содержит бизнес-логики** — он только «переводчик» между HTTP и сервисом.

2. Service (слой бизнес-логики). Сердце приложения — здесь живут **правила предметной области**. Что значит «создать заказ»? Проверить наличие товара, посчитать цену, применить скидку, зарезервировать позиции. Это бизнес-логика, и она не знает ни про HTTP (не знает, что такое запрос), ни про SQL (не знает, как устроена БД). Она работает через **интерфейсы** репозитория.

3. Repository (слой доступа к данным). Отвечает за хранение: SQL-запросы, работа с БД, кеширование. Прячет детали хранилища за интерфейсом (как мы разбирали в 4.7).

Почему именно так — направление зависимостей

Ключевой принцип: **зависимости направлены внутрь** (сверху вниз в схеме). Handler зависит от Service, Service — от Repository (точнее, от его интерфейса). Но **не наоборот**: Service не знает про Handler, Repository не знает про Service. Внутренние слои не подозревают о существовании внешних.

Зачем это: - **Бизнес-логику можно тестировать** без HTTP и без БД (подставив моки репозитория — привет DI из 4.6). - **Слой заменяемы**. Сменить HTTP на gRPC — переписать только handler. Сменить БД — только repository. Бизнес-логика (самое ценное и сложное) остаётся нетронутой. - **Понятность**. Каждый слой делает одно; новый разработчик быстро находит, где что.

Связь с чистой архитектурой

Это упрощённая версия **Clean Architecture** (и родственных Hexagonal, Onion). Их общая идея: **бизнес-логика в центре, детали (БД, фреймворки, транспорт) — снаружи, и зависимости идут снаружи внутрь**. Бизнес-правила — самое стабильное и ценное — не должны зависеть от переменчивых деталей (какая БД, какой веб-фреймворк).

DTO vs доменная модель

Практический нюанс, который отличает зрелого разработчика. На разных слоях часто используют **разные представления данных**:

- **DTO (Data Transfer Object)** — структура для передачи данных на границе (например, JSON-запрос/ответ в handler). Заточена под транспорт.
- **Доменная модель (domain model)** — структура, отражающая бизнес-сущность в слое сервиса. Заточена под логику.

Зачем разделять? Чтобы форма API не диктовала форму бизнес-модели и наоборот. Например, в JSON-ответе не должно быть внутреннего поля `passwordHash` из доменной модели — DTO его просто не содержит. Handler конвертирует DTO ↔ domain на границе. Для простых проектов этим иногда пренебрегают (одна структура на всё), но с ростом сложности разделение окупается.

Как отвечать на собеседовании про архитектуру: «Делю на слои — handler (транспорт: парсинг, валидация, ответы), service (бизнес-логика, не знает про HTTP и SQL), repository (доступ к данным за интерфейсом). Зависимости идут внутрь: service зависит от интерфейсов репозитория, а не от конкретной БД, — это позволяет тестировать логику с моками и менять инфраструктуру, не трогая бизнес-правила. На границах использую DTO, чтобы форма API не была жёстко завязана на доменную модель».

Итоги главы

Глава связала механику интерфейсов с принципами дизайна. Главное:

Устройство интерфейсов: интерфейс — это пара (тип, данные). `eface` для пустого интерфейса (`_type` + `data`), `iface` для интерфейса с методами (`tab` + `data`), где `itab` хранит таблицу методов конкретного типа. Duck typing — удовлетворение неявное, по наличию методов.

Nil interface trap: интерфейс равен `nil` только когда обе части `nil`. Nil-указатель конкретного типа, упакованный в интерфейс, даёт **не-nil** интерфейс (тип-то присутствует) — отсюда знаменитая ловушка. Возвращай буквальное `nil` или работай с переменной интерфейсного типа.

Assertion/switch: `v.(T)` извлекает конкретный тип из интерфейса (comma-ok, чтобы не паниковать); `type switch` для проверки на несколько типов. Тип должен точно совпадать (значение $\neq$ указатель).

Дизайн: наследования нет — есть композиция (встраивание для переиспользования, интерфейсы для полиморфизма). SOLID в Go выражается через маленькие интерфейсы и инверсию зависимостей. DI (передача зависимостей извне через интерфейсы) даёт тестируемость через моки. «Accept interfaces, return structs».

Паттерны: Factory (конструктор, скрывающий создание), Repository (абстракция хранилища), Strategy (сменные алгоритмы, часто просто функцией), Functional Options (гибкая конфигурация через `...Option`).

Дженерики (с 1.18) — код для многих типов без дублирования и без потери типобезопасности; параметры типа с ограничениями. Не заменяют интерфейсы: дженерики — одна логика для многих типов, интерфейсы — разное поведение за контрактом.

Слоистая архитектура: `handler` → `service` → `repository`, зависимости внутрь, бизнес-логика в центре и не знает про HTTP/SQL — тестируемо и заменяемо.

В следующей главе — обработка ошибок в Go: `error` как интерфейс, обёртки, `panic/recover/defer` и их тонкости.

Глава 5. Ошибки

Введение к главе

Обработка ошибок в Go устроена принципиально иначе, чем в большинстве языков, — и это одна из самых обсуждаемых особенностей языка. Здесь **нет исключений** (exceptions) в привычном смысле: ты не оборачиваешь код в `try/catch`. Вместо этого ошибка — это **обычное значение**, которое функция возвращает наравне с результатом, и ты проверяешь его явно.

Многие поначалу ворчат на это («сколько же `if err != nil!`»), но за подходом стоит чёткая философия: **ошибки — это часть нормального потока управления, а не что-то исключительное**. Ты видишь каждую точку, где что-то может пойти не так, прямо в коде — ошибки нельзя случайно «проглотить» или незаметно пробросить через десять уровней стека, как бывает с исключениями.

Разберём всё по порядку: от устройства `error` до тонкостей `panic / recover / defer`, на которых часто ловят на собеседовании.

5.1. error как интерфейс

Начнём с фундамента: что вообще такое ошибка в Go.

Определение

`error` — это **встроенный интерфейс**, причём предельно простой:

```
type error interface {
    Error() string
}
```

Всего один метод — `Error() string`, возвращающий текстовое описание. Любой тип, у которого есть метод `Error() string`, **является** ошибкой (вспомни duck typing из главы 4). Ничего больше не требуется.

Функции, которые могут завершиться неудачей, по соглашению возвращают `error` **последним** значением:

```
func Divide(a, b int) (int, error) {
    if b == 0 {
        return 0, errors.New("деление на ноль")
    }
    return a / b, nil // nil означает "ошибки нет"
}
```

Проверка — тот самый идиоматичный `if err != nil`:

```

result, err := Divide(10, 0)
if err != nil {
    // обрабатываем ошибку
    fmt.Println("ошибка:", err)
    return
}
// используем result — сюда попадаем только если ошибки не было

```

Ключевой момент: если ошибки нет, возвращается `nil`. И здесь всплывает всё, что мы разбирали в главе 4 про `nil` interface trap — **возвращай буквальный `nil`, а не типизированный `nil`-указатель**, иначе `err != nil` предательски сработает. Это не абстрактная тема: `nil` interface trap чаще всего кусает людей именно на возврате ошибок.

Как создавать ошибки

Два базовых способа создать простую ошибку:

```

// errors.New — из строки:
err := errors.New("файл не найден")

// fmt.Errorf — с форматированием:
err := fmt.Errorf("файл %s не найден в каталоге %s", name, dir)

```

`fmt.Errorf` удобнее, когда в сообщение нужно вставить детали (имя файла, id, значение). А ещё — и это важно — `fmt.Errorf` умеет **оборачивать** другие ошибки через глагол `%w`, о чём поговорим в 5.3.

Свой тип ошибки

Раз `error` — это интерфейс с одним методом, свою ошибку сделать элементарно. Разберём задачу из задачника: «напишите функцию, возвращающую ошибку, не импортируя никаких пакетов».

```

func handle() error {
    return &customError{}
}

type customError struct{}

func (e *customError) Error() string {
    return "Custom error!"
}

```

Здесь мы создали структуру `customError`, добавили ей метод `Error() string` — и она автоматически стала удовлетворять интерфейсу `error`. Никаких импортов: интерфейс `error` встроен в язык. Это прямая проверка понимания того, что `error` — просто интерфейс.

Зачем нужны **свои типы** ошибок, а не просто строки? Затем, что в них можно положить **дополнительные данные** — код ошибки, проблемное поле, вложенную причину:

```

type ValidationError struct {
    Field string
    Message string
}

func (e *ValidationError) Error() string {
    return fmt.Sprintf("поле %s: %s", e.Field, e.Message)
}

```

Теперь вызывающий код может не просто прочитать текст, но и **достать** структурированную информацию (какое именно поле не прошло валидацию) — через механизмы, которые разберём в 5.2.

5.2. errors.Is и errors.As

Когда ошибки начали **оборачивать** друг друга (одна ошибка содержит внутри другую — цепочка причин), понадобились инструменты, чтобы разбираться в этих цепочках. Их два: `errors.Is` и `errors.As`. Разберём, чем отличаются, — это частый вопрос.

Проблема

Представь: репозиторий вернул `sql.ErrNoRows`, сервис обернул её в «не удалось получить пользователя», handler обернул ещё раз. Получилась цепочка:

```
"ошибка обработки запроса" → "не удалось получить пользователя" → sql.ErrNoRows
```

Как в handler'е понять, что в глубине лежит именно `sql.ErrNoRows` (чтобы вернуть 404, а не 500)? Простое сравнение `err == sql.ErrNoRows` не сработает — `err` теперь это внешняя обёртка, а не сама `sql.ErrNoRows`. Нужно «размотать» цепочку. Для этого и придуманы `Is` и `As`.

errors.Is — проверка на конкретную ошибку

`errors.Is(err, target)` проверяет, есть ли **где-то в цепочке** обёрток ошибка, равная `target`. Он разматывает цепочку и сравнивает каждый уровень:

```

result, err := getUser(id)
if errors.Is(err, sql.ErrNoRows) {
    // где-то в глубине — sql.ErrNoRows, даже если сверху обёртки
    return NotFound()
}

```

`errors.Is` нужен, когда ты ищешь **конкретное значение** ошибки (обычно sentinel-ошибку, см. 5.4) — «это именно `ErrNoRows` ?».

errors.As — извлечение по типу

`errors.As(err, &target)` ищет в цепочке ошибку **определённого типа** и, найдя, **записывает** её в `target`, чтобы ты мог достать её поля:

```

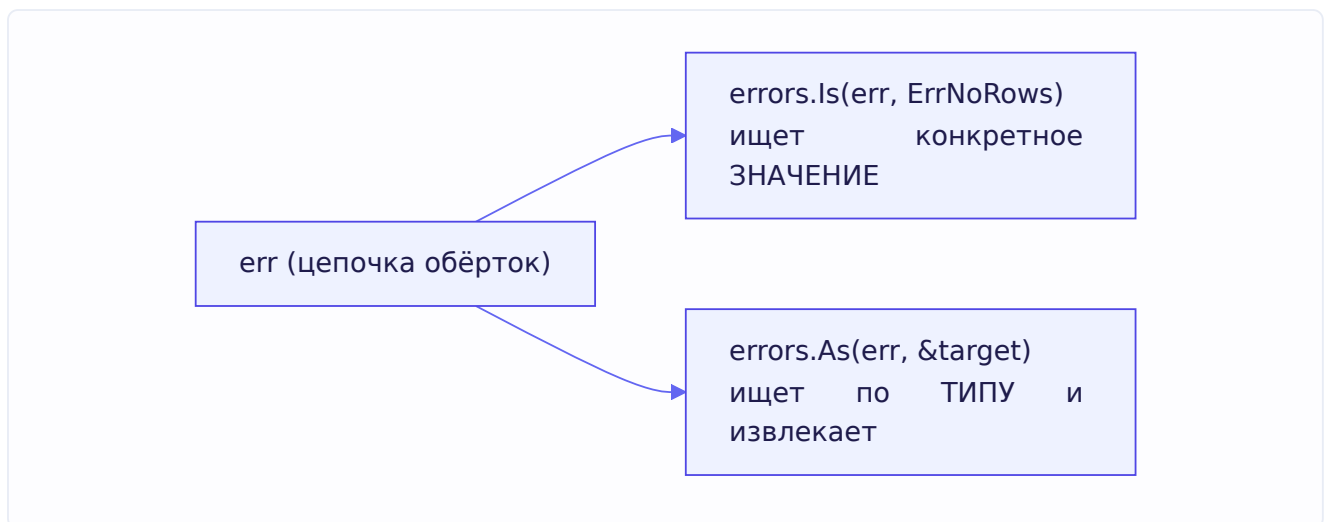
var validationErr *ValidationError
if errors.As(err, &validationErr) {
    // нашли в цепочке *ValidationError и положили в validationErr
    // теперь можно достать структурированные данные:
    fmt.Println("проблемное поле:", validationErr.Field)
}

```

`errors.As` нужен, когда тебе важен **тип** ошибки и её **данные**, а не конкретное значение.

Разница в одной фразе

Запомни так: - `errors.Is` — «в цепочке есть **вот эта конкретная** ошибка?» (сравнение по значению). - `errors.As` — «в цепочке есть ошибка **вот такого типа**? если да — дай мне её» (проверка по типу + извлечение).



Обе функции «умеют разматывать» цепочку — но для этого ошибки должны быть обёрнуты правильно, через `%w`. К этому и переходим.

5.3. Wrapping — оборачивание ошибок (%w)

Wrapping (оборачивание) — это добавление контекста к ошибке при её проброске вверх, с сохранением исходной ошибки внутри. Это то, что создаёт «цепочки» из предыдущего раздела.

Зачем оборачивать

Когда ошибка поднимается через слои, полезно на каждом уровне добавлять контекст — где именно и что пошло не так:

```

sql.ErrNoRows
→ "repository.GetUser: sql.ErrNoRows"    (репозиторий добавил, где)
→ "service.LoadProfile: ..."          (сервис добавил свой контекст)

```

В итоге по ошибке видно весь путь — как мини-трейс. Но при этом важно **сохранить** исходную `sql.ErrNoRows` внутри, чтобы `errors.Is / As` могли её найти.

Как оборачивать: %w

Оборачивание делается через `fmt.Errorf` с глаголом `%w` (от «wrap»):

```
func GetUser(id int) (*User, error) {
    user, err := db.Query(id)
    if err != nil {
        // оборачиваем: добавляем контекст, сохраняя исходную err внутри
        return nil, fmt.Errorf("GetUser(%d): %w", id, err)
    }
    return user, nil
}
```

Ключевое — разница между `%w` и `%v`:

- `%w` — **оборачивает** ошибку: добавляет её в цепочку, сохраняя доступной для `errors.Is / As`. Связь с исходной ошибкой не теряется.
- `%v` — просто **вставляет текст** ошибки в строку. Исходная ошибка «растворяется» в тексте, `errors.Is / As` её потом не найдут.

```
// С %w — цепочка сохранена, errors.Is найдёт sql.ErrNoRows:
fmt.Errorf("контекст: %w", err)

// С %v — только текст, связь потеряна, errors.Is НЕ найдёт:
fmt.Errorf("контекст: %v", err)
```

Правило: если хочешь, чтобы вызывающий код мог **распознать** исходную ошибку — оборачивай через `%w`. Если исходная ошибка — внутренняя деталь, которую не нужно распознавать снаружи, можно `%v` (или вовсе не пробрасывать).

Разворачивание: errors.Unwrap

Функция `errors.Unwrap(err)` достаёт ошибку, обернутую на **один** уровень внутрь. Обычно её не вызывают вручную — `errors.Is / As` делают это автоматически, проходя всю цепочку. Но знать про неё полезно: именно `Unwrap` — механизм, на котором держится вся система цепочек.

Не оборачивай слишком много

Практический совет: оборачивание полезно, но без фанатизма. Если на каждом уровне добавлять «функция X вызвала функцию Y», сообщение раздувается в кашу. Добавляй контекст там, где он **реально помогает** понять проблему (имя операции, ключевые параметры), и не дублируй то, что и так очевидно. Хороший контекст: `"GetUser(42): %w"`. Плохой: `"ошибка: %w"` (не добавляет ничего полезного).

5.4. Sentinel errors vs typed errors

Есть два основных подхода к тому, как определять «распознаваемые» ошибки — те, на которые вызывающий код должен как-то по-особому реагировать. Разберём оба и когда какой уместен.

Sentinel errors — ошибки-часовые

Sentinel error — это заранее объявленная переменная-ошибка на уровне пакета, которую используют как «маркер» конкретной ситуации:

```
var (
    ErrNotFound      = errors.New("не найдено")
    ErrUnauthorized = errors.New("не авторизован")
)

func GetUser(id int) (*User, error) {
    if !exists(id) {
        return nil, ErrNotFound // возвращаем sentinel
    }
    // ...
}
```

Вызывающий код распознаёт их через `errors.Is`:

```
user, err := GetUser(id)
if errors.Is(err, ErrNotFound) {
    return Response404()
}
```

Самый известный пример из стандартной библиотеки — `sql.ErrNoRows`, `io.EOF`. Это простые, узнаваемые «часовые», сигнализирующие о конкретном состоянии.

Плюсы: просто, понятно, легко проверять. **Минусы:** несут только сам факт (это «не найдено»), без деталей. И создают связанность — вызывающий пакет должен импортировать пакет с sentinel-ошибкой, чтобы на неё ссылаться.

Typed errors — типизированные ошибки

Typed error — это свой тип ошибки (структура), несущий **дополнительные данные**:

```
type NotFoundError struct {
    Resource string
    ID       int
}

func (e *NotFoundError) Error() string {
    return fmt.Sprintf("%s с id=%d не найден", e.Resource, e.ID)
}
```

Вызывающий код распознаёт их через `errors.As` и достаёт данные:

```
var nfErr *NotFoundError
if errors.As(err, &nfErr) {
    fmt.Printf("не найден %s (id=%d)\n", nfErr.Resource, nfErr.ID)
}
```

Плюсы: несут структурированную информацию (что именно не найдено, какой id, какой код).

Минусы: чуть сложнее, и — тонкость — легко случайно попасть в nil interface trap (глава 4), если вернуть nil-указатель типизированной ошибки.

Когда что выбирать

- **Sentinel** — когда ошибка это просто «флаг состояния» без деталей: «конец файла», «не найдено», «нет прав». Достаточно факта.
- **Typed** — когда вызывающему нужны **подробности** ошибки для обработки: какое поле не прошло валидацию, какой ресурс отсутствует, какой HTTP-код вернуть.

Многие проекты используют оба: sentinel для простых случаев, typed — где нужна структура. Главное — оба подхода работают через `errors.Is / As` и оборачивание, так что цепочки остаются «прозрачными».

5.5. Panic, Recover и Defer

Теперь — механизмы, которые в Go играют роль, отдалённо похожую на исключения, но с важными отличиями. Здесь много тонкостей, которые обожают на собеседовании.

Defer — отложенный вызов

`defer` откладывает вызов функции до момента, когда **завершается окружающая функция** (неважно, как — через `return`, до конца тела или из-за паники).

```
func readFile() error {
    f, err := os.Open("file.txt")
    if err != nil {
        return err
    }
    defer f.Close() // выполнится при выходе из readFile, что бы ни случилось

    // ... работаем с файлом ...
    return nil // перед возвратом сработает f.Close()
}
```

Главная польза: `defer` гарантирует освобождение ресурсов (закреть файл, разблокировать мьютекс, закрыть соединение) — даже если функция завершается неожиданно. Ты пишешь «закреть» сразу рядом с «открыть», и не забудешь про закрытие в одном из десяти путей выхода.

Порядок выполнения defer — LIFO

Если `defer`-ов несколько, они выполняются в **обратном** порядке — последний объявленный первым (LIFO, стек). Из задачника:

```
func main() {
    defer fmt.Println("1")
    defer fmt.Println("2")
    defer fmt.Println("3")
}
```

// Выведет: 3, 2, 1

Почему обратный порядок логичен: обычно ресурсы освобождают в порядке, обратном захвату (открыл А, потом В → закрыть сначала В, потом А). LIFO это обеспечивает естественно. Определение из задачника: `defer` добавляет вызовы в стек, и они срабатывают в обратном порядке.

Тонкость: аргументы defer вычисляются сразу

А вот ловушка, которую спрашивают в задачнике. Аргументы отложенной функции **вычисляются в момент объявления `defer`**, а не в момент её выполнения. Разберём две версии кода:

```
type X struct{ V int }
func (x X) S() { fmt.Println(x.V) }

func main() {
    x := X{123}
    defer x.S() // ← вот здесь
    x.V = 456
}
```

// Выведет: 123

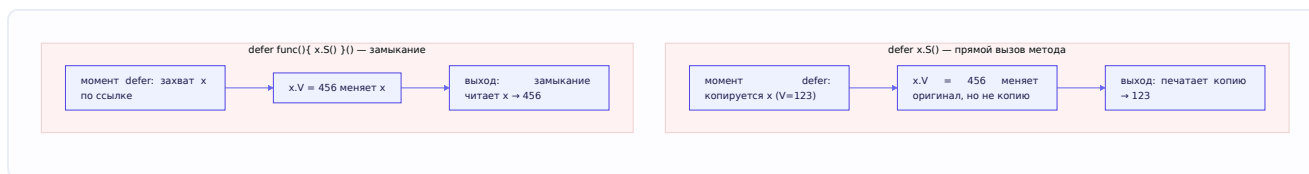
Почему **123**, а не 456? Потому что `defer x.S()` — это вызов метода на `x`. Метод `S` имеет **value receiver** (`func (x X)`), то есть получает **копию** `x`. Копия делается в момент **объявления `defer`** — а в этот момент `x.V` ещё равно 123. Последующее `x.V = 456` меняет оригинал, но копия, «замороженная» в `defer`, осталась со 123.

Теперь вторая версия из задачника:

```
func main() {
    x := X{123}
    defer func() { x.S() }() // ← обёрнуто в замыкание
    x.V = 456
}
```

// Выведет: 456

Здесь **456**! В чём разница? Теперь `defer` откладывает **анонимную функцию** (замыкание). Замыкание не получает копию `x` в момент объявления — оно **захватывает переменную `x` по ссылке** (вспомни замыкания из главы 3). Само `x.S()` вызовется только при выходе из `main` — а к тому моменту `x.V` уже равно 456. Замыкание видит актуальное значение.



Вывод, который стоит проговорить на собеседовании: `defer f(args)` фиксирует `args` в момент объявления; `defer func(){ ... }()` откладывает выполнение целиком и видит финальные значения захваченных переменных. Это прямое следствие разницы между «вычислить аргументы сейчас» и «замкнуться на переменную».

Panic — паника

panic — это механизм для **действительно исключительных** ситуаций, когда программа не может продолжать нормально. Паника останавливает обычное выполнение функции, запускает все отложенные **defer** (в порядке LIFO), затем поднимается вверх по стеку вызовов, выполняя **defer** каждой функции, — и если её не «поймать», роняет всю программу с трейсом.

```
func mustPositive(n int) {
    if n < 0 {
        panic("число должно быть положительным")
    }
}
```

Важнейшее правило Go-стиля: паника — не замена ошибкам. В Go ошибки, которые можно предвидеть и обработать (файл не найден, неверный ввод, сбой сети), возвращают через **error**. Паника — только для того, что **не должно случаться никогда** при корректной программе: выход за границы слайса, разыменование `nil`, программмерский баг, невозможность инициализироваться при старте. Не паникуй ради обычных ошибок — это дурной тон и признак непонимания идиом языка.

Recover — перехват паники

recover позволяет «поймать» панику и не дать программе упасть. Он работает **только внутри defer** — потому что `defer` выполняется во время «раскрутки» паники, и это единственное окно, где панику можно перехватить.

```
func safeCall() (err error) {
    defer func() {
        if r := recover(); r != nil {
            // поймали панику, превращаем её в обычную ошибку
            err = fmt.Errorf("восстановлено после паники: %v", r)
        }
    }()

    riskyOperation() // если здесь паника — recover её поймает
    return nil
}
```

Как это работает: если `riskyOperation` паникует, начинается раскрутка стека, выполняется наш **defer**, внутри которого `recover()` возвращает значение паники (не `nil`) — паника «погашена»,

программа продолжает нормально, а мы превращаем её в `error`. Если паники не было, `recover()` вернёт `nil`, и `defer` ничего не делает.

Где `recover` уместен на практике: на **границах**, где нельзя позволить всему приложению упасть из-за одного сбоя. Классика — HTTP-сервер: паника в обработчике одного запроса не должна ронять весь сервер. Поэтому `middleware` оборачивает обработчики в `recover`, превращая панику в ответ 500 (об этом в главе про HTTP). Но `recover` — не инструмент для повседневной обработки ошибок; это предохранитель.

Критическая тонкость: паника в горутине

Вот вопрос, который отделяет понимающих от зубрил, и он связывает эту главу с главой 3. `recover` **ловит панику только в своей горутине**.

```
func main() {
    defer func() {
        recover() // ЭТОТ recover НЕ поймает панику из горутины ниже!
    }()

    go func() {
        panic("паника в горутине") // уронит ВСЮ программу
    }()

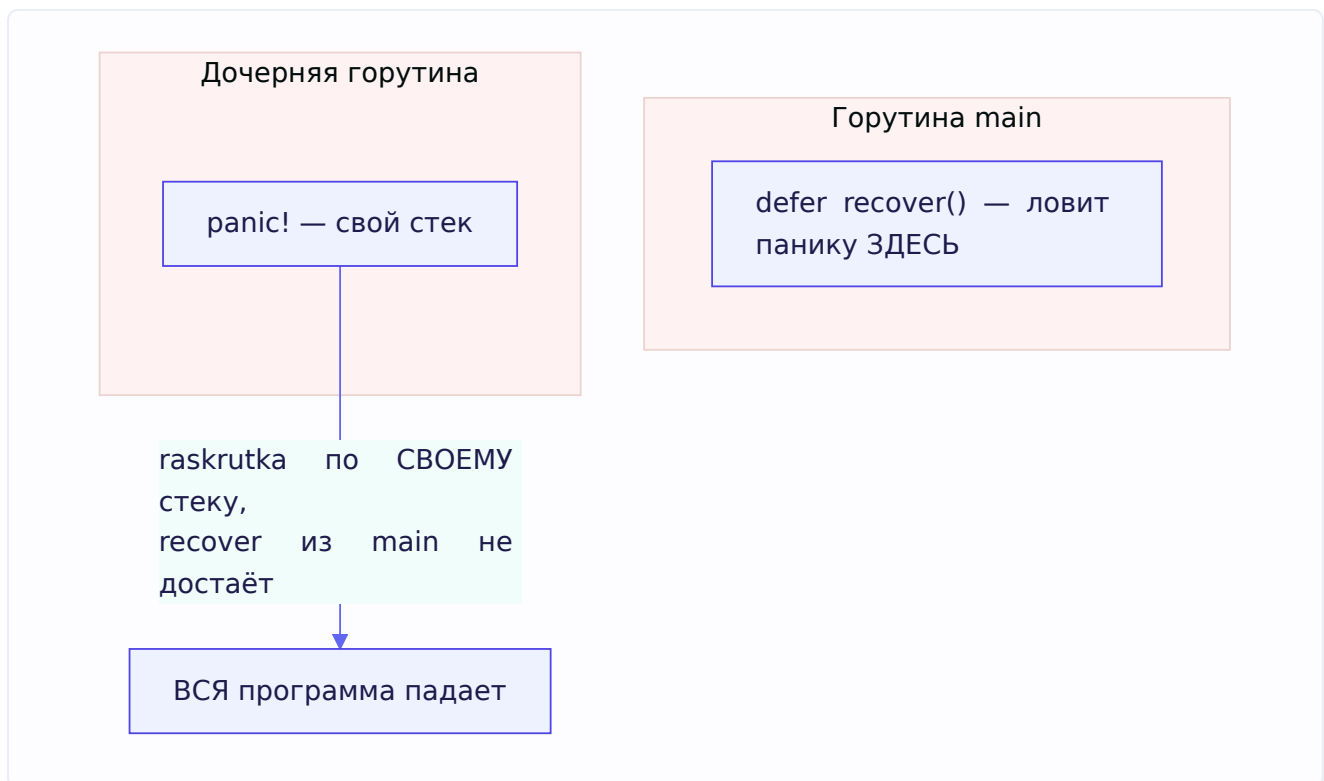
    time.Sleep(time.Second)
}
```

Почему `recover` в `main` не спасёт? Потому что каждая горутина имеет **свой** стек, и раскрутка паники идёт по стеку **той горутины**, где паника случилась. `recover` в `main` находится в стеке `main`, а паника — в стеке дочерней горутины. Они не пересекаются. Непойманная паника в **любой** горутине роняет **всю** программу целиком.

Практический вывод, критичный для реального кода: **если горутина может паниковать — ставь `recover` внутри неё самой**, а не снаружи:

```
go func() {
    defer func() {
        if r := recover(); r != nil {
            log.Printf("горутина восстановлена после паники: %v", r)
        }
    }()
    riskyWork()
}()
```

Это частая реальная проблема: где-то в фоновой горутине происходит паника, и весь сервис внезапно падает, хотя в `main` вроде бы есть `recover`. Понимание, что `recover` локален для горутины, — признак зрелого владения языком.



Когда defer НЕ выполнится

Ещё нюанс из задачника: есть случаи, когда отложенные **defer не сработают**: - `os.Exit()` — немедленно завершает программу, минуя `defer`. - `log.Fatal()` — внутри вызывает `os.Exit()`, поэтому тоже пропускает `defer`. - **Паника без recover в другой горутине**, роняющая программу, — `defer` текущей горутины могут не успеть.

Поэтому не полагайся на `defer` для критичной очистки, если возможен `os.Exit`. Обычная паника, наоборот, **выполняет** `defer` по пути вверх (именно поэтому `recover` в `defer` и работает).

5.6. Structured logging (структурированное логирование)

Ошибки нужно не только обрабатывать, но и **записывать** так, чтобы потом можно было разобраться. Здесь коротко о правильном логировании — это тоже часть работы с ошибками.

Проблема обычных логов

Традиционные текстовые логи — просто строки:

```
2024/01/15 10:23:45 ошибка при обработке запроса пользователя 42: timeout
```

Человеку читаемо, но машине — плохо. Как найти все ошибки по пользователю 42? Grep по тексту — хрупко. А в проде логи собираются централизованно (глава 2, мониторинг) и по ним нужно **искать и фильтровать**.

Структурированные логи

Structured logging — это запись логов в виде **структурированных пар ключ-значение** (обычно JSON), а не свободного текста:

```
{"level":"error","time":"2024-01-15T10:23:45Z","msg":"ошибка
обработки","user_id":42,"error":"timeout","duration_ms":5000}
```

Теперь поля (`user_id` , `error` , `level`) — это отдельные, машиночитаемые данные. Системы сбора логов (ELK, Loki и т.п.) могут по ним фильтровать, агрегировать, строить графики: «покажи все error за час по `user_id=42`» — тривиально.

slog — стандартное решение

С Go 1.21 в стандартную библиотеку входит пакет `log/slog` — официальный структурированный логгер:

```
import "log/slog"

slog.Error("не удалось обработать заказ",
    "order_id", orderID,
    "user_id", userID,
    "error", err,
)
```

Каждая пара (ключ, значение) становится отдельным полем в структурированном выводе. До `slog` использовали сторонние библиотеки (zap, zerolog) — они и сейчас популярны за скорость, но `slog` стандартизировал подход.

Что и как логировать — практика

Несколько правил, показывающих зрелость: - **Уровни логов** используй по смыслу: `Debug` (детали для отладки), `Info` (нормальные события), `Warn` (что-то подозрительное, но не критичное), `Error` (сбой, требующий внимания). - **Логируй ошибку один раз** — на том уровне, где её обрабатываешь. Если логировать на каждом уровне, куда она проходит, получишь дубли и шум. Оборачивай контекстом (`%w`) по пути, а логируй в одном месте. - **Не логируй чувствительные данные** — пароли, токены, персональные данные. Это и утечка, и нарушение приватности. - **Добавляй контекст** — id запроса (из `context`), id пользователя, имя операции. Именно контекст превращает лог из «что-то сломалось» в «вот что, где и у кого».

Связь с темами: структурированный лог хорошо сочетается с оборачиванием ошибок (`%w` строит цепочку контекста) и с `context` (откуда берут request id для трассировки через микросервисы, глава 2).

Итоги главы

Обработка ошибок в Go — про явность и ошибки-как-значения. Главное:

error — это интерфейс с единственным методом `Error() string`. Свою ошибку сделать тривиально (структура + метод `Error`). Возвращается последним значением; `nil` = нет ошибки (осторожно с `nil interface trap`).

Цепочки ошибок: оборачивай через `fmt.Errorf("...: %w", err)`, добавляя контекст и сохраняя исходную ошибку. `%w` оборачивает (сохраняет связь), `%v` — только текст (связь теряется). Разбирай цепочки: `errors.Is` — ищет конкретное значение, `errors.As` — ищет по типу и извлекает данные.

Sentinel vs typed: sentinel (`var ErrNotFound = errors.New(...)`) — простой флаг состояния, проверяется через `Is`. Typed (структура) — несёт данные, извлекается через `As`. Оба работают через оборачивание.

defer — отложенный вызов при выходе из функции (LIFO); аргументы вычисляются в момент объявления (отсюда 123 vs 456: прямой вызов копирует receiver сразу, замыкание видит финальное значение). Гарантирует освобождение ресурсов. Не выполняется при `os.Exit / log.Fatal`.

panic/recover — для действительно исключительных ситуаций, не замена ошибкам. `recover` работает только в `defer` и **только в своей горутине** — паника в горутине без внутреннего `recover` роняет всю программу.

Structured logging (`log/slog`) — логи как пары ключ-значение для машинного поиска; логируй ошибку один раз, добавляй контекст, не пиши секреты.

В следующей главе — сети и HTTP: как устроен веб под капотом, от TCP до `http.Server`, `middleware`, REST, аутентификация.

Глава 6. HTTP и сети

Введение к главе

Веб-бэкенд — это, по сути, программа, которая принимает сетевые запросы и отвечает на них. Поэтому понимание того, как устроена сеть под твоим кодом, — не теория ради теории, а фундамент профессии. На собеседовании это одна из самых спрашиваемых областей: от «чем TCP отличается от UDP» до «что происходит внутри `http.Server`, когда пришёл запрос».

Мы пойдём снизу вверх — от самых нижних уровней сети к прикладным: 1. **Сетевой фундамент** — TCP/IP, разница TCP и UDP. 2. **HTTP** — как устроен протокол, версии, и как с ним работает Go (`net/http`). 3. **Прикладные темы** — middleware, REST, аутентификация (JWT, cookies, sessions), CORS, TLS, WebSocket.

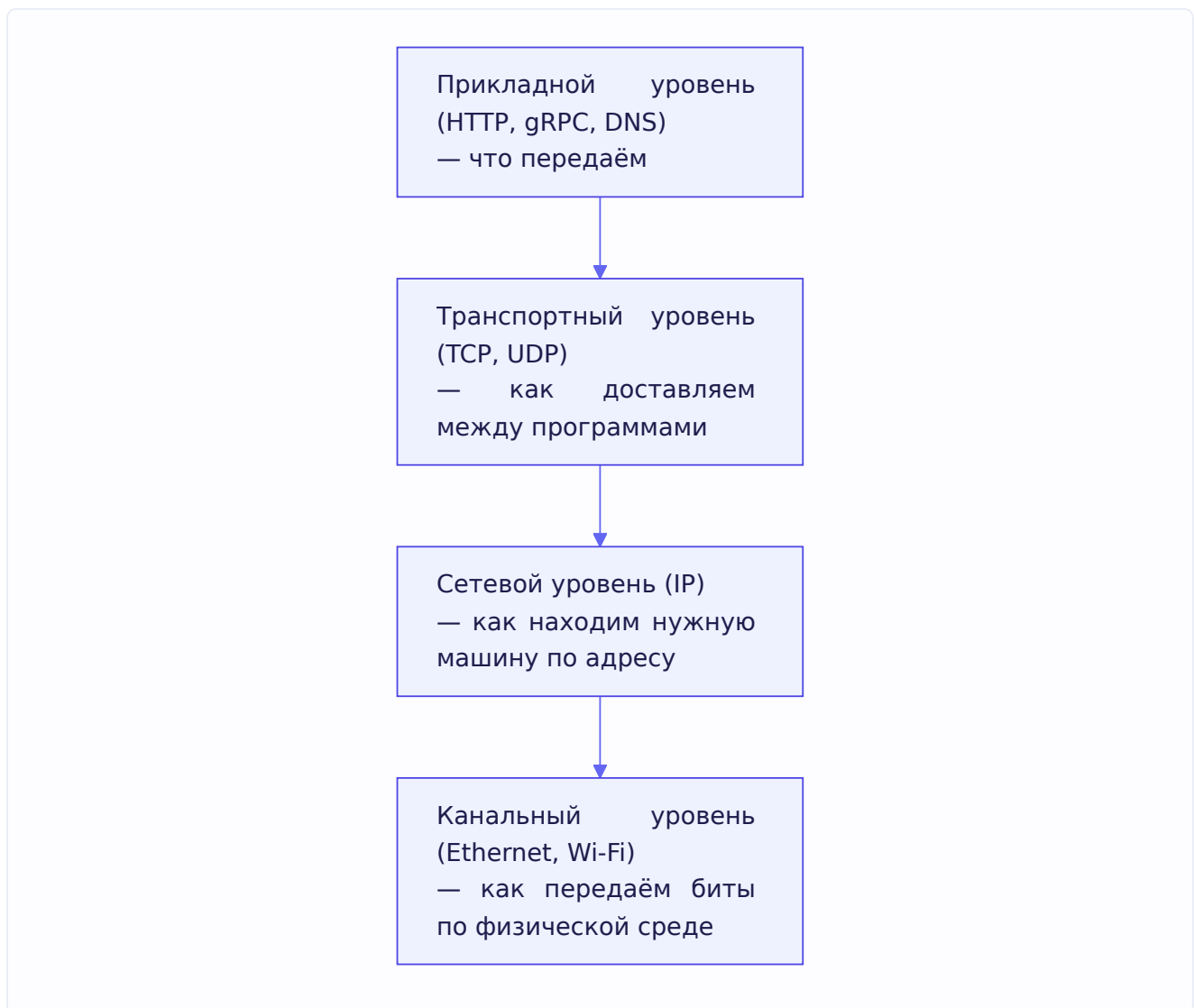
Начнём с самого низа — с того, как данные вообще путешествуют по сети.

6.1. TCP/IP basics

Модель уровней — зачем она

Сеть устроена **слоями**, где каждый слой решает свою задачу и опирается на нижний. Это как почта: ты пишешь письмо (данные), не думая, как именно грузовик повезёт его между городами. Каждый уровень абстрагирует детали нижнего.

Упрощённая модель TCP/IP (четыре уровня):



Для бэкендера важнее всего два средних уровня: **IP** (адресация — как найти машину) и **TCP/UDP** (транспорт — как доставить данные конкретной программе). На них и сосредоточимся.

IP — адресация

IP (Internet Protocol) отвечает за то, чтобы данные дошли до нужной **машины** в сети. У каждой машины есть IP-адрес (например, **192.168.1.10** для IPv4). IP разбивает данные на **пакеты** и отправляет их к адресату, но сам по себе **не гарантирует** доставку — пакеты могут потеряться, прийти не по порядку, продублироваться. IP — это «лучшая попытка» (best effort).

Раз IP ненадёжен, кто-то должен обеспечить надёжность (или сознательно от неё отказаться ради скорости). Это работа транспортного уровня — TCP или UDP.

Порты — адресация внутри машины

IP находит машину, но на машине работают десятки программ. Как понять, какой именно программе адресованы данные? Через **порт** — числовой идентификатор (0-65535). Пара «IP + порт» однозначно указывает на конкретную программу на конкретной машине. Например, веб-сервер обычно слушает порт 80 (HTTP) или 443 (HTTPS).

TCP vs UDP — ключевое различие

Это прямой вопрос из задачника, и один из самых частых на собеседовании вообще. Разберём досконально.

TCP (Transmission Control Protocol) — надёжный протокол с установлением соединения: - **Требуется установки соединения** перед передачей (рукопожатие, см. ниже). - **Надёжный**: гарантирует, что данные дойдут, в правильном порядке, без потерь и дублей. Если пакет потерялся — TCP его переотправит. - **Медленнее** — за надёжность платят: подтверждения, переотправки, контроль порядка добавляют накладные расходы.

UDP (User Datagram Protocol) — быстрый протокол без соединения: - **Не требует соединения** — просто шлёт датаграммы. - **Ненадёжный**: датаграммы могут теряться, дублироваться, приходить не по порядку — UDP этим не занимается. - **Быстрее** — за счёт отсутствия подтверждений и контроля. - Поддерживает **broadcast** (рассылку многим сразу).

Сведём в таблицу:

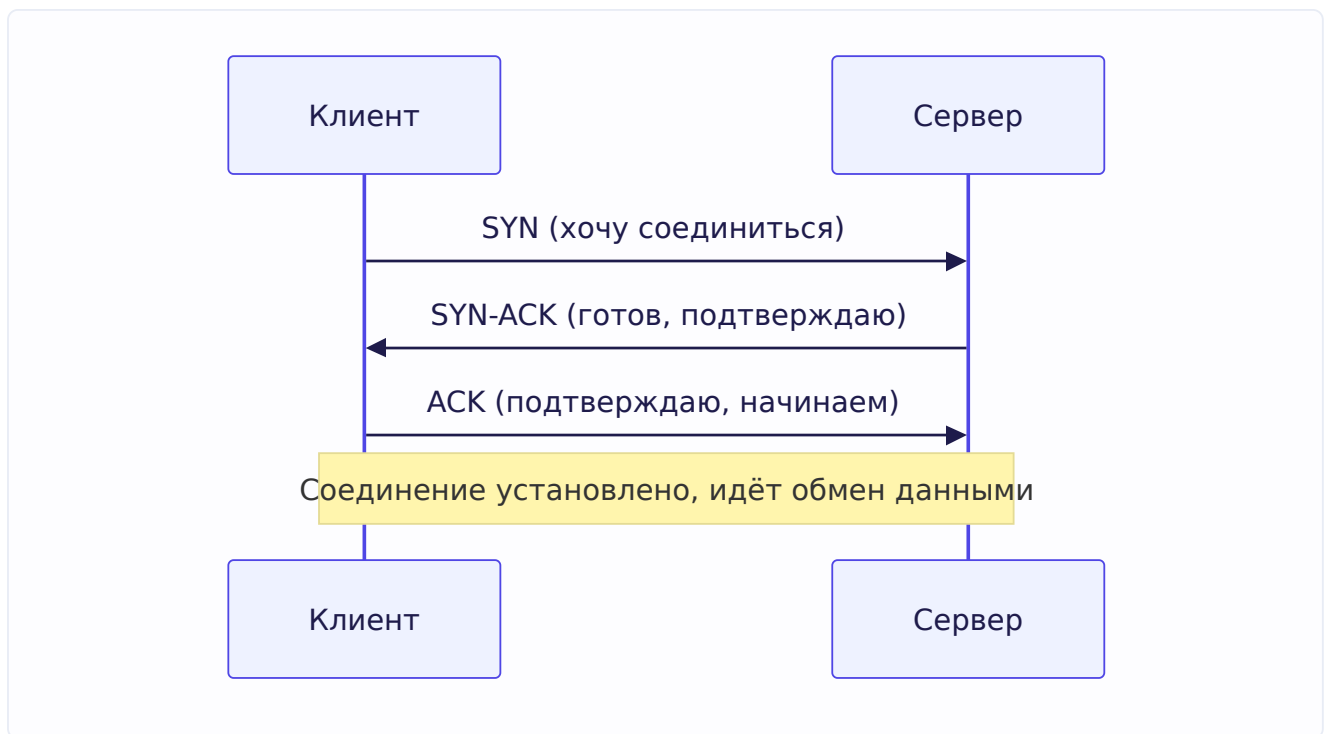
Свойство	TCP	UDP
Соединение	Требуется (рукопожатие)	Не требуется
Надёжность	Гарантирует доставку и порядок	Не гарантирует
Скорость	Медленнее	Быстрее
Переотправка потерь	Да	Нет
Применение	Web, API, БД, почта	Видео/аудио стриминг, игры, DNS, торренты

Из задачника — два хороших встречных вопроса: - «Зачем нужен UDP, если он ненадёжный?» → Затем, что он **быстрее** — нет накладных расходов на подтверждения. Для видеозвонка потерять один кадр не страшно (лучше пропустить, чем тормозить всё ради переотправки), а вот задержка критична. - «Зачем нужен TCP, если он медленный?» → Затем, что он **надёжен** — для загрузки файла, платежа, API-запроса недопустимо терять или путать данные.

Есть даже классический айтишный анекдот на эту тему (тоже в задачнике): «Рассказал бы анекдот про UDP, но не факт, что он до вас дойдёт». И про TCP: «Расскажу анекдот про TCP. Если он до вас не дойдёт — повторю снова». Он идеально иллюстрирует суть: UDP шлёт и не проверяет, TCP гарантирует доставку переотправкой.

TCP-соединение: рукопожатие и схема клиент-сервер

Раз TCP требует соединения, как оно устанавливается? Через **трёхстороннее рукопожатие** (three-way handshake):



Три шага: клиент шлёт **SYN** (запрос синхронизации), сервер отвечает **SYN-ACK** (подтверждение + свой запрос), клиент подтверждает **ACK**. После этого соединение установлено, и можно обмениваться данными. При закрытии — похожая процедура из нескольких шагов (FIN/ACK).

Полная схема взаимодействия клиент-сервер по TCP (из задачника — это ожидаемый ответ на «опишите высокоуровневую схему»):

1. **Сервер** создаёт сокет и начинает слушать входящие соединения на определённом порту: **bind** (привязать к порту) → **listen** (слушать) → **accept** (принять входящее соединение).
2. **Клиент** создаёт сокет и подключается к серверу: **connect**.
3. Идёт **обмен данными**: **send** / **recv** (отправка/получение).
4. В конце сокет **закрывается**: **close**.

Хорошая новость: в Go всю эту низкоуровневую механику (**bind** / **listen** / **accept** / **connect**) прячет стандартная библиотека. Ты работаешь с высокоуровневым **net.Listen** и **net.Dial**, а сокеты и рукопожатия происходят под капотом. Но понимать, что там внизу, — обязательно.

Сокеты в Go — базовый пример

Чтобы не быть голословным, вот минимальный TCP-сервер на чистом **net** (без HTTP), показывающий ту самую схему:

```

func main() {
    // Сервер слушает порт 8080 (bind + listen под капотом):
    listener, err := net.Listen("tcp", ":8080")
    if err != nil {
        log.Fatal(err)
    }
    defer listener.Close()

    for {
        // accept — блокируется, пока не придёт соединение:
        conn, err := listener.Accept()
        if err != nil {
            continue
        }
        // Каждое соединение обрабатываем в своей горутине:
        go handleConn(conn)
    }
}

func handleConn(conn net.Conn) {
    defer conn.Close()
    buf := make([]byte, 1024)
    n, _ := conn.Read(buf)    // recv
    conn.Write(buf[:n])       // send — эхо обратно
}

```

Обрати внимание на `go handleConn(conn)` — каждое соединение в своей горутине. Вот где сходятся главы: благодаря лёгким горутинам и netpoller'у (глава 2) Go может держать десятки тысяч одновременных соединений, каждое в своей горутине, не тратя поток ОС на ожидание. Именно это делает Go отличным языком для сетевых серверов.

6.2. HTTP: HTTP/1.1 vs HTTP/2

Поверх TCP работает **HTTP** — протокол, на котором держится веб и большинство API. Разберём его устройство и эволюцию версий.

Что такое HTTP

HTTP (HyperText Transfer Protocol) — прикладной протокол, работающий по принципу **запрос-ответ**: клиент шлёт запрос, сервер возвращает ответ. Классический HTTP/1.1 — **текстовый** протокол (запросы и ответы — читаемый текст), что удобно для отладки. Определение из задачника: HTTP — текстовый протокол, функционирующий по принципу запрос-ответ.

HTTP **без состояния** (stateless): каждый запрос независим, сервер по умолчанию не помнит предыдущие. Состояние (кто залогинен и т.п.) добавляют отдельно — через cookies/сессии/токены (см. 6.7).

Структура HTTP-запроса

Запрос состоит из трёх частей (из задачника):

1. **Стартовая строка** — метод, идентификатор ресурса (URI) и версия протокола.
2. **Заголовки (headers)** — метаданные в формате «ключ: значение».
3. **Тело (body)** — сами данные (не всегда есть; у GET обычно нет, у POST — есть).

Как это выглядит в реальном текстовом виде (можно написать руками — это спрашивают на грейде повыше):

```
POST /api/users HTTP/1.1      ← стартовая строка: метод, путь, версия
Host: example.com            ← заголовки
Content-Type: application/json
Content-Length: 27

                                ← пустая строка отделяет заголовки от тела
{"name": "Тимофей", "age": 20} ← тело
```

Структура HTTP-ответа

Ответ похож на запрос, но стартовая строка содержит не метод/URI, а **код состояния** (из задачника):

```
HTTP/1.1 200 OK              ← стартовая строка: версия, код, текст
Content-Type: application/json
Content-Length: 42

{"id": 1, "name": "Тимофей"}  ← тело ответа
```

HTTP-методы

Основные методы (из задачника — GET, HEAD, POST, PUT, DELETE) и их семантика:

- **GET** — получить ресурс. Не должен менять состояние (идемпотентный, безопасный).
- **POST** — создать ресурс / отправить данные. Меняет состояние.
- **PUT** — заменить/обновить ресурс целиком. Идемпотентный (повтор даёт тот же результат).
- **PATCH** — частично обновить ресурс.
- **DELETE** — удалить ресурс. Идемпотентный.
- **HEAD** — как GET, но без тела ответа (только заголовки) — узнать метаданные.

Понятие **идемпотентности** важно: идемпотентный метод можно безопасно повторить, результат не изменится (удалить один и тот же ресурс дважды → он всё равно удалён). POST не идемпотентен — два POST создадут два ресурса. Это влияет на дизайн API и логику повторов.

Коды состояний

Коды сгруппированы по классам (из задачника):

- **1xx** — информационные (редко используются напрямую).
- **2xx** — успех: **200 OK**, **201 Created** (создан ресурс), **204 No Content** (успех без тела).
- **3xx** — перенаправление: **301 Moved Permanently**, **304 Not Modified** (кеш актуален).

- **4xx** — ошибка **клиента**: **400 Bad Request** (кривой запрос), **401 Unauthorized** (не аутентифицирован), **403 Forbidden** (нет прав), **404 Not Found**, **429 Too Many Requests** (rate limit).
- **5xx** — ошибка **сервера**: **500 Internal Server Error**, **502 Bad Gateway**, **503 Service Unavailable**.

Ключевое различие для дизайна API: **4xx — виноват клиент** (что-то не так с запросом, повтор без изменений не поможет), **5xx — виноват сервер** (клиент может повторить позже). Правильные коды — признак качественного API.

Примеры заголовков

Полезные заголовки (из задачника — примеры): **Content-Type** (тип тела: **application/json**), **Content-Length** (размер тела), **Authorization** (данные аутентификации), **Cookie**, **User-Agent** (кто клиент), **Accept** (какой формат ответа хочет клиент).

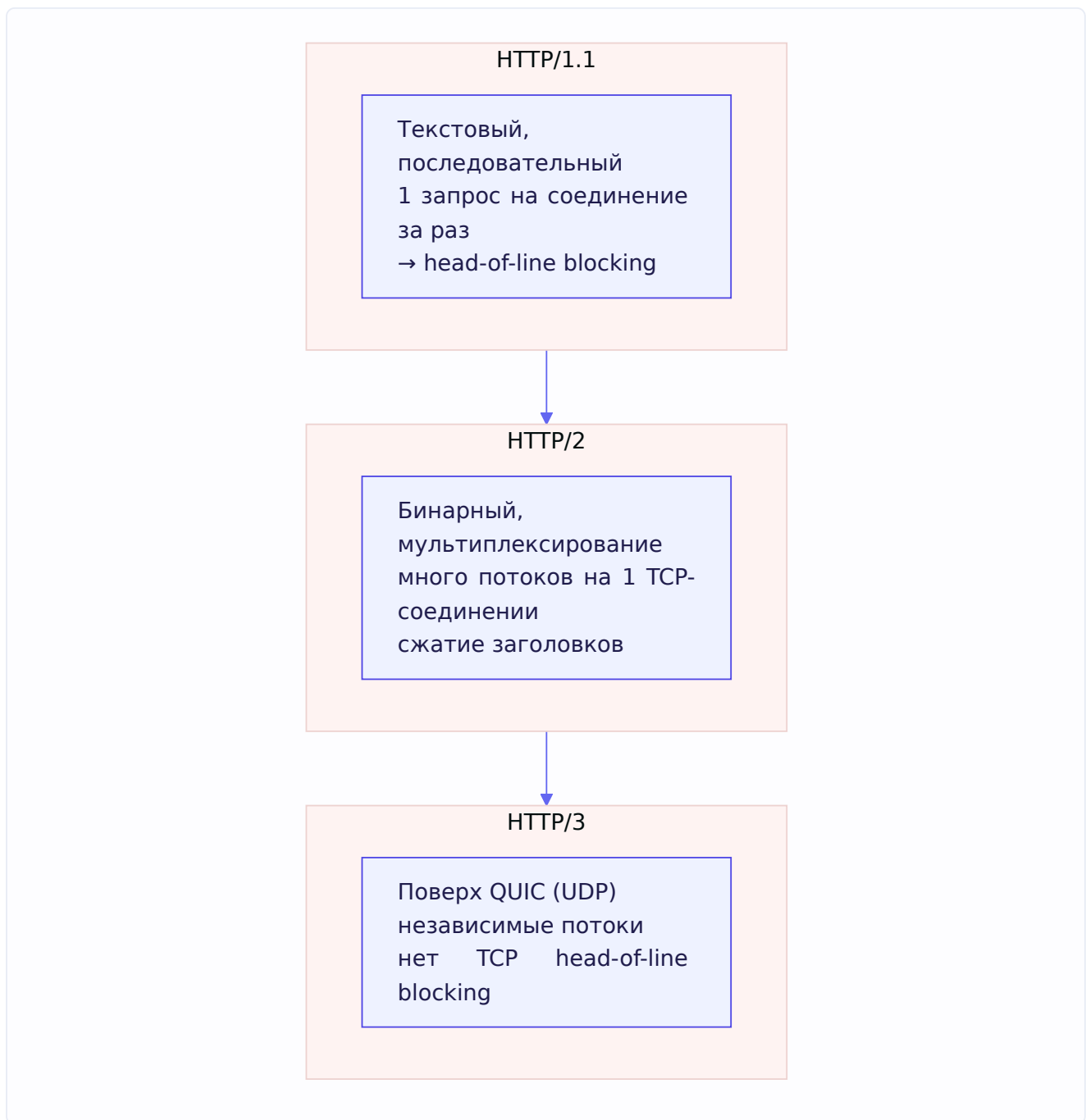
Эволюция версий: 1.1 → 2 → 3

Теперь — чем версии отличаются, это частый вопрос.

HTTP/1.1 — текстовый, одно соединение обрабатывает запросы **последовательно**. Главная проблема — **head-of-line blocking**: пока не пришёл ответ на текущий запрос, следующий по тому же соединению ждёт. Обходили это открытием нескольких TCP-соединений параллельно (браузеры открывают 6+), но это расточительно.

HTTP/2 — существенный шаг вперёд: - **Бинарный** (не текстовый) — компактнее и быстрее парсится. - **Мультиплексирование**: по **одному** TCP-соединению идёт **много** параллельных запросов/ответов (поток, streams) одновременно — нет блокировки на уровне HTTP. Это устраняет главную боль 1.1. - **Сжатие заголовков** (HPACK) — заголовки часто повторяются, их сжимают. - **Server push** — сервер может присылать ресурсы до запроса (на практике редко используется).

HTTP/3 — самое свежее: работает поверх **QUIC** (протокол на базе UDP, а не TCP). Зачем? У HTTP/2 остаётся head-of-line blocking на уровне **TCP**: потеря одного пакета тормозит все потоки в соединении. QUIC решает это, реализуя надёжность и мультиплексирование поверх UDP так, что потоки независимы.



Практическая связь: gRPC (глава 7) работает поверх HTTP/2 именно ради мультиплексирования и стриминга. А Go поддерживает HTTP/2 в `net/http` автоматически при использовании TLS — часто даже не нужно ничего настраивать.

6.3. `net/http` — работа с HTTP в Go

Go знаменит тем, что производительный HTTP-сервер пишется в несколько строк **стандартной библиотекой**, без фреймворков. Разберём `net/http`.

Минимальный сервер

```
package main

import (
    "fmt"
    "net/http"
)

func main() {
    http.HandleFunc("/hello", func(w http.ResponseWriter, r *http.Request) {
        fmt.Fprintln(w, "Привет, мир!")
    })

    http.ListenAndServe(":8080", nil) // запуск сервера на порту 8080
}
```

Всё — это рабочий веб-сервер. Разберём ключевые сущности.

Handler и ResponseWriter

Два центральных типа:

http.Request — входящий запрос: метод (`r.Method`), путь (`r.URL.Path`), заголовки (`r.Header`), тело (`r.Body`), параметры запроса (`r.URL.Query()`).

http.ResponseWriter — интерфейс для формирования ответа. Через него ты пишешь заголовки, код статуса и тело:

```
func handler(w http.ResponseWriter, r *http.Request) {
    w.Header().Set("Content-Type", "application/json") // заголовок
    w.WriteHeader(http.StatusCreated)                 // код 201
    w.Write([]byte(`{"status":"ok"}`))                 // тело
}
```

Важный порядок: сначала заголовки (`Header().Set`), потом код (`WriteHeader`), потом тело (`Write`). После записи тела заголовки менять поздно — они уже отправлены.

Интерфейс Handler

Под капотом `net/http` строится вокруг одного интерфейса:

```
type Handler interface {
    ServeHTTP(w http.ResponseWriter, r *http.Request)
}
```

Всё, что умеет обрабатывать HTTP-запрос, реализует этот интерфейс. Обычные функции превращаются в Handler через тип-адаптер `http.HandlerFunc`:

```

type HandlerFunc func(ResponseWriter, *Request)

// HandlerFunc реализует Handler, вызывая саму себя:
func (f HandlerFunc) ServeHTTP(w ResponseWriter, r *Request) {
    f(w, r)
}

```

Это красивый приём (вспомни главу 4): функция становится объектом, реализующим интерфейс, через тип-обёртку с методом. Поэтому и `http.HandlerFunc` (принимает функцию), и `http.Handle` (принимает Handler) работают одинаково — функция оборачивается в `HandlerFunc`.

ServeMux — маршрутизатор

`ServeMux` (multiplexer) — это маршрутизатор запросов: он смотрит на путь запроса и выбирает, какой handler вызвать.

```

mux := http.NewServeMux()
mux.HandleFunc("/users", usersHandler)
mux.HandleFunc("/orders", ordersHandler)

http.ListenAndServe(":8080", mux)

```

Когда ты передаёшь `nil` вместо `mux` (как в первом примере), используется глобальный `DefaultServeMux`. В реальных проектах создают свой `mux` явно (чище, без глобального состояния). С Go 1.22 стандартный `ServeMux` научился маршрутизации по методам и path-параметрам (`GET /users/{id}`), что раньше требовало сторонних роутеров (`chi`, `gorilla/mux`).

6.4. http.Server под капотом

Мы написали сервер в пару строк — но что там происходит внутри, когда приходит запрос? Это отличный вопрос на грейд повыше, и он связывает HTTP с горутинами из главы 3.

Что делает ListenAndServe

`http.ListenAndServe` под капотом:

1. Создаёт `net.Listener` на указанном порту (тот самый `bind / listen` из 6.1).
2. Входит в **бесконечный цикл** `accept` — ждёт входящие соединения.
3. На **каждое** принятое соединение запускает **отдельную горутину** для его обработки.

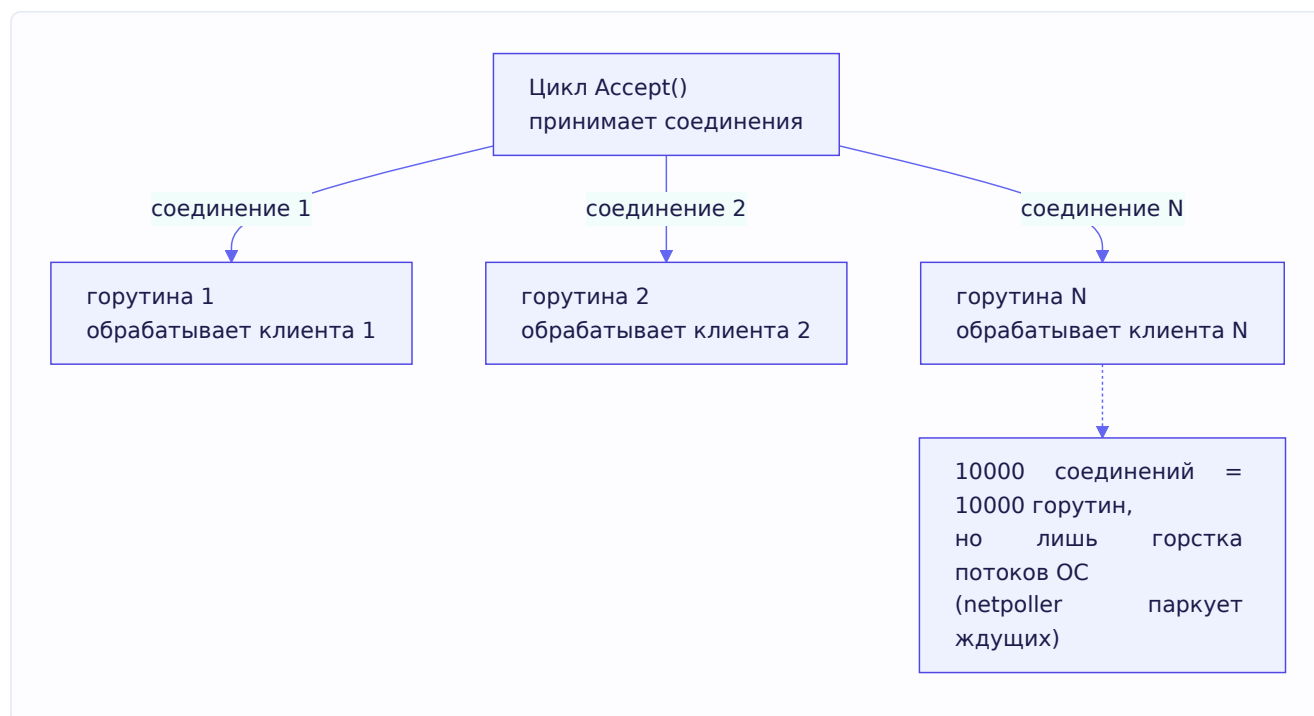
Схематично внутренний цикл выглядит так:

```
func (srv *Server) Serve(l net.Listener) error {
    for {
        conn, err := l.Accept() // ждём новое соединение
        if err != nil {
            // обработка ошибки...
            continue
        }
        go srv.handleConnection(conn) // ← ГОРУТИНА НА КАЖДОЕ СОЕДИНЕНИЕ
    }
}
```

Горутин на соединение — почему это работает

Вот ключевой момент. На **каждое** входящее соединение `http.Server` заводит свою горутину. Если у тебя 10 000 одновременных клиентов — это 10 000 горутин.

В большинстве других языков модель «поток на соединение» не масштабируется (10 000 потоков ОС убьют систему, глава 2). Но в Go это работает **именно благодаря** тому, что мы разбирали: - **Горутин дешёвы** — 10 000 горутин это нормально (килобайты на каждую, глава 2). - **Netpoller** (глава 2) — когда горутин ждёт данные из сокета, она не блокирует поток ОС; поток берёт другую работу, а горутин «паркуется» до прихода данных.



Итог: простая и понятная модель «горутин на соединение» + дешёвые горутин + netpoller = Go держит огромное число соединений малой ценой. Это и есть ответ на «как `http.Server` масштабируется».

Что делает горутин соединения

Внутри горутин соединения происходит: 1. Читается и парсится HTTP-запрос из сокета (стартовая строка, заголовки, тело — структура из 6.2). 2. `ServeMux` выбирает нужный handler по

пути. 3. Вызывается `handler.ServeHTTP(w, r)` — твой код. 4. Ответ сериализуется обратно в сокет. 5. При keep-alive соединение переиспользуется для следующего запроса (не закрывается сразу).

Важные настройки Server — производственный нюанс

Дефолтный `http.ListenAndServe(":8080", mux)` удобен для примеров, но в **проде** так делать нельзя — у него нет таймаутов, и это опасно. Правильно — создавать `http.Server` явно с таймаутами:

```
srv := &http.Server{
    Addr:      ":8080",
    Handler:   mux,
    ReadTimeout: 5 * time.Second, // макс. время на чтение запроса
    WriteTimeout: 10 * time.Second, // макс. время на запись ответа
    IdleTimeout: 120 * time.Second, // время жизни keep-alive соединения
}
srv.ListenAndServe()
```

Зачем таймауты: без них медленный или зловердный клиент может открыть соединение и держать его вечно (например, отправлять запрос по байту в час) — это утечка ресурсов и вектор атаки (slowloris). Таймауты — обязательная гигиена продакшн-сервера.

Graceful shutdown

Ещё производственная деталь, которую ты уже применял в своём Uptime Monitor. При остановке сервиса нельзя просто «убить» сервер — оборвутся запросы в обработке. Нужен **graceful shutdown** — корректное завершение: перестать принимать новые соединения, но дать доработать текущим запросам.

```
// В отдельной горутине ждём сигнал остановки:
stop := make(chan os.Signal, 1)
signal.Notify(stop, os.Interrupt, syscall.SIGTERM)

go func() {
    <-stop // ждём SIGINT/SIGTERM (глава 2 — сигналы)
    ctx, cancel := context.WithTimeout(context.Background(), 10*time.Second)
    defer cancel()
    srv.Shutdown(ctx) // корректно завершаем: даём текущим запросам доработать
}()

srv.ListenAndServe()
```

`srv.Shutdown(ctx)` перестаёт принимать новые соединения и ждёт (до дедлайна из `ctx`), пока завершатся активные запросы. Здесь сходятся сразу три темы: сигналы ОС (глава 2), `context` с таймаутом (глава 3) и HTTP-сервер. Это то, что отличает игрушечный сервер от продакшн-готового.

6.5. Middleware chain

Middleware (промежуточное ПО) — это код, который выполняется **до и/или после** основного обработчика, оборачивая его. Middleware решает **сквозные задачи** (cross-cutting concerns) — то, что нужно на многих эндпоинтах: логирование, аутентификация, восстановление после паники, CORS, ограничение частоты запросов.

Идея: обёртка вокруг Handler

В Go middleware — это функция, которая **принимает Handler и возвращает Handler**. Внешний Handler делает что-то своё, а внутри вызывает исходный:

```
func LoggingMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        start := time.Now()
        log.Printf("→ %s %s", r.Method, r.URL.Path) // ДО обработчика

        next.ServeHTTP(w, r) // вызываем следующий (основной) обработчик

        log.Printf("← %s %s за %v", r.Method, r.URL.Path, time.Since(start)) // ПОСЛЕ
    })
}
```

Разберём внимательно, потому что тут вся суть: - Middleware получает `next` — следующий обработчик в цепочке. - Возвращает **новый** Handler, который: делает что-то до (`log.Printf` «→»), вызывает `next.ServeHTTP` (передаёт управление дальше), делает что-то после (`log.Printf` «←» с замером времени). - Момент вызова `next.ServeHTTP` — это точка, где «до» переходит в «после».

Цепочка middleware

Middleware **компонуются** — оборачиваются друг в друга, образуя цепочку (chain). Запрос проходит их по очереди снаружи внутрь, ответ — изнутри наружу:

```
handler := LoggingMiddleware(
    AuthMiddleware(
        RecoverMiddleware(
            finalHandler, // основной обработчик в центре
        ),
    ),
)
```

Порядок исполнения — как «луковица»: внешний middleware начинает первым, но заканчивает последним:



Запрос «погружается» внутрь через все middleware до основного обработчика, затем ответ «всплывает» обратно. Logging начал первым — закончит последним (замерит полное время). Это ровно та же LIFO-логика, что у `defer` (глава 5) — не случайно.

Практичный способ компоновать

Ручная вложенность `A(B(C(handler)))` некрасива при многих middleware. Обычно пишут хелпер или используют роутер, поддерживающий middleware из коробки (chi, echo). Простой хелпер:

```
func Chain(h http.Handler, middlewares ...func(http.Handler) http.Handler) http.Handler {
    // применяем в обратном порядке, чтобы первый в списке был внешним:
    for i := len(middlewares) - 1; i >= 0; i-- {
        h = middlewares[i](h)
    }
    return h
}

// использование — читается как список слоёв:
handler := Chain(finalHandler,
    LoggingMiddleware,
    AuthMiddleware,
    RecoverMiddleware,
)
```

Recover-middleware — важный практический пример

Помнишь из главы 5, что непойманная паника роняет всю программу, и что паника в обработчике не должна ронять весь сервер? Вот каноничное решение — recover-middleware:

```
func RecoverMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        defer func() {
            if err := recover(); err != nil {
                log.Printf("паника в обработчике: %v", err)
                http.Error(w, "внутренняя ошибка сервера", http.StatusInternalServerError)
            }
        }()
        next.ServeHTTP(w, r) // если здесь паника — defer выше её поймает
    })
}
```

Это связывает главы: паника в одном запросе превращается в аккуратный ответ 500, а сервер продолжает обслуживать остальных. Ставь такой middleware внешним (или почти внешним) в цепочке, чтобы он покрывал все обработчики.

6.6. REST API

REST (Representational State Transfer) — архитектурный стиль для проектирования веб-API. Это не протокол и не стандарт, а **набор принципов**, как организовать API поверх HTTP. Разберём суть.

Ключевая идея: ресурсы и операции над ними

REST строится вокруг **ресурсов** — сущностей предметной области (пользователи, заказы, товары). Каждый ресурс имеет свой URL, а действия над ним выражаются **HTTP-методами** (из задачника: REST строится вокруг ресурсов и операций GET/POST/PUT/DELETE).

Вместо «эндпоинтов-глаголов» (`/getUser`, `/createUser`, `/deleteUser`) REST использует **существительные-ресурсы** + методы:

Действие	Метод + URL	Смысл
Получить список	<code>GET /users</code>	список пользователей
Получить одного	<code>GET /users/42</code>	пользователь с id=42
Создать	<code>POST /users</code>	создать нового
Обновить целиком	<code>PUT /users/42</code>	заменить пользователя 42
Частично обновить	<code>PATCH /users/42</code>	изменить часть полей
Удалить	<code>DELETE /users/42</code>	удалить пользователя 42

Один URL ресурса + разные методы = разные операции. Это чище и предсказуемее, чем россыпь глаголов-эндпоинтов.

Принципы REST

Несколько ключевых принципов, которые стоит знать:

- **Stateless (без состояния)**. Каждый запрос самодостаточен — сервер не хранит состояние клиента между запросами. Вся нужная информация (например, токен аутентификации) — в самом запросе. Это позволяет легко масштабировать сервер горизонтально (любой инстанс обработает любой запрос — вспомни главу про масштабирование).
- **Единообразный интерфейс** — предсказуемые URL-ресурсы и стандартные методы.
- **Правильные коды состояний** — `201` при создании, `404` если не найдено, `400` при кривом запросе (из 6.2).
- **Представления (representations)** — ресурс¹ передаётся в каком-то формате, обычно JSON (глава про JSON).

REST vs RPC

Раз мы обсуждаем стили API, полезно противопоставить REST и RPC (это прямой вопрос из задачника, и тема стыкуется с главой 7 про gRPC):

- **REST** строится вокруг **ресурсов** и стандартных операций над ними (GET/PUT/DELETE). «Работаем с сущностями».
- **RPC (Remote Procedure Call)** строится вокруг **вызова методов/функций** сервиса. «Вызываем процедуры».

Из задачника, тонкости: - REST обычно делают **поверх HTTP** (он на нём и построен). - RPC можно делать **поверх чего угодно** — HTTP, голого TCP, чего-то ещё. RPC — это идея «вызвать

удалённую функцию», а транспорт вторичен. - Как переложить RPC на HTTP? Передавать имя сервиса/метода в URL или заголовке, аргументы — в теле. Именно так и работает gRPC (глава 7) — поверх HTTP/2.

Практический ориентир: REST хорош для публичных API и ресурсно-ориентированных доменов; RPC (особенно gRPC) — для внутренней коммуникации между микросервисами, где важны скорость, строгие контракты и стриминг. В твоём Uptime Monitor как раз это разделение: gRPC для быстрых синхронных вызовов между сервисами.

Пример REST-обработчика в Go

```
func getUserHandler(w http.ResponseWriter, r *http.Request) {
    id := r.PathValue("id") // path-параметр (Go 1.22+)

    user, err := repo.GetByID(id)
    if errors.Is(err, ErrNotFound) {
        http.Error(w, "пользователь не найден", http.StatusNotFound) // 404
        return
    }
    if err != nil {
        http.Error(w, "внутренняя ошибка", http.StatusInternalServerError) // 500
        return
    }

    w.Header().Set("Content-Type", "application/json")
    json.NewEncoder(w).Encode(user) // сериализуем в JSON (глава про JSON)
}
```

Обрати внимание, как сходятся темы: `errors.Is` (глава 5) для распознавания «не найдено», правильные HTTP-коды (6.2), JSON-сериализация. Хороший REST-обработчик — это аккуратная стыковка всего изученного.

6.7. JWT, Cookies и Sessions — аутентификация

HTTP без состояния (stateless) — каждый запрос независим. Но сервису нужно понимать, **кто** его дёргает: залогинен ли пользователь, кто он, какие у него права. Как «помнить» пользователя между запросами в протоколе, который ничего не помнит? Разберём три связанных механизма.

Cookies — как браузер «помнит»

Cookie — небольшой кусочек данных, который сервер отдаёт клиенту, а клиент **автоматически прикладывает** к каждому последующему запросу к этому серверу. Это базовый механизм хранения состояния на стороне клиента.

Сервер устанавливает cookie заголовком `Set-Cookie` :

```
http.SetCookie(w, &http.Cookie{
    Name:     "session_id",
    Value:    "abc123xyz",
    HttpOnly: true,    // недоступна из JavaScript (защита от XSS)
    Secure:   true,    // только по HTTPS
    SameSite: http.SameSiteStrictMode, // защита от CSRF
    MaxAge:   3600,    // время жизни в секундах
})
```

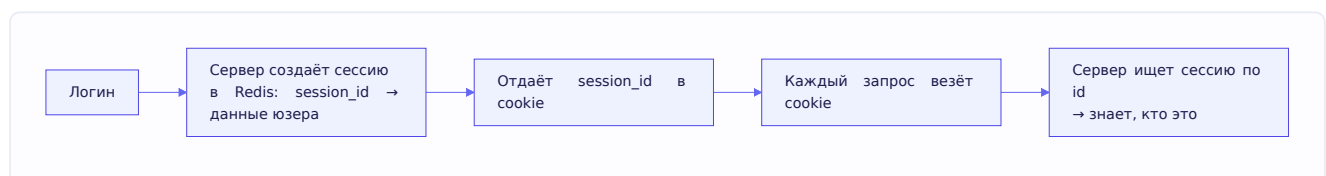
Флаги безопасности критичны: - **HttpOnly** — cookie недоступна JavaScript'у, защищает от кражи через XSS-атаки. - **Secure** — cookie передаётся только по HTTPS, не по открытому HTTP. - **SameSite** — ограничивает отправку cookie с чужих сайтов, защищает от CSRF-атак.

Браузер потом сам шлёт эту cookie в заголовке **Cookie** при каждом запросе. Сервер её читает и понимает, с кем имеет дело.

Sessions — состояние на сервере

Session (сессия) — подход, где сервер **хранит** информацию о пользователе у себя, а клиенту отдаёт только идентификатор сессии (обычно в cookie).

Как работает: 1. Пользователь логинится → сервер создаёт запись сессии (кто это, когда вошёл) в своём хранилище (память, Redis, БД) и генерирует **session_id**. 2. **session_id** отдаётся клиенту в cookie. 3. При каждом запросе клиент шлёт **session_id** → сервер находит по нему сессию → знает, кто пользователь.



Плюс: сервер полностью контролирует сессии — может мгновенно «разлогинить» (удалить сессию). **Минус:** сервер должен **хранить** состояние. При многих инстансах нужно общее хранилище (Redis), иначе сессия, созданная на одном сервере, не видна другому. Это нарушает чистую statelessness.

JWT — токен, несущий информацию в себе

JWT (JSON Web Token) — принципиально другой подход. Вместо хранения сессии на сервере, вся информация о пользователе упаковывается в **сам токен**, который подписан сервером. Сервер ничего не хранит — он лишь **проверяет подпись**.

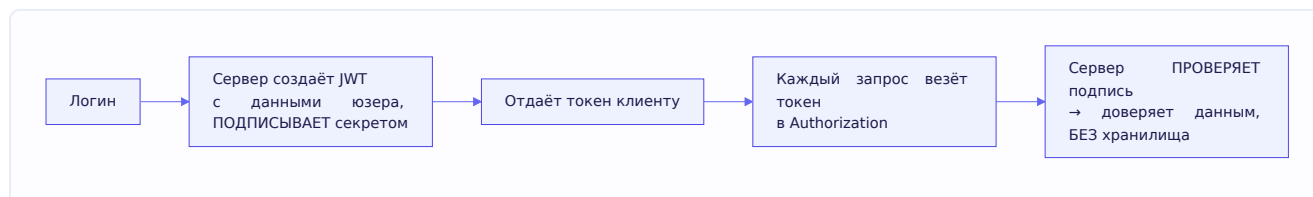
JWT состоит из трёх частей, разделённых точками: **header.payload.signature**

```
eyJhbGciOiJI... . eyJzdWIiOi... . SflKxwRj...
header      payload      signature
```

- **Header** — метаданные (алгоритм подписи, тип). Base64.
- **Payload** — сами данные (claims): id пользователя, роль, срок действия. Base64.

- **Signature** — подпись header+payload секретным ключом сервера.

Как работает: 1. Пользователь логинится → сервер создаёт JWT с данными юзера, **подписывает** его своим секретом, отдаёт клиенту. 2. Клиент шлёт JWT в каждом запросе (обычно в заголовке `Authorization: Bearer <token>`). 3. Сервер **проверяет подпись** секретным ключом. Подпись валидна → данным в токене можно доверять, сервер знает пользователя **не обращаясь ни к какому хранилищу**.



Ключевое: подпись защищает от подделки, но НЕ шифрует. Payload закодирован base64 — его **может прочитать любой** (это не шифрование!). Поэтому в JWT **нельзя класть секреты** (пароли, приватные данные). Подпись гарантирует лишь, что данные **не изменены** — если кто-то подправит payload, подпись перестанет сходиться, и сервер отвергнет токен.

JWT vs Sessions — главный компромисс

	Sessions	JWT
Где состояние	На сервере (хранилище)	В токене (у клиента)
Масштабирование	Нужно общее хранилище (Redis)	Не нужно хранилище — любой инстанс проверит подпись
Отзыв (разлогин)	Легко — удалить сессию	Сложно — токен валиден до истечения
Размер	Маленький id в cookie	Токен крупнее (везёт все данные)

Главный компромисс: **JWT избавляет от хранилища сессий** (отлично для stateless-микросервисов и масштабирования), но **труднее отзывается** — раз сервер ничего не хранит, он не может «забыть» токен до истечения его срока. Обходят это коротким временем жизни токена + refresh-токенами, или чёрным списком отозванных токенов (что частично возвращает состояние на сервер).

Практический выбор: JWT хорош для распределённых систем и API, где важна безстатусность; сессии — когда нужен строгий контроль (мгновенный разлогин) и сервер и так имеет хранилище. Часто комбинируют.

6.8. CORS

CORS (Cross-Origin Resource Sharing) — механизм, который контролирует, может ли веб-страница с одного источника (origin) обращаться к API на другом источнике. Тема, которая путает многих, поэтому разберём с самого начала — что за проблему она решает.

Same-Origin Policy — откуда растёт CORS

Браузеры по умолчанию соблюдают **Same-Origin Policy** (политику одного источника): JavaScript на странице `https://myapp.com` **не может** свободно делать запросы к `https://api.thersite.com`. Это защита: без неё зловерный сайт мог бы от твоего имени дёргать API банка, где ты залогинен, и красть данные.

Origin (источник) — это комбинация схемы + домена + порта. `https://myapp.com` и `https://api.myapp.com` — **разные** источники (разный поддомен). `http://site.com` и `https://site.com` — тоже разные (разная схема).

Что делает CORS

CORS — это способ для сервера **разрешить** определённым чужим источникам обращаться к нему. Сервер через специальные заголовки говорит браузеру: «запросам с origin `https://myapp.com` я доверяю, пропусти их».

Ключевой заголовок ответа:

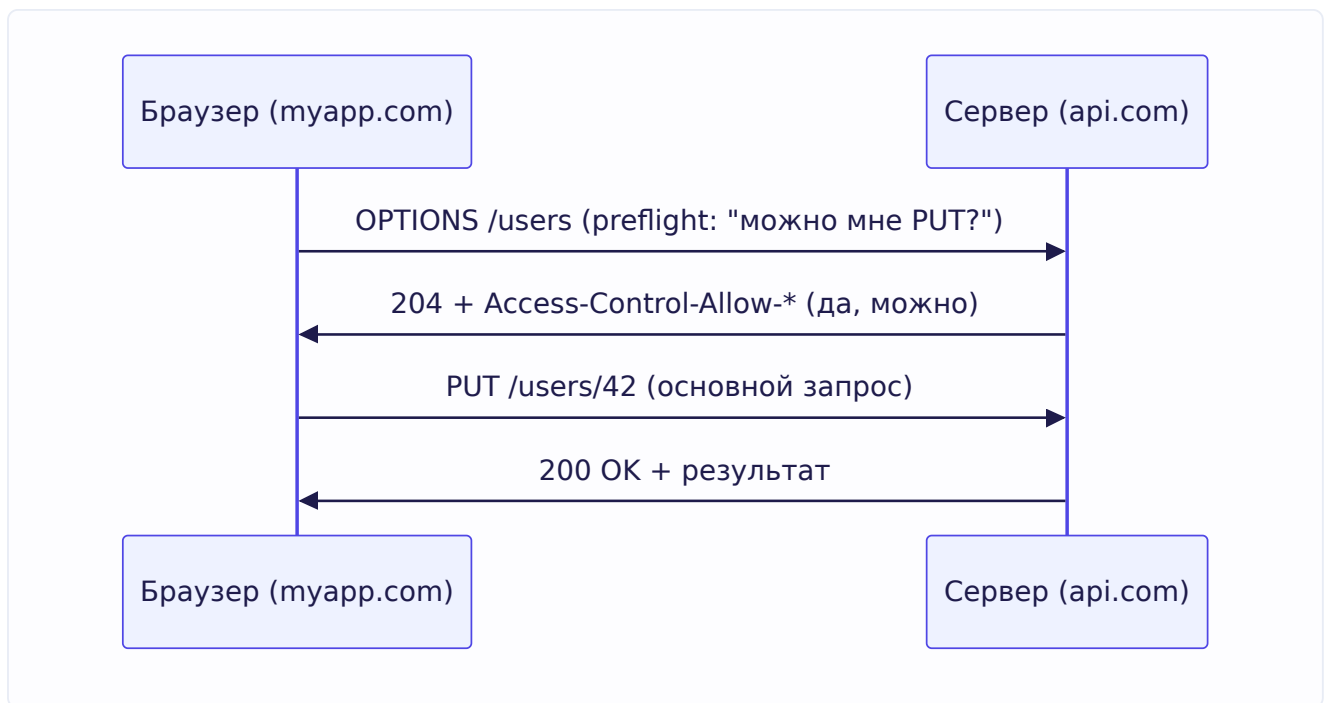
```
Access-Control-Allow-Origin: https://myapp.com
```

Это сервер сообщает браузеру: «страницам с `https://myapp.com` можно читать мои ответы». Другие заголовки уточняют, какие методы и заголовки разрешены:

```
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
Access-Control-Allow-Headers: Content-Type, Authorization
Access-Control-Allow-Credentials: true
```

Preflight-запрос

Для «непростых» запросов (например, с методом PUT/DELETE или с заголовком `Authorization`) браузер сначала шлёт **preflight-запрос** методом `OPTIONS` — спрашивает разрешение **до** основного запроса:



Браузер как бы «стучится»: можно ли мне сделать PUT с такими заголовками? Сервер отвечает разрешающими заголовками — и только тогда браузер шлёт настоящий запрос.

CORS в Go

Реализуется обычно как middleware (6.5), выставляющий нужные заголовки:

```
func CORSMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        w.Header().Set("Access-Control-Allow-Origin", "https://myapp.com")
        w.Header().Set("Access-Control-Allow-Methods", "GET, POST, PUT, DELETE, OPTIONS")
        w.Header().Set("Access-Control-Allow-Headers", "Content-Type, Authorization")

        // Preflight OPTIONS — отвечаем сразу, без вызова основного обработчика:
        if r.Method == http.MethodOptions {
            w.WriteHeader(http.StatusNoContent)
            return
        }

        next.ServeHTTP(w, r)
    })
}
```

Важная деталь безопасности: не ставь бездумно `Access-Control-Allow-Origin: *` (разрешить всем) на API с аутентификацией — это открывает данные любому сайту. Разрешай только доверенные источники. `*` уместен для полностью публичных API без приватных данных.

Ключевое понимание: **CORS — это защита браузера, а не сервера**. Это браузер отказывается отдавать ответ JavaScript'у, если сервер не разрешил origin. Запрос к серверу вне браузера (из Go, curl, мобильного приложения) CORS вообще не касается — там нет Same-Origin Policy. Поэтому CORS-ошибки видны только в браузере.

6.9. TLS и HTTPS

HTTPS — это HTTP поверх **TLS** (Transport Layer Security). TLS обеспечивает три вещи: шифрование, целостность и аутентификацию. Разберём, что это даёт и как работает.

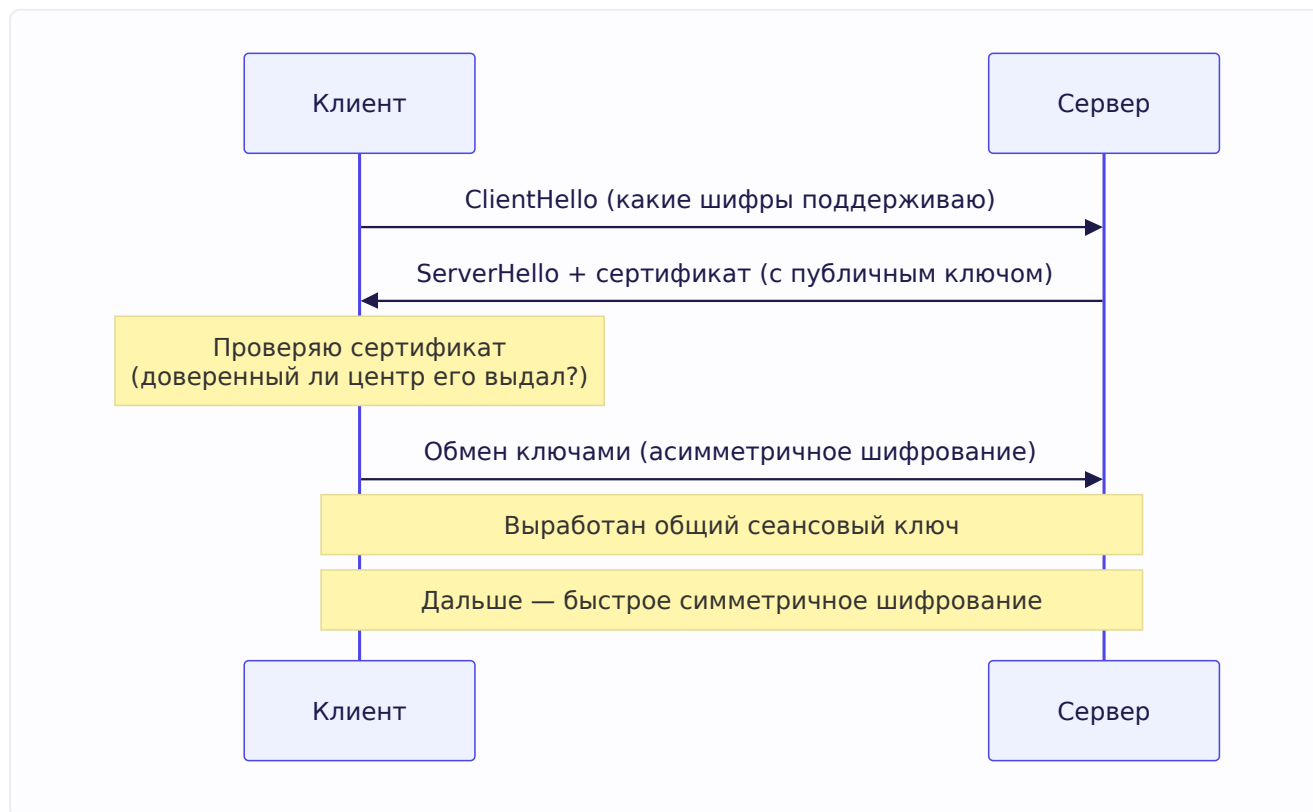
Зачем нужен TLS

Обычный HTTP передаёт данные **открытым текстом**. Любой на пути (провайдер, владелец Wi-Fi, злоумышленник в той же сети) может их прочитать и подменить. Для паролей, платежей, частных данных это недопустимо. TLS решает три проблемы:

1. **Шифрование (confidentiality)** — данные зашифрованы, перехватчик видит только «мусор».
2. **Целостность (integrity)** — данные нельзя незаметно изменить в пути.
3. **Аутентификация (authentication)** — клиент убеждается, что общается с настоящим сервером (тем, чьё имя в адресе), а не с подставным.

Как TLS устанавливает защищённое соединение

Упрощённо, TLS-рукопожатие (поверх уже установленного TCP-соединения):



Ключевая идея — **гибридное шифрование**: 1. Сначала используется **асимметричное** шифрование (пара публичный/приватный ключ) — чтобы безопасно договориться об общем секрете, даже если кто-то подслушивает. Оно надёжное, но медленное. 2. Договорившись об общем **сеансовом ключе**, стороны переходят на **симметричное** шифрование (один общий ключ) — оно быстрое. Им и шифруется весь дальнейший обмен.

Так объединяют безопасность асимметричного (для обмена ключом) и скорость симметричного (для данных).

Сертификаты и центры сертификации

Как клиент убеждается, что сервер настоящий? Через **сертификат**. Сертификат сервера подписан **центром сертификации (CA, Certificate Authority)** — доверенной организацией. Браузеры/ОС имеют список доверенных СА. Если сертификат сервера подписан доверенным СА и выдан на нужный домен — клиент ему верит. Если подпись невалидна или СА неизвестен — предупреждение «небезопасное соединение».

Это защищает от подмены: злоумышленник не сможет предъявить валидный сертификат на чужой домен, потому что не сможет получить подпись СА на него.

TLS в Go

Go делает HTTPS простым:

```
// Запуск HTTPS-сервера — нужны файлы сертификата и приватного ключа:  
srv.ListenAndServeTLS("cert.pem", "key.pem")
```

Приятный бонус: как упоминалось в 6.2, при использовании TLS Go **автоматически включает HTTP/2** — дополнительно настраивать не нужно. Для получения сертификатов в проде используют Let's Encrypt (бесплатные автообновляемые сертификаты), часто через пакет golang.org/x/crypto/acme/autocert.

6.10. WebSocket

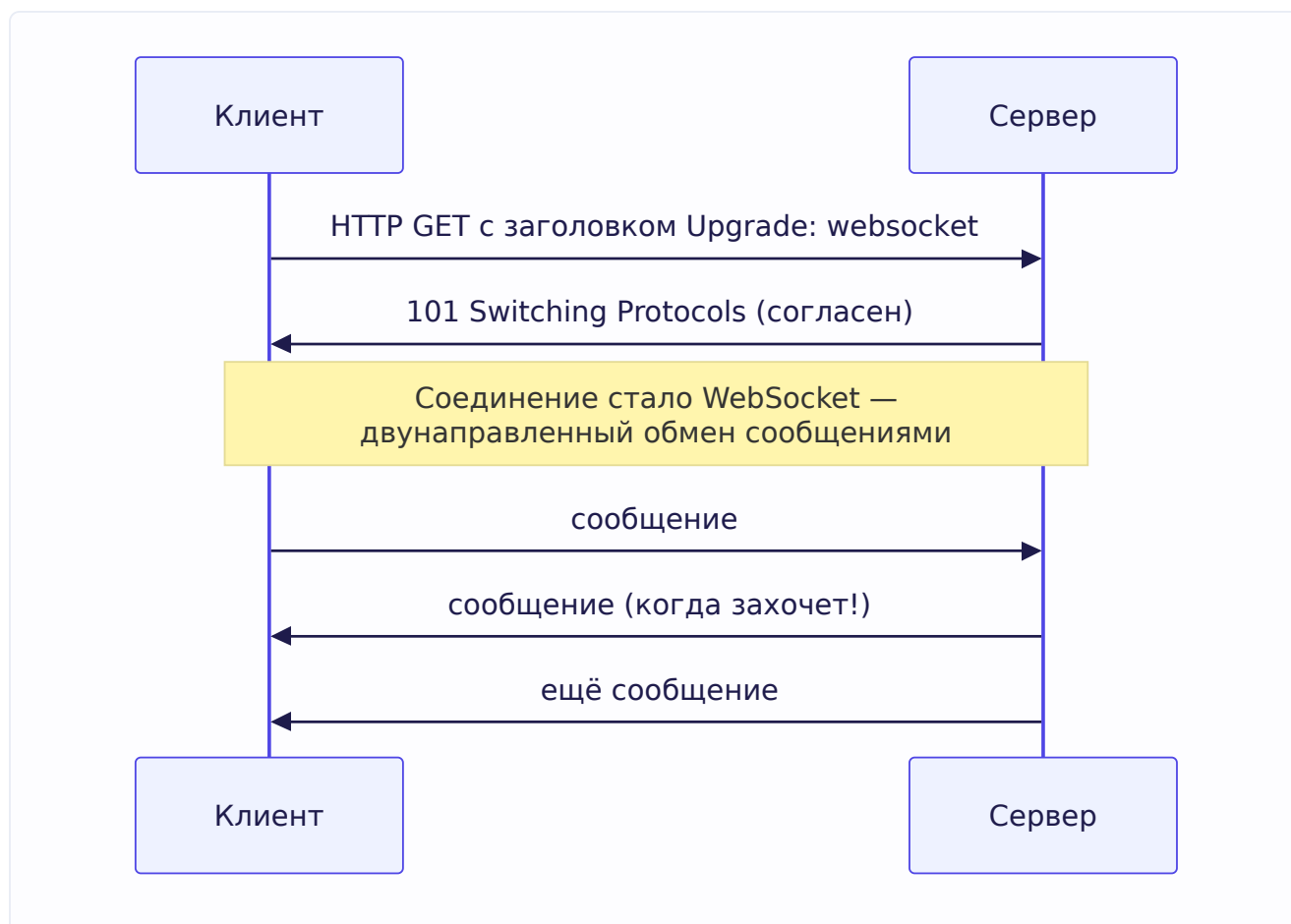
HTTP хорош для «запрос-ответ», но плох, когда серверу нужно **сам** присылать данные клиенту в реальном времени (чат, уведомления, живые обновления). Опрашивать сервер в цикле (polling) неэффективно. Решение — **WebSocket**.

Что такое WebSocket

WebSocket — протокол, дающий **постоянное двунаправленное** соединение между клиентом и сервером. В отличие от HTTP (где клиент спрашивает — сервер отвечает, и всё), WebSocket держит соединение открытым, и **обе стороны** могут слать сообщения в любой момент.

Как устанавливается — upgrade из HTTP

Красота в том, что WebSocket **начинается как обычный HTTP-запрос**, а затем «повышается» (upgrade) до WebSocket:



Клиент шлёт обычный HTTP-запрос с заголовком `Upgrade: websocket`. Сервер отвечает кодом `101 Switching Protocols` — и то же самое TCP-соединение превращается из HTTP в WebSocket. Дальше по нему летят сообщения в обе стороны, пока кто-то не закроет.

Почему через HTTP-upgrade, а не отдельный протокол? Чтобы работать через существующую веб-инфраструктуру (порты 80/443, прокси, файрволы) — WebSocket «маскируется» под HTTP на старте.

WebSocket в Go

Стандартная библиотека WebSocket не включает, используют популярные пакеты (`gorilla/websocket`, `nhooyr.io/websocket`). Схематично:


```
func wsHandler(w http.ResponseWriter, r *http.Request) {
    conn, err := upgrader.Upgrade(w, r, nil) // повышаем HTTP → WebSocket
    if err != nil {
        return
    }
    defer conn.Close()

    for {
        // читаем сообщение от клиента:
        _, msg, err := conn.ReadMessage()
        if err != nil {
            break // соединение закрыто
        }
        // отвечаем (или шлём когда угодно):
        conn.WriteMessage(websocket.TextMessage, msg) // эхо
    }
}
```

Здесь снова сходятся темы: каждое WebSocket-соединение обслуживается в своей горутине (модель из 6.4), и благодаря дешёвым горутинам + netpoller (глава 2) Go может держать **десятки тысяч** одновременных WebSocket-соединений — это делает его популярным для real-time систем (чаты, игры, живые дашборды, биржевые котировки).

Когда WebSocket, а когда нет

WebSocket не всегда нужен. Ориентир: - **WebSocket** — когда нужна **двунаправленная** связь в реальном времени: чат, совместное редактирование, игры, живые уведомления. - **Обычный HTTP** (запрос-ответ) — для обычных API-операций. - **Server-Sent Events (SSE)** — если нужен только поток **от сервера к клиенту** (уведомления, лента), без обратного канала — SSE проще WebSocket.

Не тащи WebSocket туда, где хватает REST — постоянные соединения дороже в поддержке и масштабировании.

Итоги главы

Большая практическая глава про то, как работает веб под твоим кодом. Главное:

Сетевой фундамент: данные идут слоями (IP — адрес машины, порт — адрес программы). TCP — надёжный с соединением (рукопожатие SYN/SYN-ACK/ACK), медленнее; UDP — быстрый без гарантий. Схема TCP: bind→listen→ацcept на сервере, connect на клиенте.

HTTP: текстовый (в 1.1) протокол запрос-ответ, stateless. Запрос = стартовая строка + заголовки + тело; ответ = код состояния + заголовки + тело. Методы (GET/POST/PUT/DELETE, идемпотентность), коды (4xx клиент, 5xx сервер). HTTP/2 — бинарный, мультиплексирование по одному соединению; HTTP/3 — поверх QUIC (UDP).

net/http: сервер в пару строк; всё крутится вокруг интерфейса `Handler` (`ServeHTTP`), `HandlerFunc` превращает функцию в `Handler`, `ServeMux` маршрутизирует. `http.Server` под капотом — цикл

ассерт + горутина на каждое соединение; масштабируется за счёт дешёвых горутин + netpoller (глава 2). В проде — таймауты и graceful shutdown (сигналы + context).

Прикладное: middleware — обёртки Handler для сквозных задач (логирование, recover, auth), komponуются в цепочку по принципу луковицы (LIFO, как defer). REST — ресурсы + HTTP-методы, stateless; отличается от RPC (вызов методов, любой транспорт). Аутентификация: cookies (браузер помнит), sessions (состояние на сервере), JWT (подписанный токен, состояние в нём — избавляет от хранилища, но труднее отзывается). CORS — защита браузера, сервер разрешает чужие origin заголовками. TLS/HTTPS — гибридное шифрование (асимметрия для обмена ключом, симметрия для данных) + сертификаты от CA. WebSocket — постоянное двунаправленное соединение через HTTP-upgrade, отлично ложится на горутину Go.

В следующей главе — gRPC и Protobuf: как устроена высокопроизводительная коммуникация между микросервисами поверх HTTP/2, которую ты используешь в Uptime Monitor.

Глава 7. gRPC и Protobuf

Введение к главе

В главе 6 мы разобрали REST — стиль API поверх HTTP, ориентированный на ресурсы. Но для **внутренней** коммуникации между микросервисами часто нужно другое: максимальная скорость, строгие контракты, поддержка стриминга. Здесь на сцену выходит **gRPC** — фреймворк удалённого вызова процедур от Google.

Эта глава особенно близка тебе: в твоём Uptime Monitor gRPC используется для быстрых синхронных сценариев между сервисами (в отличие от Kafka для фоновых задач). Так что здесь ты не просто готовишься к вопросам, но и укрепляешь понимание собственного проекта — что важно для его защиты на собеседовании.

Разберём: что такое Protobuf (язык описания контрактов), как устроен gRPC поверх HTTP/2, четыре типа вызовов, сравнение с REST и interceptors (аналог middleware).

7.1. Protobuf — язык описания контрактов

Прежде чем говорить о gRPC, нужно понять **Protocol Buffers (Protobuf)** — потому что gRPC на нём построен.

Что такое Protobuf

Protobuf — это, во-первых, **формат сериализации** данных (как JSON, но бинарный и компактный), и во-вторых, **язык описания** структуры этих данных (схемы). Ты описываешь свои данные и сервисы в специальном `.proto` файле, а затем инструмент генерирует из него код на нужном языке (Go, Java, Python и т.д.).

Пример .proto файла

```
syntax = "proto3"; // версия синтаксиса

package uptime;

option go_package = "./uptimepb"; // куда генерировать Go-код

// Сообщение (структура данных):
message CheckRequest {
    string url = 1;        // поле с номером 1
    int32 timeout = 2;    // поле с номером 2
}

message CheckResponse {
    bool is_up = 1;
    int32 status_code = 2;
    int64 response_time_ms = 3;
}

// Сервис (набор методов):
service Checker {
    rpc Check(CheckRequest) returns (CheckResponse);
}
```

Разберём ключевые элементы:

- **message** — описание структуры данных (аналог struct). Внутри — поля.
- **Номера полей** (**= 1**, **= 2**) — вот это важно. Каждое поле имеет **номер**, и именно номер, а не имя, используется в бинарном формате. Это даёт эволюцию схемы: можно переименовать поле (номер тот же — совместимость сохранится) или добавить новое (с новым номером — старые клиенты его проигнорируют).
- **service** — набор удалённых методов (RPC), которые можно вызывать.
- **rpc Check(CheckRequest) returns (CheckResponse)** — метод: принимает **CheckRequest**, возвращает **CheckResponse**.

Кодогенерация

Из **.proto** файла инструмент **protoc** (с плагинами для Go) **генерирует** Go-код: структуры для сообщений, интерфейсы для сервиса, клиентский и серверный код.

```
protoc --go_out=. --go-grpc_out=. checker.proto
```

Получаешь готовые Go-типы **CheckRequest**, **CheckResponse** и интерфейс **CheckerServer**, который тебе остаётся реализовать. Всю рутину сериализации/десериализации и сетевого взаимодействия сгенерированный код берёт на себя.

Это принципиальное отличие от REST+JSON: там ты вручную описываешь структуры и пишешь (де)сериализацию, а контракт между клиентом и сервером держится «на честном слове» (документация, договорённости). В Protobuf **контракт — это сам .proto файл**, и код по нему генерируется автоматически, синхронно для обеих сторон.

Почему бинарный формат компактнее JSON

Сравним. JSON — текст: `{"url":"http://ozon.ru","timeout":30}` — передаются имена полей строками, значения как текст, скобки, кавычки. Protobuf кодирует то же самое **бинарно**: вместо имени поля — его номер (одно-два байта), значения — в компактном бинарном виде. Результат:

- **Меньше размер** — нет повторяющихся имён полей и синтаксического «мусора».
- **Быстрее парсинг** — бинарные данные разбираются быстрее, чем текстовый JSON (не нужно парсить строки, искать кавычки).
- **Строгая типизация** — типы полей заданы в схеме, не нужно догадываться.

Плата — **не читается человеком** (в отличие от JSON, который можно глазами прочитать в логах). Для внутренней коммуникации между сервисами это приемлемо, ради скорости.

7.2. gRPC под капотом

Теперь — сам **gRPC**. Это фреймворк для удалённого вызова процедур (RPC), использующий Protobuf для описания контрактов и сериализации, а HTTP/2 — как транспорт.

Идея RPC

Вспомни из главы 6 разницу REST/RPC: RPC — это **вызов удалённой процедуры**, как будто ты вызываешь обычную функцию, но она выполняется на другом сервере. gRPC доводит эту идею до максимального удобства: сгенерированный код делает удалённый вызов **похожим на локальный**.

```
// Выглядит как обычный вызов функции, но идёт по сети на другой сервис:
resp, err := checkerClient.Check(ctx, &CheckRequest{
    Url:      "http://ozon.ru",
    Timeout: 30,
})
if err != nil {
    // обработка ошибки
}
fmt.Println(resp.IsUp, resp.StatusCode)
```

Ты вызываешь `checkerClient.Check(...)` — а под капотом: аргумент сериализуется в Protobuf, отправляется по HTTP/2 на сервер, там десериализуется, выполняется настоящий метод, результат едет обратно. Вся эта машинерия скрыта в сгенерированном коде.

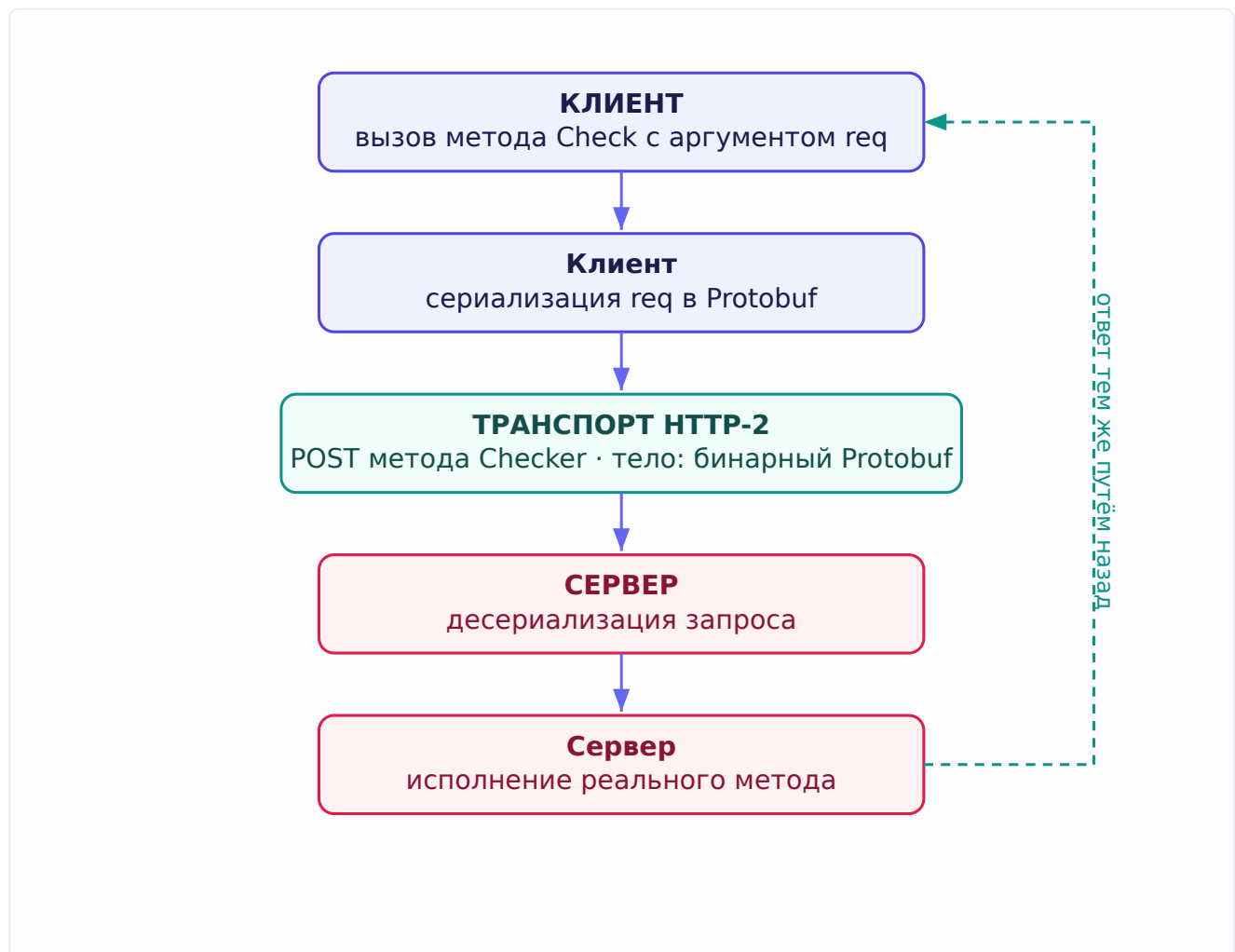
Почему поверх HTTP/2

gRPC построен на **HTTP/2** (глава 6), и это осознанный выбор — свойства HTTP/2 идеально подходят для RPC:

- **Мультиплексирование** — много параллельных вызовов по одному соединению, без head-of-line blocking. Микросервис может делать десятки одновременных gRPC-вызовов через одно соединение.

- **Бинарность** — HTTP/2 бинарный, как и Protobuf; хорошо сочетаются.
- **Стриминг** — потоки (streams) HTTP/2 позволяют реализовать потоковые вызовы (см. 7.3).
- **Сжатие заголовков** — экономия на метаданных.

Из главы 6 (задачник, вопрос про RPC поверх HTTP): gRPC как раз «перекладывает RPC на HTTP» — имя сервиса и метода кодируется в HTTP/2 (в псевдо-заголовке `:path` вида `/uptime.Checker/Check`), а тело несёт сериализованные Protobuf-данные.



7.3. Unary и streaming RPC

gRPC поддерживает **четыре типа** вызовов — это частый вопрос. Разница в том, кто и сколько сообщений шлёт: одно или поток. Возможность стриминга — прямое следствие того, что gRPC на HTTP/2.

1. Unary RPC — обычный вызов

Самый привычный: один запрос → один ответ. Как обычная функция.

```
rpc Check(CheckRequest) returns (CheckResponse);
```

Клиент шлёт один `CheckRequest`, получает один `CheckResponse`. Большинство вызовов — именно unary.

2. Server streaming — поток от сервера

Клиент шлёт один запрос, а сервер отвечает **потоком** сообщений. Полезно, когда результат большой или приходит порциями во времени.

```
rpc WatchStatus(WatchRequest) returns (stream StatusUpdate);
```

Пример: клиент подписывается на обновления статуса ресурса, сервер шлёт `StatusUpdate` каждый раз, когда статус меняется — поток обновлений по одному запросу.

3. Client streaming — поток от клиента

Клиент шлёт **поток** сообщений, сервер отвечает одним. Полезно для загрузки/агрегации.

```
rpc UploadMetrics(stream Metric) returns (UploadSummary);
```

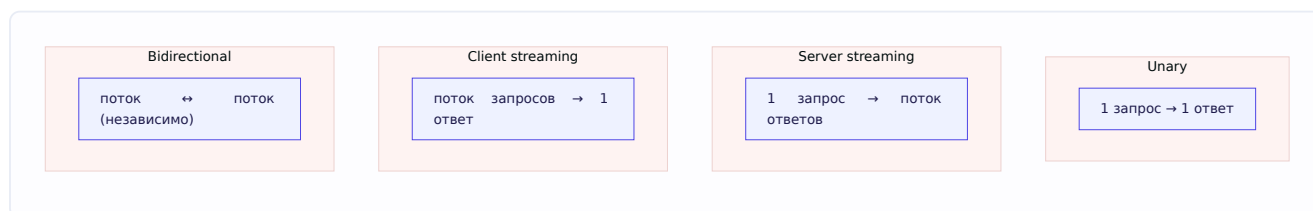
Пример: клиент шлёт поток метрик, сервер их накапливает и в конце отвечает одной сводкой.

4. Bidirectional streaming — двунаправленный поток

Обе стороны шлют потоки **независимо**. Похоже на WebSocket по возможностям.

```
rpc Chat(stream Message) returns (stream Message);
```

Пример: чат, где клиент и сервер обмениваются сообщениями в реальном времени, каждый в своём темпе.



Стриминг — сильная сторона gRPC, которой нет в обычном REST. Он возможен именно благодаря потокам HTTP/2. В Go потоковые вызовы реализуются через объект-поток с методами `Send` / `Recv`, и каждое соединение/поток естественно ложится на горутину.

7.4. gRPC vs REST — когда что выбирать

Важнейший практический вопрос, и он напрямую про твой проект (где ты сознательно выбрал gRPC для одних сценариев). Сравним честно.

Аспект	gRPC	REST
Транспорт	HTTP/2	Обычно HTTP/1.1
Формат	Protobuf (бинарный)	Обычно JSON (текст)
Контракт	Строгий <code>.proto</code> , кодогенерация	Обычно неформальный (доков/OpenAPI)
Скорость	Выше (бинарь + HTTP/2)	Ниже
Стриминг	Да (4 типа)	Ограниченно (SSE/WebSocket отдельно)
Читаемость	Бинарь, глазами не прочтёшь	JSON читается человеком
Браузеры	Нужен gRPC-Web (прослойка)	Работает везде из коробки
Кто вызывает	Сервис ↔ сервис (внутри)	Внешние клиенты, публичные API

Практический выбор

gRPC хорош для: - **Внутренней коммуникации между микросервисами** — там, где обе стороны твои, важна скорость и строгие контракты. - Высоконагруженных сценариев (бинарь быстрее JSON). - Стриминга (потокковые данные).

REST хорош для: - **Публичных API** — их вызывают разные внешние клиенты, важна простота и совместимость. - Браузерных клиентов (REST работает напрямую, gRPC — только через прослойку gRPC-Web). - Случаев, где важна читаемость и отладка «глазами».

Связь с твоим проектом

В Uptime Monitor у тебя разумное разделение, которое стоит уметь объяснить: - **gRPC** — для **синхронных** сценариев между сервисами, где нужен быстрый ответ здесь и сейчас (например, ручная проверка ресурса по требованию: Manager синхронно дёргает Checker и ждёт результат). - **Kafka** — для **асинхронных** фоновых задач (регулярные проверки: задачи публикуются в очередь, обрабатываются в фоне, результаты складываются обратно).

Это грамотный архитектурный выбор: синхронный быстрый путь (gRPC) для интерактивных операций, асинхронная очередь (Kafka) для фоновой массовой обработки. На собеседовании такое разделение «gRPC для синхронного, очередь для асинхронного» — сильный сигнал понимания, когда какой инструмент уместен.

7.5. Interceptors — аналог middleware

Помнишь middleware из главы 6 — обёртки вокруг HTTP-обработчиков для сквозных задач? В gRPC та же идея называется **interceptors (перехватчики)**.

Что это

Interceptor — код, который выполняется до и/или после вызова gRPC-метода, оборачивая его. Решает те же сквозные задачи: логирование, аутентификация, метрики, восстановление после

паники, трейсинг.

Есть два вида (параллель с типами вызовов): - **Unary interceptor** — для обычных (unary) вызовов. - **Stream interceptor** — для потоковых вызовов.

И с двух сторон: - **Серверные** — оборачивают обработку входящих вызовов на сервере. - **Клиентские** — оборачивают исходящие вызовы на клиенте.

Пример серверного unary-interceptor

```
func LoggingInterceptor(
    ctx context.Context,
    req interface{},
    info *grpc.UnaryServerInfo,
    handler grpc.UnaryHandler,
) (interface{}, error) {
    start := time.Now()

    log.Printf("→ gRPC вызов: %s", info.FullMethod) // ДО

    resp, err := handler(ctx, req) // вызываем сам метод

    log.Printf("← %s за %v, ошибка: %v", info.FullMethod, time.Since(start), err) // ПОСЛЕ
    return resp, err
}
```

Структура — точь-в-точь как HTTP middleware: что-то до, вызов `handler` (сам метод), что-то после. `info.FullMethod` даёт имя вызываемого метода, `handler(ctx, req)` — точка перехода к реальной обработке.

Регистрируется при создании сервера:

```
srv := grpc.NewServer(
    grpc.UnaryInterceptor(LoggingInterceptor),
)
```

Роль context в gRPC

Обрати внимание: первым аргументом везде идёт `context.Context` (глава 3). В gRPC context — центральный механизм: через него propagation отмены и таймаутов работает **сквозь сетевую границу**. Если клиент отменил вызов или истёк дедлайн — это распространяется на серверную обработку, и сервер может прекратить ненужную работу. Это ровно тот механизм отмены, что мы разбирали в главе 3, но теперь работающий между машинами. Красиво, как context оказывается единым языком отмены и для локальных горутин, и для распределённых вызовов.

Итоги главы

Компактная, но важная глава про высокопроизводительную межсервисную коммуникацию. Главное:

Protobuf — бинарный формат сериализации + язык описания контрактов в `.proto` файлах. Поля имеют номера (для эволюции схемы). Из `.proto` генерируется код (`protoc`). Компактнее и быстрее JSON, но не читается глазами; контракт строгий и синхронный для обеих сторон.

gRPC — RPC-фреймворк на Protobuf + HTTP/2. Удалённый вызов выглядит как локальный (сгенерированный код прячет сериализацию и сеть). HTTP/2 даёт мультиплексирование, стриминг, сжатие заголовков.

Четыре типа вызовов: unary (1→1), server streaming (1→поток), client streaming (поток→1), bidirectional (поток↔поток). Стриминг возможен благодаря потокам HTTP/2 — сильная сторона gRPC.

gRPC vs REST: gRPC — для внутренней межсервисной связи (скорость, строгие контракты, стриминг); REST — для публичных API и браузеров (простота, читаемость, совместимость). В твоём Uptime Monitor: gRPC для синхронных сценариев, Kafka для асинхронных фоновых задач — грамотное разделение.

Interceptors — аналог middleware для сквозных задач (логирование, auth, метрики), unary/stream, серверные/клиентские. Context пронизывает gRPC, распространяя отмену и таймауты через сетевую границу.

В следующей главе поднимаемся на уровень системного дизайна: rate limiting, масштабирование, проху, обработка ненадёжных зависимостей — те архитектурные задачи, что спрашивают на собеседовании.

Глава 8. Архитектура и системный дизайн

Введение к главе

Предыдущие главы были про язык и его механику. Эта — про **инженерное мышление**: как проектировать системы, которые выдерживают нагрузку, не падают от одной медленной зависимости и масштабируются. Именно эти вопросы отличают джуна от джуна+ на собеседовании — здесь редко есть один правильный ответ, важнее умение рассуждать о компромиссах.

Все темы главы взяты из архитектурного раздела задачника Озона, и у каждой — типичная структура собеседовательной задачи: дана ситуация, нужно предложить решение и обсудить его плюсы и минусы. Я буду разбирать именно так — не только «как правильно», но и «почему», и «какой ценой».

Важная установка для всей главы: в системном дизайне **почти нет однозначно правильных ответов**. Всё зависит от контекста — нагрузки, требований, инфраструктуры. Хороший ответ на собеседовании — не «делай вот так», а «есть такие варианты, вот их компромиссы, в данной ситуации я бы выбрал этот, потому что...». Этому и будем учиться.

8.1. REST vs RPC — фундаментальная разница

Мы уже касались этого в главах 6 и 7, но соберём вместе как архитектурное решение, потому что в задачнике это отдельный вопрос.

Суть различия

- **REST** строится вокруг **ресурсов** и стандартных операций над ними (GET, PUT, DELETE). Мыслишь существительными: «пользователь», «заказ», а действия — это HTTP-методы.
- **RPC** строится вокруг **вызова методов** сервиса. Мыслишь глаголами: «проверить статус», «пересчитать цену» — вызываешь удалённую процедуру.

Ключевые вопросы из задачника

Разберём тонкости, которые проверяют на собеседовании:

Обязательно ли делать REST поверх HTTP? По сути да — REST как стиль определён в терминах HTTP (методы, коды, заголовки, ресурсы через URL). REST без HTTP теряет смысл.

Обязательно ли делать RPC поверх HTTP? Нет — RPC можно строить поверх чего угодно: HTTP, голого TCP, очередей сообщений. RPC — это идея «вызвать удалённую функцию», транспорт вторичен.

Как переложить RPC-запрос на HTTP? Передавать имя сервиса и метода в URL или заголовке, а аргументы — в теле запроса. Именно так работает gRPC (глава 7): путь `/uptime.Checker/Check` кодирует сервис и метод, тело несёт Protobuf-данные.

Как сделать RPC поверх голого TCP? Придумать свой формат: например, слать длину сообщения, затем имя метода, затем сериализованные аргументы — и договориться о таком протоколе между клиентом и сервером. Это упражнение показывает, что RPC — это про соглашение о вызове, а не про конкретный транспорт.

Архитектурный выбор

Практический ориентир (повторим из главы 7, но как решение дизайна): REST — для публичных API и внешних клиентов (простота, совместимость, читаемость); RPC/gRPC — для внутренней межсервисной связи (скорость, строгие контракты, стриминг). Выбор зависит от того, кто потребитель API и что важнее — универсальность или производительность.

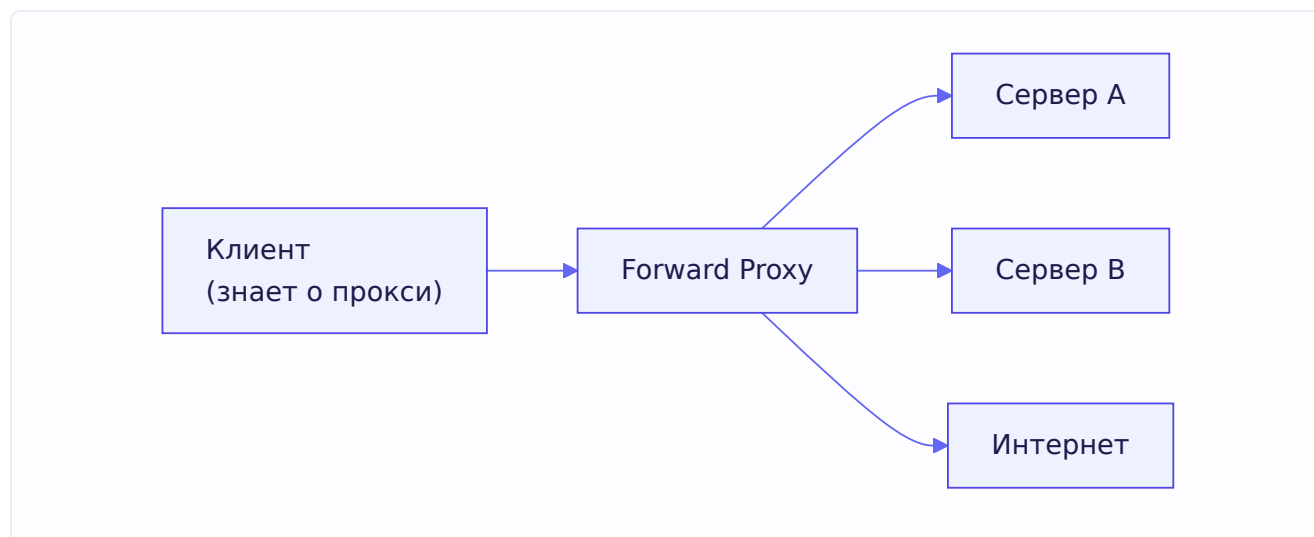
8.2. Proxy vs Reverse proxy

Два похожих по названию, но разных по назначению механизма. Их путают, поэтому разберём чётко (вопрос из задачника).

Прямой proxy (forward proxy)

Прокси (прямой прокси) стоит **на стороне клиента** и проксирует его запросы во внешний мир. Ключевое: **клиент знает** о проксе и сознательно ходит через него.

Зачем нужен: обойти ограничения (доступ к заблокированным ресурсам), кешировать, фильтровать исходящий трафик, скрыть клиента от целевого сервера (сервер видит прокси, а не реального клиента).

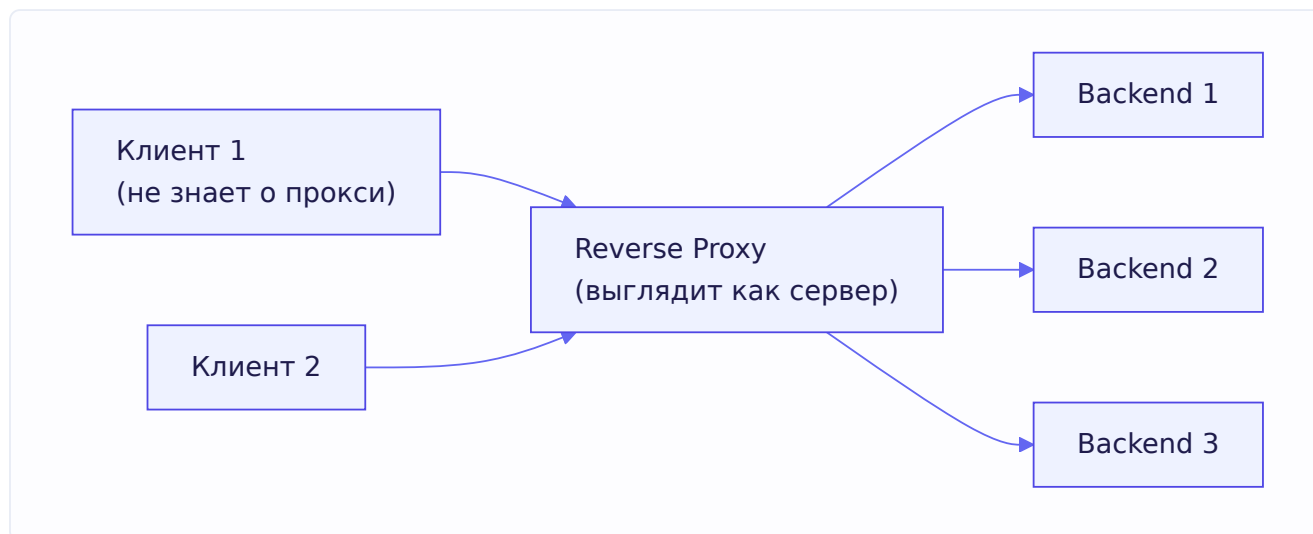


Reverse proxy (обратный прокси)

Reverse proxy стоит **на стороне сервера** и принимает запросы от клиентов, распределяя их между внутренними серверами. Ключевое: **клиент НЕ знает** о его существовании — для клиента reverse proxy выглядит как сам сервер.

Из задачника, суть различия: в случае прямого прокси клиент знает, что запрос идёт через прокси в точку назначения. При reverse proxy клиент ничего не знает о его существовании и не

знает, кто конкретно обработает запрос.



Зачем нужен reverse proxy — целый набор важнейших задач: - **Балансировка нагрузки** — распределять запросы между несколькими backend-серверами (см. 8.4). - **Терминация TLS** — расшифровывать HTTPS в одном месте, разгружая backend'ы. - **Кеширование** — отдавать закешированные ответы, не беспокоя backend. - **Единая точка входа** — скрыть внутреннюю структуру, дать один адрес наружу. - **Защита** — фильтровать вредоносные запросы до backend'ов.

Типичные reverse proxy: nginx, HAProxy, Envoy. Разница в одной фразе: **forward proxy защищает/обслуживает клиента, reverse proxy защищает/обслуживает сервер.**

8.3. Rate limiting — ограничение частоты запросов

Rate limiter ограничивает, сколько запросов можно сделать за единицу времени. Прямая задача из задачника: сервис использует внешний API с лимитом по RPS, нужно спроектировать ограничитель. Разберём подходы и алгоритмы.

Зачем нужен

- **Защита от перегрузки** — не дать (себе или клиентам) завалить сервис запросами.
- **Соблюдение чужих лимитов** — если внешний API разрешает N запросов в секунду, нельзя его превышать (иначе бан).
- **Справедливость** — не дать одному клиенту забрать все ресурсы.
- **Защита от атак** — ограничить перебор паролей, DDoS.

Два архитектурных подхода (из задачника)

Вариант 1: с внешним хранилищем (например, Redis). Счётчик запросов хранится централизованно в Redis. Все экземпляры сервиса обращаются к нему.

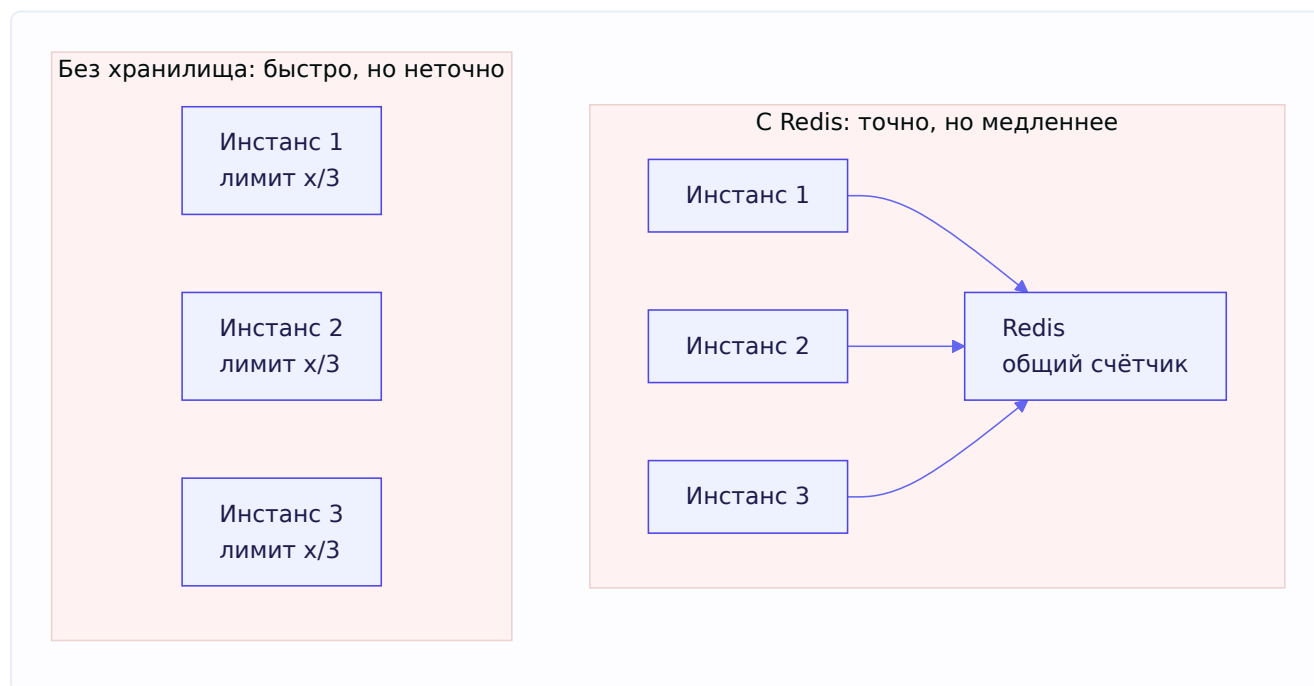
Плюсы: - **Не важно, как сбалансированы запросы** по экземплярам — счётчик общий, лимит соблюдается точно глобально.

Минусы: - **Увеличение времени запроса** — каждый запрос идёт в Redis (сетевой round-trip). - **Единая точка отказа** — упал Redis, сломался rate limiting.

Вариант 2: без внешнего хранилища (per-instance). Каждый инстанс имеет свой локальный лимит x / n , где x — общий лимит, n — число инстансов.

Плюсы: - **Нет единой точки отказа** — каждый считает сам. - **Нет дополнительных задержек** — счётчик локальный, в памяти.

Минусы: - **Сложно конфигурировать** — нужно знать число инстансов и делить лимит. - **Балансировка запросов критична** — если запросы неравномерно распределены по инстансам, суммарный лимит может нарушаться (один инстанс исчерпал свою квоту, другой простаивает, а глобально лимит вроде не достигнут).



Оптимизация работы с Redis

Из задачника: как оптимизировать работу с Redis для rate limiting? Использовать **Lua-скрипт** — процедуру, которая атомарно проверяет счётчик и возвращает true/false. Почему это важно: без атомарности между «прочитать счётчик» и «увеличить» может вклиниться другой запрос (гонка, как в главе 3, но распределённая). Lua-скрипт в Redis выполняется атомарно целиком — вся проверка-и-инкремент как одна операция, гонки нет. Плюс это один round-trip вместо нескольких.

Алгоритмы rate limiting

Стоит знать основные алгоритмы (это углубляет ответ):

Token Bucket (ведро токенов). В «ведро» с постоянной скоростью добавляются токены (до максимума). Каждый запрос забирает токен. Нет токенов — запрос отклоняется. Позволяет **всплески** (burst): накопленные токены можно потратить разом. Гибкий, популярный алгоритм.

Leaky Bucket (дырявое ведро). Запросы попадают в очередь («ведро»), из которого «вытекают» с постоянной скоростью. Сглаживает нагрузку до равномерной, но не допускает всплесков.

Fixed Window (фиксированное окно). Считаем запросы в фиксированном окне (например, за текущую секунду). Просто, но есть проблема на границах окон (можно сделать двойную порцию на стыке двух окон).

Sliding Window (скользящее окно). Учитывает запросы за последний интервал относительно текущего момента, а не в фиксированных границах. Точнее fixed window, но чуть сложнее.

В Go для локального rate limiting есть готовый golang.org/x/time/rate — реализация token bucket. Для распределённого — Redis + Lua.

Backpressure — умное снижение нагрузки

Из задачника (задача про аналитику): если просто «уменьшить RPS в конфигах» при перегрузке зависимости — плохо, начнёт копиться очередь. Правильно — **адаптивное** управление: экспоненциально снижать нагрузку, когда зависимость под давлением, и плавно повышать, когда она восстановилась. Это называется **backpressure** — обратное давление, когда система сама подстраивает темп под возможности потребителя. К этому вернёмся в 8.5.

8.4. Масштабирование сервисов

Задача из задачника: сервис А ходит в сервис В по HTTP, как масштабировать В? Разберём направления. Ключевая мысль: однозначного ответа нет, всё зависит от ситуации, но есть основные подходы.

Вертикальное vs горизонтальное масштабирование

Сначала базовое различие:

- **Вертикальное (scale up)** — дать серверу больше ресурсов (мощнее CPU, больше RAM). Просто, но есть потолок (нельзя бесконечно наращивать одну машину) и единая точка отказа.
- **Горизонтальное (scale out)** — добавить больше серверов, распределяя нагрузку между ними. Масштабируется почти неограниченно, устойчиво к отказу отдельных машин, но сложнее (нужна балансировка, синхронизация состояния).

Современный подход — обычно горизонтальный, и задача про сервис В — про него.

Два способа балансировки (из задачника)

Server-side balancing (балансировка на стороне сервера). Между А и В ставим промежуточный балансировщик (nginx, HAProxy) — это тот самый reverse proxy из 8.2. Сервис А шлёт запрос балансировщику, тот распределяет по экземплярам В.



Client-side balancing (балансировка на стороне клиента). Сервис А сам знает про все инстансы В (через service discovery) и сам выбирает, к какому обратиться. Список инстансов А получает из системы обнаружения — DNS, Consul, etcd.



Тонкости из задачника

Разберём цепочку дополнительных вопросов, которую задают:

Сколько балансировщиков в server-side схеме? Как минимум **два** — потому что один балансировщик сам становится единой точкой отказа. Нужен резерв.

Если балансировщиков несколько, как приложение узнаёт, где они? Через **DNS**. Приложение обращается к доменному имени, DNS возвращает адреса балансировщиков.

Как выглядит discovery через DNS? Через **A-записи** (доменное имя → IP-адреса) или **SRV-записи** (которые несут ещё и порт, и приоритеты). Клиент запрашивает DNS, получает список адресов, выбирает один.

Компромисс между подходами

- **Server-side** проще для клиента (он просто шлёт на один адрес), но балансировщик — дополнительный сетевой хоп и его самого надо резервировать.

- **Client-side** убирает лишний хоп (клиент идёт прямо к нужному инстансу), но усложняет клиента (ему нужна логика discovery и выбора). gRPC часто использует client-side балансировку.

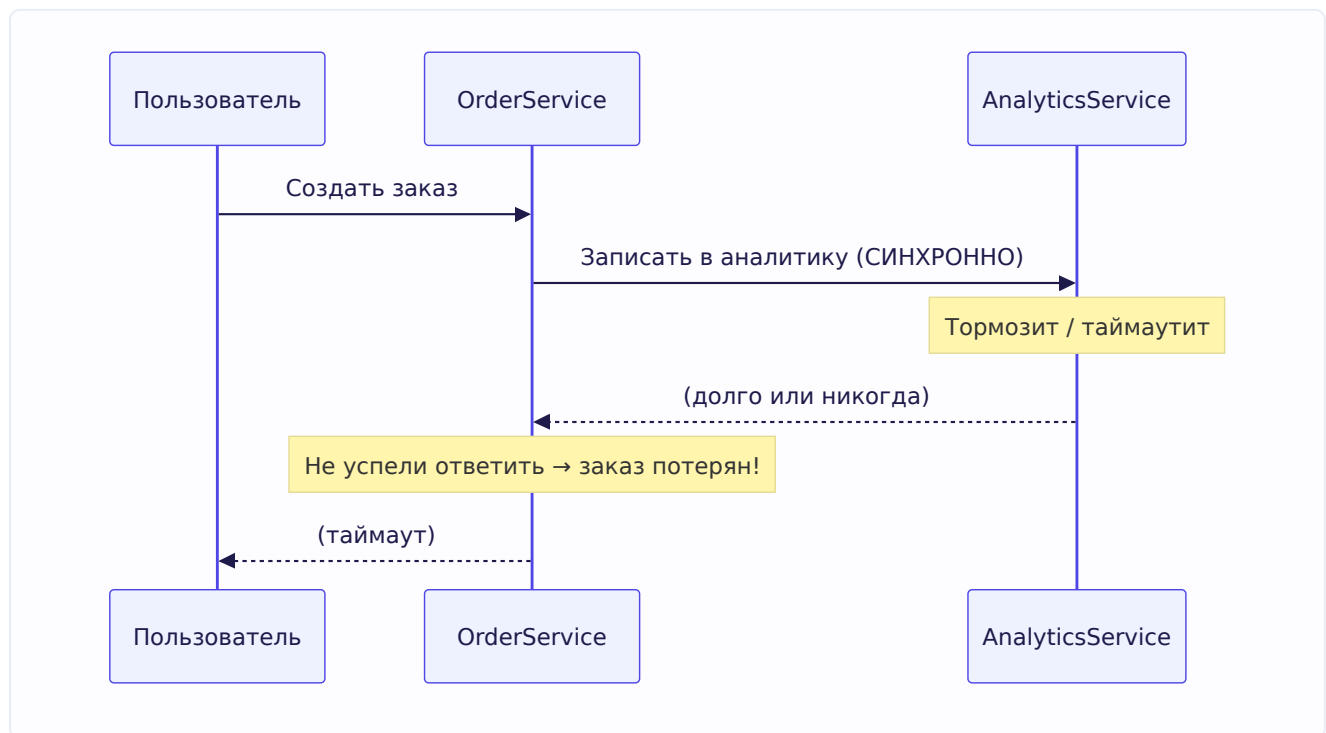
Выбор зависит от того, где терпимее сложность — в инфраструктуре (server-side) или в клиентах (client-side).

8.5. Обработка медленных и ненадёжных зависимостей

Финальная и, пожалуй, самая ценная тема — реальная production-задача из задачника, разобранный как история с развитием. Она учит мыслить об устойчивости системы.

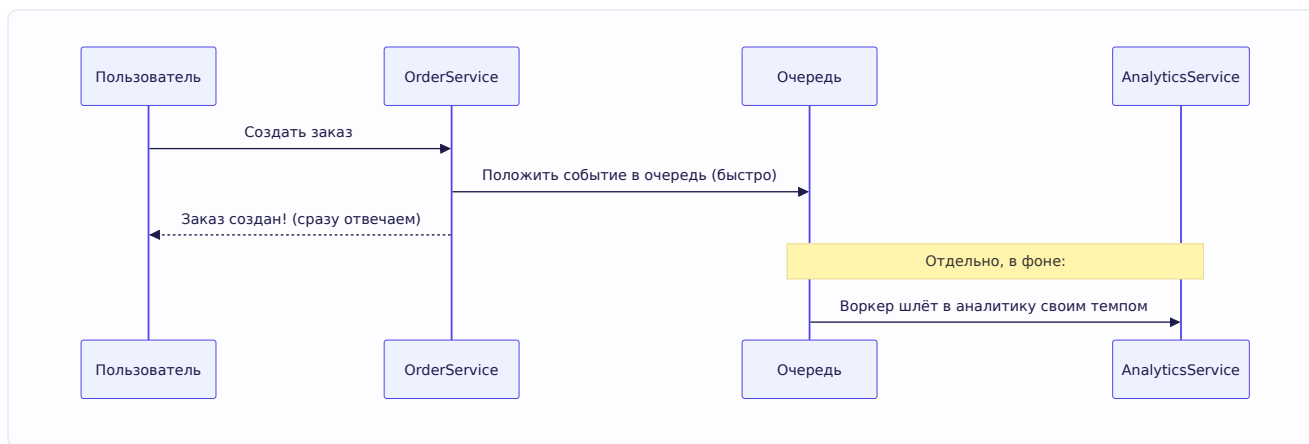
Постановка задачи

Есть создание заказа. Между запросом и ответом мы синхронно складываем товары в сервис аналитики (для подсчёта популярности). Сервис аналитики периодически тормозит или таймаутит — и мы не успеваем ответить, **теряем заказы и деньги**. Что делать?



Шаг 1: развязать заказ и аналитику

Корневая проблема — заказ **зависит** от аналитики, хотя не должен. Аналитика — второстепенная задача, а мешает основной. Решение: **сделать аналитику асинхронной**. Заказ обрабатывается сам по себе, а данные для аналитики складываются в **очередь** (или свою БД) и обрабатываются отдельно, в фоне.



Теперь ответ пользователю не ждёт аналитику — заказы не теряются. Это ровно тот паттерн, что ты применил в Uptime Monitor: Kafka для асинхронных фоновых задач, чтобы не блокировать основной путь.

Шаг 2: воркеры не должны дублировать работу

Дополнительный вопрос из задачника: как организовать воркеры, чтобы они не брали в работу одни и те же данные? Нужно, чтобы одно событие обработал только один воркер. Подходы: - **Мастер-процесс**, раздающий задачи воркерам (координатор). - Механизм, при котором воркеры не заселектят одни данные — например, `SELECT ... FOR UPDATE SKIP LOCKED` в БД (глава 9), или разделение по ключу/партиции (как в Kafka — каждая партиция читается одним консьюмером).

Шаг 3: сами пушим, если аналитика не читает

Вопрос: сервис аналитики не будет читать из нас или нашей очереди — что делать? Тогда **мы сами пушим** данные в аналитику из своей очереди (инвертируем поток: не они забирают, а мы отправляем, своим темпом).

Шаг 4: backpressure — не завалить аналитику

Вопрос: аналитика жалуется на высокий RPS от нас, под нагрузкой мы её кладём. Как быть? Просто уменьшить RPS в конфигах — плохо (будет копиться очередь). Нужно **адаптивное** управление: экспоненциально **повышать или понижать** нагрузку в зависимости от состояния аналитики. Тормозит — снижаем темп; ожила — плавно повышаем. Это и есть backpressure из 8.3. Хорошая аналогия — как TCP управляет окном перегрузки: наращивает скорость, пока всё хорошо, и резко сбрасывает при потерях.

Шаг 5: ограничены ресурсы очереди

Финальный вопрос: очередь занимает слишком много места, инфра ограничила ресурсы (расширять некуда), на аналитику повлиять не можем. Что делать? Раз данные не помещаются, нужно **просеивать** их в зависимости от квоты и загруженности очереди — **вытеснять** или **прореживать** данные для аналитики на входе. То есть сознательно жертвовать частью аналитических данных (они второстепенны), чтобы система оставалась работоспособной.

Это важный урок системного дизайна: когда ресурсы исчерпаны, приходится **осознанно деградировать** — жертвовать менее важным (полнота аналитики) ради более важного (работоспособность и заказы). Это называется **graceful degradation** — управляемая деградация вместо полного отказа.

Связанные паттерны устойчивости

Раз тема про устойчивость, стоит знать несколько классических паттернов (углубляет ответ на собеседовании):

- **Circuit Breaker (предохранитель)**. Если зависимость постоянно падает, «размыкаем цепь» — перестаём её дёргать на время, сразу возвращая ошибку/заглушку. Даём ей восстановиться, не добивая запросами. Как автоматический предохранитель в электросети.
- **Retry с backoff**. Повторять неудавшийся запрос, но с **растущими** паузами (и лучше со случайным разбросом — jitter), чтобы не завалить зависимость лавиной повторов.
- **Timeout**. Всегда ставить таймаут на внешние вызовы (через context, глава 3), чтобы не ждать вечно зависшую зависимость.
- **Fallback**. Если зависимость недоступна — вернуть запасной ответ (кеш, значение по умолчанию), а не падать.
- **Bulkhead (переборка)**. Изолировать ресурсы под разные задачи, чтобы сбой одной не утянул всё (как переборки в корабле не дают воде затопить весь трюм).

Все они служат одной цели: **сбой одной части не должен ронять всю систему**.

Итоги главы

Глава про инженерное мышление и устойчивость систем. Главное:

Установка: в системном дизайне нет однозначных ответов — важно рассуждать о компромиссах в контексте задачи.

REST vs RPC — ресурсы vs вызовы методов; REST практически всегда на HTTP, RPC — на чём угодно.

Proxy vs reverse proxy — forward стоит у клиента (клиент знает о нём), reverse — у сервера (клиент не знает); reverse proxy делает балансировку, TLS-терминацию, кеширование.

Rate limiting — с Redis (точно, но медленнее и единая точка отказа) или per-instance (быстро, но неточно при неравномерной балансировке); атомарность через Lua-скрипт; алгоритмы (token/leaky bucket, fixed/sliding window); backpressure для адаптивного управления.

Масштабирование — вертикальное (мощнее машина, есть потолок) vs горизонтальное (больше машин); server-side балансировка (nginx, минимум 2 балансировщика, discovery через DNS A/SRV-записи) vs client-side (клиент сам выбирает инстанс через Consul/etcd).

Устойчивость к медленным зависимостям — развязать через очередь (асинхронность), не дублировать работу воркеров (SKIP LOCKED / партиции), backpressure (адаптивно менять нагрузку), graceful degradation при исчерпании ресурсов (прореживать второстепенное).

Паттерны: circuit breaker, retry с backoff, timeout, fallback, bulkhead — сбой части не должен ронять целое.

В следующей главе — базы данных: SQL, индексы, транзакции, блокировки и всё, что спрашивают про хранение данных.

Глава 9. SQL и базы данных

Введение к главе

База данных — сердце большинства бэкенд-систем. И именно по SQL и БД на собеседовании задают больше всего практических задач: спроектировать схему, написать запрос с JOIN и группировкой, объяснить, почему тормозит запрос, разобраться с блокировками. В задачнике Озона раздел «Базы данных» — один из самых объёмных.

Эта глава — про то, как БД устроена и как ей правильно пользоваться. Мы разберём: SQL-основы и JOIN, устройство индексов (B-Tree) и проектирование составных индексов, ACID и транзакции, уровни изоляции и аномалии, блокировки и MVCC, constraints, а также практические темы — N+1, connection pool, EXPLAIN, пагинацию и миграции.

Примеры будут на **PostgreSQL** (самая частая БД в Go-бэкенде), но большинство концепций универсальны. Где поведение специфично для Postgres или отличается в MySQL — я это отмечу.

9.1. SQL basics

Что такое SQL и реляционная модель

SQL (Structured Query Language) — язык для работы с реляционными базами данных. **Реляционная** модель означает, что данные хранятся в **таблицах** (отношениях), состоящих из строк (записей) и столбцов (полей), а между таблицами есть **связи**.

Ключевая идея реляционной модели — **нормализация**: данные не дублируются, а разносятся по таблицам и связываются через ключи. Например, вместо того чтобы в каждом заказе хранить полное имя и адрес пользователя, мы храним `user_id` — ссылку на таблицу пользователей. Один пользователь, много заказов, данные не дублируются.

Основные группы команд

SQL-команды делят на группы по назначению:

- **DDL (Data Definition Language)** — определение структуры: `CREATE`, `ALTER`, `DROP` (создать/изменить/удалить таблицу).
- **DML (Data Manipulation Language)** — работа с данными: `SELECT`, `INSERT`, `UPDATE`, `DELETE`.
- **DCL (Data Control Language)** — права доступа: `GRANT`, `REVOKE`.
- **TCL (Transaction Control Language)** — управление транзакциями: `BEGIN`, `COMMIT`, `ROLLBACK`.

Анатомия SELECT

`SELECT` — самая используемая команда. Разберём её полную структуру и, что важно, **порядок выполнения** (он отличается от порядка написания!):

```

SELECT user_id, COUNT(*) AS order_count -- 5. что вернуть
FROM orders -- 1. откуда
WHERE price_total >= 1000 -- 2. фильтр СТРОК (до группировки)
GROUP BY user_id -- 3. группировка
HAVING COUNT(*) > 2 -- 4. фильтр ГРУПП (после группировки)
ORDER BY order_count DESC -- 6. сортировка
LIMIT 10; -- 7. ограничение количества

```

Порядок логического выполнения: **FROM** → **WHERE** → **GROUP BY** → **HAVING** → **SELECT** → **ORDER BY** → **LIMIT**. Это важно понимать, потому что объясняет многие вещи — например, почему в **WHERE** нельзя использовать алиас из **SELECT** (алиас ещё не «существует», **SELECT** выполняется позже), а в **ORDER BY** можно (оно после **SELECT**).

WHERE vs HAVING — частый вопрос

Это прямой вопрос из задачника (задача про заказы с группировкой), и его любят гонять. Разница вытекает из порядка выполнения выше:

- **WHERE** фильтрует **строки** — работает **до** группировки, с отдельными записями.
- **HAVING** фильтрует **группы** — работает **после** **GROUP BY**, с агрегированными результатами.

```

-- WHERE: отобрать заказы дороже 1000 (фильтр строк),
-- HAVING: оставить пользователей, у кого таких заказов больше 2 (фильтр групп):
SELECT user_id, COUNT(*) AS cnt
FROM orders
WHERE price_total >= 1000 -- сначала отсеиваем дешёвые заказы
GROUP BY user_id
HAVING COUNT(*) > 2; -- потом отсеиваем пользователей с малым числом заказов

```

Ключевое правило: **агрегатные функции** (**COUNT**, **SUM**, **AVG**, **MAX**, **MIN**) нельзя использовать в **WHERE** — они появляются только после группировки, поэтому для них есть **HAVING**. А обычные условия по полям лучше ставить в **WHERE** (эффективнее — отсеиваем строки раньше, до группировки).

Агрегация и GROUP BY

GROUP BY объединяет строки с одинаковым значением в группы, а агрегатные функции считают что-то по каждой группе:

```

-- Количество заказов и сумма по каждому пользователю:
SELECT user_id, COUNT(*) AS orders, SUM(price_total) AS total
FROM orders
GROUP BY user_id;

```

Важное правило (частая ошибка): если есть **GROUP BY**, то в **SELECT** можно указывать только **поля из GROUP BY** и **агрегатные функции**. Нельзя выбрать обычное поле, которого нет в группировке, — БД не знает, какое из значений группы взять. Из задачника прямо отмечено: агрегирующие функции без **GROUP BY** не работают так, как ожидается, а поля вне группировки вызовут ошибку.

9.2. JOIN и команды SELECT

JOIN — соединение таблиц по условию. Это то, ради чего существует реляционная модель: данные разнесены по таблицам, а JOIN их соединяет обратно при запросе. Разберём типы и когда какой.

Зачем нужен JOIN

Вспомни: мы храним `user_id` в заказах, а не дублируем данные пользователя. Но когда нужно показать «заказы с именами покупателей», надо **соединить** таблицу заказов с таблицей пользователей по `user_id`. Это и делает JOIN.

Типы соединений

INNER JOIN — только строки, у которых **есть совпадение в обеих** таблицах. Самый частый.

```
SELECT u.name, o.total
FROM users u
INNER JOIN orders o ON u.id = o.user_id;
-- вернёт только пользователей, у которых ЕСТЬ заказы
```

LEFT JOIN (LEFT OUTER JOIN) — **все** строки из левой таблицы + совпадения из правой. Если совпадения нет — поля правой таблицы будут `NULL`.

```
SELECT u.name, o.total
FROM users u
LEFT JOIN orders o ON u.id = o.user_id;
-- вернёт ВСЕХ пользователей; у кого нет заказов — total будет NULL
```

RIGHT JOIN — зеркально: все строки правой таблицы + совпадения левой. На практике используется редко (обычно переписывают через LEFT JOIN, поменяв таблицы местами).

FULL OUTER JOIN — все строки обеих таблиц; где нет совпадений — `NULL` с соответствующей стороны.

CROSS JOIN — декартово произведение: каждая строка левой с каждой строкой правой. Редко нужен намеренно.



Классическая ловушка: LEFT JOIN + WHERE

Тонкий момент из задачника (задача про покупки до бана). Разберём, потому что на нём часто попадаются.

Задача: вывести покупки пользователей, совершённые **до** того, как их забанили (а забанены не все). Нужен LEFT JOIN к таблице банов, но с осторожностью:

```
SELECT DISTINCT u.id, u.firstname, p.item_id
FROM users u
JOIN purchase p ON u.id = p.user_id
LEFT JOIN ban_list bl ON u.id = bl.user_id -- LEFT, чтобы не потерять НЕзабаненных
WHERE bl.user_id IS NULL                  -- пользователь не забанен
     OR bl.date_from > p.date;           -- ИЛИ забанен позже покупки
```

Почему именно **LEFT JOIN**, а не обычный **INNER**? Если сделать **INNER JOIN** с **ban_list**, в результат попадут **только забаненные** пользователи (у остальных нет строки в **ban_list**, и INNER их отбросит). А нам нужны все. LEFT JOIN сохраняет незабаненных, у которых **bl.user_id** будет **NULL** — и условие **bl.user_id IS NULL** их пропускает.

Общее правило: **если условие в WHERE ссылается на поле из правой таблицы LEFT JOIN без проверки на NULL — LEFT JOIN «превращается» в INNER**, потому что **NULL** не проходит обычные сравнения. Помни про **IS NULL**.

Связь N-M через промежуточную таблицу

Из задачника (задача про чаты): пользователь может быть в нескольких чатах, чат — содержать нескольких пользователей. Это связь **многие-ко-многим (N-M)**, и в реляционной модели она реализуется через **промежуточную (junction) таблицу**:

```
CREATE TABLE user_chats (      -- таблица-связка для N-M
    user_id INT,
    chat_id INT,
    PRIMARY KEY (user_id, chat_id) -- составной ключ из двух FK
);

-- Выбрать все чаты пользователя "Вася":
SELECT c.id, c.name
FROM users u
JOIN user_chats uc ON uc.user_id = u.id
JOIN chats c ON c.id = uc.chat_id
WHERE u.name = 'Вася';
```

Промежуточная таблица **user_chats** хранит пары (пользователь, чат). Каждая связь — отдельная строка. Так реализуется N-M: один пользователь → много строк в **user_chats** → много чатов, и наоборот.

Работа с NULL

NULL — это «отсутствие значения», и он ведёт себя коварно (из задачника, вопросы про constraints): - Любое сравнение с **NULL** даёт не **true / false**, а **NULL** (то есть «неизвестно»). **NULL = NULL** — это **NULL**, а не **true**! - Поэтому для проверки на NULL используют **IS NULL / IS NOT NULL**, а не **= NULL**. - Арифметика с NULL даёт NULL: **5 + NULL = NULL**. - Чтобы подставить значение вместо NULL, используют **COALESCE(поле, значение_по_умолчанию)**.


```
SELECT b - COALESCE(c, 0) AS delta FROM t; -- если c = NULL, берём 0
```

9.3. Индексы и B-Tree

Индексы — тема, которую ты уже глубоко проработал в своих конспектах (физическое хранение, TID, Heap vs Clustered, ESR-правило). Я не буду это дублировать, а соберу суть и добавлю то, что важно для цельной картины и чего в конспектах меньше.

Кратко: что такое индекс и зачем

Индекс — вспомогательная структура данных, которая хранит значения ключа в **отсортированном** виде вместе с указателями на физические строки, чтобы находить данные быстро — за $O(\log N)$ вместо $O(N)$.

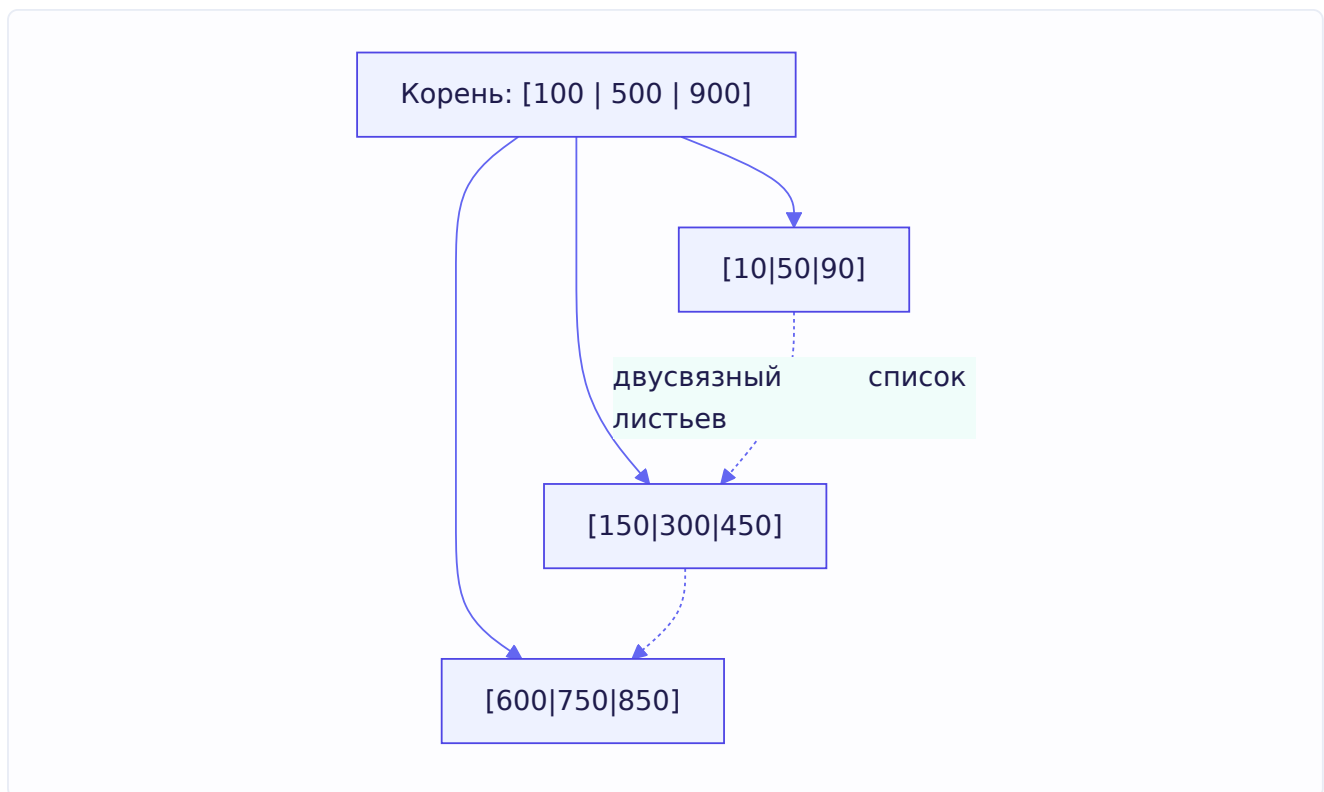
Без индекса БД делает **Seq Scan** (последовательное чтение всей таблицы) — проверяет каждую строку. На миллионах строк это катастрофа. С индексом — **Index Scan**: бинарный поиск/проход по дереву находит нужное за считанные шаги. (Детали механики — в твоём конспекте про Heap и TID.)

Почему именно B-Tree

Стандартный индекс — **B-Tree** (точнее B+Tree). Разберём, почему именно эта структура, а не, скажем, обычное бинарное дерево или хеш-таблица.

B-Tree — это сбалансированное дерево, где: - Каждый узел содержит **много** ключей (не один, как в бинарном дереве) — сотни. - Дерево **широкое и низкое** — высота всего 3-4 уровня даже для миллиардов записей. - Листья связаны в **двусвязный список** — для быстрого прохода по диапазонам.

Почему «широкое и низкое» критично? Вспомни из главы 9 (и своих конспектов) — БД читает данные **страницами** (блоками по 8 КБ). Каждый переход между узлами дерева — это потенциальное чтение страницы с диска (дорого!). Обычное бинарное дерево на миллиард элементов имело бы высоту $\sim 30 \rightarrow 30$ обращений к диску. B-Tree с сотнями ключей в узле имеет высоту 3-4 $\rightarrow$ всего 3-4 обращения. Разница колоссальна.



Почему B-Tree лучше **хеш-индекса** для большинства задач: хеш даёт $O(1)$ для точного поиска (`WHERE id = 5`), но **бесполезен** для диапазонов (`WHERE age > 20`), сортировки (`ORDER BY`) и поиска по префиксу (`LIKE 'abc%'`). B-Tree же хорош и для точного поиска, и для диапазонов (благодаря сортировке и связанным листьям), и для сортировки. Поэтому B-Tree — универсальный выбор по умолчанию.

Другие типы индексов

Кратко (детали — в твоих конспектах): - **Hash** — только точный поиск за $O(1)$, не умеет диапазоны/сортировку. - **GIN** — для поиска внутри составных объектов (массивы, JSONB, полнотекстовый поиск). - **B-Tree** — универсальный, по умолчанию.

Составные индексы и правило ESR

Ты подробно разобрал ESR (Equality → Sort → Range) в конспекте, поэтому здесь — только связь с задачей из задачника, чтобы закрепить.

Задача из задачника: построить оптимальный индекс для

```
SELECT * FROM employee
WHERE sex = 'm' AND salary > 300000 AND age = 20
ORDER BY created_at;
```

Применяем ESR: - **Equality (=)**: `sex = 'm'` и `age = 20` → идут первыми. - **Sort (ORDER BY)**: `created_at` → после равенств. - **Range (>)**: `salary > 300000` → в конце.

Оптимальный индекс: `(sex, age, created_at, salary)`. Почему `salary` в конце, хотя по нему фильтр? Потому что диапазон «ломает» порядок последующих колонок (как ты записал в

конспекте: диапазон — «убийца» сортировки). Если поставить `salary` перед `created_at`, сортировка по `created_at` внутри диапазона зарплат сломается, и БД придётся досортировывать вручную (filesort).

Когда индекс НЕ работает

Ты отлично разобрал это в конспекте (функции над колонкой, низкая селективность, `LIKE '%...'`, неявное приведение типов). Добавлю лишь связку с одним из вопросов задачника про EXPLAIN: именно эти причины проявляются в плане запроса как неожиданный **Seq Scan** там, где ты ждал Index Scan. Умение по EXPLAIN увидеть «ага, тут Seq Scan, потому что я обернул колонку функцией» — практический навык, к которому вернёмся в 9.11.

Цена индексов

Важный нюанс, который стоит проговорить: индексы **не бесплатны**. Каждый индекс: - **Занимает место** на диске (это отдельная структура). - **Замедляет запись** (`INSERT` / `UPDATE` / `DELETE`) — при изменении данных нужно обновлять и все индексы таблицы.

Поэтому индексы ставят обдуманно: под реальные запросы, а не «на всякий случай на каждую колонку». Из твоего конспекта — не пихать больше 4-5 колонок в составной индекс без необходимости. Баланс между скоростью чтения и стоимостью записи — часть инженерного решения.

9.4. ACID

Ты подробно разобрал ACID в конспекте, поэтому здесь — сжатая суть плюс акцент на том, как это связано с Go-кодом. **ACID** — четыре свойства, которые делают транзакции надёжными.

Транзакция — это группа SQL-команд, которую БД воспринимает как **единое целое**: либо выполняются все, либо ни одной. Классика — перевод денег: списать у одного, зачислить другому. Если между этими шагами что-то сломается, недопустимо, чтобы деньги списались, но не пришли.

Четыре свойства ACID:

- **A — Atomicity (Атомарность)**. «Всё или ничего». Споткнулся хоть один шаг → откат всей транзакции (`ROLLBACK`), данные возвращаются в исходное состояние. Всё прошло → `COMMIT`.
- **C — Consistency (Согласованность)**. БД переходит из одного корректного состояния в другое; все правила (типы, ограничения, связи) соблюдаются. Транзакция не может оставить данные в невалидном состоянии.
- **I — Isolation (Изоляция)**. Параллельные транзакции не мешают друг другу; результат такой, как будто они шли по очереди. Насколько строго — регулируется уровнями изоляции (9.6).
- **D — Durability (Стойкость)**. Если сказано «COMMIT успешен» — данные записаны намертво, переживут даже сбой сервера. В Postgres это обеспечивает **WAL (Write-Ahead Log)**: изменения сначала пишутся в лог на диск, и по нему данные восстанавливаются после сбоя.

9.5. Транзакции на практике

Разберём, как транзакции работают в SQL и как ими управлять из Go — это то, чего в конспектах меньше, а на практике критично.

Управление транзакциями в SQL

```
BEGIN;                                -- начать транзакцию
UPDATE accounts SET balance = balance - 1000 WHERE id = 1; -- списать
UPDATE accounts SET balance = balance + 1000 WHERE id = 2; -- зачислить
COMMIT;                                -- зафиксировать (или ROLLBACK для отмены)
```

Если между `UPDATE`-ами что-то пойдёт не так, вместо `COMMIT` делается `ROLLBACK`, и оба изменения отменяются — атомарность в действии.

Savepoint — частичный откат

Внутри транзакции можно ставить **точки сохранения** (`SAVEPOINT`) и откатываться к ним, не отменяя всю транзакцию:

```
BEGIN;
INSERT INTO orders ...;
SAVEPOINT sp1;
INSERT INTO order_items ...; -- если тут ошибка,
ROLLBACK TO sp1;             -- откатим только это, заказ останется
COMMIT;
```

Транзакции в Go — паттерн с defer

Вот важная практика для бэкендера — как правильно работать с транзакциями через `database/sql`. Каноничный паттерн:

```

func Transfer(ctx context.Context, db *sql.DB, from, to int, amount int) error {
    // Начинаем транзакцию (с контекстом — глава 3):
    tx, err := db.BeginTx(ctx, nil)
    if err != nil {
        return err
    }
    // defer с откатом — сработает, если не дойдём до Commit:
    defer tx.Rollback() // если Commit уже прошёл, Rollback вернёт ошибку, которую игнорируем

    // Списываем:
    _, err = tx.ExecContext(ctx,
        "UPDATE accounts SET balance = balance - $1 WHERE id = $2", amount, from)
    if err != nil {
        return err // defer сделает Rollback
    }

    // Зачисляем:
    _, err = tx.ExecContext(ctx,
        "UPDATE accounts SET balance = balance + $1 WHERE id = $2", amount, to)
    if err != nil {
        return err // defer сделает Rollback
    }

    return tx.Commit() // всё ок — фиксируем
}

```

Разберём приём с `defer tx.Rollback()` — он элегантен и связан с главой 5: - `defer tx.Rollback()` ставится **сразу** после начала транзакции. - Если на любом шаге вернём `err` — `defer` выполнит `Rollback`, транзакция откатится. Не нужно писать `Rollback` в каждой ветке ошибки. - Если дошли до `tx.Commit()` — транзакция уже зафиксирована, и последующий `tx.Rollback()` из `defer` просто вернёт ошибку «транзакция уже завершена», которую мы игнорируем (это безопасно).

Это идиоматичный, защищённый от ошибок паттерн: транзакция **гарантированно** либо закоммитится, либо откатится, что бы ни случилось. Здесь сходятся `context` (отмена/таймаут запроса, глава 3), `defer` (глава 5) и транзакции.

Prepared statements — и защита от SQL-инъекций

Обрати внимание на `$1`, `$2` в запросах выше — это **параметры-плейсхолдеры**, а не конкатенация строк. Это критически важно для безопасности. Сравни:

```

// ОПАСНО — SQL-инъекция! Никогда так не делай:
query := "SELECT * FROM users WHERE name = '" + userInput + "'"
// если userInput = "'; DROP TABLE users; --" — катастрофа

// ПРАВИЛЬНО — параметризованный запрос:
db.QueryContext(ctx, "SELECT * FROM users WHERE name = $1", userInput)

```

Почему параметры защищают: БД получает текст запроса и данные **отдельно**. Данные никогда не интерпретируются как SQL-код — они всегда только значения. Даже если в `userInput` будет `'; DROP TABLE users; --`, это станет просто строкой для поиска имени, а не командой. **Всегда**

используй **плейсхолдеры для пользовательских данных** — это главная защита от SQL-инъекций.

9.6. Уровни изоляции и аномалии

Ты детально разобрал аномалии и уровни изоляции в конспектах (Lost Update, Dirty Read, Non-repeatable Read, Phantom Read, и три уровня Postgres). Сведу это в компактную опорную структуру для собеседования и добавлю пару акцентов.

Четыре аномалии — кратко

Когда транзакции идут параллельно, возможны аномалии (из задачника — их надо уметь перечислить и кратко описать):

1. **Lost Update (потерянное обновление)** — две транзакции читают, меняют и записывают одну строку; вторая затирает изменения первой. (Пример со складом: продали 2 телефона, а списался 1.)
2. **Dirty Read (грязное чтение)** — транзакция читает незакоммиченные данные другой, которая потом делает ROLLBACK. Прочитали то, чего «не было».
3. **Non-repeatable Read (неповторяющееся чтение)** — читаешь одну строку дважды, между чтениями другая транзакция её изменила и закоммитила → видишь разные значения.
4. **Phantom Read (фантомное чтение)** — то же, но про **набор** строк: между твоими чтениями по условию другая транзакция добавила/удалила строки → «появились фантомы».

Разница non-repeatable vs phantom: первое — про **изменение существующей** строки, второе — про **появление/исчезновение** строк в диапазоне.

Уровни изоляции

Уровни изоляции — это компромисс между защитой от аномалий и производительностью. Чем строже — тем безопаснее, но медленнее. Стандарт SQL определяет 4 уровня; в Postgres реально работают 3 (READ UNCOMMITTED ведёт себя как READ COMMITTED — Postgres принципиально не допускает грязного чтения):

Уровень	Dirty Read	Non-repeatable	Phantom
READ UNCOMMITTED	возможно*	возможно	возможно
READ COMMITTED (по умолчанию)	нет	возможно	возможно
REPEATABLE READ	нет	нет	нет**
SERIALIZABLE	нет	нет	нет

В Postgres `READ UNCOMMITTED = READ COMMITTED` (грязного чтения нет никогда). По стандарту SQL на `REPEATABLE READ` фантомы допускаются, но Postgres строже стандарта* — на `REPEATABLE READ` он блокирует и фантомы тоже (за счёт снапшота, см. MVCC).

Разберём три рабочих уровня Postgres:

- **READ COMMITTED (по умолчанию).** Видишь только закоммиченные данные. Но каждый запрос внутри транзакции видит свежий снимок на момент **начала запроса** — поэтому возможны non-repeatable и phantom reads.
- **REPEATABLE READ.** Транзакция видит снимок данных на момент **своего начала** — база для неё «заморозилась». Отсюда защита от non-repeatable и (в Postgres) phantom. При конфликте обновления выдаёт ошибку сериализации (`could not serialize access`), которую приложение должно обработать повтором.
- **SERIALIZABLE.** Самый строгий — эмулирует последовательное выполнение всех транзакций. Защищает даже от сложных аномалий (write skew). Цена — высокая вероятность ошибок сериализации, приложение должно быть готово повторять транзакции.

Практический вывод

Выбор уровня — компромисс. `READ COMMITTED` (дефолт) подходит большинству случаев. Повышай до `REPEATABLE READ` / `SERIALIZABLE`, когда логика чувствительна к согласованности (финансы, инвентарь), но будь готов обрабатывать ошибки сериализации повтором транзакции. Это прямая связь с getгу-паттерном из главы 8.

9.7. Блокировки и MVCC

Ты глубоко разобрал MVCC (xmin/xmax, версии строк, VACUUM) и блокировки (FOR UPDATE, FOR SHARE, deadlock detector) в конспектах. Сведу суть и добавлю связки с темами книги.

MVCC — многоверсионность

MVCC (Multi-Version Concurrency Control) — механизм, позволяющий читателям и писателям не блокировать друг друга. Главное правило: **читающие не блокируют пишущих, а пишущие не блокируют читающих**.

Как (кратко, детали в твоём конспекте): при `UPDATE` Postgres не меняет строку на месте, а создаёт **новую версию**, пометчая старую служебными полями `xmin` (какая транзакция создала) и `xmax` (какая удалила/обновила). Разные транзакции видят разные версии в зависимости от своего снимка (snapshot). Мёртвые версии потом убирает `VACUUM`.

Связь с главой 1: помнишь, в конце главы про типы данных мы упоминали, что append-only архитектура избегает конфликтов? MVCC — это, по сути, тоже про версионность вместо перезаписи на месте. И твой Uptime Monitor с append-only `check_logs` использует похожую идею — новые записи вместо обновления существующих, что снимает конфликты конкурентной записи.

Блокировки строк

Иногда MVCC мало, и нужно явно заблокировать данные (из твоего конспекта и задачника):

- `SELECT ... FOR UPDATE` — берёшь строку «в заложники»: никто не изменит и не заблокирует её до твоего COMMIT/ROLLBACK. Другие могут только читать. Идеально против Lost Update.

- **SELECT ... FOR SHARE** — разделяемая блокировка: другие тоже могут читать через FOR SHARE, но никто не может изменить, пока ты не закончишь.
- Модификаторы: **NOWAIT** (не ждать, сразу ошибка, если занято), **SKIP LOCKED** (пропустить заблокированные строки).

SKIP LOCKED — связь с очередями задач

Вот отличная связка с главой 8. Помнишь задачу «воркеры не должны брать одни и те же данные»? **SELECT ... FOR UPDATE SKIP LOCKED** — классическое решение:

```
-- Воркер берёт задачу, пропуская уже взятые другими воркерами:
SELECT * FROM tasks
WHERE status = 'pending'
ORDER BY created_at
LIMIT 1
FOR UPDATE SKIP LOCKED; -- если строка заблокирована другим воркером — пропускаем её
```

Каждый воркер блокирует свою строку, а **SKIP LOCKED** заставляет остальных пропускать занятые — так реализуется распределение задач между воркерами через БД без дублирования. Это ровно то, что обсуждалось в 8.5.

Блокировки таблиц

LOCK TABLE ... IN ... MODE блокирует таблицу целиком — для тяжёлых административных операций (изменение структуры). В обычном коде приложения **избегай** — это останавливает всю параллельную работу с таблицей.

Deadlock в БД

Как и в Go (глава 3), в БД возможен **deadlock**: транзакция 1 заблокировала строку А и ждёт В, транзакция 2 заблокировала В и ждёт А — вечное ожидание. Postgres имеет встроенный **deadlock detector**: он мониторит граф ожиданий, обнаруживает цикл и **убивает одну** из транзакций (делает ROLLBACK), чтобы вторая прошла. Приложение получит ошибку и должно повторить упавшую транзакцию.

Профилактика та же, что в главе 3: **захватывать блокировки в одном порядке** во всех транзакциях. Если все берут строки в порядке возрастания id — цикла ожидания не возникнет.

9.8. Constraints — ограничения целостности

Constraints (ограничения) — правила на уровне БД, гарантирующие корректность данных. Это реализация свойства Consistency из ACID: база **физически не примет** данные, нарушающие правила. Разберём каждый тип подробно — это важная тема из задачника.

PRIMARY KEY (первичный ключ)

PRIMARY KEY — главный идентификатор строки (обычно **id**). Это ограничение объединяет **два** правила сразу: - Поле **NOT NULL** — не может быть пустым. - Поле **UNIQUE** — значение

уникально во всей таблице.

Плюс — важнейшая деталь: под капотом для первичного ключа **автоматически создаётся уникальный B-Tree индекс**. Поэтому поиск по РК всегда быстрый, и отдельный индекс на него создавать не нужно.

```
CREATE TABLE users (  
  id SERIAL PRIMARY KEY, -- NOT NULL + UNIQUE + авто-индекс  
  name TEXT NOT NULL  
);
```

Можно составной первичный ключ (из нескольких колонок) — как в junction-таблице `user_chats` из 9.2:

```
PRIMARY KEY (user_id, chat_id) -- уникальна КОМБИНАЦИЯ
```

Из задачника есть тонкая задача на внимательность: если сделать `PRIMARY KEY (a, c)` и вставить строку, где `c` не задано (NULL), — вставка **провалится** с ошибкой, потому что колонки первичного ключа не могут быть NULL.

FOREIGN KEY (внешний ключ)

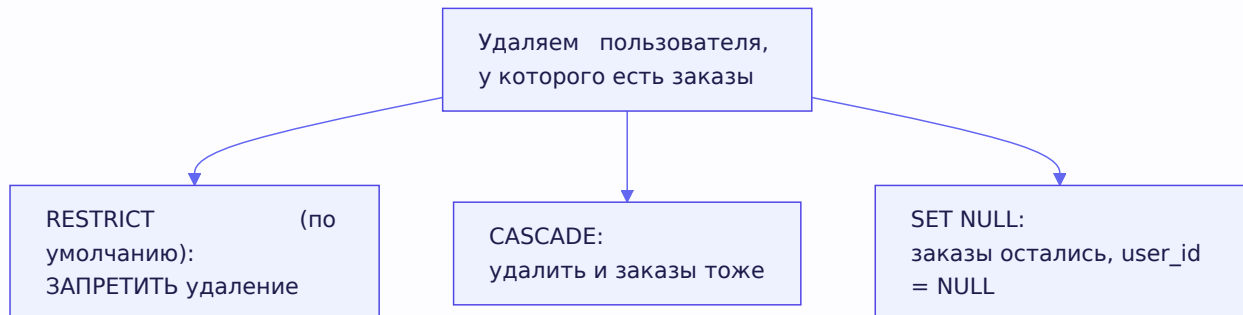
FOREIGN KEY связывает две таблицы, гарантируя **ссылочную целостность**: нельзя сослаться на несуществующую запись. Например, нельзя создать заказ с `user_id = 999`, если пользователя с `id=999` нет.

```
CREATE TABLE orders (  
  id SERIAL PRIMARY KEY,  
  user_id INT REFERENCES users(id), -- внешний ключ на users  
  total INT  
);  
-- INSERT с user_id, которого нет в users → ошибка нарушения FK
```

Теперь — самое важное и частый вопрос: **что делать при удалении** записи, на которую ссылаются? Это задаётся опцией `ON DELETE`:

- `ON DELETE RESTRICT` (по умолчанию) — **запретить** удаление, пока есть ссылающиеся записи. Нельзя удалить пользователя, у которого есть заказы. Самый безопасный вариант.
- `ON DELETE CASCADE` — **каскадно удалить** и ссылающиеся записи. Удалили пользователя → автоматически удалились все его заказы. Удобно, но опасно (можно случайно снести много данных).
- `ON DELETE SET NULL` — оставить ссылающиеся записи, но записать `NULL` в поле внешнего ключа. Удалили пользователя → заказы остались, но `user_id` в них стал NULL (заказ «ничей»).

```
CREATE TABLE orders (
  id SERIAL PRIMARY KEY,
  user_id INT REFERENCES users(id) ON DELETE CASCADE, -- удалить заказы вместе с юзером
  total INT
);
```



Из задачника — важное предостережение про FOREIGN KEY. Плюсы: гарантированный контроль целостности силами БД. Но минусы, которые надо знать: - Можно поймать неожиданное поведение CASCADE/RESTRICT и потерять данные или сломать контроль на уровне приложения. - Сложности при работе с шардами (связанные данные могут быть на разных шардах). - **Самое важное для нагруженных систем:** каскадные операции вызывают блокировки страниц всех связанных таблиц, что бьёт по производительности при массовых изменениях, и повышают риск deadlock'ов.

Поэтому в высоконагруженных системах иногда сознательно **отказываются** от FK, контролируя целостность на уровне приложения, — это осознанный компромисс между надёжностью и производительностью.

UNIQUE

UNIQUE запрещает дубликаты в столбце (или комбинации столбцов). Как и PK, скрытно создаёт уникальный индекс.

```
CREATE TABLE users (
  id SERIAL PRIMARY KEY,
  email TEXT UNIQUE -- два пользователя с одинаковым email невозможны
);

-- UNIQUE на комбинацию:
UNIQUE (first_name, last_name) -- уникальна пара
```

NOT NULL

NOT NULL запрещает пустое значение — поле обязательно должно быть заполнено.

```
name TEXT NOT NULL -- вставка без name → ошибка
```

CHECK

CHECK задаёт кастомное условие, которое должны удовлетворять данные:

```
CREATE TABLE products (  
  price NUMERIC CHECK (price > 0),      -- цена только положительная  
  age INT CHECK (age >= 18)             -- возраст от 18  
);  
-- вставка price = -100 → ошибка нарушения CHECK
```

Из задачника — полезные детали: - При нарушении CHECK происходит ошибка модификации и откат транзакции. - Можно **дать ограничению имя** — тогда в ошибке будет это имя (понятнее): `CONSTRAINT positive_price CHECK (price > 0)`. - CHECK может ссылаться на **несколько столбцов**: `CHECK (a - b >= c)` — проверяется при любом изменении.

DEFAULT

DEFAULT задаёт значение по умолчанию, которое подставится, если при вставке поле не указано:

```
CREATE TABLE orders (  
  status TEXT DEFAULT 'new',             -- если не указан — 'new'  
  created_at TIMESTAMP DEFAULT NOW()     -- если не указан — текущее время  
);
```

Очень удобно для полей вроде `created_at` (автоматически ставится время создания) или `status` (стартовый статус). DEFAULT — не совсем «ограничение» в строгом смысле, но задаётся там же и решает похожую задачу — гарантировать осмысленное значение.

EXCLUDE (Postgres-специфично)

EXCLUDE — продвинутое ограничение Postgres: гарантирует, что при сравнении любых двух строк по заданным столбцам с заданными операторами хотя бы одно сравнение вернёт false или NULL. Используется редко — например, чтобы запретить пересечение временных интервалов бронирования. На собеседовании достаточно знать, что оно существует.

9.9. N+1 проблема

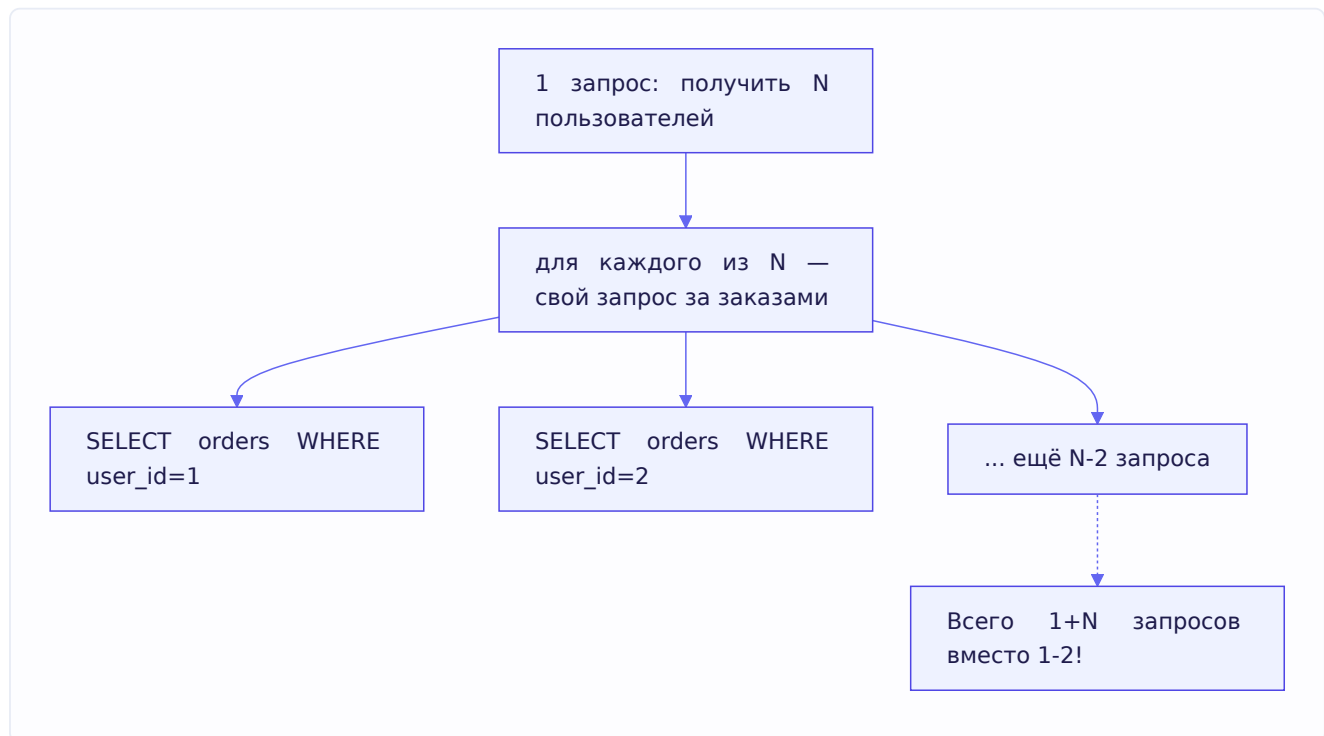
N+1 — одна из самых частых проблем производительности при работе с БД, особенно через ORM. Разберём, потому что её любят спрашивать и она реально встречается в проде.

Суть проблемы

Представь: нужно вывести 10 пользователей и их заказы. Наивный код делает так:

1 запрос: получить список из N пользователей (`SELECT * FROM users LIMIT 10`)
затем для КАЖДОГО пользователя — отдельный запрос за его заказами:
N запросов: `SELECT * FROM orders WHERE user_id = 1`
`SELECT * FROM orders WHERE user_id = 2`
... (ещё 8 раз)

Итого: **1 + N** запросов (отсюда название). Для 10 пользователей — 11 запросов, для 100 — 101. Каждый запрос — round-trip к БД (сетевая задержка), и это убивает производительность.



Решения

1. JOIN — одним запросом. Получить всё сразу соединением:

```
SELECT u.id, u.name, o.id, o.total
FROM users u
LEFT JOIN orders o ON o.user_id = u.id
WHERE u.id IN (...); -- один запрос вместо 1+N
```

2. Загрузка пачкой (IN). Сначала получить пользователей, потом **все** их заказы одним запросом через **IN**:

```
SELECT * FROM users LIMIT 10; -- запрос 1: пользователи
SELECT * FROM orders WHERE user_id IN (1,2,...,10); -- запрос 2: ВСЕ заказы разом
```

Итого 2 запроса вместо 1+N — потом в коде разложить заказы по пользователям.

Оба подхода превращают N+1 в 1-2 запроса. Какой лучше — зависит от ситуации (JOIN может дублировать данные пользователя в каждой строке заказа; batch-загрузка чище, но два запроса). Главное — **распознавать** N+1: если видишь запросы в цикле — это красный флаг.

9.10. Connection pool

Connection pool (пул соединений) — критически важная вещь для производительности, и напрямую связанная с Go.

Зачем нужен пул

Установка соединения с БД — **дорогая** операция: сетевое рукопожатие (глава 6), аутентификация, выделение ресурсов на стороне БД. Если открывать новое соединение на каждый запрос и закрывать после — накладные расходы съедят производительность.

Пул соединений решает это: держим **набор** открытых соединений наготове и **переиспользуем** их. Нужно выполнить запрос — берём свободное соединение из пула, отработали — возвращаем обратно (не закрываем).



Ключевой факт про database/sql в Go

Вот что **обязательно** знать для Go-собеседования: тип `sql.DB` в стандартной библиотеке — это **не одно соединение, а пул соединений**. Это удивляет новичков.

```
db, err := sql.Open("postgres", dsn)
// db — это НЕ соединение! Это пул. Open даже не открывает соединение сразу.
```

`sql.DB` — потокобезопасный пул. Ты можешь спокойно использовать один `db` из множества горутин одновременно — пул сам раздаёт соединения. Не нужно (и не надо!) создавать `sql.DB` на каждый запрос — создай один на всё приложение.

Настройка пула

У пула есть важные параметры, которые в проде нужно настраивать:

```
db.SetMaxOpenConns(25) // максимум одновременно открытых соединений
db.SetMaxIdleConns(5)  // сколько держать в простое (готовыми к переиспользованию)
db.SetConnMaxLifetime(5 * time.Minute) // как долго живёт соединение до пересоздания
```

Зачем настраивать: - **MaxOpenConns** — ограничивает нагрузку на БД. Без лимита при всплеске горутин можно открыть тысячи соединений и завалить БД (у неё тоже лимит). Если все соединения заняты, новый запрос **ждёт** освобождения (или падает по таймауту context). - **MaxIdleConns** — баланс между «держать соединения наготове» (быстрее) и «не занимать ресурсы зря». - **ConnMaxLifetime** — периодически пересоздавать соединения (полезно при работе через балансировщики, смене IP, и чтобы БД не копила старые соединения).

Связь с главой 3: пул — это, по сути, тот же паттерн ограничения параллелизма, что и семафора/worker pool. Число соединений ограничивает, сколько запросов реально идут в БД одновременно, защищая её от перегрузки.

9.11. EXPLAIN — анализ плана запроса

EXPLAIN — инструмент, показывающий, **как** БД собирается выполнить запрос. Незаменим для поиска проблем производительности (в задачнике упоминается в связке с индексами).

Что показывает

EXPLAIN перед запросом выводит **план выполнения** — дерево операций, которые БД применит, с оценкой их стоимости:

```
EXPLAIN SELECT * FROM users WHERE email = 'test@mail.com';
```

EXPLAIN ANALYZE — то же, но **реально выполняет** запрос и показывает фактическое время (не только оценку):

```
EXPLAIN ANALYZE SELECT * FROM users WHERE email = 'test@mail.com';
```

Ключевые узлы плана

Что искать в плане (из задачника — «какой узел однозначно показывает нехватку индекса»):

- **Seq Scan (Sequential Scan)** — последовательное чтение **всей** таблицы. Красный флаг на больших таблицах: почти всегда значит, что **не хватает индекса** (или он не используется — причины из твоего конспекта: функция над колонкой, низкая селективность и т.д.).
- **Index Scan** — поиск через индекс. Хорошо: БД нашла данные по индексу, не читая всю таблицу.
- **Index Only Scan** — ещё лучше: все нужные данные взяты **прямо из индекса**, без обращения к таблице (покрывающий индекс из твоего конспекта).
- **Bitmap Scan** — промежуточный вариант для средней селективности.
- **Nested Loop / Hash Join / Merge Join** — способы соединения таблиц в JOIN.

Как использовать

Практический алгоритм: запрос тормозит → делаешь `EXPLAIN ANALYZE` → видишь `Seq Scan` на большой таблице → понимаешь, что нужен индекс (или запрос мешает его использовать) → добавляешь индекс / переписываешь запрос → снова `EXPLAIN`, убеждаешься, что теперь `Index Scan`. Это ровно тот навык, что проверяют: увидеть по плану, где узкое место, и исправить.

9.12. Pagination — постраничная выдача

Пагинация — разбиение большого результата на страницы. Кажется простой темой, но есть важный нюанс производительности.

OFFSET/LIMIT — простой, но медленный на глубине

Классический способ:

```
SELECT * FROM products ORDER BY id LIMIT 20 OFFSET 40; -- страница 3 (по 20)
```

`OFFSET 40` пропускает первые 40 строк, `LIMIT 20` берёт следующие 20. Просто и понятно. Но есть проблема: чтобы пропустить 40 строк, БД должна их **прочитать и отбросить**. На странице 1000 (`OFFSET 20000`) БД прочтает 20000 строк, чтобы выбросить, и только потом вернёт 20. Чем глубже страница — тем медленнее. На больших таблицах это серьёзная проблема.

Keyset pagination (курсорная) — быстрый способ

Решение — пагинация «по ключу» (keyset / cursor): вместо пропуска по номеру используем **условие по последнему виденному значению**:

```
-- Вместо OFFSET – «дай следующие 20 после id=40»:
SELECT * FROM products WHERE id > 40 ORDER BY id LIMIT 20;
```

Здесь БД по индексу на `id` **сразу прыгает** к нужному месту (`id > 40`) и берёт 20 строк — не читая и не отбрасывая предыдущие. Скорость **не зависит** от глубины страницы. Клиент запоминает последний `id` и передаёт его для следующей страницы.

Keyset (быстро всегда)

по индексу прыгает к
`WHERE id > last`,
берёт `LIMIT` — без
отбрасывания

OFFSET (медленно на глубине)

читает и **ОТБРАСЫВАЕТ**
`OFFSET` строк,
потом берёт `LIMIT`

Компромисс: OFFSET проще и позволяет прыгать на произвольную страницу («страница 5»), но медленный на глубине. Keyset быстрый и стабильный, но позволяет только «вперёд/назад» (нельзя прыгнуть на произвольный номер). Для бесконечной прокрутки (лента) — keyset идеален; для нумерованных страниц с переходом на любую — OFFSET (если данных не слишком много).

9.13. Миграции

Миграции — управляемое изменение схемы БД во времени. Тема практическая и важная для командной работы.

Проблема, которую решают

Схема БД меняется по мере развития проекта: добавили таблицу, новую колонку, индекс. Как применять эти изменения согласованно — на своей машине, у коллег, на тестовом сервере, в проде? Вручную выполнять SQL опасно и невоспроизводимо. Решение — **миграции**: версионированные, повторяемые изменения схемы.

Как работают

Каждая миграция — это пара скриптов (обычно): - **up** — применить изменение (создать таблицу, добавить колонку). - **down** — откатить изменение (для отката, если что-то пошло не так).

```
-- 001_create_users.up.sql
CREATE TABLE users (id SERIAL PRIMARY KEY, name TEXT NOT NULL);

-- 001_create_users.down.sql
DROP TABLE users;
```

Миграции **нумеруются** и применяются по порядку. БД хранит, какие миграции уже применены (в специальной служебной таблице), — поэтому инструмент применяет только новые. Так схема на любой машине приводится к одному состоянию воспроизводимо.

Инструменты в Go

Популярные: `golang-migrate/migrate`, `goose`, `atlas`. Они умеют применять/откатывать миграции, отслеживать версию схемы. Часто миграции запускают при старте сервиса или отдельной командой в CI/CD.

Правила безопасных миграций

Несколько принципов, показывающих зрелость: - **Обратная совместимость**. В проде с нулевым простоем миграция не должна ломать работающий старый код. Например, удаление колонки делают в несколько шагов: сначала перестать её использовать в коде, потом удалить в отдельной миграции. - **Осторожно с блокировками**. Изменение большой таблицы (`ALTER TABLE`) может взять долгую блокировку и остановить работу. В Postgres некоторые операции

(например, создание индекса) можно делать `CONCURRENTLY`, чтобы не блокировать. - **Тестируй откат**. Миграция down должна реально работать — проверяй.

9.14. Проектирование схем

Соберём практику проектирования — это частый формат задач из задачника («опишите предметную область в виде таблиц»).

Типы связей

Три вида связей между сущностями:

- **Один-к-одному (1-1)** — редко; например, пользователь и его расширенный профиль. Реализуется внешним ключом с UNIQUE.
- **Один-ко-многим (1-N)** — самый частый; например, пользователь и его заказы. Реализуется внешним ключом на стороне «многих» (в заказе хранится `user_id`).
- **Многие-ко-многим (N-M)** — например, пользователи и чаты. Реализуется **промежуточной таблицей** (junction), как `user_chats` из 9.2.

Пример: схема сообщений (из задачника)

Задача: пользователь, чат, сообщение. Пользователь может быть в нескольких чатах; сообщение принадлежит ровно одному чату.

```
CREATE TABLE users (  
    id SERIAL PRIMARY KEY,  
    name TEXT NOT NULL,  
    created_at TIMESTAMP NOT NULL DEFAULT NOW()  
);  
  
CREATE TABLE chats (  
    id SERIAL PRIMARY KEY,  
    name TEXT NOT NULL,  
    created_at TIMESTAMP NOT NULL DEFAULT NOW()  
);  
  
-- Связь N-M пользователей и чатов:  
CREATE TABLE user_chats (  
    user_id INT REFERENCES users(id),  
    chat_id INT REFERENCES chats(id),  
    PRIMARY KEY (user_id, chat_id)  
);  
  
CREATE TABLE messages (  
    id SERIAL PRIMARY KEY,  
    content TEXT NOT NULL,  
    author_id INT NOT NULL REFERENCES users(id), -- связь N-1: много сообщений от одного автора  
    chat_id INT NOT NULL REFERENCES chats(id),   -- связь N-1: сообщение в одном чате  
    created_at TIMESTAMP NOT NULL DEFAULT NOW()  
);
```

Обрати внимание: сообщение имеет `chat_id` (одно сообщение — один чат, связь N-1), а связь пользователей и чатов — через `user_chats` (N-M). Правильный выбор между FK и junction-таблицей — суть проектирования схем.

UUID vs автоинкремент — выбор первичного ключа

Частый вопрос из задачника: числовой (автоинкремент) или UUID первичный ключ?

Автоинкремент (SERIAL/BIGSERIAL): - Плюсы: компактный (4-8 байт), последовательный (хорошо для B-Tree и кеша), читаемый. - Минусы: угадываемый (можно перебрать id), выдаёт количество записей, проблемы при слиянии данных из разных источников и при шардировании (конфликты id).

UUID: - Плюсы: уникален глобально (можно генерировать на клиенте/в разных сервисах без координат), не угадывается, удобен при шардировании. - Минусы: больше (16 байт), случайный (хуже для B-Tree — вставки в случайные места дерева, менее эффективно для кеша).

Выбор: автоинкремент — для простых монолитных случаев; UUID — для распределённых систем, где id генерируются в разных местах или важна непредсказуемость. Существуют компромиссы (UUIDv7, ULID) — упорядоченные во времени, сочетающие плюсы обоих.

9.15. Датафикс на большой таблице

Практическая задача из задачника: таблица на миллиард строк активно используется в проде, нужно изменить 10 миллионов строк. Как?

Проблема прямого UPDATE

Наивный `UPDATE ... WHERE условие` на 10 млн строк — катастрофа в проде: - Один огромный `UPDATE` возьмёт блокировки на множество строк надолго, мешая работе пользователей. - Гигантская транзакция раздуёт WAL, нагрузит БД, при откате всё придётся откатывать. - Может привести к простоям.

Решение: батчами

Обновлять **порциями** (батчами), по индексируемым полям:

```
-- Обновляем по 1000-10000 строк за раз, в отдельных транзакциях:
UPDATE big_table SET field = new_value
WHERE id IN (
    SELECT id FROM big_table WHERE условие LIMIT 1000
);
-- повторять, пока есть что обновлять
```

Принципы (из задачника): - **Батчи** по 1000-10000 строк — каждая порция в своей транзакции, короткие блокировки, БД «дышит» между ними. - **По индексируемым полям** — чтобы находить нужные строки быстро, не сканируя всю таблицу. - **Часто помогает житейское наблюдение:** обычно надо поправить только свежие данные (например, за сегодня) — а на

старые забить. Если есть коррелирующий с проблемой индекс (например, дата), можно радикально сократить перебор.

Если индекса нет

Если данные размазаны по всей таблице и нужного индекса нет: сначала отдельным проходом собрать `id` проблемных строк (читая со слеява/реплики, чтобы не грузить мастер), сохранить в файл, потом накатывать изменения пачками по `WHERE id IN (...)`, беря `id` из файла. Так тяжёлое чтение отделяется от записи и не блокирует прод.

Это отличный пример инженерного подхода: не «сделать в лоб», а разбить на управляемые части, минимизируя влияние на работающую систему — та же философия, что в главе 8.

9.16. Шардирование

Шардирование (sharding) — горизонтальное разбиение данных по нескольким БД/серверам. Финальная тема, связанная с масштабированием из главы 8.

Зачем

Когда данные не помещаются на один сервер или нагрузка на одну БД слишком велика, данные **разбивают на шарды** — части, живущие на разных серверах. Каждый шард хранит свою долю данных.

Как выбирать, что шардировать (из задачника)

Задача из задачника: чаты популярны, число пользователей растёт, скоро не хватит места. Что делать? **Шардировать наиболее быстро растущие данные** — в примере это таблица `messages`, потому что она хранит генерируемый пользователями текст и растёт быстрее всего. Пользователи и чаты растут медленнее — их можно оставить.

Принцип: шардируют то, что **растёт быстрее всего** и создаёт основную нагрузку. Нет смысла шардировать маленький справочник.

Ключ шардирования

Данные распределяют по шардам по **ключу шардирования** (shard key) — например, по `user_id` (все данные пользователя на одном шарде) или по хешу `id`. Выбор ключа критичен: - Хороший ключ распределяет данные и нагрузку **равномерно** по шардам. - Плохой ключ создаёт «горячие» шарды (перекос) — один перегружен, другие простаивают.

Сложности

Шардирование — мощно, но дорого по сложности: - **Запросы между шардами** усложняются (JOIN данных с разных шардов — боль). - **Транзакции между шардами** практически невозможны в обычном виде. - **ФК между шардами** не работают (связь с 9.8 — одна из причин отказа от ФК в шардированных системах). - **Ребалансировка** (добавление шарда) сложна.

Поэтому шардирование — крайняя мера, когда вертикальное масштабирование и реплики уже не справляются. Сначала оптимизируют запросы, добавляют индексы, реплики для чтения — и только потом шардируют.

Итоги главы

Большая глава про хранение данных. Главное:

SQL: реляционная модель с нормализацией; порядок выполнения SELECT (FROM→WHERE→GROUP BY→HAVING→SELECT→ORDER BY→LIMIT); WHERE фильтрует строки, HAVING — группы. JOIN соединяет таблицы (INNER — пересечение, LEFT — вся левая); ловушка LEFT JOIN + WHERE без IS NULL; N-M через junction-таблицу.

Индексы: B-Tree (широкое низкое дерево — мало обращений к диску) универсален для точного поиска, диапазонов и сортировки; составные по правилу ESR; индексы ускоряют чтение, но замедляют запись.

Транзакции и ACID: атомарность/согласованность/изоляция/стойкость; в Go — паттерн `defer tx.Rollback()` + `tx.Commit()`; параметры-плейсхолдеры (`$1`) защищают от SQL-инъекций. Аномалии (lost update, dirty/non-repeatable/phantom read) и уровни изоляции (READ COMMITTED по умолчанию → REPEATABLE READ → SERIALIZABLE, в Postgres 3 уровня).

Конкурентность в БД: MVCC (версии строк, читатели не блокируют писателей); блокировки (FOR UPDATE, FOR SHARE, SKIP LOCKED для очередей задач); deadlock detector; порядок блокировок против дедлоков.

Constraints: PRIMARY KEY (NOT NULL + UNIQUE + авто-индекс), FOREIGN KEY (ON DELETE RESTRICT/CASCADE/SET NULL, риски каскадов в нагруженных системах), UNIQUE, NOT NULL, CHECK, DEFAULT.

Практика: N+1 (1+N запросов → JOIN или batch); connection pool (`sql.DB` — это пул, не соединение; настройка MaxOpenConns и т.д.); EXPLAIN (Seq Scan = нет индекса, Index Scan = хорошо); пагинация (OFFSET медленный на глубине, keyset быстрый); миграции (версионированные up/down); проектирование схем (1-1, 1-N, N-M; UUID vs автоинкремент); датафикс батчами; шардирование быстрорастущих данных.

Это была последняя большая теоретическая глава. В следующей — разбор алгоритмических и практических задач с собеседований, где всё изученное применяется на конкретных примерах.

Глава 10. Алгоритмические и скрининг-задачи

Введение к главе

Финальная глава — практическая. Здесь мы разберём типовые задачи, которые дают на скрининге и собеседованиях (все — из задачника Озона), и увидим, как всё изученное в книге применяется на конкретном коде. Это не новая теория, а **тренировка** — умение быстро и правильно писать код под давлением собеседования.

Разберём: LRU cache, in-memory cache с эволюцией под нагрузку, задачи на слайсы, brute force паролей, и разбор реальных code-review задач. По каждой — не только решение, но и разбор ловушек, оценка сложности и «что хотят услышать».

Важный совет по формату скрининга: на таких задачах ценят не только правильный ответ, но и **ход мысли** — проговаривай, что делаешь, оценивай сложность, замечай крайние случаи (пустой ввод, переполнение). Пустой слайс, nil, одинаковые элементы — всегда проверяй их в уме.

10.1. LRU cache

LRU (Least Recently Used) — кеш фиксированного размера, который при переполнении вытесняет **самый давно использованный** элемент. Классическая задача, которую любят на собеседовании, потому что она проверяет владение структурами данных.

Постановка

Кеш имеет максимальный размер. Операции: - **Get(key)** — получить значение (и пометить элемент как «недавно использованный»). - **Put(key, value)** — добавить/обновить. Если превышен размер — удалить самый давно использованный элемент.

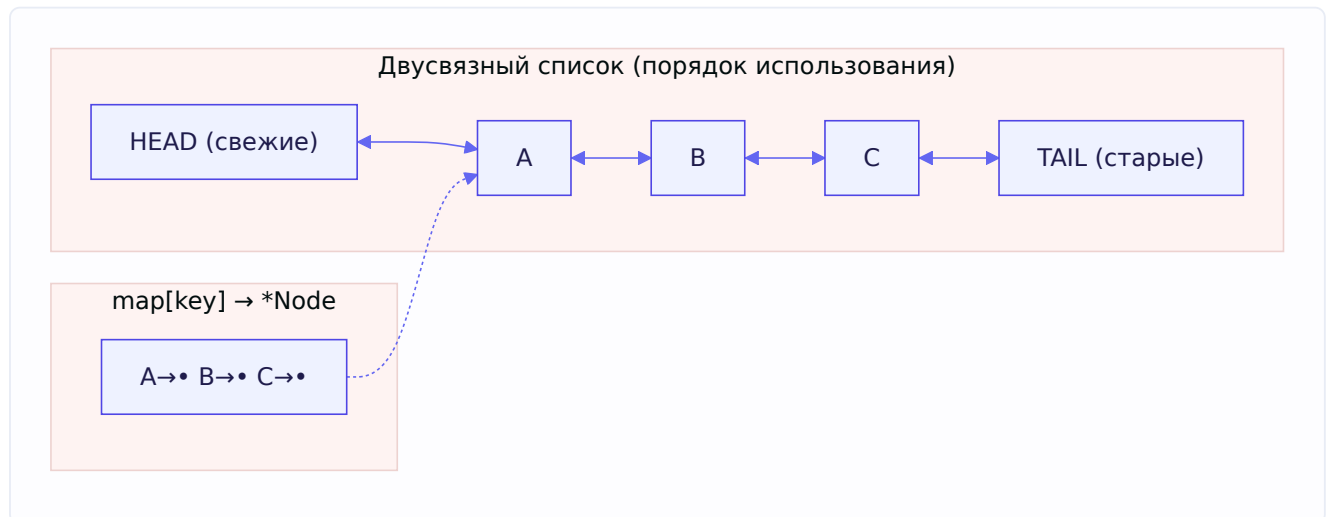
Обе операции должны работать за **O(1)**. Вот пример поведения из задачника:

```
cache = LRU(capacity=2)
put(A, 1) // {A=1}
put(B, 2) // {A=1, B=2}
get(A)    // вернёт 1, теперь A — свежий
put(C, 3) // {A=1, C=3} — B вытеснен (был самым старым)
get(B)    // -1 (не найден)
```

Идея решения: hash map + двусвязный список

Нужны две вещи одновременно: 1. **Быстрый доступ по ключу** за $O(1)$ → хеш-таблица (**map**). 2. **Быстрое отслеживание порядка использования** — кто самый старый, кто самый свежий, с перемещением за $O(1)$ → **двусвязный список**.

Комбинируем: `map` хранит ключ → указатель на узел двусвязного списка. Сам узел лежит в списке, где порядок = порядок использования. Самый свежий — с одного конца, самый старый — с другого.



Почему именно **двусвязный** список? Потому что при обращении к элементу его нужно **вырвать из середины** и переместить в «свежий» конец за $O(1)$. Для этого нужны ссылки и на предыдущий, и на следующий узел — иначе удаление из середины было бы $O(N)$.

Механика операций

- **Get(key):** нашли узел через `map` за $O(1)$ → переместили его в «свежий» конец списка за $O(1)$ (обновили ссылки соседей) → вернули значение.
- **Put(key, value):** если ключ есть — обновили значение, переместили в свежий конец. Если нет — создали узел, добавили в свежий конец, положили в `map`. Если превысили размер — удалили узел с «старого» конца (и из `map`).

Реализация на Go

Go даёт готовый двусвязный список в `container/list`, но на собеседовании часто просят свой. Покажу с `container/list` (короче и идиоматичнее):

```

type entry struct {
    key, value int
}

type LRUCache struct {
    capacity int
    cache    map[int]*list.Element // ключ → элемент списка
    list     *list.List        // порядок использования (front = свежие)
}

func NewLRUCache(capacity int) *LRUCache {
    return &LRUCache{
        capacity: capacity,
        cache:    make(map[int]*list.Element),
        list:     list.New(),
    }
}

func (c *LRUCache) Get(key int) (int, bool) {
    if elem, ok := c.cache[key]; ok {
        c.list.MoveToFront(elem) // пометить как свежий — O(1)
        return elem.Value.(*entry).value, true
    }
    return 0, false
}

func (c *LRUCache) Put(key, value int) {
    if elem, ok := c.cache[key]; ok {
        // ключ уже есть — обновляем и освежаем:
        elem.Value.(*entry).value = value
        c.list.MoveToFront(elem)
        return
    }
    // новый ключ — добавляем в свежий конец:
    elem := c.list.PushFront(&entry{key, value})
    c.cache[key] = elem

    // превысили размер — вытесняем самый старый (с конца):
    if c.list.Len() > c.capacity {
        oldest := c.list.Back()
        if oldest != nil {
            c.list.Remove(oldest)
            delete(c.cache, oldest.Value.(*entry).key)
        }
    }
}

```

Что важно проговорить

- **Сложность $O(1)$** для обеих операций — за счёт связки map (доступ) + двусвязный список (порядок).
- **Почему двусвязный, а не односвязный** — нужно удалять из середины за $O(1)$.
- **Зачем хранить `key` в узле (`entry.key`)** — при вытеснении с конца списка нужно узнать ключ, чтобы удалить его из map. Без этого пришлось бы искать ключ по значению — $O(N)$.

Тут применяются знания из главы 1 (устройство map) и понимание указателей — узлы связаны указателями, перемещение = переприсваивание ссылок.

10.2. In-memory cache — эволюция под нагрузку

Задача из задачника: написать in-memory кеш под интерфейс `Cache { Set(k,v string); Get(k string) (v string, ok bool) }`. Но самое интересное — не базовая реализация, а как она **развивается** по мере роста нагрузки. Это отражает реальный инженерный процесс и связывает главы 1 и 3.

Шаг 1: простейшая реализация с мьютексом

Начинаем с очевидного — map под защитой мьютекса (помним из главы 1: map не concurrent-safe):

```
type Cache struct {
    mu    sync.Mutex
    data  map[string]string
}

func NewCache() *Cache {
    return &Cache{data: make(map[string]string)} // ВАЖНО: инициализировать map!
}

func (c *Cache) Set(k, v string) {
    c.mu.Lock()
    defer c.mu.Unlock()
    c.data[k] = v
}

func (c *Cache) Get(k string) (string, bool) {
    c.mu.Lock()
    defer c.mu.Unlock()
    v, ok := c.data[k]
    return v, ok
}
```

Из задачника — что важно на базовом уровне: правильно **инициализировать map в конструкторе** (иначе паника при записи в nil-map), защитить мьютексом. И уметь подсветить проблемы: переполнение памяти (ключи живут вечно), нет механизма инвалидации.

Шаг 2: RWMutex — раз чтений больше

Если чтений больше, чем записей (типичный кейс кеша — 80/20 из задачника про ревью кода), переходим на `RWMutex` — читатели работают параллельно (глава 3):


```
func (c *Cache) Get(k string) (string, bool) {
    c.mu.RLock()           // много читателей одновременно
    defer c.mu.RUnlock()
    v, ok := c.data[k]
    return v, ok
}
```

Но помним нюанс из главы 3: RWMutex не всегда быстрее — при высокой частоте и коротких операциях накладные расходы могут перевесить. Из задачника прямой вопрос: когда RWMutex будет работать медленно? Ответ: при большом количестве запросов lock contention растёт, и любая запись тормозит чтение.

Шаг 3: шардирование — против lock contention

Когда один мьютекс становится узким местом (все горютины дерутся за него), применяем **шардирование** (главы 1 и 3): разбиваем данные на N частей, у каждой свой мьютекс:

```
type ShardedCache struct {
    shards []*Cache
}

func (c *ShardedCache) getShard(key string) *Cache {
    h := fnv32(key)           // хешируем ключ
    return c.shards[h%uint32(len(c.shards))] // выбираем шард
}

func (c *ShardedCache) Set(k, v string) {
    c.getShard(k).Set(k, v) // лочим только нужный шард
}
```

Из задачника — тонкости шардирования: - **Как шардировать** — по хешу ключа (md5, crc, fnv). - **Сколько шардов** — не меньше числа потоков (GOMAXPROCS), а лучше больше и кратно, чтобы вероятность попадания двух горютин в один шард была мала.

Идея: горютины с разными ключами попадают в разные шарды и не мешают друг другу — contention падает.

Шаг 4: рассуждения продвинутого уровня

Из задачника, что отличает высокий грейд — умение порассуждать о компромиссах: - **Нужна ли инвалидация** — ключи не должны жить вечно; варианты: TTL (время жизни) или LRU (вытеснение, см. 10.1). Что лучше — зависит от задачи. - **Кешировать ли ошибки/отсутствие данных** — иногда полезно кешировать «данных нет», чтобы не долбить хранилище повторно (защита от cache penetration). - **Локальный vs распределённый кеш** — in-memory быстрый, но у каждого инстанса свой (дублирование, рассогласование); централизованный (Redis/memcached) общий, но медленнее (сеть). Компромисс из главы 8. - **Сравнение с memcached** — свой кеш проще, но memcached даёт готовые вытеснение, распределённость, устойчивость. - **Оптимизация для известных ключей** — если ключи заранее известны и это числа от 1 до 10000, можно вместо map использовать **слайс** нужного размера (доступ по индексу ещё быстрее, без хеширования) — привет глава 1.

Эта задача — отличная иллюстрация того, что на собеседовании ценят не «идеальный код сразу», а **эволюцию решения** с пониманием, зачем каждый шаг.

10.3. Задачи на слайсы

Слайсы — любимая тема скрининга (глава 1 объясняет почему — там много ловушек). Разберём типовые задачи из задачника. Все они на первый взгляд простые, но проверяют аккуратность.

Проверка на монотонность

Задача: определить, монотонен ли слайс (везде не убывает ИЛИ везде не возрастает).

```
func isMonotonic(s []int) bool {
    isUp, isDown := true, true
    for i := 1; i < len(s); i++ {
        if s[i-1] > s[i] {
            isUp = false // где-то убывало → не возрастающий
        }
        if s[i-1] < s[i] {
            isDown = false // где-то возросло → не убывающий
        }
    }
    return isUp || isDown
}
```

Изыщество решения: за **один проход** проверяем обе гипотезы сразу. Флаг `isUp` остаётся true, если нигде не было убывания; `isDown` — если нигде не было возрастания. Монотонность = хотя бы одно из них true. Крайние случаи (пустой слайс, один элемент) обрабатываются автоматически — цикл не выполнится, оба флага останутся true. Сложность $O(N)$, память $O(1)$.

Фильтрация: удалить нули (in-place)

Задача: удалить все нули из слайса. Красивое решение — **two pointers** (два указателя), без выделения новой памяти:

```
func remove(in []int) []int {
    j := 0 // указатель на позицию для записи
    for i := 0; i < len(in); i++ {
        if in[i] != 0 {
            in[j] = in[i] // переносим ненулевой элемент вперёд
            j++
        }
    }
    return in[:j] // обрезаем до числа ненулевых
}
```

Как работает: `i` идёт по всем элементам, `j` отмечает, куда писать следующий ненулевой. Ненулевые «сдвигаются» в начало, в конце обрезаем слайс до `j`. Это **in-place** (на месте) — не создаём новый слайс, память $O(1)$. Здесь применяется понимание слайсов из главы 1: `in[:j]` — новый слайс на том же массиве, с длиной `j`.

Zip двух слайсов

Задача: соединить два слайса в слайс пар (по длине меньшего).

```
func zip(s1, s2 []int) [][]int {
    minLen := len(s1)
    if len(s2) < minLen {
        minLen = len(s2)
    }
    res := make([][]int, 0, minLen) // сразу выделяем нужную вместимость!
    for i := 0; i < minLen; i++ {
        res = append(res, []int{s1[i], s2[i]})
    }
    return res
}
```

Обрати внимание на `make([][]int, 0, minLen)` — сразу задаём вместимость (глава 1: избегаем реаллокаций при `append`). Идём до длины **меньшего** слайса — иначе выход за границы.

Из задачника есть усложнение — zip произвольного числа слайсов (variadic):

```
func zip(slices ...[]int) [][]int {
    if len(slices) == 0 {
        return [][]int{}
    }
    minLen := len(slices[0])
    for _, s := range slices[1:] {
        if len(s) < minLen {
            minLen = len(s)
        }
    }
    res := make([][]int, 0, minLen)
    for i := 0; i < minLen; i++ {
        pair := make([]int, 0, len(slices))
        for _, s := range slices {
            pair = append(pair, s[i])
        }
        res = append(res, pair)
    }
    return res
}
```

Генерация N уникальных случайных чисел

Задача: сгенерировать слайс из N уникальных случайных чисел. Ключевая идея — использовать **map** как **множество** для проверки уникальности:

```
func uniqRandn(n int) []int {
    res := make([]int, 0, n)
    seen := make(map[int]struct{}, n) // множество виденных (struct{} = 0 байт!)
    for len(res) < n {
        val := rand.Int()
        if _, ok := seen[val]; ok {
            continue // уже было — пропускаем
        }
        res = append(res, val)
        seen[val] = struct{}{} // отмечаем как виденное
    }
    return res
}
```

Здесь прямая связь с главой 1: `map[int]struct{}` — идиома «множество», где `struct{}` занимает **0 байт** (значение не нужно, важен факт наличия ключа). Проверка `_, ok := seen[val]` за $O(1)$. Цикл крутится, пока не наберём N уникальных.

Задачи-ловушки на слайсы

Помимо алгоритмических, на скрининге дают «что выведет» — они разобраны в главе 1, но напомним суть (это чистая проверка понимания `SliceHeader`): - `append`-ловушки (`y, z := append(x, ...)` → делят массив) — глава 1, раздел 1.5. - Изменение слайса при итерации — принцип «everything passed by value». - Слайс `nums[0:2]` в функции с `append` — изменения данных видны, удлинение нет.

Если ты уверенно решаешь эти «что выведет» — это сигнал глубокого понимания слайсов, который ценят выше алгоритмической задачи.

10.4. Brute force — перебор паролей

Задача из задачника: восстановить пароль по его хешу, зная алфавит символов. Проверяет понимание хеширования, перебора и вычислительной сложности.

Постановка

Дан хеш пароля (например, MD5) и алфавит допустимых символов. Нужно найти пароль перебором — генерировать комбинации, хешировать, сравнивать с целевым хешем.

Идея решения

Перебираем все возможные строки в порядке возрастания длины: сначала все однобуквенные, потом двухбуквенные и т.д. Каждую хешируем и сравниваем:

```

var alphabet = []rune{'a', 'b', 'c', 'd', '1', '2', '3'}

func RecoverPassword(target []byte) string {
    for step := 0; ; step++ {
        guess := genPassword(step)
        if bytes.Equal(hashPassword(guess), target) {
            return guess // нашли!
        }
    }
}

// Генерирует step-ю по счёту комбинацию (как число в системе счисления по базе len(alphabet)):
func genPassword(step int) string {
    var res string
    for {
        res = string(alphabet[step%len(alphabet)]) + res
        step = step/len(alphabet) - 1
        if step < 0 {
            break
        }
    }
    return res
}

```

Идея `genPassword`: трактуем номер комбинации `step` как число в системе счисления с основанием = размер алфавита. Каждая «цифра» этого числа — индекс символа в алфавите. Так последовательно перебираем все комбинации всех длин.

Вычислительная сложность — важно проговорить

Из задачника — ключевые вопросы: - **Сложность перебора**: $O(a^n)$, где `a` — размер алфавита, `n` — длина пароля. Экспоненциальная — каждый дополнительный символ умножает число вариантов на размер алфавита. Вот почему длинные пароли неперебираемы за разумное время. - **Для n-битной хеш-функции** сложность нахождения первого прообраза — $O(2^n)$.

Как защищаются (и как атакуют) — из задачника

Это отличная возможность показать знание безопасности: - **Timing attack** — атака по времени: если сравнение хешей завершается раньше при несовпадении первых байтов, по времени ответа можно подбирать посимвольно. Защита — **сравнение за константное время** (`hmac.Equal`, `subtle.ConstantTimeCompare`), которое всегда проверяет все байты. - **Rainbow tables** — предвычисленные таблицы «хеш → пароль» для быстрого обратного поиска. Если длину пароля ограничить, подбор становится почти константным по времени через такую таблицу. - **Как усложнить подбор** (защита разработчиком): криптостойкое хеширование (bcrypt/argon2 вместо MD5 — они специально медленные), **соль** (случайная добавка к паролю — делает rainbow tables бесполезными), ограничение попыток, сравнение за константное время.

Эта задача связывает алгоритмику (перебор), сложность ($O(a^n)$) и безопасность (соль, timing attack) — комплексная проверка.

10.5. Code review — разбор чужого кода

Отдельный формат задач — дать код с багами и попросить провести ревью. Проверяет не «написать», а «увидеть проблемы» — навык, критичный в реальной работе. Разберём главный пример из задачника.

Задача: ревью кода кеша

Дан код кеша, который будет работать под высокой нагрузкой (чтение/запись 80%/20%):

```
var cache = make(map[string]string)

func GetOrCreate(key, value string) string {
    var m sync.Mutex // ← проблема 1
    m.Lock()
    value = cache[key]
    m.Unlock()
    if value != "" {
        return value
    }
    m.Lock() // ← проблема 2
    cache[key] = value
    m.Unlock()
    return value
}
```

Что нужно заметить (из задачника)

Хороший ревью находит целый набор проблем — перечислю в порядке серьёзности:

1. **Мьютекс — локальная переменная!** `var m sync.Mutex` создаётся **внутри** функции, значит у каждого вызова **свой** мьютекс. Он никого ни от чего не защищает — горутины лочат разные мьютексы. Это фатальный баг: гонки на общей map остаются. Мьютекс должен быть **общим** — вынесен в структуру рядом с map.
2. **Два отдельных лока → гонка между ними.** Между `m.Unlock()` после чтения и `m.Lock()` перед записью есть «окно», где другая горутина может вклиниться. Проверка «есть ли ключ» и запись не атомарны вместе — возможна конкурентная запись. Нужно лочить **один раз** на всю операцию.
3. **Нет defer для Unlock.** Если между Lock и Unlock случится паника — мьютекс останется заблокированным навсегда (глава 3). Идиома — `defer m.Unlock()`.
4. **map не concurrent-safe** — при конкурентной записи будет `fatal error: concurrent map writes` (глава 1).

Рекомендации ревьюера (из задачника)

- Оформить кеш и мьютекс в **структуру** (общий мьютекс, а не локальный).
- Использовать `defer` для Unlock.
- Лочить **один раз** в GetOrCreate (объединить проверку и запись под одной блокировкой).

- Рассмотреть **RWMutex** (раз чтений 80%) — но помнить про его нюансы (глава 3).
- Для высокой нагрузки — **шардирование** или `sync.Map`.

Исправленная версия:

```
type Cache struct {
    mu    sync.RWMutex
    data  map[string]string
}

func (c *Cache) GetOrCreate(key, value string) string {
    // Сначала пробуем прочитать под R-блокировкой (чтений больше):
    c.mu.RLock()
    if v, ok := c.data[key]; ok {
        c.mu.RUnlock()
        return v
    }
    c.mu.RUnlock()

    // Не нашли — берём полную блокировку для записи:
    c.mu.Lock()
    defer c.mu.Unlock()
    // Повторная проверка под Lock (другая горутинa могла успеть записать):
    if v, ok := c.data[key]; ok {
        return v
    }
    c.data[key] = value
    return value
}
```

Обрати внимание на **двойную проверку** (double-checked locking): проверяем под RLock, потом ещё раз под Lock. Это защищает от гонки, когда между отпусканьем RLock и взятием Lock другая горутинa уже записала ключ. Тонкий, но важный паттерн.

Другой пример: баг с типом в assertion

Из задачника — задача «про звёздочку» (разбирали в главе 4): в кеш кладётся **значение** `warehouse.Warehouse`, а assertion делается на **указатель** `*warehouse.Warehouse`. Типы не совпадают → assertion всегда `ok=false` → кеш всегда возвращает `nil` (промах) → кеш не работает. Урок ревью: следить за соответствием типов при работе с `interface{}`.

Что проверяет формат code review

Такие задачи проверяют: - **Внимательность** — заметить локальный мьютекс, несовпадение типов легко пропустить. - **Знание конкурентности** — понять, что два лока не атомарны, что map не потокобезопасен. - **Практический опыт** — предложить `defer`, `RWMutex`, шардирование к месту.

Это ровно та recurring слабость, которую стоит держать в фокусе: **внимание к точным требованиям и деталям**. На code-review задачах особенно важно не пробежать код по диагонали, а вдумчиво проверить каждую строчку на предмет гонок, крайних случаев и несоответствий.

Итоги главы и книги

Эта глава показала, как теория превращается в код на собеседовании:

- **LRU cache** — map + двусвязный список для $O(1)$; почему двусвязный (удаление из середины), зачем хранить ключ в узле.
- **In-memory cache** — эволюция от mutex+map через RWMutex к шардированию; ценят не идеальный код сразу, а понимание, зачем каждый шаг.
- **Задачи на слайсы** — монотонность (один проход, два флага), фильтрация (two pointers, in-place), zip (с предвыделением cap), уникальные числа (map как множество, `struct{}`).
- **Brute force** — перебор как счёт в системе счисления, сложность $O(a^n)$, защита (соль, константное сравнение, timing attack).
- **Code review** — находить баги: локальный мьютекс, неатомарные локи, несовпадение типов; двойная проверка; внимание к деталям.

Финальное слово по всей книге. Мы прошли путь от битов в памяти до системного дизайна: типы и память → рантайм и GMP → конкурентность → интерфейсы и дизайн → ошибки → HTTP и сети → gRPC → архитектура → базы данных → практические задачи. Сквозная идея, которая связывает всё: **понимать, как работает под капотом**, а не заучивать. Когда ты понимаешь механику (почему слайс делит массив, почему горутины лёгкие, почему map не потокобезопасен, почему RWMutex не всегда быстрее), ты можешь вывести ответ на почти любой вопрос, а не вспоминать заученное.

На собеседовании это главное преимущество: не «я знаю ответ», а «я понимаю, почему так, и могу рассуждать». Ты проделал большую работу — теперь у тебя есть и глубина, и практика. Удачи на собеседовании в Ozon — ты к нему хорошо подготовлен.